

RTL Design Sherpa

DDR2 / LPDDR2 Family Controller Micro-Architecture Specification 0.8

October 8, 2026

Table of Contents

List of Figures

No figures in this document.

List of Tables

No tables in this document.

1 Pumice Mas Index

Generated: 2026-10-10

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 mem-ctrl-ip Common IP — Micro-Architecture Specification

Version: 0.1 **Date:** 2026-10-10 **Status:** v0.1. One chapter per common block. The architecture these blocks implement — the layer stack, the descriptor seam, the doctrines — is the family HAS (common_has); this book is the per-block microarchitecture and never restates the architecture. Where the two disagree, the HAS wins and this book is the defect.

> **Read this first.** Every claim in a chapter is one of: (a) read from the RTL it documents (cited `file:line`), (b) inherited from a named rock's measured history (cited to the rock's book or bug record), or (c) an explicitly marked open question. Nothing else is asserted. Silicon-bug history (`BUG-*`, `ISSUE-*`, measured regression numbers) is invariant documentation and is carried verbatim from the RTL comments.

2.1 Document Organization

Note: All chapters linked below for automated document generation.

2.1.1 Front Matter

- Document Information

2.1.2 Chapter 1: The Family Package

- `mc_common_pkg` — types, memtype enum, shared opcodes

2.1.3 Chapter 2: `mc_axi4_layer` and the Host-Side FUBs

- `mc_axi4_layer` — AXI slaves, intakes, coalescing, return ring

2.1.4 Chapter 3: `mc_storage_layer` and the Entry-Pool FUBs

- `mc_storage_layer` — CAMs, read snooping, bank image

2.1.5 Chapter 4: `mc_scheduler_layer` and the Pick Machinery

- `mc_scheduler_layer` — schedulable-entry view, timers, arbiter

2.1.6 Chapter 5: `mc_training_layer`

- `mc_training_layer` — shell, maintenance slot, rock wrappers

2.1.7 Chapter 6: `mc_ras_layer`

- `mc_ras_layer` — skeleton, transform pair, reserved CSR surface

2.1.8 Chapter 7: `mc_dfi_2p1_layer` and the Shared DFI Pipeline

- `mc_dfi_2p1_layer` — DFI 2.1 command/data pipeline + CDC

2.1.9 Chapter 8: `mc_dfi_3p1_layer`

- `mc_dfi_3p1_layer` — the 3.1 generation delta

2.1.10 Chapter 9: `mc_dfi_4p0_layer`

- `mc_dfi_4p0_layer` — the 4.0 generation delta

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Document Information

3.1 DDR2 / LPDDR2 Family Controller Micro-Architecture Specification

Document Number: DDR2-LPDDR2-MAS-001 **Version:** 0.8 **Status:** Draft - reconciled with rearchitected RTL **Classification:** Open Source - MIT License

---

3.2 Document Purpose

This Micro-Architecture Specification (MAS) is the implementation-level companion to the DDR2 / LPDDR2 Hardware Architecture Specification (HAS). Where the HAS answers "what is the architecture and why," this MAS answers "what is in the RTL and how does it work."

The MAS is the document RTL designers, verification engineers, and integrators consult when writing or reviewing SystemVerilog code, building testbenches, and stitching the controller into an SoC.

Target Audience:

- RTL designers implementing the FUBs described in this document
- Verification engineers writing per-FUB cocotb tests
- Integrators stitching the controller into an SoC
- Software engineers writing low-level memory configuration code (driver authors, bring-up engineers)

Companion Documents:

- DDR2/LPDDR2 Family Controller HAS (`../pumice_has/`) — high-level architecture and design rationale
- DDR2/LPDDR2 DFI BFM documentation (in `RTLDesignSherpa-DV`) — verification-side reference

---

3.3 Document Scope

The MAS is the **micro-architecture** view: per-FUB inputs/outputs, internal state, FSM diagrams, datapath timing, register-level pipeline stages, and per-block timing budgets. It assumes the reader has read the HAS for context.

This document covers:

- Top-level integration wiring (`pumice_ctrl`)
- Each FUB's interface signal list, internal storage, datapath flow, FSM, and timing budget
- AXI4 and DFI v2.1 wire-level protocol details specific to this controller
- APB programming interface and the full CSR register map
- Programming sequences (init, refresh, power-down, multi-rank, error handling)
- Build-time and runtime configuration reference

This document does **not** cover:

- Higher-level architectural rationale (see HAS)
- Verification plans or coverage models (see the YAML testplans in `dv/testplans/`)
- Floorplan or layout guidance (project-specific)
- Bit-level CSR definitions — the PeakRDL source `rtl/macro/pumice_csr.rdl` is the source of truth for CSR generation (generated collateral in `regs/generated/`)

3.4 Revision History

Version	Date	Author	Notes
0.1	2026-06-14	RTL Design Sherpa	Initial skeleton — chapter outline, FUB list, port list scaffold
0.2	2026-07-07	RTL Design Sherpa	Narrow-device (x16) support: `DRAM_DEVICE_WIDTH` param; burst-length scaling to device-word units (§15); `addr_mapper` column granularity = device word via `BYTE_OFFSET_WIDTH` (§3); `DFI_PHASE` CSR (rd_phase/wr_phase). Fixes the on-silicon DDR2 read failure (Nexys A7 x16).
0.3	2026-07-07	RTL Design Sherpa	§20 mode_register: document burst length as a fixed-per-instance init constant, decoded (not hardcoded) so the same RTL retargets DDR2 BL4 / DDR3-DDR4 BL8; BC4/burst-chop declared out of

Version	Date	Author	Notes
			scope (always issue full BL).
0.4	2026-07-12	RTL Design Sherpa	Full reconciliation with the rearchitected RTL. ch02 block chapters rewritten to the live FUBs: retired txn_queue/bank_machine/xbank_timers/cmd_encoder/odt_ctrl/standalone page_predictor -> CAMs / FSM-free bank_timer / global_timers / dfi_cmd_formatter / arbiter inline open-page. `addr_mapper` = single `ADDR_MAP.bank_lsb` knob (scheme mux retired). LPDDR2 fully functional (bit-exact JESD209-2F CA + JEDEC MR init). `pumice_top_geared` host-width gearing. CSR map reconciled to the RDL. All block diagrams regenerated.
0.5	2026-09-xx	RTL Design Sherpa	(No revision-history row was recorded when v0.5 was cut; its content is whatever the 2026-07-22 build captured. Noted rather than reconstructed.)
0.6	2026-09-26	RTL Design Sherpa	Paging modes 6/7 (`rbl_static`/`rbl_dyn`) RETIRED and the `pumice_rbl_table` FUB removed — measured on silicon on a workload built to suit them, the mechanism worked (thrash 100% -> 57.8%) and still lost: mode 6 -26% bandwidth, mode 7 bit-identical to plain open page. `PAGE_RBL_CFG` @ 0x07C left a documented HOLE, not reused. Newly documented: the bank timers' advisory lookahead (`safe_*_la_o`), the arbiter's in-flight shadow and FINAL-STAGE timing authority, and the TASK-006 stall counters + `REF_STATS_REF_BUSY`, none of which appeared in any prior revision.
0.7	2026-09-26	RTL Design Sherpa	**Page-policy characterization; the default change is RECOMMENDED but BLOCKED (BUG-003).** Mode 3 (`fixed_open`) with `tr_init = 2` measured strictly dominant over open page on the board at txn_scale=1000: +41.2% `col_major_interleaved_bl4`, +8.6..11.6% `col_major`, exactly flat on `incremental`/`row_major`,

Version	Date	Author	Notes
			no scenario regressing, zero integrity failures. The RDL resets were changed to mode 3 / TR=2 and REVERTED the same day: the component gate caught a read-return-ring assertion (“DFI return beat with NO ticket in flight”) at `rd_gap >= 8`, proven by controlled A/B to be caused by the new reset – a short timeout fires inside a read’s DFI return latency and precharges under an in-flight read (BUG-003, P1). The mechanism finding stands regardless: auto-precharge costs 4.9x the activations and 2x the read latency on identical traffic (160,006 vs 32,400 ACT) because it is uncancellable, whereas a background precharge gated on bank idle is implicitly cancellable. Mode 4 (`adapt_time`) documented as SUBSUMED by mode 3 – its TR decays to `tr_min` and it equals `fixed_open(tr_min)` exactly at three different floors – and `policy_scope=0`’s “per-bank TR” documented as a fiction (`r_mc` is one global counter). Mode 5 (`adapt_access`) documented as unproven and mis-plumbed. Two CSR traps now documented: `tr_init = 0` DISABLES the timeout, and `PAGE_ADAPT_CFG.check_interval = 0` gates the entire mode-4 adjustment off – which is why mode 4 measured inert on every campaign before this one.
0.8	2026-10-08	RTL Design Sherpa	LPDDR2 calibration and training layer documented from committed RTL. New ch02 block pages: `pumice_training_layer` macro, `pumice_zq_ctrl` (periodic ZQCS/ZQCL as MRW-to-MR10 maintenance), `pumice_lp_cal` (one-shot MRR MR32/MR40 DQ-cal sequencer). ch04 contract rows: cmd-FIFO MRR flag (`{mrr, ap, col, row, bank, rank, op}`), formatter MRR CA word (JESD209-2F §5.12, CA3r=H), rd_aligner cal-capture sideband, `trn_cmd` arbiter priority (init > refresh > trn >

Version	Date	Author	Notes
			demand, all-banks-idle grant, `cal_busy` folded into `w_out_safe` per the BUG-003 fire==valid discipline), and the seven `CAL_*` CSRs (0x0A0–0x0F0) transcribed from the RDL.

4 Architecture and Datapath

This chapter is the implementation-level architectural orientation: the live four-layer hierarchy, the read/write datapath flow, and the clocking topology. Per-macro detail is in section 2; per-interface detail is in section 3.

4.1 Layer Hierarchy

The controller is a **four-layer core** wrapped by a top that adds the PeakRDL-generated register block. The top-of-tree is `pumice_top` (`rtl/top/pumice_top.sv`), with an optional host-width wrapper `pumice_top_geared` above it:

```
pumice_top_geared (rtl/top/pumice_top_geared.sv)   -- OPTIONAL host-
width wrapper
  |- axi4_dwidth_converter_wr/_rd (formal IP; only when
HOST_AXI_DATA_WIDTH != DW)
  |- pumice_top (rtl/top/pumice_top.sv)
       |- pumice_csr (regs/generated/rtl/, PeakRDL passthrough cpuif)
       |- pumice_core (rtl/top/pumice_core.sv)
            |- pumice_axi4_layer
(rtl/macro/pumice_axi4_layer.sv)
            |- pumice_scheduler_layer
(rtl/macro/pumice_scheduler_layer.sv)
            |- pumice_training_layer
(rtl/macro/pumice_training_layer.sv, LPDDR2 cal/training)
            |- pumice_dfi_layer
(rtl/macro/pumice_dfi_layer.sv)
```

`pumice_core` wires the four macros, built bottom-up:

Macro (module)	Role	Chapter
`pumice_axi4_layer`	Host AXI4 slave + burst splitters + write/read CAMs + snarf	2.1
`pumice_scheduler_layer`	Arbiter + per-bank/global timers + refresh + init + mode reg	2.2
`pumice_training_layer`	ZQ calibration + DQ-cal (MRR) sequencer + `trn_cmd` mux + cal CDC (LPDDR2 only)	2.5
`pumice_dfi_layer`	Single async-FIFO CDC + DFI-clock command/write/read datapath	2.4

The data path is **not** a standalone macro. Write and read burst buffers live in the CAMs inside `pumice_axi4_layer` (SRAM-backed, de-FSM'd streaming readers); the DFI-clock serializer /

aligner live in pumice_dfi_layer. Section 2.3 documents this split in place of the old data_path_macro.

Configuration (timings, DFI phases, page policy, address map) is delivered to pumice_core on ports, driven **by name** from the CSR hwif_out.* fields in pumice_top. There are no config ports on pumice_top other than the register cpuif – software programs every timing/phase/policy through the register bus.

4.2 Read / Write Datapath: AXI -> split -> intake -> CAM

The host AXI4 face is pumice_axi4_layer. Each host burst is first split at DRAM-burst-byte boundaries by the shared axi_master_wr_splitter / axi_master_rd_splitter (one DRAM burst per split command), then handed to the dumb 1:1 intakes:

- pumice_wr_intake (axi4_slave_wr + AW-meta FIFO + wr-data FIFO + addr_mapper) pushes (bank, row, col, id) plus write data into pumice_wr_data_cam.
- pumice_rd_intake (axi4_slave_rd + addr_mapper + snarf probe) pushes (bank, row, col, id) into pumice_rd_cmd_cam, and probes the write CAM for read-your-write forwarding.

The salient property: there is exactly **one** stage where AXI-layer concepts (burst, ID, write strobe) cross into DRAM-layer concepts (rank, bank, row, column). That stage is addr_mapper inside each intake. Upstream everything is AXI; downstream of the CAMs everything is DRAM.

4.2.1 Why Two CAMs, Not One

The CAM split is between the write and read paths because the two CAMs hold **different metadata**, carry **different data**, and have **different lifetimes**:

CAM	Key	Data / storage	Retired on
pumice_wr_data_cam	(bank, row)	(col, id, age, slot) + write-data SRAM; fill/commit-drain/snarf movers	commit-done (B push)
pumice_rd_cmd_cam	(bank, row)	(col, id, age, slot) + read-return SRAM; return-fill/drain movers	last drained R beat

A single unified CAM would either carry both data sets (the write CAM needs a write-data SRAM the read CAM does not) or force an awkward “is_write” predicate on every scheduler lookup. Keeping them separate lets the read and write paths size independently.

Both CAMs are **de-FSM’d**: each stores burst data in an SRAM and exposes a streaming read engine that is FIFO-fed / oldest-pick beat-counter driven – no active/slot state latch. The r_fdone fill-complete flag gates schedulability (and, on the write side, snarf).

4.2.2 Snarf (read-your-write forwarding)

pumice_rd_intake probes the write CAM before scheduling a read. On a hit the read is streamed directly from the write CAM's SRAM via the **snarf mover** in pumice_wr_data_cam – there is no standalone wr2rd_forward block in the live path. Snarf is limited to unscheduled writes with the same id and same burst length.

4.3 Scheduler and Command Stream

pumice_scheduler_layer queries both CAMs in the (bank, row) dimension via N_LU = NUM_BANKS parallel lookup ports, picks one command with pumice_cmd_arbiter (the open-page decision is **inline** in the arbiter – there is no separate page_predictor in the issue path), and emits a single abstract DRAM command {op, rank, bank, row, col, ap} into an output command FIFO. The scheduler is single-issue, PHY/nphases-agnostic, and runs entirely on aclk. Per-bank JEDEC readiness is stamped by pumice_bank_timers (wrapping the FSM-free bank_timer); cross-bank turnaround windows (tFAW/tRRD/tWTR/tRTW/tCCD) come from global_timers.

4.4 Clocking Topology

Two external clocks; exactly one CDC.

Clock	Domain members
aclk	Host AXI, burst splitters, both CAMs, and the whole command scheduler
dfi_clk	DFI command path, write serializer, read aligner, DFI 2.1 pin bus

The single CDC lives inside pumice_dfi_layer (pumice_dfi_cdc) and is built from **async gaxi FIFOs only** – the command, write-data, and read-data streams plus the init handshake cross there. No other clock crossing exists in the controller. If the SoC's AXI master is on a different clock than aclk, an external clock converter is required upstream. See section 1.3 for the reset topology.

4.5 DFI Phase / Rate Topology

The internal data unit is the **DFI word**: DW = DRAM_BEAT_WIDTH * DFI_RATE (128 by default). One AXI beat == one DFI word == DFI_RATE DRAM beats; one AXI burst (BL/DFI_RATE beats) == one DRAM burst (BL beats). The DFI command path in pumice_dfi_layer phase-packs the abstract command onto the DFI_RATE phases, placing the command and CS/ODT on wr_phase / rd_phase as programmed by the DFI_PHASE CSR; pumice_dfi_wr_serializer and pumice_dfi_rd_aligner handle the DFI-word to per-phase data split. The scheduler never sees the phase dimension – its issue rate is one command per aclk.

The old "AXI_DATA_WIDTH == DRAM_BEAT_WIDTH" coupling is gone. Host-width freedom is a separate concern handled by the optional pumice_top_geared wrapper, which inserts the

formally-verified `axi4_dwidth_converter_wr/_rd` between a host-width AXI slave and the fixed-`DW` core (HOST == DW is a bit-identical generate bypass). See `docs/AXI_DRAM_GEARING_SCOPE.md`.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Top-Level Port List

This chapter is the **wire-level** port list of the `pumice_top` module (`rtl/top/pumice_top.sv`) – the exact signal names, directions, widths, and parameter dependencies that the integration script and SoC top-level use. The optional host-width wrapper `pumice_top_geared` re-exports the same set with a free `HOST_AXI_DATA_WIDTH` on the AXI slave face; see `docs/AXI_DRAM_GEARING_SCOPE.md`.

5.1 Parameters

`pumice_top` is fully parameterized – there is no `MEMTYPE`/`N_PHASES` string build any more; memory type is a runtime CSR field (`PHY_TIMING.memtype`) and the phase count is the integer `DFI_RATE`.

```
module pumice_top #(
    parameter int AXI_ID_WIDTH    = 8,
    parameter int AXI_ADDR_WIDTH  = 32,
    parameter int NUM_RANKS       = 1,
    parameter int NUM_BANKS       = 8,
    parameter int ROW_WIDTH       = 14,
    parameter int COL_WIDTH       = 10,
    parameter int DFI_RATE        = 2,
    parameter int DRAM_BEAT_WIDTH = 64,
    parameter int BL              = 8,   // DRAM beats per burst
    parameter int NUM_ENTRIES     = 8,   // CAM depth
    parameter int N_SRAM_SLOTS    = 8,   // per-CAM burst-data SRAM
slots
    parameter int CSR_ADDR_W      = 12
);
```

5.2 Parameter-Dependent Widths

Derived locally in the module (see `rtl/top/pumice_top.sv`):

- `DW` = `DRAM_BEAT_WIDTH * DFI_RATE` (host AXI data width == DFI word)
- `SW` = `DW/8`
- `IW` = `AXI_ID_WIDTH`
- `AW` = `AXI_ADDR_WIDTH`
- `PHW` = `(DFI_RATE > 1) ? $clog2(DFI_RATE) : 1`
- `DFI_DATA_WIDTH` = `DW`
- `DFI_STRB_WIDTH` = `DW/8`
- `DFI_EN_WIDTH` = `DFI_RATE`

- DFI_VALID_WIDTH = DFI_RATE
- DFI_ADDR_BUS_W = ROW_WIDTH * DFI_RATE
- DFI_BANK_BUS_W = $clog2(NUM_BANKS) * DFI_RATE
- DFI_CTRL_BUS_W = 1 * DFI_RATE
- DFI_CS_BUS_W = NUM_RANKS * DFI_RATE

5.3 Clocks and Reset

Two clocks, two active-low resets. There is no APB CSR clock – the register block runs on aclk via a passthrough cpuif.

Port	Dir	Width	Domain	Notes
aclk	in	1	aclk	Host AXI + CAMs + scheduler
aresetn	in	1	aclk	Active-low async reset
dfi_clk	in	1	dfi_clk	DFI datapath + PHY pin bus
dfi_rstn	in	1	dfi_clk	Active-low async reset
init_done_o	out	1	aclk	Init sequencer done (STATUS)

5.4 Register cpuif (PeakRDL passthrough)

The register block pumice_csr is generated from rtl/macro/pumice_csr.rdl (via bin/peakrdl_generate.py). The top presents the raw PeakRDL passthrough cpuif – an SoC bridges its APB/AXI-Lite onto these signals.

Port	Dir	Width
s_cpuif_req	in	1
s_cpuif_req_is_wr	in	1
s_cpuif_addr	in	CSR_ADDR_W
s_cpuif_wr_data	in	32
s_cpuif_wr_biten	in	32
s_cpuif_req_stall_wr	out	1
s_cpuif_req_stall_rd	out	1
s_cpuif_rd_ack	out	1
s_cpuif_rd_err	out	1
s_cpuif_rd_data	out	32

Port	Dir	Width
s_cpuif_wr_ack	out	1
s_cpuif_wr_err	out	1

All config (timings, DFI phases, page policy, address map) is programmed through this bus by name via the generated dv/tbclasses/pumice_regmap.py; pumice_top fans hwif_out.* into pumice_core's config ports. See section 4 of this MAS (or the RDL) for the register map.

5.5 AXI4 Slave Port List

Standard AXI4 slave, single ID width IW, data width DW (= DFI word). All five channels are present (AW/W/B/AR/R) with the usual side-band (awcache/awprot/awqos/awregion/awlock/awuser and the AR equivalents). s_axi_awuser / s_axi_aruser / s_axi_wuser are single-bit at the top. Widths:

- Address: s_axi_awaddr / s_axi_araddr = AW
- Data: s_axi_wdata / s_axi_rdata = DW; s_axi_wstrb = SW
- ID: s_axi_awid / s_axi_bid / s_axi_arid / s_axi_rid = IW

The exact per-signal declaration is the AW/W/B/AR/R block in rtl/top/pumice_top.sv. AXI side-band awqos/arqos/awregion/arregion are carried through the burst splitters but do not drive scheduler priority.

5.6 DFI 2.1 Master Port List

The DFI pin bus is the wide (per-phase x DFI_RATE) form. Command control strobes are 1-bit-per-phase; address/bank are ROW_WIDTH/$clog2(NUM_BANKS) per phase.

Port	Dir	Width
dfi_address_o	out	DFI_ADDR_BUS_W
dfi_bank_o	out	DFI_BANK_BUS_W
dfi_cas_n_o	out	DFI_CTRL_BUS_W
dfi_ras_n_o	out	DFI_CTRL_BUS_W
dfi_we_n_o	out	DFI_CTRL_BUS_W
dfi_cs_n_o	out	DFI_CS_BUS_W
dfi_odt_o	out	DFI_CS_BUS_W
dfi_wrdata_o	out	DFI_DATA_WIDTH
dfi_wrdata_en_o	out	DFI_EN_WIDTH
dfi_wrdata_mask_o	out	DFI_STRB_WIDTH

Port	Dir	Width
dfi_rddata_en_o	out	DFI_EN_WIDTH
dfi_rddata_i	in	DFI_DATA_WIDTH
dfi_rddata_valid_i	in	DFI_VALID_WIDTH
dfi_init_start_o	out	1
dfi_init_complete_i	in	1

Notes:

- The DDR2 ras/cas/we strobes are present in all builds. For LPDDR2 (PHY_TIMING.memtype = 1) the command rides the CA bus packed onto dfi_address_o and these strobes idle; the memtype is a runtime field, so the same SV header serves both flavors.
- The DFI bus is driven in the dfi_clk domain. a7ddrphy (LiteDRAM's Xilinx 7-series PHY, the board target) handles the internal DFI-to-DRAM gearing; the controller does not drive per-phase read gearing beyond rd_phase.

5.7 SystemVerilog Header Snippet

The canonical source is rtl/top/pumice_top.sv – the module declaration there is authoritative. The parameter block above and the port groups (cpuif, AXI4, DFI 2.1) are a human-readable mirror of that file.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

6 Clocks and Reset

6.1 Clock Domains

The controller has exactly two clock domains. Everything from the host AXI face through the command scheduler runs on `aclk`; the DFI datapath and PHY pin bus run on `dfi_clk`. The register block `pumice_csr` runs on `aclk` (there is no separate APB CSR clock).

Clock	Polarity	Domain members
`aclk`	Posedge	`pumice_csr`, `pumice_axi4_layer` (AXI slave, burst splitters, both CAMs), `pumice_scheduler_layer` (arbiter, bank/global timers, refresh, init, mode register)
`dfi_clk`	Posedge	`pumice_dfi_layer` command path, write serializer, read aligner; the DFI 2.1 pin bus

`aclk` and `dfi_clk` are independent. The DFI-word datapath unit means one FIFO word equals one `dfi_clk` cycle, so the crossing is bubble-free at rate.

6.2 Reset Topology

Reset	Polarity	Type	Domain
`aresetn`	Active-low	Async assert	`aclk`
`dfi_rstn`	Active-low	Async assert	`dfi_clk`

Each domain has its own reset. `pumice_csr` takes `rst = ~aresetn` (the PeakRDL block uses active-high internally). All flops use the repo reset macros (`reset_defs.svh`); SRAMs inside the CAMs have no reset port. The SoC's PMU is expected to drive clean, de-glitched resets – there is no reset-glitch filter in the controller.

6.3 CDC

There is exactly **one datapath** clock-domain crossing in the controller, and it lives in `pumice_dfi_cdc` inside `pumice_dfi_layer`. It is built from **async gaxi FIFOs only** (`N_FLOP_CROSS` = 2 by default). Four things cross it:

Stream	Direction	Payload
Command	`aclk` -> `dfi_clk`	`{mrr, ap, col, row, bank, rank, op}` (`CMD_DW`)
Write data	`aclk` -> `dfi_clk`	`{last, strb, data}` DFI-word

Stream	Direction	Payload
		(WD_DW)
Read data	dfi_clk -> aclk	{last, resp, data} DFI-word (RD_DW)
Init handshake	both	init_start out, init_complete back

There is **no** APB-to-MC CSR crossing, no quiet-point override-staging crossing, and no CDC in the AXI4 datapath – pumice_axi4_layer runs entirely on aclk. If the SoC's AXI master is on a different clock, an external clock converter is required upstream. The register block also runs on aclk, so CSR writes take effect combinationally through hwif_out.* into the config ports of pumice_core (no staging register). Timing/phase/policy fields should be programmed while the controller is idle (before init_start, or at a quiet point) since they feed the timers and phase-packers directly.

Sideband note: the training layer owns a second, non-datapath crossing for the read-aligner calibration capture (cal_expect down, captured beat up) — see ch02/25. It touches no AXI or DRAM data stream.

6.4 Reset / Init Sequence

On power-on:

1. aresetn and dfi_rstn are asserted (both low). The PHY drives dfi_init_complete_i = 0.
2. The SoC deasserts both resets. pumice_csr becomes R/W-able on aclk; software programs the timing, DFI-phase, page-policy, and address-map fields by name (never by hardcoded offset – see dv/tbclasses/pumice_regmap.py). The controller idles with init_done_o = 0; AXI traffic is held off.
3. The init_sequencer (inside pumice_scheduler_layer) drives dfi_init_start_o and walks the per-memtype JEDEC MRS init sequence, emitting init commands into the same arbiter path as normal traffic and waiting on the programmed init timings (t_init_wait, t_dll_wait, t_mrd_wait, t_rp_wait, t_rfc_wait) plus dfi_init_complete_i from the PHY. The mode_register shadow captures the MRS writes and exposes CL/CWL/BL to the DFI layer.
4. On completion the sequencer asserts init_done_o; the controller begins honoring host AXI traffic and the refresh controller is enabled.

Because the CSR block and the scheduler share aclk, no inter-clock handshake gates step 2; software only needs both resets released and the register bus alive.

7 Pipeline Latency and Mux-Level Schematics

Every figure on this page is **derived from the elaborated netlist**, not written by hand, so it cannot drift from the RTL without the generator disagreeing. Regenerate with the commands in the last section; if a number here disagrees with a fresh run, the page is stale and the run is right.

7.1 How the numbers are obtained

Yosys elaborates each block and stops after `proc`, so the netlist is still word-level (`$mux`, `$eq`, `$add`, `$dff`) and maps back to the SystemVerilog rather than to LUTs. `gen_latency.py` then reduces that netlist to **state only** — flops, memories and ports. Combinational logic is never a node; it is the edge between two pieces of state. Nodes are ranked into columns by flop-distance from the input ports, so **one column is one clock**.

Two properties matter for trusting the result:

- The netlist is **flattened** for this view. A parameterised submodule instance carries a Yosys type of `$paramod\<name>\<params>`, and treating that as a primitive makes its internal registers invisible — `pumice_bank_timers` reported 0 registers and 0 clocks before flattening, which is impossible for a timer block. Flattening removes the class of error rather than adding another special case.
- **Feedback is condensed** (Tarjan SCC) before ranking. Without it any register that feeds itself makes the longest path infinite. A path whose longest route runs through a real loop is reported as a lower bound.

Clock and reset nets are excluded before the graph is built; they connect everything to everything and would destroy the staging.

7.2 Per-block pipeline depth

Depth is the longest flop-distance from an input port. Registers counts the merged registers (a bit-blasted `r_bank[2:0]` is one), not raw flop cells.

Block	Depth (clocks)	Registers	State nodes
`addr_mapper`	0	0	8
`dfi_cmd_formatter`	0	0	17
`dfi_signal_pack`	0	0	24
`global_timers`	1	5	24

Block	Depth (clocks)	Registers	State nodes
mode_register	1	1	16
pumice_dfi_cdc	2	53	78
pumice_dfi_rd_aligner	2	3	14
pumice_dfi_wr_serializer	2	2	12
pumice_wr_splitter	2	13	51
powerdown_ctrl	3	5	18
pumice_bank_timers	3	48	72
pumice_cmd_arbiter	3	57	128
pumice_rd_cmd_cam	3	11	49
pumice_rd_intake	3	24	75
refresh_ctrl	3	6	26
pumice_row_pred_table	4	8	18
pumice_rd_return_ring	5	23	41
pumice_wr_data_cam	5	60	120
pumice_page_policy	7	39	70
pumice_wr_intake	7	31	79

Three blocks are purely combinational (`addr_mapper`, `dfi_cmd_formatter`, `dfi_signal_pack`): they transform a command in place and add no clock. The two deepest are the paging predictors and the write intake.

7.3 Datapath latencies worth knowing

Read return, the path that sets read bandwidth:

Path	Flops
dfi_ret_data_i ->	3

Path	Flops
`drain_data_o` (return ring)	

That is slot write, then the block RAM, then the skid — the three register boundaries the ring is built from.

Scheduling, from a CAM entry becoming schedulable to the command issuing:

Path	Flops
`*_sch_*` / `bank_*_ready_i` -> `rd_issue_slot_o`	3
`*_sch_*` / `bank_*_ready_i` -> `wr_commit_slot_o`	3

This is the arbiter's pick pipeline, and it is the reason strict in-order scheduling costs bandwidth under the auto-precharge paging modes: those modes need two dependent commands per access (activate, then column-with- auto-precharge), so each access pays the pick-pipeline traversal twice. See the scheduler chapter and pumice ISSUE-006 (was PUMICE-021).

7.4 Combinational feedthroughs

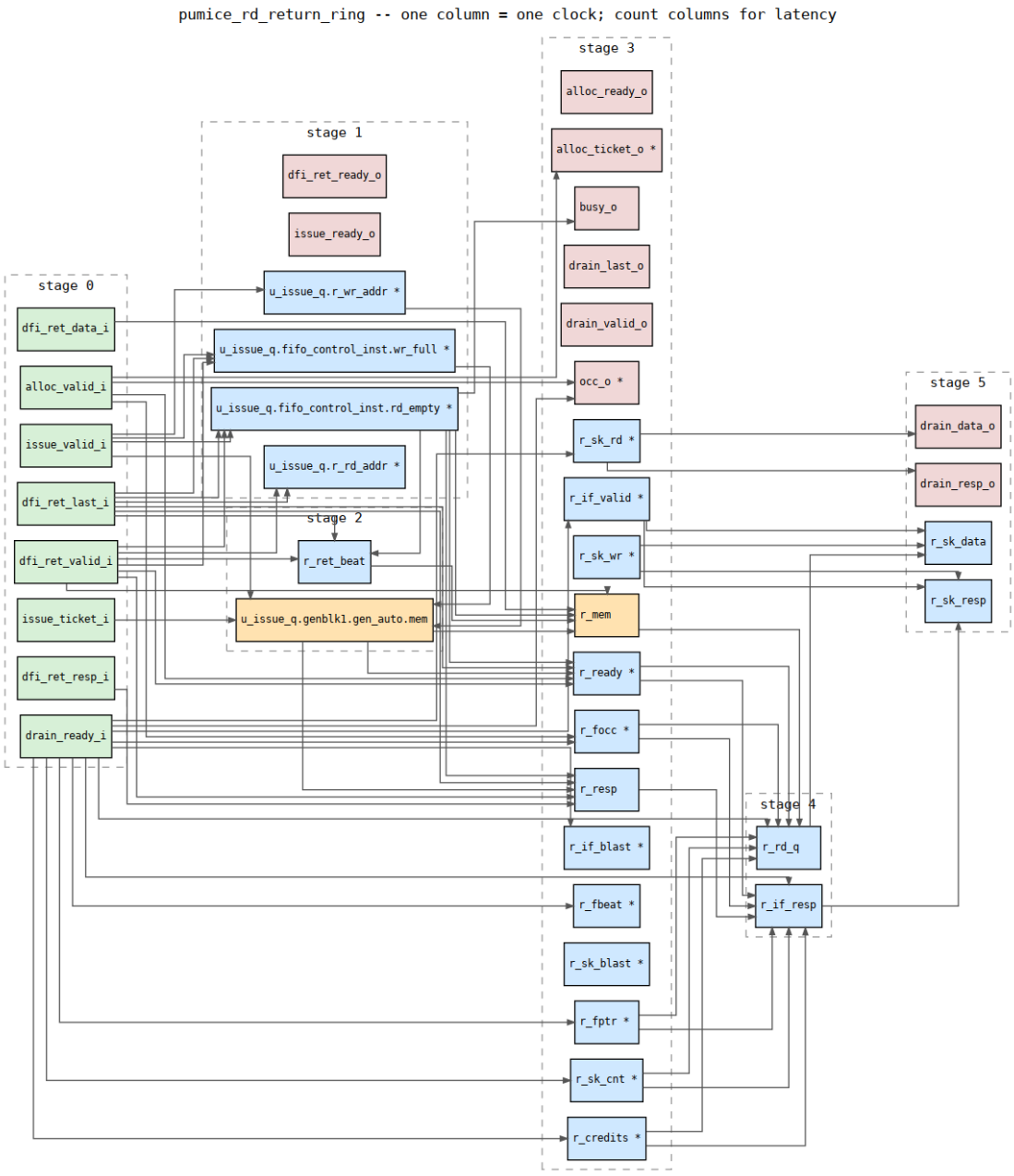
A zero-flop input-to-output path is **not latency, it is a timing path** — typically a ready/valid handshake. The generator reports these separately for exactly that reason. In the return ring, for example, `alloc_ready_o`, `issue_ready_o` and `dfi_ret_ready_o` are all combinational from their respective valids. They belong in timing review, not in a latency budget.

7.5 The schematics

Three views live in `rtl/schematics/`, each answering a different question:

View	File	Question
Mux-level	`<block>.png`	what does this cell-for-cell look like
Dataflow	`<block>.dataflow.png`	which combinational cone is deep (timing)
Latency	`<block>.latency.png`	how many flops from here to there

The mux-level view is faithful but becomes unreadable above roughly 1200 cells, so the largest blocks are deliberately black-boxed rather than rendered into a hairball. The latency view has no such limit, because its node count is the number of registers rather than the number of cells.



Read return ring — one column is one clock; the read data path crosses three

Symbols come from a custom netlistsvg skin. The shipped default aliases the one-hot case primitive onto the two-input mux symbol, so every `case` statement rendered as a 2:1 mux, and it has no symbol at all for the async-reset flop — the element a clock-domain review most needs to see. Both are fixed in `skin/rds-skin.svg`.

7.6 Regenerating

```
cd rtl/schematics
python3 make_skin.py                                    # once, or
after an npm bump
python3 gen_schematics.py --module <block>              # mux-level +
the Yosys JSON
python3 gen_dataflow.py   --module <block> --min-depth 3
python3 gen_latency.py    --module <block> --min-flops 1
```

`gen_latency.py` prunes with `--from` / `--to` (port regexes), `--only` / `--hide` (node regex), `--no-feedback` (drop backward edges from the picture) and `--max-nodes`. Above `--max-nodes` the picture is skipped and the table still prints, which is the right behaviour for the largest blocks.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 Top-Level Integration (pumice_core / pumice_top)

Modules: pumice_core.sv, pumice_top.sv **Location:** rtl/top/ **Category:** Integration (structural wiring) **Status:** Implemented

> **Note:** the early SWAG used a five-*_macro hierarchy (pumice_core_macro, axi_frontend_macro, command_scheduler_macro, data_path_macro, dfi_v21_interface_macro). That naming is retired. The live controller is a four-layer stack instantiated by pumice_core, wrapped by pumice_top (which adds the PeakRDL CSR block) and optionally by pumice_top_geared (which adds a host-width AXI dwidth shim).

8.1 Purpose

The controller top is structural wiring only — every behavioral statement lives in a FUB or a layer macro. Two files make up the top:

- **pumice_core** — wires the four functional layers on their two clocks and exposes host AXI4 plus the DFI 2.1 pin bus. Config arrives on ports.
- **pumice_top** — instantiates pumice_core plus the PeakRDL-generated pumice_csr register block, and drives every core config port **by name** from the CSR hwif_out.* decode. It exposes the register cpuif, host AXI4, and the DFI pin bus.

An optional third file, pumice_top_geared, wraps pumice_top with a free HOST_AXI_DATA_WIDTH and inserts the repo's formally-verified axi4_dwidth_converter_wr / axi4_dwidth_converter_rd between a host-width AXI slave and the fixed-DW core. HOST == DW is a generate bypass (bit-identical). See docs/AXI_DRAM_GEARING_SCOPE.md.

8.2 pumice_core — the four layers

pumice_core instantiates exactly four layer modules and the nets between them. There is no FSM, no CSR decode, and no arithmetic beyond the packed-bus assign for the command word.

```
pumice_core
├── u_ifc       : pumice_axi4_layer          (host AXI + wr/rd CAMs)
[aclk]
├── u_sched     : pumice_scheduler_layer (arbiter + timers +
refresh/init)[aclk]
├── u_training  : pumice_training_layer    (ZQ + lp_cal + trn_cmd mux
```

```
+ CDC)[aclk→dfi_clk]
└── u_dfi       : pumice_dfi_layer         (single async CDC + DFI
datapath)[aclk→dfi_clk]
```

- Host AXI, the scheduler, and both CAMs run on **`aclk`** (`aresetn`).
- The DFI phase-packer and PHY interface run on **`dfi_clk`** (`dfi_rstn`).
- The datapath clock crossing is inside `pumice_dfi_layer` (async gaxi FIFOs only); `pumice_dfi_layer` is instantiated with `.ctl_clk(aclk)` / `.ctl_rstn(aresetn)` on the control side and `.dfi_clk` / `.dfi_rstn` on the PHY side. The training layer owns a second, sideband-only crossing (cal_expect down / captured data up for the read-aligner calibration capture) — see ch02/25.

The internal data unit is the **DFI word** (`DFI_DATA_WIDTH = DRAM_BEAT_WIDTH * DFI_RATE`, default 128). The host AXI data width equals the DFI word; a host that runs a different AXI width uses the external `pumice_top_geared` shim (a separate edge concern — the core does not gear internally).

8.2.1 Layer instantiation inventory (`pumice_core`)

Instance	Module	Count	Clock	Role
`u_ifc`	`pumice_axi4_layer`	1	`aclk`	Host AXI4 face; wr-data CAM + rd-cmd CAM
`u_sched`	`pumice_scheduler_layer`	1	`aclk`	Command arbiter + bank/global timers + refresh + init + mode-register shadow
`u_dfi`	`pumice_dfi_layer`	1	`aclk` → `dfi_clk`	Single async CDC + DFI cmd/wr/rd datapath

There are **no** `generate` fan-out blocks in `pumice_core`. Per-(rank, bank) fan-out (the bank timers) happens one level down, inside `pumice_scheduler_layer`.

8.2.2 Inter-layer nets

`pumice_core` declares and wires four net groups:

1. **Scheduler ↔ IFC CAM ports** — the per-bank scheduler lookup buses (`w_wr_lu_*`, `w_rd_lu_*`), the oldest-entry ports (`w_wr_old_*`, `w_rd_old_*`), the write commit handshake (`w_wr_commit_*`), and the read issue handshake (`w_rd_issue_*`). `N_LU = NUM_BANKS` (one lookup per bank).

2. **IFC commit-data → DFI wrdata** (`w_cm_*`: valid/ready/data/strb/last) and **DFI rddata → IFC rd-return** (`w_ret_*`: valid/ready/data/resp/last).
3. **Scheduler command stream → DFI** — the abstract command `{op, rank, bank, row, col, ap}`, flattened into `w_cmd_data` (width `CMD_DW = 4 + RKW + BKW + ROW_WIDTH + COL_WIDTH + 1`) by a single `assign`.
4. **Init handshake** — `w_init_start` / `w_init_complete` between the scheduler's init sequencer and the DFI layer's PHY-init side.

Note the parameter mapping at the IFC boundary: `pumice_core` passes `.BL(BL / DFI_RATE)` to `pumice_axi4_layer` (the IFC/CAM view of burst length is in DFI words), while `pumice_dfi_layer` receives the full `.BL(BL)` (DRAM beats).

8.3 `pumice_top` — CSR + by-name config

`pumice_top` adds the register block and the config fan-out. It does two things:

1. Instantiates `pumice_csr` (PeakRDL passthrough cpuif) with the `hwif_in` / `hwif_out` struct pair. `hwif_in` is currently tied to `'{default:'0}` — status/observability readback is a follow-up; config-drive is wired first.
2. Instantiates `pumice_core` and drives **every** config port by name from the decoded `hwif_out.*` fields. Representative mappings:

Core port	CSR field (`hwif_out.*`)
`memtype_i`	`PHY_TIMING.memtype` (0 = DDR2, 1 = LPDDR2)
`page_policy_i`	`REFRESH_TUNING.page_policy_or`
`bank_lsb_i`	`ADDR_MAP.bank_lsb`
`hash_en_i`	`ADDR_MAP.hash_en`
`hash_seed_i`	`ADDR_MAP.hash_seed`
`t_rcd_i/t_rp_i/ t_ras_i/t_rc_i`	`TIMINGS_RC_RCD_RP_RAS.*`
`t_wr_i`	`TIMINGS_CL_CWL_WR.tWR`
`t_rtp_i/t_rtw_i`	`TIMINGS_RTP_RTW.*`
`t_faw_i/t_rrd_i/ t_wtr_i/t_ccd_i`	`TIMINGS_RRD_FAW_WTR_CCD.*`
`t_refi_i`	`TIMINGS_RFC_REFI.tREFI`
`refresh_burst_i`	`PHY_TIMING.refresh_burst`
`t_init_wait_i/ t_dll_wait_i`	`INIT_TIMING0.*`
`t_mrd_wait_i/ t_rp_wait_i/ t_rfc_wait_i`	`INIT_TIMING1.*`

Core port	CSR field (`hwif_out.*`)
`rd_phase_i/ wr_phase_i`	`DFI_PHASE.*` (sliced to `[PHW-1:0]`)
`t_phy_wrlat_i/ t_rddata_en_i`	`PHY_TIMING.*`

The CSR block is clocked on `aclk` with `.rst(~aresetn)` (PeakRDL uses an active-high reset; the top inverts `aresetn`). See `rtl/macro/pumice_csr.rdl` for the full field list.

8.4 What the top does not do

- No combinational logic beyond the single command-word `assign` in `pumice_core` and the by-name field selects in `pumice_top`.
- No CSR field expansion in `pumice_core` (that is `pumice_csr`'s job; the top only reads decoded `hwif_out` fields).
- No clock gating (a synthesis-script concern).
- No reset synchronization in the top — each layer takes its own domain reset (`aresetn` for `aclk` logic, `dfi_rstn` for the PHY domain); the CDC inside `pumice_dfi_layer` owns the cross-domain reset discipline.

This intentional poverty makes the top the obvious place for the structural-only sanity check: every line is an instantiation, a net declaration, or a port/field mapping.

9 AXI front-end macro — mc_axi4_layer instantiation

Module: mc_axi4_layer (common IP)
Location: projects/components/mem-ctrl-ip/common-ip/rtl/macro/mc_axi4_layer.sv
Category: Layer-1 macro in pumice_core
Instance: u_ifc (rtl/top/pumice_core.sv:376-499)
Child FUBs: mc_wr_splitter, mc_axi_burst_chopper, mc_wr_intake, mc_rd_intake, mc_rd_return_ring

9.1 Purpose

“Host AXI4 to storage-layer descriptors and the read-return ring.” pumice_core instantiates the common mc_axi4_layer on aclk as the SoC-facing AXI4 slave boundary. It splits host bursts into DFI-word-sized sub-commands, pushes write and read descriptors to mc_storage_layer, sources read-your-write snarf data from the write-data CAM, and forwards the DFI read-return stream to the read path.

Until 2026-10-09 the pumice MAS described a pumice-local pumice_axi4_layer macro (rtl/macro/pumice_axi4_layer.sv) that bundled the splitters, intakes, write-data CAM, and read-command CAM; Phase-2 extracted the shared logic into the common mc_axi4_layer and mc_storage_layer, and the pumice-local module no longer exists.

9.2 Parameter map in pumice_core

Parameter	Bound value in pumice_core	Notes
AXI_ID_WIDTH	IW	IW = AXI_ID_WIDTH (pumice_core.sv:97)
AXI_ADDR_WIDTH	AW	AW = AXI_ADDR_WIDTH (pumice_core.sv:98)
AXI_DATA_WIDTH	DW	DW = DFI_DATA_WIDTH (pumice_core.sv:9

Parameter	Bound value in pumice_core	Notes
		5); one AXI beat == one DFI word
AXI_USER_WIDTH	UW	UW = 1 (pumice_core.sv:99)
DRAM_BEAT_WIDTH	DW	Intentionally set to the AXI/DFI word width, not the physical DRAM beat width (pumice_core.sv:385)
NUM_RANKS	NUM_RANKS	Passed through from core
NUM_BANKS	NUM_BANKS	Passed through from core
ROW_WIDTH	ROW_WIDTH	Passed through from core
COL_WIDTH	COL_WIDTH	Passed through from core
BYTE_OFFSET_WIDTH	BYTE_OFFSET_WIDTH	Passed to the

Parameter	Bound value in pumice_core	Notes
		common address mapper
AXI_BEATS_PER_BURST	BURST_WORDS	DFI words per burst group (pumice_core.sv:391); sub-DFI-word expansion is handled downstream
NUM_ENTRIES	NUM_ENTRIES	Scheduler/storage entry count
N_SRAM_SLOTS	N_SRAM_SLOTS	= NUM_ENTRIES by default
RD_RET_DEPTH	RD_RET_DEPTH	In-flight read count (return-ring depth)
N_SCHED_LU	N_LU	N_LU = NUM_BANKS (pumice_core.sv:103)
AGE_WIDTH	AGE_WIDTH	Free-

Parameter	Bound value in pumice_core	Notes
		running age width

9.3 External boundaries

- **Upstream:** the host AXI4 slave port (`s_axi_*`), the only SoC-facing interface.
- **To `mc_storage_layer` (`u_storage`):** the storage seam — `aw_push_*` + `wd_*` for writes, `snarf_probe_*` / `snarf_rd_*` for read-your-write, `ar_push_*` for reads, and the return-ring ticket ports (`rt_alloc_*`, `rd_iss_*`, `rd_cam_ins_ready_*`).
- **From `mc_dfi_2p1_layer` (`u_dfi`):** the read-return stream `rd_dfi_ret_*` (`valid/ready/data/resp/last`).
- **Status:** `busy_o` is left unconnected in `pumice_core` (`pumice_core.sv:498`).

9.4 Pumice-specific integration notes

Gearing wrapper. The AXI4 layer is built for the compile-time maximum DFI word width and does not know `gear_i` or `bl_i` directly. It frames one AXI beat per DFI word and passes `AXI_BEATS_PER_BURST = BURST_WORDS`. Runtime sub-DFI-word burst packing (`N_SUBCMD > 1`) is expanded later in `mc_dfi_2p1_layer` (`pumice_core.sv:766-836`).

Observed buses. The signals observed at this layer are the host AXI4 channels, the `mc_storage_layer` storage-seam nets, and the `rd_dfi_ret_*` stream from the DFI layer. No pumice-local CAM or datapath buses exist here anymore.

Backpressure thresholds. Write and read acceptance is gated by `mc_storage_layer` ready signals, not by local FIFOs. Read allocation is additionally gated by the return ring (`ar_push_ready_o = rd_cam_ins_ready_o && rt_alloc_ready_i` inside `mc_storage_layer`; see the common layer documentation). The DFI-side read-return FIFO is sized in `mc_dfi_2p1_layer` as `RD_RET_DEPTH * BURST_WORDS` entries (`pumice_core.sv:784`).

9.5 Common-layer reference

For the common layer architecture, child-instance map, burst-chop/split semantics, read-return-ring ordering contract, and complete port list, see common_mas chapter 2: `mc_axi4_layer`.

9.6 Tests

Wrapper-level tests drive `pumice_core` through the AXI4 BFMs (`AXI4MasterWrite` / `AXI4MasterRead` + `AXI4Sequence`). The underlying `mc_axi4_layer` FUBs are covered by the common unit tests referenced in the common_mas chapter.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 pumice_scheduler_layer (command scheduler macro)

Module: pumice_scheduler_layer.sv **Location:** rtl/macro/ **Category:** Layer-2 wrapper around the common mc_scheduler_layer

10.1 Purpose

The pumice_scheduler_layer header comment describes it verbatim:

> Thin pumice wrapper around the common mc_scheduler_layer core. Instantiates the rock-specific init_sequencer, mode_register, refresh_ctrl, page_policy, and command-history checker, and maps the pumice training channel onto the generic maintenance slot.

This macro is no longer the owner of the pick machinery, bank/global timers, refresh controller, init sequencer, mode-register shadow, or output command FIFO. Until 2026-10-09 it was a pumice-local layer that bundled exactly those functions; the Phase-2 extraction moved that logic into the common mc_scheduler_layer in projects/components/mem-ctrl-ip/common-ip/rtl/macro/. What remains in pumice is the thin wrapper that supplies rock-specific children, adapts the pumice training channel to the common maintenance slot, and ties off unused common features.

The common scheduler microarchitecture is documented in Chapter 4 of the common MAS; the family layer-stack and descriptor-seam architecture are defined in the common HAS.

10.2 What the wrapper adds

10.2.1 Rock-specific children

Instance	Module	Key parameters	Role
u_init	init_sequencer	NUM_BANKS, ROW_WIDTH	DDR2 + LPDDR2 JEDEC MRS init microprogram. Drives dfi_init_start_o, emits init commands into the common core, and writes the MR shadow. Gates

Instance	Module	Key parameters	Role
			traffic until `init_done_o`.
`u_mode_reg`	`mode_register`	`NUM_RANKS`	MR shadow updated by init writes; decodes `cl_o` / `cwl_o` / `bl_o` for the DFI layer.
`u_refresh`	`refresh_ctrl`	`NUM_BANKS`	`tREFI` down-counter, postpone/pull-in, and per-bank refresh-kind selection (enabled only after init).
`u_page_policy`	`pumice_page_policy`	`NUM_RANKS`, `NUM_BANKS`, `ROW_WIDTH`	Runtime page-policy engine producing `ap_mode_en_o`, `ap_close_o`, `timeout_pre_req_o`, and `timeout_pre_bank_o`.
`u_cmd_history`	`pumice_cmd_history_checker`	`NUM_RANKS`, `NUM_BANKS`, timing windows	Optional JEDEC same-bank sequencing audit, generated only when `CMD_HISTORY_EN != 0`.

10.2.2 Maintenance-slot mapping

The pumice training layer presents one maintenance request at a time on `{trn_cmd_req_i, trn_cmd_op_i, trn_cmd_bank_i, trn_cmd_row_i, trn_cmd_mrr_i}` plus a `cal_busy_i` gate. The wrapper maps this onto the common `mc_scheduler_layer` generic maintenance slot:

- `maint_req_i` ← `trn_cmd_req_i`

- `maint_op_i` ← `trn_cmd_op_i`
- `maint_bank_i` ← `trn_cmd_bank_i`
- `maint_row_i` ← `trn_cmd_row_i`
- `maint_col_i` ← `'0`
- `maint_extra_i[0]` ← `trn_cmd_mrr_i`
- `maint_busy_i` ← `cal_busy_i`

The common core returns `maint_grant_o`; the wrapper forwards it as `trn_cmd_grant_o` and recovers the MRR flag as `cmd_mrr_o = cmd_extra_o[0]`. The common arbiter priority inside `mc_scheduler_layer` is init > refresh > maintenance > demand; see the pumice training-layer chapter and the training interface contracts.

10.2.3 Tie-offs and constants

Because pumice is a DDR2/LPDDR2 single-rank rock, several common features are disabled or tied off:

- `NUM_BG = 1`; `t_ccd_l_i`, `t_ccd_s_i`, `t_rrd_l_i`, `t_rrd_s_i` are tied to `8'd0`; `cmd_bg_o` is left open.
- `HAS_ZQ = 1'b0`: `zq_req_i` is tied to `1'b0`, `t_zqcs_i` to `16'd0`, and `stall_zq_o` is left open. (ZQ calibration is handled by the pumice training layer, not the scheduler.)
- `MAINT_EXTRA_WIDTH = 1`: the single-bit MRR flag is the only maintenance extra payload.
- Page/policy telemetry is computed by `pumice_page_policy`, so the wrapper ties off the common core's `stat_*` outputs.
- `mode_register` unused rank/ODT/drive-strength outputs and the init sequencer's `zqcl_req_o`/`init_busy_o` are left open with their grant inputs tied low.

10.3 External boundaries

- **CAM sched ports (to `pumice_axi4_layer`):** per-direction `N_LU = NUM_BANKS` lookup buses, `oldest_*` snapshots, and write `commit_*` / read `issue_*` handshakes.
- **Command stream out (to `mc_dfi_2p1_layer`):** `cmd_valid/ready` + `{mrr, ap, col, row, bank, rank, op}`.
- **Maintenance channel in (from `pumice_training_layer`):** `trn_cmd_req` + payload + `trn_cmd_grant` back.

- **Mode-register shadow out:** `cl_o` / `cwl_o` / `bl_o` to the DFI layer.
- **Init handshake:** `dfi_init_start_o` out and `dfi_init_complete_i` in (routed through the DFI layer); `init_done_o` status out.
- **Config in:** JEDEC timings, refresh/programming parameters, `page_policy_i`, `page_mode_i`, `page_tr_init_i`, and scheduler knobs — all delivered from the CSR by name.

`busy_o` is generated by the common core and asserts while not init-done, on a pending refresh, while a command is buffered in the FIFO, or while any bank row is active.

10.4 Pumice integration details that stay

- **Single controller clock:** the wrapper and the common core run on `aclk`; the only controller-to-PHY crossing is inside `mc_dfi_2p1_layer`.
- **External CAMs:** the CAM sched-lookup / oldest / commit / issue ports remain external; the CAMs live in `pumice_axi4_layer`.
- **Training priority and gating:** demand commands are blocked while `cal_busy_i` is asserted and the training request obeys the init > refresh > training > demand priority cone.

10.5 Tests

The common `mc_scheduler_layer` owns the macro-level tests for the pick machinery, bank/global timers, and command FIFO; see the common MAS. The pumice wrapper is covered by integration tests that exercise the training-channel mapping, init-sequencer handshake, and maintenance-slot priority against the common core.

11 Data path macro — mc_storage_layer instantiation

Module: mc_storage_layer (common IP)
Location: projects/components/mem-ctrl-ip/common-ip/rtl/macro/mc_storage_layer.sv
Category: Layer-1b macro in pumice_core
Instance: u_storage (rtl/top/pumice_core.sv:504-596)
Child FUBs: mc_wr_data_cam, mc_rd_cmd_cam, mc_addr_mapper

11.1 Purpose

"Schedulable entry pools and data buffers for writes and reads." pumice_core instantiates the common mc_storage_layer on aclk between u_ifc (mc_axi4_layer) and u_sched (pumice_scheduler_layer). It holds the write-data CAM / write buffer and the read-command scheduling window, exposes the per-entry scheduler vectors, and streams committed write data toward the DFI layer.

Until 2026-10-09 the write-data and read-command CAMs lived inside the pumice-local pumice_axi4_layer and were documented as part of the AXI front-end macro; Phase-2 moved them into the common mc_storage_layer, and the old data_path_macro (wr_beat_sequencer + rd_cl_aligner) is retired.

11.2 Parameter map in pumice_core

Parameter	Bound value in pumice_core	Notes
AXI_ID_WIDTH	IW	IW = AXI_ID_WIDTH (pumice_core.sv:97)
AXI_DATA_WIDTH	DW	DW = DFI_DATA_WIDTH (pumice_core.sv:95)
NUM_RANKS	NUM_RANKS	Passed through from core
NUM_BANKS	NUM_BANKS	Passed

Parameter	Bound value in `pumice_core`	Notes
		through from core
`ROW_WIDTH`	`ROW_WIDTH`	Passed through from core
`COL_WIDTH`	`COL_WIDTH`	Passed through from core
`AXI_BEATS_PER_BURST`	`BURST_WORDS`	DFI words per burst group (`pumice_core.sv:511`)
`NUM_ENTRIES`	`NUM_ENTRIES`	CAM / scheduling-window depth
`N_SRAM_SLOTS`	`N_SRAM_SLOTS`	Write-data SRAM slots
`N_SCHED_LU`	`N_LU`	`N_LU = NUM_BANKS` (`pumice_core.sv:103`)
`AGE_WIDTH`	`AGE_WIDTH`	Age-counter width
`RD_RET_DEPTH`	`RD_RET_DEPTH`	Return-ring depth / ticket

Parameter	Bound value in pumice_core	Notes
		space

11.3 External boundaries

- **From `mc_axi4_layer` (`u_ifc`):** the storage seam — `aw_push_*` + `wd_*` for writes, `snarf_probe_*` for read-your-write probes, `ar_push_*` for reads, and the return-ring ticket handshake (`rt_alloc_*`, `rd_iss_*`, `rd_cam_ins_ready_*`).
- **To `pumice_scheduler_layer` (`u_sched`):** per-entry scheduling vectors (`wr_sch_*`, `rd_sch_*`), plus the write `commit` and read `issue` handshakes.
- **To `mc_dfi_2p1_layer` (`u_dfi`):** committed write-data stream `wr_cm_rd_*` (`valid/ready/data/strb/last`).
- **Status:** `busy_o` is left unconnected in `pumice_core` (`pumice_core.sv:595`).

11.4 Pumice-specific integration notes

Gearing wrapper. The storage layer operates on whole DFI words. `AXI_BEATS_PER_BURST` is bound to `BURST_WORDS`, so one AXI burst maps to one buffered CAM entry. Runtime sub-DFI-word packing (`N_SUBCMD > 1`) and the PHY serialization timing are handled in `mc_dfi_2p1_layer`; the CAMs see only the `BURST_WORDS` framing.

Observed buses. The buses visible at this layer are the storage-seam nets from `mc_axi4_layer`, the scheduler vectors / commit/issue ports, and the `wr_cm_rd_*` commit-data stream to the DFI layer. Returned read data enters through `mc_axi4_layer`'s `rd_dfi_ret_*` port, not directly through the storage layer.

Backpressure thresholds. Write commit data is staged so that it reaches the PHY before its command. The effective command-release delay is computed in `pumice_core` as

```
localparam int CMD_DELAY_EFF = (CMD_DELAY == 0) ? (5 + 2 *
BURST_WORDS) : CMD_DELAY;
```

Measured: BL8/4-word bursts held up to 6 cycles at `CMD_DELAY=6 -> 13`; BL4 x16 (1 word) needs 7 (`pumice_core.sv:273-275`). The DFI write-data FIFO is sized to 32 entries (`WD_FIFO_DEPTH = 32`, `pumice_core.sv:788`) to absorb that staging without stalling the CAM drain. The read-return FIFO is sized to `RD_RET_DEPTH * BURST_WORDS` entries (`pumice_core.sv:784`). Inside the common write-data CAM, `WR_DRAIN_AHEAD = 2` bounds how far the command stream can run ahead of the data drain; the common_mas chapter records that the old FIFO-room gate allowed a command to run ~20 `aclk` ahead and produced a measured 29373-cycle DFI hold.

`tCCD` is clamped to at least `BURST_WORDS` in `pumice_core` (`w_t_ccd_eff = (t_ccd_i < 8'(BURST_WORDS)) ? 8'(BURST_WORDS) : t_ccd_i`; `pumice_core.sv:281-282`) because the DFI command path no longer paces columns.

11.5 Common-layer reference

For the common storage-layer architecture, CAM internals, address-mapper behavior, scheduling-vector layout, and complete port list, see common_mas chapter 3: `mc_storage_layer`.

11.6 Tests

The common CAM FUB tests and the `mc_dfi_2p1_layer` FUB tests cover the datapath building blocks; `pumice_core` wrapper-level tests exercise the end-to-end buffered write and read data flow through the common layers.

12 Pumice DFI v2.1 interface (mc_dfi_2p1_layer)

Module: mc_dfi_2p1_layer (common IP) **Location:** common-ip/rtl/macro/ **Category:** Layer-3 PHY-facing layer, instantiated directly in pumice_core

12.1 Purpose

pumice_core consumes the common mc_dfi_2p1_layer directly; there is no pumice-specific DFI wrapper. The common layer translates the controller-clock command, write-data, and read-data streams into DFI v2.1 pins and holds the single controller-to-PHY clock-domain crossing.

Until 2026-10-09 pumice used a local pumice_dfi_cmd_path / pumice_dfi_cdc / pumice_dfi_wr_serializer / pumice_dfi_rd_aligner stack, corresponding to the old dfi_v21_interface_macro; the Phase-2 extraction moved that logic into mc_dfi_2p1_layer.

The common DFI 2.1 microarchitecture is documented in Chapter 7 of the common MAS; the family layer-stack and descriptor-seam architecture are defined in the common HAS.

12.2 Pumice integration details

12.2.1 Instantiation in pumice_core

pumice_core instantiates mc_dfi_2p1_layer as u_dfi (rtl/top/pumice_core.sv:766-836). The parameters pumice supplies are:

Parameter	Pumice value / source	Purpose
NUM_RANKS	NUM_RANKS	CS/rank fan-out
NUM_BANKS	NUM_BANKS	Bank count
ROW_WIDTH / COL_WIDTH	top-level params	Address widths
DFI_RATE	top-level param	Compile-time phase count
DRAM_BEAT_WIDTH	top-level param	One DRAM beat data width
DFI_BEATS_PER_BURST	BL_PUMICE	DRAM beats mapped to one DFI word
N_SUBCMD	derived	Sub-column commands packed per DFI word
SUB_COL_STRIDE	DRAM_BL	Column stride between packed sub-bursts

Parameter	Pumice value / source	Purpose
SUB_PHASE_STRIDE	derived	Phase stride between packed sub-bursts
SUBW_MAX	derived	Width holding N_SUBCMD
BURST_WORDS	derived	DFI words per burst group
RD_MAX_OUTSTANDING	RD_RET_DEPTH	Aligner outstanding-read depth
RD_FIFO_DEPTH	RD_RET_DEPTH * BURST_WORDS	Read-return CDC FIFO depth
WD_FIFO_DEPTH	32	Write-data CDC FIFO depth
CMD_HISTORY_EN / HIST_T_*	top-level params	DFI-wire command-history audit

The controller-side clock and reset are wired to aclk and aresetn; the PHY side uses dfi_clk and dfi_rstn. The abstract command stream comes from pumice_scheduler_layer; write data comes from the write CAM, and read returns go to the read CAM.

12.2.2 Rock FUBs feeding the layer

The common mc_dfi_cmd_path instantiates the pumice dfi_cmd_formatter rock FUB (rtl/fub/dfi_cmd_formatter.sv) to place the JEDEC command on the selected DFI phase. The dfi_signal_pack FUB remains in the pumice tree (rtl/fub/dfi_signal_pack.sv) as the legacy output-packing partner but is not currently instantiated inside pumice_core or the common layer.

12.2.3 PHY timing programming

The PHY-integration knobs are programmed through the PHY_TIMING and DFI_PHASE CSRs:

Knob	CSR / bits	Measured pumice value
t_phy_wrlat_i	PHY_TIMING 7:0	1 on Nexys A7 a7ddrphy (CSR map)
t_rddata_en_i	PHY_TIMING 15:8	6 on Nexys A7 a7ddrphy (CSR map)
rd_phase_i / wr_phase_i	DFI_PHASE	0 / 0 on Nexys A7 (error-handling chapter)

These are not datapath constants; pumice is the measured customer (Nexys A7, 66.67 / 75 MHz). The board-validated tuple (2026-07-21) is `rd_phase=0`, `t_rddata_en=6`, `t_phy_wrlat=1`, plus harness `DFI_TUNING.rddata_delay=7` (runtime overrides).

12.2.4 CDC clocking

The single controller↔PHY crossing is inside `mc_dfi_2p1_layer` and uses async `gaxi_fifo_async` FIFOs for command, write-data, and read-data, plus token crossings for `init_start` and `init_complete`. `pumice_core` does not add a second crossing.

12.3 Tests

Unit tests for the common CDC, command path, write serializer, and read aligner live in `common-ip`; the pumice integration is exercised by the `pumice_core` layer-level test against the DFI slave BFM.

> **Status (2026-10-10): this block lives in common-ip now.** The RTL is `mc_axi4_layer`; its microarchitecture is owned by common_mas mc_axi4_layer — AXI4 host interface and read-return ring. This chapter keeps the pumice provenance and measured history; for current interfaces and parameters, read the common chapter.

13 AXI4 Slave + Intakes (axi4_slave_wr / axi4_slave_rd)

Modules: axi4_slave_wr / axi4_slave_rd (repo AMBA IP) inside pumice_wr_intake / pumice_rd_intake **Location:** intakes in rtl/fub/; slaves in the shared AMBA library **Category:** FUB (intakes) **Parent:** pumice_axi4_layer **Status:** Implemented

> **Rearchitected:** the SWAG had a single axi4_slave_fub that also owned transaction state and an R-emit FSM. That block is retired. In the live design the AXI4 protocol engine is the repo's axi4_slave_wr / axi4_slave_rd skid IP, and each is wrapped by a thin **intake** FUB (pumice_wr_intake, pumice_rd_intake) that is FIFO-based and FSM-free. All in-flight transaction state lives downstream in the two CAMs (see ch02/04, ch02/05).

13.1 Purpose

The host-facing AXI4 protocol handling is split by direction and factored into two layers each:

- **axi4_slave_wr** / **axi4_slave_rd** — the repo's standard AXI4 slave skid buffers. They do the channel handshakes, provide SKID_DEPTH_* buffering, and present a clean post-skid "fub" face (fub_axi_*).
- **pumice_wr_intake** / **pumice_rd_intake** — the pumice-specific intake FUBs that sit on the post-skid face. They decode the burst address via addr_mapper, push a decoded command downstream to a CAM, stream write/read data through FIFOs, and return B/R responses.

Ahead of both intakes, pumice_axi4_layer places the repo's axi_master_wr_splitter / axi_master_rd_splitter so that each burst delivered to an intake is **exactly one DRAM burst** (aligned to DRAM_BURST_BYTES = BL * DRAM_BEAT_WIDTH/8). The intakes therefore assume "one AXI burst == one DFI burst" and carry no burst-splitting logic.

13.2 pumice_wr_intake — structure

Exactly three things plus the address decoder, per the locked μarch spec:

1. **axi4_slave_wr (u_slave_wr)** — AXI protocol / skid buffering. Parameters SKID_DEPTH_AW=2, SKID_DEPTH_W=4, SKID_DEPTH_B=2.

2. **AW-meta FIFO (`u_aw_meta_fifo`)** — a `gaxi_fifo_sync`, depth `AW_FIFO_DEPTH` (4), width `AWM_W = 1 + IW + AW`, capturing `{err, awid, awaddr}` per burst.
3. **wr-data FIFO (`u_wr_data_fifo`)** — a `gaxi_fifo_sync`, depth `WDATA_FIFO_DEPTH` (16), width `WD_W = 1 + SW + DW`, capturing `{wlast, wstrb, wdata}` per AXI beat.
4. **`addr_mapper` (`u_addr_mapper`)** — combinational decode of the FIFO-head address into `{rank, bank, row, col}` for the downstream `aw_push`.

Plus a **B-response FIFO (`u_b_fifo`)**, depth `B_FIFO_DEPTH` (4), width `B_W = 2 + IW`.

13.2.1 Write data flow

1. **AW intake** — `fub_awvalid` writes `{err, awid, awaddr}` into the AW-meta FIFO. `fub_awready = w_awm_wr_ready` (accept while the FIFO has room).
2. **W intake** — `fub_wvalid` writes `{wlast, wstrb, wdata}` into the wr-data FIFO; `fub_wready = w_wd_wr_ready`. AW and W are decoupled — each has its own FIFO.
3. **Decode + push** — the AW-meta FIFO head address is decoded by `addr_mapper`; the decoded `{bank, row, col, id, err}` is presented on `aw_push_*` and valid whenever the AW-meta FIFO is non-empty. It pops on the downstream `aw_push_ready_i` handshake. (`aw_push_rank_o` is driven but left unconnected at the IFC — single-rank pick.)
4. **wr-data pop** — the wr-data FIFO head is exposed on `wdata_*` and drained by the wr-data CAM (`wdata_ready_i`) in commit order.
5. **B response** — a completed commit strobes `wr_done_valid_i` / `wr_done_id_i` from the CAM; that pushes `{resp, id}` into the B FIFO, which drives the AXI B channel.

13.2.2 Ragged-burst handling

The intake computes `w_aw_err = ((awlen+1)*GEAR != BL)` where `GEAR = AXI_DATA_WIDTH / DRAM_BEAT_WIDTH`. On a ragged burst the intake **self-generates** a `SLVERR` B response at the `aw_push` pop (err takes priority over a `wr_done`), and the downstream CAM drops the illegal command. A simulation `$error` fires when `RAGGED_ASSERT != 0` (a guardrail under `translate_off`); a companion assertion catches a B-push collision between the err path and a same-cycle `wr_done`.

13.3 `pumice_rd_intake` — structure

Mirror of the write intake, plus the read **snarf** path:

1. **axi4_slave_rd (u_slave_rd)** — AXI protocol / skid buffering (SKID_DEPTH_AR=2, SKID_DEPTH_R=4).
2. **addr_mapper (u_addr_mapper)** — decode fub_araddr → {rank, bank, row, col} at the AR inlet (combinational).
3. **Snarf probe** — the decoded {bank, row, col, id, len} is presented on the snarf_probe_* ports every cycle fub_arvalid is high; the wr-data CAM returns a combinational snarf_hit_i.
4. **Order FIFO (u_order_fifo)** — a gaxi_fifo_sync, depth ORDER_FIFO_DEPTH (8), width ORD_W = 1 + IW, storing {source, id} per admitted read to preserve AR order (SRC_SNARF=1, SRC_DFI=0).
5. **Source arbiter** — a combinational mux selecting the snarf stream or the DFI read-return stream by the order-FIFO head's source tag.
6. **rd-data FIFO (u_rd_data_fifo)** — a gaxi_fifo_sync, depth RD_FIFO_DEPTH (16), width RD_W = 2 + 1 + IW + DW = {resp, last, id, data}, feeding the AXI R channel.

13.3.1 Read admission and ordering

- An AR is admitted (w_can_admit) when the order FIFO has room **and**, for a MISS, ar_push_ready_i is asserted. fub_arready = w_can_admit.
- On a **HIT** (snarf_hit_i) the read is tagged SRC_SNARF, snarf_accept_o fires, and no ar_push is emitted (the data is streamed from the write CAM's SRAM — DRAM is stale for the in-flight write).
- On a **MISS** the read is tagged SRC_DFI and ar_push_valid_o fires with the decoded {bank, row, col, id} toward the scheduler / rd CAM.
- The source arbiter drains one burst at a time in AR order: the order-FIFO head's source selects which input stream is connected to the rd-data FIFO, and the head is popped only when the last beat of that burst is accepted. So a snarf-ready read still waits behind an earlier DFI read at the head (in-order R per the AXI contract).

13.4 External interface

Both intakes present a full AXI4 write (AW/W/B) or read (AR/R) slave face plus:

Port group	Direction	Purpose
bank_lsb_i / hash_en_i / hash_seed_i	in	ADDR_MAP config forwarded to addr_mapper

Port group	Direction	Purpose
`aw_push_*` / `ar_push_*`	out	Decoded command to the wr/rd CAM
`wdata_*` (wr)	out	Write-data pop to the wr-data CAM
`wr_done_*` (wr)	in	Commit-completion strobe from the CAM → B response
`snarf_probe_*` / `snarf_hit_i` / `snarf_accept_o` (rd)	in/out	Read-your-write probe into the wr CAM
`snarf_rd_*` (rd)	in	Snarf data stream from the wr CAM SRAM
`dfi_rd_*` (rd)	in	DFI read-return stream (MISS reads) from the rd CAM
`busy_o`	out	Any FIFO non-empty / slave busy

13.5 Notes

- There is **no** AXI ID side table, no `w_buf`/`b_fifo` in the SWAG sense, and no R-emit FSM. Per-ID metadata that must survive into the DRAM layer is carried in the CAM entries; everything else is dropped at the intake.
- Outstanding-count limits are structural: writes are bounded by the wr-data CAM depth (`NUM_ENTRIES`), reads by the rd-cmd CAM depth plus the order FIFO depth. There are no `MAX_PER_ID_*` parameters.
- The intakes are strict-FIFO and back-pressure cleanly: an intake stalls its AXI channel when the relevant FIFO or the downstream CAM is full.

14 Address Mapper (addr_mapper)

Module: addr_mapper.sv **Location:** rtl/fub/ **Category:** FUB (combinational; no clock) **Parent:** pumice_wr_intake / pumice_rd_intake **Status:** Implemented

> **Rearchitected:** the SWAG described a three-scheme decoder (ROW_MAJOR / BANK_INTERLEAVE / XOR_HASH) selected by a runtime scheme_active_i mux. That is retired. The live addr_mapper has **one** placement knob — bank_lsb_i — that slides the bank field through the (byte-stripped) column region, plus an optional bank XOR-hash. The classic schemes are now just settings of bank_lsb. There is no scheme selector, no SYNTH_* param, and no un-synthesized-scheme tie-off.

14.1 Purpose

addr_mapper decodes a flat AXI_ADDR_WIDTH-bit address into the DRAM-layer (rank, bank, row, col) tuple. It is **pure combinational** — no clock, no flop, no FSM. The output is valid the same cycle as the input. One instance sits at the head-address decode of each intake (pumice_wr_intake, pumice_rd_intake), so the CAMs store the decoded tuple rather than the raw AXI address.

The mapper is driven entirely from the ADDR_MAP CSR register (see rtl/macro/pumice_csr.rdl) via three runtime inputs; there are no per-scheme build parameters.

14.2 Parameters

Parameter	Default	Purpose
AXI_ADDR_WIDTH	32	Flat AXI byte-address width
NUM_RANKS	1	1, 2, or 4 (rank field width KW)
NUM_BANKS	8	4 or 8 (bank field width BW = $clog2(NUM_BANKS))
ROW_WIDTH	14	Row field width RW
COL_WIDTH	10	Column field width CW
BYTE_OFFSET_WIDTH	3	log2(beat byte size); low bits stripped to word addr

14.3 Interface

Signal	Direction	Width	Description
axi_addr_i	input	AXI_ADDR_WIDTH	Flat AXI byte address
bank_lsb_i	input	5	Bank-field LSB in the word address (ADDR_MAP.bank_lsb)
hash_en_i	input	1	Enable the bank XOR-hash (ADDR_MAP.hash_en)
hash_seed_i	input	8	XOR-hash seed (ADDR_MAP.hash_seed)
rank_o	output	$clog2(max(NR,2))	Decoded rank
bank_o	output	$clog2(NUM_BANKS)	Decoded bank
row_o	output	ROW_WIDTH	Decoded row
col_o	output	COL_WIDTH	Decoded column

14.4 The single-knob layout

The address is first stripped of its byte offset to a **word address**:

```
w_word = axi_addr_i[AXI_ADDR_WIDTH-1 : BYTE_OFFSET_WIDTH]   (zero-
extended to 32b)
```

The bank field is BW bits wide and sits at bit position bank_lsb inside the word address. The column is split **around** the bank into a low part below it and a high part above it; the row and rank stack above the whole column region at **invariant** positions:

```
word address (low → high):
  [ col_lo(bank_lsb) | bank(BW) | col_hi(CW - bank_lsb) | row(RW) |
rank(KW) ]


  col = { col_hi, col_lo }
  row LSB is always at bit CW + BW      (only the bank slides; the
column
                                         auto-splits around it)
```

14.4.1 Field extraction (variable-base shifts/masks)

The RTL clamps bank_lsb to [0, COL_WIDTH] first (w_blsb), so col_hi's width CW - bank_lsb never goes negative and the row/rank slices stay legal. Then, over 32-bit intermediates:

```
col_lo = w_word & ((1 << bank_lsb) - 1)
bank   = (w_word >> bank_lsb) & ((1 << BW) - 1)
col_hi = (w_word >> (bank_lsb + BW)) & ((1 << (CW - bank_lsb)) - 1)
row    = (w_word >> (CW + BW)) & ((1 << RW) - 1)
rank   = (w_word >> (CW + BW + RW)) & ((1 << KW) - 1)   // 0 when
NUM_RANKS == 1
col    = col_lo | (col_hi << bank_lsb)
```

14.5 The classic schemes are just bank_lsb settings

Legacy scheme	Equivalent setting
ROW_MAJOR	`bank_lsb == COL_WIDTH` — bank sits **above** the whole column
max BANK_INTERLEAVE	`bank_lsb == log2(DRAM_BL)` — minimal `col_lo`, so a burst's column walk stays inside one bank while consecutive lines round-robin banks
partial interleave	any `bank_lsb` in between
XOR_HASH	`hash_en == 1`, folded on top of **any** placement above

The default `ADDR_MAP.bank_lsb = 0x0A = COL_WIDTH` is therefore ROW_MAJOR. Software is expected to keep `log2(DRAM_BL) <= bank_lsb <= COL_WIDTH` so a DRAM burst's column walk never crosses a bank boundary; the RTL clamp enforces the upper edge. (`DRAM_BL`, not `BL/DFI_RATE` — the mapper indexes device words, so a burst spans `DRAM_BL` columns here. Both older formulas were wrong, in opposite directions; measured in `dv/tests/fub/test_addr_mapper.py::minimum_bank_lsb_is_measured`. pumice BUG-022.)

14.6 Bank XOR-hash

When `hash_en_i` is set, each bank bit is XOR-folded with two row slices and the seed (a `genvar` loop of width `BW`):

```
bank_hashed[i] = bank_raw[i] ^ row[i] ^ row[MID] ^ hash_seed[i]
  where MID = (i + BW < ROW_WIDTH) ? (i + BW) : (ROW_WIDTH - 1)   //
clamped
w_bank = hash_en ? bank_hashed : bank_raw
```

This is the identical fold to the legacy XOR_HASH scheme and defeats power-of-two-stride hot-banking. `hash_seed_i` is runtime-modifiable, so a bring-up engineer can change the hash without rebuilding.

14.7 Mirror to the DV address model

This RTL is the bit-exact mirror of the DV-side address model; both must produce identical `(rank, bank, row, col)` for the same input and the same `bank_lsb`/`hash_en`/`hash_seed`. The

mirror eliminates the class of bugs where the BFM decodes one way and the controller decodes another. It is exercised by the `addr_mapper` FUB test, which sweeps addresses and `bank_lsb` values and asserts bit-equality against the model.

14.8 Timing

Purely combinational. The longest path is the hash fold (a few XOR levels) plus the variable-base shift/mask extraction. At the board target this is comfortably within a cycle; if a future high-frequency target cannot meet timing, the mapper can be treated as a multi-cycle path (the placement knob only changes at quiet points).

14.9 Notes / flags

- `bank_lsb_i` is 5 bits wide, matching `ADDR_MAP.bank_lsb[4:0]`.
- `rank_o` width uses `$clog2(NUM_RANKS > 1 ? NUM_RANKS : 2)` so a single-rank build still has a legal 1-bit port; `rank_o` is tied to `'0` when `NUM_RANKS == 1`.
- There is **no** `bg_field_pos` / bank-group input (DDR2/LPDDR2 have no bank groups), no `scheme_active_i`, and no CSR `pslverr` path for unsupported schemes — those SWAG concepts are gone.

> **Status (2026-10-10): this block lives in common-ip now.** The RTL is `mc_storage_layer`; its microarchitecture is owned by common_mas mc_storage_layer — Storage Layer and Entry-Pool FUBs. This chapter keeps the pumice provenance and measured history; for current interfaces and parameters, read the common chapter.

15 Read Command CAM (pumice_rd_cmd_cam)

Module: pumice_rd_cmd_cam.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** pumice_axi4_layer **Status:** Implemented (de-FSM'd streaming drain)

> **Rearchitected:** the SWAG CAM was keyed by AXI ID and drove a match_pending vector with per-slot beat counters and an issue FSM. The live pumice_rd_cmd_cam is keyed by {bank, row} with a free-running age and exposes N_SCHED_LU parallel scheduler lookups plus an oldest port.
>
> **Corrected 2026-09-09.** This CAM is a **scheduling window only**. It no longer buffers returned read data and is no longer the reorder buffer: it carries the read's *ticket* (ins_ticket_i), hands that ticket to pumice_rd_return_ring when the column issues, and **frees the entry at issue**. The ring holds the read for its DRAM round trip and drains in AR order.
>
> Why the split: with the CAM holding a read from insert to R-drain, its entry count was the in-flight read limit, and Little's law capped read bandwidth at roughly 180 MB/s on the board (pre-ring-fix state, 2026-09; the return-ring fix took board reads to ~572 MB/s) regardless of scheduling quality. The CAM now only has to be as deep as the *scheduling* window; in-flight depth is the ring's RD_RET_DEPTH.

15.1 Purpose

pumice_rd_cmd_cam tracks every outstanding DRAM read (the MISS path — snarf hits never enter this CAM). It is the mirror of pumice_wr_data_cam on the issue side, but the data direction is opposite: data comes **in** from the DFI read return and drains **out** to pumice_rd_intake. There is no snarf port (that is a write-CAM concept).

It acts as a reorder buffer because DRAM read data returns in the order the scheduler *issued* commands (which is row-hit reordered), whereas the AXI R channel must see reads in AR order. Data is buffered per entry, then released oldest-first once complete.

15.2 Parameters

Parameter	Default	Purpose
NUM_ENTRIES	8	In-flight read slots (PTRW = $clog2)
N_SCHED_LU	4	Parallel scheduler lookup ports

Parameter	Default	Purpose
NUM_BANKS	8	BKW = $clog2(NUM_BANKS)
ROW_WIDTH	14	
COL_WIDTH	10	
AXI_ID_WIDTH	8	IW
AXI_DATA_WIDTH	64	DW (the DFI word at the IFC)
BL	4	Beats per burst (BCW = $clog2(BL))
AGE_WIDTH	16	Free-running age counter width
N_SRAM_SLOTS	NUM_ENTRIES	SRAM data slots (may be < NUM_ENTRIES)

15.3 Entry state (per slot)

Field	Description
r_valid	Slot occupied
r_issued	Scheduler has issued this read to DRAM
r_ready	Data complete (last DFI return beat seen)
r_bank	Decoded bank (key)
r_row	Decoded row (key)
r_col	Decoded column
r_id	AXI ID (echoed on drain)
r_resp	Captured DFI return response
r_age	Insert-time snapshot of r_age_ctr
r_ptr / r_pv	SRAM slot index (set on first return beat) + its valid flag

Relative age is w_rel[i] = r_age_ctr - r_age[i] (wrap-safe); larger rel = older. The burst data lives in a distributed-RAM array r_sram[N_SRAM_SLOTS*BL].

15.4 Interfaces

15.4.1 Insert (from pumice_rd_intake ar_push, AR order)

ins_valid_i / ins_ready_o with {bank, row, col, id}. ins_ready_o is w_have_free (any !r_valid[i]). On fire the slot is allocated, r_issued / r_ready cleared, key/id captured, and r_age <= r_age_ctr.

15.4.2 Scheduler lookups (N, keyed {bank, row})

For each of N_SCHED_LU ports the CAM returns the **oldest NOT-ISSUED** entry matching {bank, row} (max rel among valid && !issued), with its slot, col, id, and age. This lets the arbiter row-hit-schedule reads exactly like writes.

15.4.3 Oldest not-issued port

oldest_valid_o + {bank, row, col, id, slot} — the oldest valid && !issued entry, the arbiter's fallback ACT target.

15.4.4 Issue notify

issue_valid_i / issue_ready_o / issue_slot_i. The scheduler tells the CAM which slot it issued. On fire, r_issued[slot] <= 1 **and** the slot is pushed into an **issue-order FIFO** (u_issue_q, depth NUM_ENTRIES) so returns fill the right slot in issue order.

15.4.5 DFI read return (data in, issue order)

dfi_ret_valid_i / dfi_ret_ready_o with {data, resp, last}. The return-fill engine writes each beat into the SRAM slot of the **issue-FIFO head** (w_iq_rd_slot):

- On the first beat (r_ret_beat == 0) it pre-allocates a free SRAM slot (w_slot_free) and records it in r_ptr[head], marking r_sram_occ.
- dfi_ret_ready_o is gated on the issue FIFO being non-empty and (first-beat only) a free SRAM slot existing.
- On dfi_ret_last_i the entry is marked r_ready, r_resp captured, and the issue FIFO popped (w_iq_rd_ready).

15.4.6 Drain to pumice_rd_intake (AR order, oldest-first, ready-gated)

drain_valid_o / drain_ready_i with {data, id, resp, last}. The draining slot **is** the combinational oldest-valid pick w_dro_slot (max rel among all valid), which is stable across a burst: the entry stays valid until its own last-beat evict, and later inserts are always younger, so no other entry can become "more oldest" mid-burst. Drain is enabled when w_dr_go = oldest-found && r_ready[oldest] (data staged). Only a burst beat-counter r_dr_beat is registered; drain_last_o = (r_dr_beat == BL-1).

15.5 De-FSM'd sequencing

There is **no** active/slot FSM on the return or drain path. The return path targets the issue-FIFO head; the drain path targets the oldest-valid pick. Both "active" slots are combinational functions of registered state and remain stable until their burst's last beat. The only registered burst state is the two beat-counters (r_ret_beat, r_dr_beat) plus the age counter and per-entry flags. This mirrors the write CAM and removes the SWAG's issue FSM entirely.

15.6 Eviction

On the drain last beat (`w_dr_fire && drain_last_o`): `r_valid[oldest] <= 0`, `r_pv` cleared, `r_dr_beat` reset, and the SRAM slot freed (`r_sram_occ[r_ptr[oldest]] <= 0`). AR-order release is a natural consequence of always draining the oldest entry.

15.7 Reset

`ALWAYS_FF_RST(aclk, aresetn, ...)`: clears `r_age_ctr`, both beat-counters, `r_sram_occ`, and per-entry `r_valid` / `r_issued` / `r_ready` / `r_pv`. `busy_o` is `oldest-found || issue-FIFO non-empty`.

15.8 Notes / flags

- Reads are keyed on `{bank, row}` only (not AXI ID); ID is carried for the R echo and drained with the data. There is no per-ID beat table.
- The `drain_id_o` port is driven but left unconnected at the IFC (the intake's order FIFO already carries the R-channel id); `oldest_id_o` and `sched_lu_id_o` are likewise informational — the arbiter's `unused` sink absorbs the id/slot buses it does not consume.
- No CSR/QoS priority in this CAM — selection is purely by wrap-safe age.

> **Status (2026-10-10): this block lives in common-ip now.** The RTL is `mc_axi4_layer`; its microarchitecture is owned by common_mas mc_axi4_layer — AXI4 host interface and read-return ring. This chapter keeps the pumice provenance and measured history; for current interfaces and parameters, read the common chapter.

16 Read Return Ring (pumice_rd_return_ring)

Module: pumice_rd_return_ring.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** pumice_axi4_layer **Status:** Implemented

> **New block (2026-09-08).** Before it, pumice_rd_cmd_cam held a read from insert all the way to R-drain and buffered the returned data per entry. That made the CAM's entry count the in-flight read limit, and Little's law then bounds read bandwidth: eight entries over a ~27-cycle DRAM round trip is 8 x 8 B / 27 cyc, about 180 MB/s on the board (pre-ring-fix state, 2026-09; the return-ring fix took board reads to ~572 MB/s), no matter how good the scheduler is.
>
> The ring splits the two jobs. The CAM entry now lives **insert -> issue only** and is purely a scheduling window; the ring holds the read for its whole DRAM round trip. In-flight capacity is DEPTH (32), decoupled from the CAM depth, which only has to be as deep as the scheduling window.

16.1 Purpose

A read is given a **ticket** — a ring slot — when it is admitted, in AR order. The CAM entry that schedules it is freed the cycle its column **issues**, and the ticket alone follows the read through DRAM. Returns arrive in **issue** order, because the DFI path is in order; each lands in its ticket's slot. The ring drains from its **head in AR order** once the head's data is complete.

That gives the ordering guarantee AXI needs (per-ID reads return in request order) without an age matrix and without an oldest-entry pick: **AR order is the ring order**, structurally.

16.2 Structure

FSM-free. The state is:

Element	Role
head / tail pointers	tail allocates a ticket, head drains in AR order
per-slot ready + resp	is this slot's burst complete, and its RRESP
issue-order ticket FIFO	maps the next DFI return to the slot that issued

Element	Role
BRAM	the burst data, one slot per in-flight read
fetch pointer + 2-deep skid	runs ahead of head across the synchronous-read BRAM

The prefetch skid is the same one `pumice_rd_cmd_cam` uses, under the same 1-cycle BRAM read-latency contract.

16.3 Interfaces

Group	Direction	Purpose
`alloc_*`	in / out	with the CAM insert, in AR order; returns `alloc_ticket_o` (= ring tail). `alloc_ready_o` is the ring-not-full backpressure that gates AR acceptance
`issue_*`	in	from the CAM when the column issues: pushes the ticket into the issue-order FIFO
`dfi_ret_*`	in	the in-order DFI return stream: data, last, resp
`drain_*`	out	AR-order drain to `pumice_rd_intake`
`occ_o` / `busy_o`	out	occupancy and any-in-flight, for observation

16.4 Sizing

`RD_RET_DEPTH` (default 32) is the in-flight read count and therefore the read bandwidth ceiling: sustaining 8 B/cycle across a 27-cycle round trip needs 32 in flight. It is plumbed from `pumice_core` and sizes the DFI layer's read FIFO alongside it (`RD_FIFO_DEPTH = RD_RET_DEPTH * BURST_WORDS`). Raising the CAM depth without raising this does nothing for read bandwidth; that is the whole point of the split.

16.5 Latency

Measured from the elaborated netlist (see ch01 “Pipeline Latency and Mux-Level Schematics”):

Path	Flops
`dfi_ret_data_i` -> `drain_data_o`	3

Slot write, then the BRAM, then the skid. `alloc_ready_o`, `issue_ready_o` and `dfi_ret_ready_o` are combinational from their respective valids — those are timing paths, not latency.

16.6 Verification

`dv/tests/fub/test_pumice_rd_return_ring.py` covers depth-8 and depth-32 configurations at 1 and 4 beats per burst. The block asserts on a return with no outstanding ticket and on an issue notify when the issue FIFO is empty — both would be a ticket-accounting break rather than a data error, so they are caught structurally rather than by a data mismatch downstream.

> **Status (2026-10-10): this block lives in common-ip now.** The RTL is `mc_storage_layer`; its microarchitecture is owned by common_mas mc_storage_layer — Storage Layer and Entry-Pool FUBs. This chapter keeps the pumice provenance and measured history; for current interfaces and parameters, read the common chapter.

17 Write Data CAM (pumice_wr_data_cam)

Module: pumice_wr_data_cam.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** pumice_axi4_layer **Status:** Implemented (fill / commit-drain / snarf movers)

> **Rearchitected:** the SWAG wr_cmd_cam was an ID-keyed metadata CAM with w_buf_ptr / strb_ptr pointers into an external AXI write buffer and a b_pending / b_complete window driven by xbank_timers. The live pumice_wr_data_cam **owns** the write-data SRAM and is keyed on {bank, row, col} with a free-running age. It has three data movers over the SRAM — **fill** (on insert), **snarf-stream** (read-your-write), and **commit-drain** (to DFI write) — plus associative query ports. Commit completion drives B directly (no external write-window timer). All movers are FIFO-fed with beat-counters — no active/slot FSM.

17.1 Purpose

pumice_wr_data_cam holds every pending write: the decoded command key {bank, row, col} **and** the burst data in an SRAM slot. It presents the scheduler the same lookup/oldest interface as the read CAM so writes row-hit schedule identically, forwards data to in-flight reads (snarf), streams committed data to the DFI write path, and self-generates the B-completion strobe on evict.

17.2 Parameters

Parameter	Default	Purpose
NUM_ENTRIES	8	In-flight write slots (PTRW = $clog2)
N_SCHED_LU	4	Parallel scheduler lookup ports
NUM_BANKS	8	BKW = $clog2(NUM_BANKS)
ROW_WIDTH	14	
COL_WIDTH	10	
AXI_ID_WIDTH	8	IW
AXI_DATA_WIDTH	64	DW (the DFI word at the IFC); SW = DW/8
BL	4	Beats per burst (BCW = $clog2(BL))
AGE_WIDTH	16	Free-running age counter width
N_SRAM_SLOTS	NUM_ENTRIES	SRAM data slots (may be < NUM_ENTRIES)

17.3 Entry state (per slot)

Field	Description
r_valid	Slot occupied
r_bank / r_row / r_col	Decoded command key
r_id	AXI ID (echoed on commit-done → B)
r_age	Insert-time snapshot of r_age_ctr
r_ptr / r_pv	SRAM slot index (set on first fill beat) + its valid flag
r_fdone	**Fill complete** — set on the last fill beat. Only then may the entry be scheduled or snarfed (otherwise commit-drain could outrun a gapped fill and read stale SRAM beats).
r_sched	Arbiter has committed this slot — excludes it from sched_lu / oldest the *next* cycle so the arbiter can pick another entry immediately (clean 1 cmd/clock).

Relative age w_rel[i] = r_age_ctr - r_age[i] (wrap-safe). The data and strobes live in distributed-RAM arrays r_sram / r_strb, each N_SRAM_SLOTS*BL deep.

17.4 Age selectors

Port	Pick rule	Rationale
Snarf lookup	**youngest** match (min rel)	Latest data for a read
Scheduler lookup	**oldest** match (max rel)	In-order commit per row
Oldest port	**oldest** valid (max rel)	Scheduler fallback

All three selectors are gated on r_valid && r_fdone && !r_sched.

17.5 Interfaces

17.5.1 Insert (from pumice_wr_intake aw_push)

ins_valid_i / ins_ready_o with {bank, row, col, id}. ins_ready_o is w_have_free && u_fill_q.wr_ready. On fire the slot is allocated (via the priority-encoded free slot), key/id/age captured, and r_fdone cleared. The slot index is pushed into the **fill FIFO** (u_fill_q, depth NUM_ENTRIES).

17.5.2 Fill (from pumice_wr_intake wdata pop)

wd_valid_i / wd_ready_o with {data, strb, last}, written to SRAM in insert order. The fill engine reads its slot from the fill-FIFO head:

- On the first beat (r_fill_beat == 0) a free SRAM slot is pre-allocated and recorded in r_ptr[head]; wd_ready_o requires a free slot on the first beat.
- Each beat writes r_sram / r_strb at r_ptr[head]*BL + r_fill_beat.
- On wd_last_i the entry's r_fdone is set and the fill FIFO popped.

17.5.3 Snarf lookup + stream (from pumice_rd_intake)

snarf_probe_* returns a combinational snarf_hit_o. Read-your-write forwarding is **limited to the safe case** — the hit requires all of:

1. !r_sched — a scheduled write is draining/evicting to DRAM, so its CAM data is racy.
2. r_id == snarf_probe_id_i — same-id W-before-R is the only AXI-ordered case where the read is *required* to see the write (cross-id has no ordering guarantee → that read takes the DRAM path).
3. snarf_probe_len_i == BL-1 — every admitted write is exactly BL beats, so a short/long read must not snarf a full-BL write.

On snarf_accept_i (the read was admitted as a hit) the youngest matching slot is pushed into a **snarf-request FIFO** (u_snarf_q). The snarf read engine streams snarf_rd_* from r_sram at the FIFO-head slot, beat-counter r_sn_beat, popping on the last beat.

17.5.4 Oldest port + scheduler lookups

Same shape as the read CAM: oldest_* presents the oldest schedulable entry; each of N_SCHED_LU ports returns the oldest {bank, row} match with its slot, col, id, and age.

17.5.5 Commit (from arbiter) + drain to DFI write

commit_valid_i / commit_ready_o / commit_slot_i. Commit **marks** the slot r_sched (immediate exclusion) and enqueues it into the **drain FIFO** (u_drain_q); commit_ready_o is the drain FIFO's write-ready. The commit-drain read engine streams cm_rd_* ({data, strb, last}) from r_sram at the drain-FIFO-head slot, beat-counter r_cm_beat, to the DFI write path. Marking is decoupled from draining so the arbiter is never stalled by the DFI back-end.

17.5.6 Commit-done → B

On the commit-drain last beat, commit_done_valid_o + commit_done_id_o strobe the write intake's B-response path, and the entry is evicted: r_valid/r_pv/r_fdone/r_sched cleared and

the SRAM slot freed. There is no separate `b_pending`/`b_complete` window and no `xbank_timers` completion driver — completion is simply "the committed burst finished streaming to DFI."

17.6 De-FSM'd sequencing

Each of the three movers (fill, snarf, commit-drain) is just a burst beat-counter. The "active" slot for each is its request-FIFO head, stable until popped on the last beat. There is no active/slot latch and no per-slot beats counter beyond the shared engine counters.

17.7 Difference from the read CAM

Aspect	Read CAM (`pumice_rd_cmd_cam`)	Write CAM (`pumice_wr_data_cam`)
Data direction	DFI return **in** → drain **out** to intake	Fill **in** from intake → commit **out** to DFI
Movers	return-fill + drain	fill + snarf-stream + commit-drain
Forwarding	none	snarf (read-your-write) — the wr2rd path
Extra flags	`r_issued`, `r_ready`	`r_fdone`, `r_sched`
Completion	AR-order drain to R channel	commit-done strobe → B channel

17.8 Reset / busy

`ALWAYS_FF_RST` clears the age counter, all three beat-counters, `r_sram_occ`, and per-entry `r_valid`/`r_pv`/`r_fdone`/`r_sched`. `busy_o` is oldest-found or any request FIFO non-empty.

17.9 Notes / flags

- The wr2rd (read-your-write) forwarding is **entirely** the snarf mover here; there is no standalone `wr2rd_forward` block in the live path (see ch02/21).
- `N_SRAM_SLOTS` may be smaller than `NUM_ENTRIES`: an entry can exist in the CAM (key + age) before its SRAM slot is allocated on the first fill beat, so more commands can be tracked than there are data slots.

Status (2026-10-10): this block lives in common-ip now. The RTL is `mc_storage_layer`; its microarchitecture is owned by common_mas mc_storage_layer — Storage Layer and Entry-Pool FUBs. This chapter keeps the pumice provenance and measured history; for current interfaces and parameters, read the common chapter.

18 Write-to-Read Forward — RETIRED as a standalone block

Status: RETIRED. There is no `wr2rd_forward.sv` in the live RTL. **Replaced by:** the **snarf mover inside `pumice_wr_data_cam`** (see §17, the Write Data Path chapter). **Probe driver:** `pumice_rd_intake` (the snarf probe on the AR path).

18.1 Why it was retired

The rearchitected AXI4 interface (`pumice_axi4_layer`) folded write-to-read forwarding into the write CAM rather than keeping it as a separate FUB on the AR path. The old `wr2rd_forward` block sat between `addr_mapper` and `rd_cmd_cam` and compared each AR against a `wr_cmd_cam` snapshot bus; that snapshot bus, the standalone comparator, and the separate `w_buf` storage it read from no longer exist. Forwarding is now a native operation of `pumice_wr_data_cam`, which already holds the pending writes and their burst data in its SRAM.

18.2 Where forwarding lives now

Write-to-read forwarding ("snarf") is the **snarf mover** of `pumice_wr_data_cam`. The flow is:

1. `pumice_rd_intake` presents an incoming AR's decoded key `{bank, row, col}` plus its AXI id and `arlen` on the CAM's `snarf_probe_*` port.
2. `pumice_wr_data_cam` combinationally searches its entries and asserts `snarf_hit_o` for a **youngest** match, subject to the safe-case restrictions.
3. On `snarf_accept_i`, the matched slot is queued into the snarf request FIFO and the snarf mover streams the write's SRAM beats back on `snarf_rd_*` (non-destructively — the write still drains to DRAM normally). Otherwise the read takes the DRAM miss path through `pumice_rd_cmd_cam` (§18).

18.3 Restrictions (narrower than the old block)

The live snarf is limited to the safe case — a write is snarfable only if **all three** hold:

- **Unscheduled** (`!r_sched`): a scheduled write is draining/evicting to DRAM, so its CAM data is racy.
- **Same AXI id**: same-id write-before-read is the only AXI-ordered case where the read is *required* to see the write. Cross-id reads have no ordering guarantee and take the DRAM path.

- **Same burst length** (`arlen == BL-1`): every admitted write is exactly `BL` beats (ragged bursts are rejected in `pumice_wr_intake`), so a short or long read cannot snarf a full-BL write.

The match must also be fill-complete (`r_fdone`); among candidates the **youngest** wins (latest data). This differs from the old block's "last-write-wins by highest slot index, any matching id, conservative all-1 strb-coverage" policy — the live rule is youngest + same-id + same-BL + unscheduled.

18.4 Memory Ordering

Forwarding preserves AXI per-ID in-order semantics: the snarf path returns the same data a DRAM read would have, just earlier. Because it is restricted to same-id matches, it never forwards across an unordered cross-id pair.

18.5 Tests

Covered by the `pumice_wr_data_cam` FUB test and the `pumice_axi4_layer` integration tests (snarf stream, snarf-vs-DRAM-path selection, and the same-id / same-BL / unscheduled exclusion scenarios). See §17 for the detailed test plan.

19 Command Arbiter (pumice_cmd_arbiter)

Module: pumice_cmd_arbiter.sv **Location:** rtl/fub/ **Category:** FUB (the pick core) **Parent:** pumice_scheduler_layer **Status:** Implemented (single-issue, single-rank v1)

> **Rearchitected:** the SWAG scheduler.sv was a multi-mask priority-encoder pipeline (Stages 1–4) over a wide txn_queue snapshot with a column-op FSM (S_NEED_PRE → S_NEED_ACT → S_NEED_RDWR → S_DONE) and a round-robin W/R toggle. That block is retired. The live pick core is pumice_cmd_arbiter — a **combinational, single-issue, FSM-free** priority picker that reads the CAM lookup/oldest ports and the per-bank / global readiness inputs and emits ONE abstract DRAM command per cycle. It is instantiated inside pumice_scheduler_layer alongside the timers, refresh, init, and mode-register (see ch02/19, ch02/09, ch02/11, ch02/12).

19.1 Purpose

pumice_cmd_arbiter picks one abstract DRAM command each cycle and pushes it into the scheduler → DFI command interface. It is PHY/nphases-agnostic and single-issue; JEDEC timing is enforced by the per-bank (pumice_bank_timers) and global (global_timers) readiness inputs it consumes. Open-page vs auto-precharge is decided **inline** from page_policy_i — there is no standalone page runtime page-policy engine (see ch02/08).

The wider pumice_scheduler_layer wrapper wires the arbiter to those timers, the refresh controller, the init sequencer + mode-register shadow, and an output command FIFO; the CAM sched-lookup / oldest / commit / issue ports pass through to pumice_axi4_layer.

19.2 Priority (each cycle)

The pick is a single combinational if/else cascade producing an abstract command plus its side-effects (event strobes, CAM commit/issue, refresh grant), all gated on the command sink accepting the push:

1. **INIT** (!init_done_i) — forward the init_sequencer command verbatim (init_cmd_op/bank/row).

2. **REFRESH** (refresh_req_i || refresh_drain_i) — if any bank is active, precharge the lowest active, precharge-ready, un-guarded bank (one/cycle); once no bank is active, issue OP_REF and assert refresh_grant_o. The REF fires only under **w_ref_safe**: the registered view shows every row closed AND no row-affecting command is in flight or

inside its 2-cycle guard window AND the previous REF's tRFC recovery has elapsed. The registered bank view alone is 2-3 cycles stale, which let a REFab collide with a just-issued ACT (no PRE) — silicon-confirmed as refresh-rate-correlated row corruption. **Mission-mode tRFC** (`TIMINGS_RFC_REFI.tRFC` -> `t_rfc_i`) is enforced by an arbiter down-counter loaded on each fired REF; while non-zero, ACT picks and further REFs are blocked (init-time refreshes use `INIT_TIMING1.t_rfc_wait` separately). Column picks also carry a **PRE-only 3-cycle bank guard** (`w_pre_col_guard`): without it a conflict-PRE could fire in a column-readiness gap and a column then pick against the stale row-open image (up to 3 cycles end-to-end) — the RD lands on the just-closed row, its data never returns, and the rd reorder CAM's AR-order drain wedges behind it forever (found by the parked-victim pattern; latent since the bank-parallel refactor; PRE-only because the general `w_guarded` also covers RD/WR fires and would throttle same-bank column streaming). The `SCHED_POLICY.order_mode` overlay (Axis 1) NARROWS the FR-FCFS class masks before the pick. `1 in_order` keeps only each CAM's head-of-queue entry, so every channel is strictly FIFO; the arbiter's normal read-versus-write preference then chooses the side. `3 age_threshold` narrows every class to aged entries whenever any exist (per-entry 1-bit flags from the CAMs at `SCHED_POLICY.age_thresh`, and those flags are registered inside the CAMs so the 16-bit age compare never enters the mask cone).

Build tier (2026-09-09). Both modes are in the BASE build. What is gated on `+define+PUMICE_ENHANCED` is *global* in_order: comparing the two CAMs' oldest relative ages (`w_rd_head_wins`) so the younger head waits. That compare — CAM older-matrix, oldest select, 16-bit rel-age mux, export, compare, mask, pick — is the cone that misses 75 MHz by about 20 ps, so it is compiled out by default and the export constant-propagates away. On a base bitstream in_order therefore orders *within* each channel and may issue a read behind a younger write at the arbiter's preference. AXI orders nothing between AR and AW and the read-after-write hazard is the write CAM's snarf, so this is a legal ordering; it is simply not the global one. Software cannot tell by readback — check the build. `SCHED_POLICY.qos_en` makes AxQOS the OUTER pick key: each class first narrows to its max-QoS candidates (the CAMs carry AxQOS per entry from the AR/AW handshake), then row_sel/col_sel and the oldest tie-break run inside that set. `SCHED_POLICY.prio_sub` selects the read-vs-write key WITHIN a class: 0/2 load_over_store (default), 1 none (alternating direction toggle, flipped on each fired demand op), 3 age_boost (an age-boosted write winner outranks a non-boosted read winner — the `age_thresh` flags feed this directly). `SCHED_WR_WM` write batching: wr-CAM occupancy >= high_wm arms a registered drain in which WRITES outrank reads in every demand class, released at <= low_wm (high_wm = 0 disables — the default read-priority is bit-identical). `SCHED_POLICY.access_pref` reorders the demand-CLASS preference itself (0/1 column_first = the legacy order, 2 row_first, 3 precharge_first); the class is chosen from the possibly-narrowed per-class picks, then read-over-write applies within it. `SCHED_POLICY.row_sel/col_sel` further steer the SELECTION within the (possibly narrowed) activate/column classes: population-first (most/fewest schedulable entries sharing the candidate's {bank,row}, an 8x8 match triangle per CAM) with oldest tie-break; 0 = pure oldest, and precharge picks are always oldest. `w_ref_safe` also carries **`! r_grant`**: the grant-to- request-drop round trip is 2 cycles, so without it the branch re-

picked a SECOND REF while the first still sat in the output register and the refresh DOUBLE-ISSUED on the wire — a silent tRFC-between-REFs violation for REFab (present in every build before 2026-08-26), fatal for REFpb (each command advances the device's bank rotor). With `refresh_kind_i` set (REFpb, LPDDR2 `REF_CTRL.mode=2`) the branch instead precharges ONLY the device's rotor bank (`refresh_bank_i`, the controller's mirror of the device-internal counter) and issues `OP_REFPB` under `w_refpb_safe` (same terms, rotor-bank-scoped row check); the recovery counter loads `trfc_pb` instead of tRFCab, and the rank-wide ACT block during that (shorter) window also provides the JEDEC spacing between consecutive REFpb commands. The sequencing is audited by `rtl/fub/pumice_cmd_history_checker.sv` (generate-gated by `CMD_HISTORY_EN` inside the scheduler macro, `$fatal` on violation).

3. **COLUMN row-hit** — an issuable RD/WR to an open row. **READ-PRIORITY**: a ready read wins over a ready write. Within each direction, **oldest** (max CAM relative age) wins the tie-break.

4. **FALLBACK** — no ready row-hit: `ACT` the oldest pending op's row on an idle, act-ready, un-guarded bank; or `PRE` a bank that is open on the wrong row.

19.2.1 Column issuability

For each bank `j` the arbiter drives the CAM lookups with `{bank j, that bank's open row}` (`bank_open_row_i`), gated valid by `bank_row_active_i`. A returned hit is issuable when:

- **RD**: `rd_lu_hit && bank_rdwr_ready && w_tccd_fwd_ok && twtr_ok`
- **WR**: `wr_lu_hit && bank_rdwr_ready && w_tccd_fwd_ok && trtw_ok`

plus three guards added with the 2026-09-08/09 bandwidth work:

- **Auto-precharge stale-image guard** — `!(f_ap(b) && w_col_inflight_bank[b]) && !r_ap_closing[b]`. A column to a bank with an auto-precharge column already in flight would be picked against a row image the AP is about to invalidate. The mask is gated on `f_ap(b)` so it costs nothing when the access is not auto-precharging; an ungated version throttled same-bank column streaming.
- **Forward tCCD** — `w_tccd_fwd_ok` replaces the flopped global `tccd_ok` on the column masks. The counter reloads on column **selection** (not on fire) and compares `<= 1`, so the period is exactly tCCD. Stacking the old fire-reloaded gate on top of it made the period tCCD+4 and halved read throughput at tCCD=4.
- **Refresh column block** — `w_ref_col_block[b]`: columns to the banks a pending refresh is going to close are not selectable (all banks for REFab, the rotor bank for REFpb). Without it the x16 reorder case livelocked — refresh

pending, bank 0 open, eight row-hit entries, and the pick pipeline re-selecting a bank-0 column every cycle whose in-flight guard blocked the very PRE the refresh was waiting for.

The oldest issuable RD (max `rd_lu_age`) and oldest issuable WR are scanned in parallel across all `N_LU = NUM_BANKS` lookups; read-priority then selects RD over WR if both exist.

The output stage re-checks `wr_commit_ready_i`, `rd_issue_ready_i` and the ACT gate **live**, because the pick pipeline is three stages deep and a readiness that was true at selection can be false at issue. The ACT case was found by probe: an ACT selected in the one cycle between a refresh's tRFC expiring and the next pulled-in REF firing reached the output with `rfc_busy` set.

19.2.2 Auto-precharge (inline page policy)

```
w_ap = (page_policy_i == PAGE_POLICY_CLOSE);
```

A column op becomes `OP_RDA` / `OP_WRA` when the auto-precharge decision is set, else `OP_RD` / `OP_WR`; `evt_ap_o` tells the bank timers to model the auto-precharge. The decision is the legacy flat `w_ap` (`page_policy_i == CLOSE`) unless the runtime page-policy engine is active, in which case it is per-bank (`ap_close[bank]`) and the engine may also request idle-timeout closes as the arbiter's lowest-priority pick — see ch02/08. (`PAGE_POLICY_HAPPY_HYBRID` was retired 2026-08-25; the enum encoding maps to build default.)

19.3 The in-flight shadow, and where timing is actually enforced

Reworked 2026-09-25. The two paragraphs below describe **one** mechanism; they used to be nine ad-hoc masks.

19.3.1 Why any guard exists at all

`bank_timer` is meant to BE the authority — its `safe_*` outputs are a combinational AND of the counters, a single register stage, no FSM lag. But the arbiter drives the `set_*` strobes at **FIRE** (`evt_act_o = w_fire_out && r_do_act`) while it **SELECTS** three to four registered stages earlier. So up to four commands are in flight that the timers have never been told about, and `safe_*` reports safe because it has not seen them. Every bank guard in this block covers that single gap.

They were built in separate `always_comb` blocks spread over ~200 lines, which hid the fact that they are the SAME QUESTION asked over different stage subsets. They now derive from one matrix — `w_if_{preact,col}_{sel,pre,out}` over the SEL / PRE / OUT stages — and the composites (`w_prepick_guard`, `w_col_inflight_guard`, `w_preact_bank_guard`, `w_guarded`) keep their exact previous subsets. Nothing downstream changed; what changed is that "what is in flight" has ONE definition instead of nine.

19.3.2 Final-stage timing authority

The op sitting in the output register is re-checked against the **LIVE** `bank_*_ready_i`, and may not fire until that says safe:

```
w_out_safe   = live bank readiness for the op in the output register
w_out_reject = r_pick_valid && !w_out_safe
w_out_ready  = !r_pick_valid || cmd_ready_i || w_out_reject
w_fire_out   = r_pick_valid && cmd_ready_i && w_out_safe
```

A rejected pick is **DROPPED, never held.** Holding freezes `w_out_ready` and head-of-line blocks the whole pipeline behind one bank — measured as a regression (close-page 30.77% -> 28.57%) when an earlier speculative-column attempt did exactly that. Dropping is lossless: every CAM commit is qualified by `w_fire_out` (`wr_commit_valid_o` / `rd_issue_valid_o`), so an unfired entry stays schedulable and is simply re-picked. Refresh and init carry no bank and are never gated here.

This is what makes the bank timers' **advisory lookahead** safe to act on (see `10_xbank_timers.md`): the pick pipeline may decide optimistically on `safe_*_la`, because the last stage enforces the truth. Rejects are attributed to `stall_banktimer_o`, not `stall_bp_o`.

In practice the gate never rejects — the front end is conservative enough that it measures zero on every workload run. That is the correct outcome, not a dead path: it is what allows the front end to be relaxed at all.

19.4 Per-bank ACT/PRE re-issue guard

Because `pumice_bank_timers` register their readiness outputs (a 2-cycle latency from an `evt` to `act/pre_ready` dropping), a purely combinational arbiter would re-issue ACT/PRE to the same bank before the timers reflect it. The arbiter keeps a **2-cycle per-bank guard** (`r_guard0`, `r_guard1`; `w_guarded = guard0 | guard1`) set on any accepted **ACT, PRE or column**, and skips guarded banks in the refresh-PRE and fallback paths. Columns are included because a column fired <2 cycles ago has not yet dropped the bank's registered `pre_ready` (tRTP/tWR load), so an unguarded PRE — normal or refresh-drain — could precharge on stale readiness.

Column ops still need no guard **against each other**: both CAMs exclude a just-committed/issued slot from their lookup/oldest ports the next cycle (`r_sched` on the write side, `r_issued` on the read side), so the arbiter never re-issues the same slot. The shared-DQ-bus occupancy (a BL burst owns the DQ bus for `BL/DFI_RATE` dfi cycles) is a `dfi_clk`-domain constraint enforced **downstream** in `pumice_dfi_cmd_path` (`COL_BURST_CYC`), not here — the CDC decouples `aclk` command issue from `dfi_clk` DQ timing.

19.5 Write data must lead the command

The command stream cannot be back-pressured to wait for write data. JEDEC spacing (tRFC, tRP, tCCD) is enforced by the timers at the arbiter, but the command then crosses the in-order scheduler command FIFO and the command CDC. Any stall downstream of the timers **compresses** the spacing of everything already queued behind it: commands that the arbiter separated correctly arrive at the DRAM bunched together. Holding a write command until its data showed up is exactly such a stall, and it produced real tRFC and tRP violations.

So the invariant is inverted: the data is made to lead the command, and nothing downstream ever has to wait. Four mechanisms, all landed 2026-09-09:

1. **Rate-matched commit.** `pumice_wr_data_cam` only accepts a commit when the DFI write-data path has room for it, evaluated at **decision time** rather than after the fact (`WR_DRAIN_AHEAD`, with the occupancy counter registered). An earlier version predicated on the *current* occupancy and, with the one-cycle decision-to-fire latency, refused commits it had already promised — the CAM and the arbiter deadlocked for 29373 cycles.
2. **Fixed command delay.** The scheduler's command FIFO releases each command a fixed `CMD_DELAY` cycles after the arbiter's decision. Because *every* command is delayed alike, spacing is preserved exactly; only latency moves, by `+CMD_DELAY`. `CMD_DELAY = 0` selects the automatic `5 + 2 * BURST_WORDS`, which is the write-data lead a burst needs; a fixed 6 was not enough at BL8.
3. **Staged-token gate as a checked invariant.** `pumice_dfi_cmd_path` still holds a write command until its data is staged, but with (1) and (2) in place that gate should never engage. It is instrumented rather than trusted: the block counts the cycles it holds and the throughput tests assert the count is **zero**. A non-zero count means the lead has been lost upstream.
4. **Free at fetch-last.** The write CAM frees its entry when the data is *fetched* out, not when it is consumed at the DFI. With the command delayed, consume-last held each entry about 30 cycles, and eight entries cannot cover a burst every four — W back-pressured for 222 cycles in the write ceiling test. The B response rides the skid tag, so nothing needs the entry after the fetch.

This is structurally what LiteDRAM does: its multiplexer drives the DFI directly and only chooses a write command once that write's data is at the head of the write FIFO. There is no command queue downstream of the timing decision.

19.6 Interface (arbiter)

Group	Direction	Notes
page_policy_i	in	OPEN / CLOSE / HAPPY_HYBRID (HAPPY == OPEN in v1)
init passthrough	in	init_done_i, init_cmd_{valid,op,bank,row}_i
refresh	in/out	refresh_req_i, refresh_drain_i, refresh_grant_o
per-bank readiness	in	bank_{act,rdwr,pre}_ready_i, bank_row_active_i, bank_open_row_i ([NUM_RANKS][NUM_BANKS])
global readiness	in	tfaw_ok_i, trrd_ok_i (per-rank), twtr_ok_i, trtw_ok_i, tccd_ok_i
wr CAM lookup + oldest + commit	in/out	wr_lu_* (drive + read), wr_oldest_*, wr_commit_{valid,slot}_o
rd CAM lookup + oldest + issue	in/out	rd_lu_*, rd_oldest_*, rd_issue_{valid,slot}_o
event strobes	out	evt_{act,rd,wr,pre,ap}_o, evt_{rank,bank,row}_o (to timers)
command push	out/in	cmd_{valid,op,rank,bank,row,col,ap}_o, cmd_ready_i

19.7 Side-effects (all gated on w_fire = w_valid && cmd_ready_i)

- evt_act/rd/wr/pre_o — strobe the bank + global timers on an accepted issue.
- wr_commit_valid_o / wr_commit_slot_o — on an accepted WR/WRA, mark the wr CAM slot scheduled (enqueue to its drain FIFO).
- rd_issue_valid_o / rd_issue_slot_o — on an accepted RD/RDA, mark the rd CAM slot issued (enqueue to its issue FIFO).
- refresh_grant_o — on an accepted OP_REF.

cmd_valid_o is asserted whenever a command is picked; cmd_rank_o is tied to rank 0 (RK0) — v1 is a single-rank pick.

19.8 v1 scope / TODO

- **Single-rank** pick (`RK0 = 0`); multi-rank arbitration is a TODO.
- **HAPPY_HYBRID** == **OPEN**; the per-bank page predictor is not in the build.
- Write-drain watermark and powerdown are TODO.
- Unused CAM buses (`wr_lu_id_i`, `rd_lu_id_i`, `*_oldest_slot_i`) are absorbed by an explicit `unused` sink.

19.9 `pumice_scheduler_layer` wrapper

The macro instantiates, on a single `aclk`:

Instance	Module	Role
`u_init`	`init_sequencer`	JEDEC MRS init; gates traffic until `init_done`
`u_mode_reg`	`mode_register`	MRS-updated CL/CWL/BL shadow → `cl_o/cwl_o/bl_o`
`u_refresh`	`refresh_ctrl`	tREFI + postpone; enabled after init
`u_bank_timers`	`pumice_bank_timers`	per-(rank,bank) safe timers + open-row
`u_global_timers`	`global_timers`	tFAW/tRRD (per-rank), tWTR/tRTW/tCCD (global)
`u_arbiter`	`pumice_cmd_arbiter`	the pick core (above)
`u_cmd_fifo`	`gaxi_fifo_sync`	output command FIFO, packs `{op,rank,bank,row,col,ap}` (`CMD_W = 4 + RKW + BKW + ROW_WIDTH + COL_WIDTH + 1`, depth `CMD_FIFO_DEPTH=8`)

The arbiter pushes into `u_cmd_fifo`; the FIFO read side is the macro's `cmd_*_o` command stream to the DFI layer. `busy_o` is `!init_done || refresh_req || cmd-fifo-non-empty || any-bank-active`.

Issue rate is **one command per `aclk`**. The arbiter never sees the DFI multi-phase dimension — the DFI layer packs phases downstream. Selection is oldest-first by wrap-safe CAM age; there is no QoS, no lookahead window, and no `SCHEDULER_MODE` OOO/INORDER synthesis switch.

20 Page-Policy Engine (pumice_page_policy)

20.1 Overview

pumice_page_policy is the runtime Axis-2 (paging) engine. It replaced the HAPPY address-hash predictor (page_predictor.sv, retired 2026-08-25: it was never wired into the rearchitected core — PAGE_POLICY_HAPPY_HYBRID degraded to OPEN in the arbiter). The engine watches the arbiter's ISSUED command stream — the same valid && ready tap the command-history checker audits — together with the registered per-bank row state, and owns two decisions plus the page telemetry.

All behaviour is selected at runtime by PAGE_POLICY_CFG.policy_mode; encoding 0 is the build default (bit-identical to the pre-engine controller, where REFRESH_TUNING.page_policy_or alone selects OPEN/CLOSE).

The reset value is 0. Mode 3 (fixed_open) with PAGE_TIMEOUT_CFG.tr_init = 2 is the measured best default and is RECOMMENDED, but the reset change was made and reverted on 2026-09-26 and is **blocked on BUG-003** (a short timeout precharges under an in-flight read; the read-return ring loses a ticket at rd_gap >= 8). The measurement below stands. Measured on the board at txn_scale=1000 it is strictly dominant over open page across every scenario in the suite – +41.2% on col_major_interleaved_bl4, +8.6..11.6% on col_major, and exactly flat on incremental/row_major – with no scenario regressing and zero integrity failures.

> **Vintage:** these board measurements are 2026-09-27, pre-sdpram-fix (amba 71d48b6f7, 2026-10-02); the fix is on the harness UART/debug SRAM, off this datapath, and the 2026-10-03 post-fix re-run reproduced close_page col_major read to the digit (195.2 MB/s on both images; FINDINGS_litedram_ab_2026-10-03.md).

Two notes for anyone changing it:

- tr_init = 0 **disables the timeout entirely** (f_tr/r_idle below). It is not a "build default" sentinel on this field, which is why the reset had to move to 2 and not stay at 0.
- TR=2 is a measured optimum, not a guess: TR=1 is identical on every scenario and TR=4 already falls back to open-page numbers on plain col_major, so the cliff sits between 2 and 4.

See vault/Tasks/projects/components/mem-ctrl-ip/research-ip/pumice-ddr2-lpddr2/task/open/TASK-013.md for the full campaign.

20.2 Modes

policy_mode	Name	Behaviour
0	build default	Engine inert; legacy flat auto-precharge from `page_policy_or`.
1	`static_open`	Per-bank ap mask forced 0 — rows stay open.
2	`static_close`	Per-bank ap mask forced 1 — every column op auto-precharges.
3	`fixed_open`	**THE SHIPPING DEFAULT** – `policy_mode` resets to 3 and `tr_init` to 2 as of 2026-09-27. ap=0; per-bank idle countdown from `PAGE_TIMEOUT_CFG.tr_init`. On expiry the engine REQUESTS a close; the arbiter issues the PRE as its strictly lowest-priority pick. This background-precharge path is where the measured win comes from – see “Mechanism” below.
4..7	*retired*	**Fall through to the build default** — engine inert, no auto-precharge. 4 (`adapt_time`) and 5 (`adapt_access`) RETIRED 2026-09-27 (TASK-014); 6 (`rbl_static`) and 7 (`rbl_dyn`) RETIRED 2026-09-26 (TASK-011).

policy_mode	Name	Behaviour
		policy_mode is a 3-bit field, so software can still write these; the fallthrough is a contract, regressed by test_page_predictor and the scheduler matrix.

20.3 Decision interfaces to the arbiter

1. **Auto-precharge override** — ap_mode_en + a per-bank ap_close mask. When enabled, the arbiter's column picks take ap_close[bank] instead of the legacy flat w_ap.
2. **Timeout close** — timeout_pre_req + bank. A new arbiter branch below the conflict-precharge path issues the PRE, gated identically (registered row_active + pre_ready + the 2-cycle re-issue guard), so demand, refresh and JEDEC timing always outrank a housekeeping close.

20.4 The retired adaptive modes (4 and 5)

Both were implemented 2026-08-25 and removed 2026-09-27 (TASK-014).

adapt_time was measured to BE fixed_open(tr_min). It carried a mistake taxonomy — premature close (MC++) when an ACT re-opened the row a timeout PRE had just closed, held-too-long (MC−−) when a conflict PRE closed a bank whose timer had not expired — and walked TR by tr_step within [tr_min, tr_max] every check_interval. In practice MC is dominated by the held-too-long case, so TR decayed monotonically to the floor and stayed: moving only tr_min moved the result onto the matching fixed point every time (2 → 436.8 MB/s, 8 → 338.5, 16 → 327.7). MC was also a single GLOBAL counter driving all eight r_tr[b], so policy_scope's "per-bank TR" could not diverge from global. Mode 3 is already its close path, so retiring it cost nothing functional.

adapt_access was removed as a decision, not a measurement. A per-row 2-bit saturating counter (tagless, {bank, XOR-folded row}) voted at ACT time and drove ap_close_o — auto-precharge. That mechanism measures 4.9x the activations of a background precharge on identical traffic (160,006 vs 32,400 ACT) and double the read latency, and it commits at the column op, before it is known whether more requests to that row are coming. On a controller whose value is FR-FCFS reordering to batch same-row columns, AP fights the reordering that justifies the design, so even a perfect predictor is bounded by a losing mechanism. It was unproven and mis-plumbed

rather than disproven; re-plumbing it onto the background precharge was the alternative not taken.

20.5 Telemetry

Always on, mode-independent, feeding the read-only *_STATS CSRs:

Counter	Event
PAGE_STATS_HIT	Column op issued (columns only issue on row hits in this arbiter).
PAGE_STATS_MISS	ACT to a bank whose previous close was a conflict PRE.
PAGE_STATS_EMPTY	ACT to a simply-closed bank (timeout / refresh closes count here).
SCHED_STATS_ACT / SCHED_STATS_PRE / REF_STATS_REF	Command-class counts.

20.6 Verification

test_pumice_core_dfi.py::test_pumice_core_fixed_open — self-checking in both directions: mode-0 arms assert zero idle precharges before and after the mode arms (inertness and disarm) and the fixed_open arm asserts the idle-timeout close and a clean golden-data reopen. The close request was mutation-checked (engine forced off → the test fails at “row never closed”).

test_page_predictor.py asserts the RETIREMENT contract: mode 0 and every retired encoding (4..7) leave ap_mode_en_o low and ap_close_o at zero, while mode 2 (static_close) does auto-precharge — so the test still distinguishes “no AP” from a block that has stopped driving ap_close_o at all. test_pumice_sched_matrix.py sweeps the retired encodings as fallthrough arms across all 12 operating points.

20.7 Related

- ch02/07 scheduler — the arbiter pick order this engine feeds.
- docs/design-requirements.md “Advanced modes”, Axis 2 — the mode catalog and the serial landing order.

  Modes 6/7 (rbl_static/rbl_dyn) were **RETIRED 2026-09-26**: measured on silicon at txn_scale=1000 on a workload built specifically to suit them (TASK-011), mode 6 lost 26% of bandwidth (195.2 -> 144.2 MB/s) by paying +22,827 ACTs to save precharges that never materialised, and mode 7’s hill-climb drove its threshold to “never close early”, landing bit-identical

to plain open page. A write of 6 or 7 now falls through to the build default.

20.8 Mechanism: why the close path matters more than the policy

One live mode closes a row with **auto-precharge** (`ap_close_o`, fused into the column command) and one with a **background precharge** (`timeout_pre_req_o`, a separate command the arbiter issues at lowest priority). This distinction is why the adaptive modes were retired rather than tuned — it dominates the policy that selects it:

mode	close mechanism
2 `static_close`	auto-precharge
3 `fixed_open`	background precharge

Measured on the board, same stimulus and the same close-on-sight policy, differing only in mechanism (txn_scale=1000, peak 600 MB/s):

mechanism	rd MB/s	ACT	PRE	thrash	rd latency
auto-precharge	144.1	**160,006**	8	0.0%	192.0
background precharge (TR=1)	**436.8**	**32,400**	40,400	24.7%	96.0

Auto-precharge costs 4.9x the activations for identical traffic and doubles read latency. Confirmed independently on both AP paths – the legacy `w_ap` route (160,018 ACT) and `ap_close_o` (160,002 ACT) land on the same number, so this is a property of auto-precharge and not of one implementation.

The reason is that **auto-precharge is uncancellable.** It commits to closing at the column op, before it is known whether more requests to that row are coming. A background precharge fires only once the bank goes IDLE, and if more same-row requests are pending the bank is not idle – so the row survives and serves them. That idle gate is an implicit cancellation. On a controller whose value is FR-FCFS reordering to batch same-row columns, auto-precharge destroys the row before the batching can happen: it fights the reordering that justifies the design.

Corollary for the adaptive modes: `fixed_open` at TR=1 and TR=2 are identical, so the grace period contributes nothing measurable. The idle gate alone does the work.

20.9 Standing of the adaptive modes (measured 2026-09-26)

Mode 4 `adapt_time` is subsumed by mode 3. Its adaptation earns nothing measurable. `r_mc` is dominated by the held-too-long case (a conflict PRE on a bank whose timer had not expired), so TR decays monotonically to `tr_min` and stays there. Proven by moving only the floor:

config	tr_min	rd MB/s	equals
adapt_time	2	436.8	fixed_open TR=2 -> 436.8
adapt_time	8	338.5	fixed_open TR=8 -> 338.5
adapt_time	16	327.7	fixed_open TR=16 -> 327.7

Exact equality at three different floors: mode 4 is `fixed_open(tr_min)`. Note also that `r_mc` is a single GLOBAL counter driving every `r_tr[b]` from one decision, and all eight start at `tr_init` – so `policy_scope = 0`'s "per-bank TR" cannot diverge and is a fiction. Retiring mode 4 costs nothing functional, because mode 3 *is* its close path.

Mode 5 `adapt_access` is unproven and mis-plumbed, not disproven. `close_pred_o` does not assert on hardware: `PRE ~= ACT` and thrash falls only 100% -> 97% at best, against 60% of activations converted in the DFI-loopback sim on identical stimulus. The divergence is not run length (board engagement is 0% at txn_scale 1 and 10, saturating at 3% from 100) and not refresh rate (it is non-monotonic – zero at both refresh extremes). The `ap_close_o` port itself is proven good: mode 2 driving it returns ACT=160,002 / PRE=2.

Even with that bug fixed, mode 5 would drive auto-precharge – the mechanism above, costing 4.9x the activations. **The change worth making is routing the per-row verdict to `timeout_pre_req_o` instead of `ap_close_o`**, keeping the idle-gate cancellation that makes mode 3 fast while adding per-row selectivity. That is the only version of mode 5 that could beat `fixed_open`.

> **Status (2026-10-10): this block lives in common-ip now.** The RTL is `mc_scheduler_layer`; its microarchitecture is owned by common_mas Chapter 4 — Scheduler Layer and Pick Machinery. This chapter keeps the pumice provenance and measured history; for current interfaces and parameters, read the common chapter.

21 Global Timers (global_timers)

Module: global_timers.sv **Location:** rtl/fub/ **Category:** FUB **Parent macro:** pumice_scheduler_layer **Status:** Implemented (per-rank tFAW/tRRD; global tWTR/tRTW/tCCD; strict-flop outputs)

> **Replaces the retired cross-bank timer block.** The original xbank_timers FUB owned a mix of per-bank AND cross-bank counters. That mix has been split:
>
> - **Per-bank** timing (tRCD, tRP, tRAS, tRC, tWR/tRTP) moved into bank_timer, stamped per (rank, bank) by pumice_bank_timers.
> - **Cross-bank / channel-wide** turnaround (tFAW, tRRD, tWTR, tRTW, tCCD) lives here in global_timers.
>
> The retired block's cross-rank extras (tRTRS, tCS, the last_rd_rank / last_cmd_rank chains, and the wide blocks_*[NR][NB] broadcast) are **not** in the live RTL. global_timers emits compact per-rank / global *_window_ok readiness bits that the arbiter ANDs with bank_timer's safe_* outputs.

21.1 Purpose

global_timers holds the controller-wide DRAM constraints that span banks and therefore cannot live inside a single bank's timer:

- **tFAW** — no more than four ACT commands within a rolling t_faw_i window. Tracked **per rank** (each rank has its own 4-deep sliding window) so a multi-rank build enforces the device-local four-activate limit independently.
- **tRRD** — minimum cycles between any two ACTs, **per rank**.
- **tWTR** — cycles since the last WR (global; the DQ bus is shared).
- **tRTW** — cycles since the last RD (global; shared DQ bus).
- **tCCD** — cycles since the last column command (global; paces back-to-back RD/WR across banks on the shared DQ bus).

The arbiter reads five readiness outputs — tfaw_window_ok_o[r], trrd_window_ok_o[r] (per rank), and twtr_global_ok_o, trtw_window_ok_o, tccd_window_ok_o (global) — and ANDs the relevant subset into its eligibility decision alongside the per-bank safe_* bits.

21.2 Constraint Scope

Constraint	Scope	Why
tFAW	per rank	Four-Activate-Window — device-local thermal/power limit
tRRD	per rank	Minimum ACT-to-ACT stagger inside one DRAM device
tCCD	global (channel)	Column-to-column pacing on the shared DQ bus
tWTR	global (channel)	Write → read turnaround on the shared DQ bus
tRTW	global (channel)	Read → write turnaround on the shared DQ bus

There is **no cross-rank (tRTRS / tCS) tracking** in the live module. NUM_RANKS only sizes the per-rank tFAW/tRRD arrays; for NUM_RANKS == 1 those arrays are a single element and the per-rank outputs collapse to a single bit. The DQ-bus turnaround counters (tWTR/tRTW/tCCD) are single global counters shared across all ranks because the DQ bus is physically shared.

21.3 Synthesis Parameters

Parameter	Default	Effect
NUM_RANKS	1	Number of per-rank tFAW windows and tRRD counters
NUM_BANKS	8	Present for interface symmetry; not used to size counters
RKW	clog2(NUM_RANKS)	Width of evt_act_rank_i (min 1)
BKW	clog2(NUM_BANKS)	Bank-select width (interface symmetry)

All timing reload values are runtime CSR-backed 8-bit inputs (t_faw_i, t_rrd_i, t_wtr_global_i, t_rtw_i, t_ccd_i) — not parameters.

21.4 Per-Rank Trackers

21.4.1 tFAW window — `r_faw_slots[NUM_RANKS][4]`

Each rank owns a 4-deep pool of 8-bit down-counters (`logic [NUM_RANKS-1:0][3:0][7:0]`). Every cycle each non-zero slot decrements. On an ACT to a rank, the slot with the **smallest remaining count** is reloaded with `t_faw_i`:

```
if (evt_act_i) begin
    // pick the slot with the smallest remaining count for this rank
    automatic int unsigned slot_pick = 0;
    automatic logic [7:0] slot_min = 8'hFF;
    for (int unsigned i = 0; i < 4; i++) begin
        if (r_faw_slots[evt_act_rank_i][i] < slot_min) begin
            slot_min  = r_faw_slots[evt_act_rank_i][i];
            slot_pick = i;
        end
    end
    r_faw_slots[evt_act_rank_i][slot_pick] <= t_faw_i;
    r_trrd_cnt [evt_act_rank_i]            <= t_rrd_i;
end
```

The window is "OK" (an ACT may issue) when **at least one** of the rank's four slots is at 0 — i.e. fewer than four of the most-recent ACTs are still inside the tFAW window:

```
w_tfaw_ok[k] = 1'b0;
for (int unsigned i = 0; i < 4; i++)
    if (r_faw_slots[k][i] == 8'd0) w_tfaw_ok[k] = 1'b1;
```

The "pick smallest count" load policy is equivalent to "replace the oldest entry," which is the correct rolling-window behavior. It avoids a timestamp FIFO (which would need a subtractor at check time); the check is a 4-way compare-to-zero, one LUT layer.

21.4.2 tRRD — `r_trrd_cnt[NUM_RANKS]`

One 8-bit down-counter per rank, reloaded with `t_rrd_i` on each ACT to that rank (see the ACT block above). `trrd_window_ok_o[r]` is registered from the counter's NEXT value, so it reads `(w_trrd_nxt[r] == 0)` – written in the RTL as `w_act_rank[r] ? (t_rrd_i == 0) : (r_trrd_cnt[r] <= 1)`, which keeps the late `evt_act_i` on a one-bit mux select. It is **not** `(r_trrd_cnt[r] == 0)`: that was the old form, and flopping it left the gate open for a cycle after the ACT that should have closed it (pumice ISSUE-018). An ACT on rank 0 does not affect `r_trrd_cnt[1]`.

21.5 Global Trackers (channel-wide)

Three single 8-bit counters shared across all ranks:

Counter	Reload trigger	Reload value	Drives
r_twtr_cnt	evt_wr_i (WR event)	t_wtr_global_i	twtr_global_ok_o
r_trtw_cnt	evt_rd_i (RD event)	t_rtw_i	trtw_window_ok_o
r_tccd_cnt	evt_wr_i OR evt_rd_i (any column command)	t_ccd_i	tccd_window_ok_o

```
if (evt_wr_i) begin r_twtr_cnt <= t_wtr_global_i; r_tccd_cnt <=
t_ccd_i; end
if (evt_rd_i) begin r_trtw_cnt <= t_rtw_i;        r_tccd_cnt <=
t_ccd_i; end
```

Each *_ok output is asserted when its counter is 0. tCCD reloads on **either** a read or a write, so it paces every column-to-column gap on the shared bus. tWTR loads on writes and gates reads; tRTW loads on reads and gates writes.

At DDR2-800 tRTW is typically 0 cycles, but the counter is still synthesized for parameter consistency and forward-compatibility with DDR3+ where tRTW matters.

> **Note on the tWTR trigger.** In the live RTL, tWTR reloads on the WR *command* event (evt_wr_i), not on a separate last-write-data-beat strobe. The CSR-supplied t_wtr_global_i value therefore carries whatever additional WL+BL/2 offset the timing programming folds in; there is no data-path wr_last_beat input to this module.

21.6 Readiness Outputs (strict-flop)

The *_window_ok_o outputs are **registered** (strict-flop house style). Each cycle the module computes the next-cycle window-ok values combinationally, then flops them:

```
// next-cycle tFAW-ok (combinational, per rank)
tfaw_window_ok_o <= w_tfaw_ok;
for (int unsigned k = 0; k < NUM_RANKS; k++)
    trrd_window_ok_o[k] <= (r_trrd_cnt[k] == 8'd0);
twtr_global_ok_o <= (r_twtr_cnt == 8'd0);
trtw_window_ok_o <= (r_trtw_cnt == 8'd0);
tccd_window_ok_o <= (r_tccd_cnt == 8'd0);
```

Out of reset, every *_ok output is driven to its permissive value ('1 / 1'b1) so the arbiter is not spuriously blocked before any command has issued.

Because the outputs are flopped, the arbiter sees a one-cycle-registered view of the counters — the same single-stage discipline as bank_timer's combinational safe_*, one flop deeper. The arbiter accounts for this uniform one-cycle readiness latency across both timer FUBs.

21.7 Advisory lookahead — `safe_{act,rdwr,pre}_la_o`

Added 2026-09-25. The pick pipeline decides on a **registered** bank image and the command fires four register stages later (`r_bank_*_ready` -> STAGE-1a -> pre-pick -> output), so a bank inside a tRCD/tRP/tRAS window read as not-ready even when that window would have closed before the command reached the DRAM. The arbiter idled through the latency instead of pipelining into it.

Every constraint here is a **saturating down-counter** that decrements once per cycle, so `r_X <= LA` is exactly "reaches 0 within LA cycles" — the lookahead is the same AND tree with a different comparator threshold, not a second timer. `BANK_LA` (default 4) is the select-to-fire stage count.

These outputs are **ADVISORY**. A reload can only come from a command the scheduler itself issues, so the prediction can be wrong; the scheduler's final stage re-checks the LIVE `safe_*` before anything leaves (see `07_scheduler.md`). A lookahead that guessed wrong therefore costs one dropped pick, never a JEDEC violation.

Two terms are deliberately **not** extrapolated, and both omissions are conservative — they can only under-report safety:

- `r_row_valid` is set by an ACT and cleared by a PRE that the scheduler itself issues. Predicting it would mean predicting the scheduler from inside the timer.
- The auto-precharge self-close **is** autonomous, but it RELOADS tRP when it fires, so `r_rp <= LA` is wrong for an AP-pending bank — the true condition is `max(preblk, ras) + 1 + t_rp <= LA`. An arm for this was built and measured: it changed nothing at any LA, because that ETA (5–7 cycles) exceeds the select-to-fire distance (3–4), so it predicts readiness LATER than the command arrives and the pick is dropped. Reverted, not shipped.

`LA=0` collapses each term to its `safe_*_o` twin exactly, which is how the plumbing was gated before the depth was turned up.

Measured on silicon (isolated A/B, same bitstream, only `BANK_LA` 0 -> 4): close-page reads +32.8%, page-hostile col_major +19.2%, streaming +0.5%. The gain lands where bank timers bind and nowhere else, which is the mechanism.

21.8 Observability

All observability outputs are combinational "counter non-zero" flags:

Signal	Width	Meaning
obs_faw_nz_o	NR	Rank's tFAW window currently full (! w_tfaw_ok[r])
obs_trrd_nz_o	NR	r_trrd_cnt[r] != 0
obs_twtr_nz_o	1	r_twtr_cnt != 0
obs_trtw_nz_o	1	r_trtw_cnt != 0
obs_tccd_nz_o	1	r_tccd_cnt != 0

21.9 Interface

21.9.1 Clocks / Reset

Signal	Direction	Description
mc_clk	input	Controller clock
mc_rst_n	input	Active-low async reset

21.9.2 Timing config (CSR-backed)

Signal	Width	Constraint
t_faw_i	8	tFAW
t_rrd_i	8	tRRD
t_wtr_global_i	8	tWTR
t_rtw_i	8	tRTW
t_ccd_i	8	tCCD (CAS-to-CAS)

21.9.3 Events from the arbiter

Signal	Width	Description
evt_act_i	1	ACT issued this cycle
evt_act_rank_i	RKW	Rank of the ACT (selects tFAW/tRRD)
evt_rd_i	1	RD issued this cycle
evt_wr_i	1	WR issued this cycle

21.9.4 Readiness to the arbiter

Signal	Width	Description
`tfaw_window_ok_o`	NR	Per-rank: rank may issue another ACT
`trrd_window_ok_o`	NR	Per-rank: tRRD elapsed
`twtr_global_ok_o`	1	Global: tWTR elapsed (RD may follow WR)
`trtw_window_ok_o`	1	Global: tRTW elapsed (WR may follow RD)
`tccd_window_ok_o`	1	Global: tCCD elapsed (next column cmd OK)

21.9.5 Observability

`obs_faw_nz_o[NR]`, `obs_trrd_nz_o[NR]`, `obs_twtr_nz_o`, `obs_trtw_nz_o`, `obs_tccd_nz_o`.

21.10 Timing Budget

Event → readiness is single-cycle plus the output flop:

```
cycle T:   arbiter asserts evt_act_i, evt_act_rank_i = r
cycle T:   r_trrd_cnt[r] <- t_rrd_i; picked tFAW slot <- t_faw_i
cycle T+1: trrd_window_ok_o[r] = 0 (registered); another ACT to rank r
blocked
```

The window-ok comparators are one LUT layer (compare-to-zero) feeding the output flop, so the path is short. The arbiter combines these with the per-bank `safe_*` bits; the aggregate eligibility AND is the arbiter's own critical path (§2.7), not this module's.

21.11 Verification Notes (cocotb test plan)

Scenario	What it proves
Two ACTs to same rank, gap < tRRD	Second blocked; `trrd_window_ok_o[r]` low
Two ACTs to same rank, gap == tRRD	Second issues exactly at tRRD
Five ACTs to one rank inside tFAW window	Fifth blocked; all four slots non-zero
Five ACTs spanning the tFAW boundary (4 in + 1 aged out)	Fifth issues; oldest slot returned to 0
Multi-rank: ACTs on rank 0 do not block ACTs	Per-rank tRRD / tFAW isolation

Scenario	What it proves
on rank 1	
WR then RD before tWTR elapses	RD blocked; `twtr_global_ok_o` low
RD then WR before tRTW elapses	WR blocked; `trtw_window_ok_o` low
Back-to-back column commands before tCCD elapses	Second column blocked; `tccd_global_ok_o` low
Reset → all `*_ok` outputs assert permissively	Reset default correctness
tFAW “pick smallest” evicts the oldest slot	Rolling-window load policy

21.12 Open Questions / Future Work

- **Cross-rank turnaround (tRTRS / tCS).** The live module has no rank-switch DQ-handoff or chip-select-setup tracking. On the DDR2/LPDDR2 board targets (single rank, or same-rank streaming) these are met by IOB margin. A true multi-rank DDR3/DDR4 build would add a `last_rd_rank` / `last_cmd_rank` pair and cross-rank gate here — reserved but not implemented.
- **tWTR from data-path event.** tWTR currently reloads on the WR command event, relying on the CSR value to fold in WL+BL/2. A more precise implementation would load from the last-write-data-beat strobe; add a `wr_last_beat_i` input if characterization shows the command-relative approximation costs bandwidth.
- **Bank-group tCCD (DDR4).** DDR4 splits tCCD into same-group (tCCD_L) vs different-group (tCCD_S). DDR2/LPDDR2 have no bank groups, so the single global tCCD is exact here; the DDR4 family controller adds per-group tracking.

> **Status (2026-10-10): this block lives in common-ip now.** The RTL is `mc_scheduler_layer`; its microarchitecture is owned by common_mas

Chapter 4 — Scheduler Layer and Pick Machinery. This chapter keeps the pumice provenance and measured history; for current interfaces and parameters, read the common chapter.

22 Global Timers (global_timers)

Module: global_timers.sv **Location:** rtl/fub/ **Category:** FUB **Parent macro:** pumice_scheduler_layer **Status:** implemented (v2 H)

22.1 Purpose

Controller-wide constraint trackers that span banks. Where pumice_bank_timers holds per-(rank, bank) JEDEC "safe" countdowns, global_timers holds the **windows that apply across banks** — some per-rank (device-local limits), some truly global (shared DQ bus):

Constraint	Meaning	Tracked as
tFAW	At most 4 ACT commands within t_faw_i cycles	Per-rank: a 4-deep sliding window of countdown timers
tRRD	Minimum cycles between any two ACT commands	Per-rank: single countdown, reloaded on each ACT
tWTR	Cycles after a WR before any RD	Global: single countdown, reloaded on each WR
tRTW	Cycles after a RD before any WR	Global: single countdown, reloaded on each RD
tCCD	Cycles between back-to-back column (RD/WR) commands	Global: single countdown, reloaded on each RD *and* WR

tFAW and tRRD are **per-rank** because they enforce device-local thermal/power limits — each rank has its own 4-deep tFAW slot array and its own tRRD countdown, so multi-rank silicon meters each device independently. tWTR, tRTW, and tCCD are **global** because they gate the shared DQ bus, which every rank contends for.

The FUB exposes _window_ok flags to the scheduler:

```
tfaw_window_ok_o [NUM_RANKS]   // per-rank: 1 = at least one tFAW slot
is at 0
trrd_window_ok_o [NUM_RANKS]   // per-rank: 1 = tRRD has elapsed
twtr_global_ok_o               // global : 1 = tWTR has elapsed
trtw_window_ok_o               // global : 1 = tRTW has elapsed
tccd_window_ok_o               // global : 1 = tCCD has elapsed
```

All five _window_ok outputs are strict-flop registered, and they are registered from the **next** state of their counters, not the current one. That distinction is the block's contract: a consumer may gate on these outputs alone and every JEDEC window is satisfied. Nothing further is required of it.

It was not always so. The outputs used to be flopped from `r_*_cnt == 0`, the state they were about to replace, which left every gate open for exactly one cycle after the command that should have closed it – a tCCD of 2 and a tRTW of 1 both honoured as 1. Both consumers had grown compensating logic for it (`pumice_cmd_arbiter` stopped using `tccd_ok_i` for gating altogether, and added fire-history terms to the turnarounds after a board ILA capture showed a tRTW of 20 honoured as 1), while tFAW and tRRD had no compensation and were simply exposed. The counters and the readiness flags are now derived from one next-state function, so there is no second derivation to fall out of step. See pumice ISSUE-018 and `formal/pumice/global_timers`, whose proof assumes only what these outputs publish.

Default after reset is `'1` / `1'b1` (all windows open), which is correct given no ACTs/WRs/RDs have been issued.

22.2 Timer mechanics

All timers are 8-bit down-counters that saturate at 0 (they decrement each `mc_clk` while non-zero). Reload values come in on the `t_*_i` ports:

- **`evt_act_i`** (with `evt_act_rank_i`): installs `t_faw_i` into the targeted rank's tFAW slot with the smallest remaining count (the freed slot), and reloads that rank's tRRD with `t_rrd_i`.
- **`evt_wr_i`**: reloads the global tWTR (`t_wtr_global_i`) *and* tCCD (`t_ccd_i`).
- **`evt_rd_i`**: reloads the global tRTW (`t_rtw_i`) *and* tCCD (`t_ccd_i`).

`tfaw_window_ok_o[r]` is high when rank `r` will have at least one tFAW slot at 0 next cycle (i.e. fewer than 4 ACTs will still be inside the window when a command issued now would land). On an ACT the slot being reloaded does not count as free, since it is about to hold `t_faw_i`.

22.3 Observability

Combinational `obs_*` "non-zero" flags mirror each counter for CSR/debug observation. They read the current counter state, and because the `_window_ok` outputs are now registered from the next state, the two views are **aligned in the same cycle** – `*_window_ok` equals `!obs_*_nz` throughout. They used to be one cycle apart, which is the same lag that was the bug; a debug read that disagreed with the gate would misdirect any investigation of a spacing violation. Asserted in `formal/pumice/global_timers`.

```
obs_faw_nz_o  [NUM_RANKS]   obs_trrd_nz_o [NUM_RANKS]
obs_twtr_nz_o               obs_trtw_nz_o               obs_tccd_nz_o
```

22.4 Parameters

Parameter	Default	Purpose
`NUM_RANKS`	1	Per-rank tFAW/tRRD

Parameter	Default	Purpose
NUM_BANKS	8	array depth (BKW derivation)

22.5 Scope

- **Per-rank tFAW / tRRD** are implemented (v2 H). Multi-rank silicon may still need the per-rank windows tightened against rank-level scheduler coordination.
- **tCCD is tracked here** now (it was previously baked into burst-length pacing in the scheduler).

22.6 Tests

Verified by `dv/tests/fub/test_global_timers.py`: `smoke`, `tfaw_4_acts`, `trrd_spacing`, `twtr_after_wr`, `trtw_after_rd` (plus tCCD coverage as the suite is extended for the v2 H additions).

23 Refresh Controller (refresh_ctrl)

Module: refresh_ctrl.sv **Location:** rtl/fub/ **Category:** FUB **Parent macro:** pumice_scheduler_layer **Status:** Implemented — FSM-free tREFI/pending/drain accumulator with a REFab/REFpb selector and REFpb bank rotor.

> Refresh **recovery** (tRFC) is not this block's job: the arbiter (pumice_cmd_arbiter) loads a down-counter from TIMINGS_RFC_REFI.tRFC on each fired REF and blocks ACT/REF picks until it expires — see ch02/07. refresh_ctrl only meters *when* refreshes are owed.
>
> Scope note: this module is refresh_ctrl. It is a compact, FSM-free accumulator design — three registered counter blocks plus a bank rotor, all strict-flopped at the outputs. It tracks the tREFI interval, keeps a JEDEC-capped count of postponed refreshes, meters how many REF commands the scheduler should burst back-to-back, and (for LPDDR2 REFpb) rotates a target bank across grants.
>
> Features that earlier drafts of this chapter described as current architecture are **not implemented** and are not part of this module: DARP per-bank age selection, periodic ZQCS piggyback, per-rank PASR mask propagation, self-refresh coordination, LPDDR2 temperature (MR4) scaling, per-rank REFab round-robin, a multi-state FSM, and per-(rank, bank) refresh_req/refresh_gnt handshake arrays. Refresh does not use a bank-machine handshake: it raises a single refresh_req_o and wins against other scheduler commands by priority inside pumice_cmd_arbiter. Those retired features are collected under Open Questions / Future Work.

23.1 Purpose

refresh_ctrl owns DRAM refresh scheduling. It produces:

- refresh_req_o — a single request line that elevates refresh to highest priority in the scheduler. Strict default: high while any refresh is pending; the v3 postpone/pull-in credits (section 2b) reshape when it asserts.
- pending_refreshes_o — the count of postponed refreshes (JEDEC max 8).

- `refresh_drain_active_o` — a hint that the scheduler should keep granting REF back-to-back to work down a burst quota.
- `refresh_kind_o` / `refresh_bank_o` — the REFab-vs-REFpb selector and, in REFpb mode, the target bank produced by the bank rotor.
- `obs_*` — observability taps on internal state for future CSR readout.

This is the heart of the controller's correctness story: get refresh wrong and DRAM forgets data. The design is deliberately simple — three registered counter blocks and a bank rotor, with no control FSM.

23.2 Synthesis Parameters

Parameter	Default	Effect
`NUM_BANKS`	`8`	Number of banks the REFpb rotor cycles through (`0..NUM_BANKS-1`)
`BA_W`	`$clog2(NUM_BANKS)`	Width of `refresh_bank_o` / the bank rotor (derived)

The module imports `pumice_pkg::*` and uses the standard reset macros (`reset_defs.svh`). Clock is `mc_clk`; reset is `mc_rst_n` (active-low).

23.3 Behavioral Blocks

There is no FSM. The module is four registered blocks feeding a strict-flop output stage.

23.3.1 1. tREFI Counter (`r_refi_cnt`)

A 16-bit down-counter that measures the refresh interval in MC cycles.

- Only ticks when `enable_i` is high (`enable_i` comes from `init_sequencer` — refresh is gated off until init completes). While `enable_i` is low, the counter is held reloaded at `t_refi_i`.
- When it reaches 0 (`w_refi_expired`), it reloads `t_refi_i` and that expiry event drives one increment of the pending accumulator.

23.3.2 2. Pending Accumulator (`r_pending`)

A 4-bit count of postponed refreshes, capped at `MAX_PENDING = 8` (the JEDEC ceiling of postponed refreshes).

- A tREFI expiry (while enabled) adds 1, saturating at 8. At saturation the RTL comment flags a looming DRAM data-retention violation — the workload has blocked refresh longer than JEDEC allows. With pull-in credit banked (below), an expiry consumes a credit instead of adding to the backlog.
- A grant retires a pending refresh if any (`w_grant_accept`); a grant with no backlog banks a pull-in credit (`w_grant_early`).
- A simultaneous expiry and grant net to zero change.

23.3.3 2b. JEDEC +-8 Credits (`REF_CTRL.postpone_limit` / `pullin_limit`, v3)

Two 4-bit CSR knobs shape when the request line asserts; 0/0 is the strict baseline (request the moment anything is pending), bit-identical to v2.

- **Postpone** (`postpone_limit`, clamped to 7): while demand persists, `refresh_req_o` is withheld until the backlog EXCEEDS the limit. The clamp guarantees the saturating backlog (8) can always exceed it, so the retention ceiling forces refresh even under unbroken demand.
- **Pull-in** (`pullin_limit`, clamped to 8): once idle is confirmed, refreshes run AHEAD of tREFI, banking up to the limit in `r_pullin`; each later expiry consumes a credit, giving a following demand burst a refresh-free window.
- **Idle confirmation**: `demand_i` (scheduler CAM occupancy) blinks off for a few cycles between bursts; a 16-cycle hysteresis counter (`IDLE_CONFIRM`) keeps micro-gaps from releasing postponed refreshes or triggering pull-ins mid-stream.
- `refresh_req_o` is therefore `idle ? (backlog > 0 || credit < pullin_limit) : (backlog > postpone_limit)`, registered.

23.3.4 3. Drain Quota (`r_burst_remaining`)

A 4-bit counter that meters how many REFs the scheduler should issue back-to-back once refresh wins arbitration.

- When the previous burst is fully drained (`r_burst_remaining == 0`) and there is pending work (`r_pending > 0`), it (re)loads `min(refresh_burst_i, r_pending)`. If that clamp evaluates to 0 it loads 1 instead, so a burst always makes forward progress.
- Each accepted grant decrements it.
- `w_drain_active = (r_burst_remaining > 0) && (r_pending > 0) && refresh_req_o` is exported as `refresh_drain_active_o`. While it is high, the scheduler should keep granting REF back-to-back rather than yielding

to reads/writes. The `refresh_req_o` term is load-bearing: a postponed backlog (request withheld) must not open the drain window, or the arbiter's drain preemption would defeat the postpone credit entirely.

`refresh_burst_i` is a 4-bit input (1..8) that sets the desired drain count per request cycle.

23.3.5 4. REFpb Bank Rotor (`r_bank_rotor`)

Mirrors the DEVICE'S internal per-bank refresh counter (JESD209-2 6.6 — the REFpb command carries no bank address; the device walks its own strict rotor and the controller can only track it).

- The rotor advances exactly when a REFpb COMMAND is granted onto the wire (`grant_was_pb_i`, driven by the scheduler from the ARBITER-side granted op — not the output-FIFO head, which is an older command, and not `refpb_mode_i`, which desynchronizes at every mode boundary).
- It HOLDS through REFab mode: the device's counter persists across mode changes, and clearing the mirror would desynchronize it.
- In REFpb mode the tick interval switches to tREFIpb (`REF_TIMING_PB.trefi_pb`, 0 = derive `t_refi_i >> 3`).
- `refresh_bank_o` is the registered rotor value; it is only meaningful in REFpb mode.
- `r_grants_total` (16-bit) counts total accepted grants for observability.

`refresh_kind_o` is simply the registered copy of `refpb_mode_i` (0 = REFab, 1 = REFpb), forwarded to the scheduler / `dfi_cmd_formatter`.

All final outputs are strict-flopped (registered) in a single output stage.

23.4 Interface

23.4.1 Parameters and Clocking

Signal	Direction	Width	Description
`mc_clk`	input	1	Memory-controller clock
`mc_rst_n`	input	1	Active-low asynchronous reset

23.4.2 Inputs

Signal	Width	Description
`t_refi_i`	16	Refresh interval in MC cycles (tREFI). Reload value for

Signal	Width	Description
		r_refi_cnt.
refresh_burst_i	4	Desired drain count per request cycle (1..8). Loads the burst quota.
refpb_mode_i	1	0 = REFab, 1 = REFpb (LPDDR2). Selects refresh_kind_o and enables the rotor.
enable_i	1	From init_sequencer (init done). Gates the tREFI counter and accumulation.
postpone_limit_i	4	REF_CTRL.postpone_limit — defer under demand, clamped to 7. 0 = strict.
pullin_limit_i	4	REF_CTRL.pullin_limit — run ahead on confirmed idle, clamped to 8. 0 = off.
demand_i	1	Scheduler CAM occupancy (any read/write waiting). Feeds the idle hysteresis.
refresh_grant_i	1	Pulsed by the scheduler when it issues a REF on the DFI bus.

23.4.3 Outputs

Signal	Width	Description
refresh_req_o	1	Credit-shaped request (see 2b), registered.
pending_refreshes_o	4	Current postponed-refresh count (0..8).
refresh_drain_active_o	1	Burst quota owed, pending nonzero AND request asserted — keep granting REF.
refresh_kind_o	1	Registered refpb_mode_i (0 = REFab, 1 = REFpb).
refresh_bank_o	BA_W	REFpb target bank from the rotor (valid in REFpb mode).

23.4.4 Observability (obs_*, future CSR readout)

Signal	Width	Source
obs_refi_cnt_o	16	Current r_refi_cnt value
obs_drain_remaining_o	4	Current r_burst_remaining value
obs_bank_rotor_o	BA_W	Current r_bank_rotor value

Signal	Width	Source
obs_grants_total_o	16	Total accepted grants (r_grants_total)
obs_pullin_credit_o	4	Current pull-in credit (r_pullin)

These are wired out for future CSR/telemetry hookup; no CSR block consumes them yet.

23.5 Timing / Behavior

- **Reset.** All registers clear to 0. refresh_req_o, refresh_drain_active_o, refresh_kind_o, refresh_bank_o, and every obs_* output reset to 0.
- **Init gating.** With enable_i low, r_refi_cnt is held at t_refi_i and no pending refreshes accumulate — refresh is inert until init_sequencer releases enable_i.
- **Single-cycle latency.** Because there is no FSM, request/drain/bank outputs simply track the counter state one flop behind. When tREFI expires, r_pending rises the next cycle and refresh_req_o follows the cycle after (output flop stage).
- **Grant handshake.** The module never blocks on a grant. It exports its request and drain hint; the scheduler decides when to pulse refresh_grant_i, and the module bookkeeps against those pulses.
- **Priority, not handshake.** Refresh wins against other scheduler commands by priority in pumice_cmd_arbiter; there is no per-bank refresh_req/refresh_gnt array and no bank-machine handshake.

23.6 Verification Notes (cocotb test plan)

Scenario	What it proves
enable_i low: t_refi_i loaded, no ticks, r_pending stays 0, refresh_req_o stays low	Init gating
enable_i high: r_refi_cnt counts down and reloads on expiry	tREFI countdown + reload
One tREFI expiry with no grant: r_pending == 1, refresh_req_o asserts	Pending accumulate + request
Withhold grants across 8+ expiries: r_pending	JEDEC max-8 postpone cap

Scenario	What it proves
saturates at 8, does not overflow	
Expiry and grant in the same cycle: `r_pending` unchanged	Simultaneous tick+grant net-zero
Grants with pending > 0: `r_pending` decrements; `refresh_req_o` drops at 0	Grant accounting
`refresh_burst_i = N` with `r_pending >= N`: quota loads N, `refresh_drain_active_o` high	Drain quota load + active hint
`refresh_burst_i > r_pending`: quota clamps to `r_pending`	Burst clamp (no overcount)
`refresh_burst_i` clamp resolves to 0 but pending > 0: quota loads 1	Forward-progress floor
Grants during drain: `r_burst_remaining` counts down; `refresh_drain_active_o` deasserts at 0	Drain metering
REFab mode (`refpb_mode_i = 0`): `refresh_bank_o` stays 0, `refresh_kind_o == 0`	REFab selector + rotor held
REFpb mode (`refpb_mode_i = 1`): `refresh_bank_o` rotates `0..NUM_BANKS-1` across grants	REFpb bank rotor wrap
`obs_*` taps track internal counters	Observability wiring

23.7 Open Questions / Future Work

The following were described in earlier SWAG/HAS drafts but are **not implemented** in `refresh_ctrl`. They are recorded here as possible future scope, not as current behavior:

- **DARP per-bank age selection.** REFpb currently uses a plain round-robin bank rotor. An age-aware selector (refresh the oldest idle non-masked bank first, falling back to oldest-first) would need bank-state and last-refresh-age inputs the module does not have today.
- **Periodic ZQCS piggyback.** No ZQCS interval counter or ZQCS sequencing exists. If added, it could piggyback on the refresh window (all banks idle) rather than triggering a separate bus-blocking event.
- **Per-rank PASR mask propagation.** No PASR bank/segment masks, no MR16/MR17 propagation. LPDDR2 partial-array self-refresh is unsupported.

- **Self-refresh coordination.** No `sr_entry_req`/`sr_entry_gnt` handshake with a power-state block; the tREFI counter is not paused for self-refresh.
- **LPDDR2 temperature scaling.** No MR4-driven tREFI derating. `t_refi_i` is the single reload value regardless of temperature class.
- **Per-rank REFab round-robin.** Single-rank only; there is no rank pointer or per-rank REF dispatch. `NUM_BANKS` (banks), not ranks, is the only structural parameter.
- **Multi-state FSM / per-bank handshake.** The design is FSM-free and uses a single priority-based request rather than per-(rank, bank) `refresh_req`/`refresh_gnt` arrays. Any of the above features would likely reintroduce sequencing state.

---

24 Init Sequencer (`init_sequencer`)

Module: `init_sequencer.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent macro:** `pumice_scheduler_layer` **Status:** Implemented — DDR2 and LPDDR2 sequences both complete (DDR2 validated on the Nexys A7 board; LPDDR2 passes the full sim suite for family reuse)

> **Renamed / re-scoped:** an early sketch called this an `init_engine_fub` built on a microprogram step-record ROM (opcodes, a step interpreter FSM, `SIM_INIT_SCALE`, `WAIT_FOR_BIT` polling, ZQ retry loops, an `init_error` path, IRQ outputs, and a CSR MR-override path). **None of that ROM machinery was built.** The implemented module is a small hard-coded FSM that issues a fixed JEDEC bring-up sequence. It drives MR-write strobes into `mode_register` (which owns the MR shadow + live CL/CWL/BL decode), and it issues the real DRAM commands (PRECHARGE, REFRESH, MRS/MRW) through the command path while `init_busy_o` is high.

24.1 Purpose

`init_sequencer` walks the JEDEC cold-boot DRAM initialization sequence — the deterministic recipe of DFI init handshake, PRECHARGE-ALL, mode-register loads, and REFRESH settling that must complete before any normal traffic can issue. It is memtype-specific: `memtype_i` selects between the DDR2 sequence (mirrors LiteDRAM's proven Nexys A7 recipe) and the LPDDR2 MRW sequence (JEDEC JESD209-2F).

Two things happen for each mode-register step:

1. A real DRAM command is issued. The sequencer drives a single-cycle command request (`init_cmd_valid_o` / `init_cmd_op_o` / `init_cmd_bank_o` / `init_cmd_row_o`) that the parent scheduler forwards to `dfi_cmd_formatter`. Because the scheduler stays idle during init (it owns nothing else while `init_busy_o` is high) and the formatter is always `cmd_ready`, a single-cycle pulse issues exactly one command — no grant handshake is needed.
2. The `mode_register` shadow is updated in lockstep (`mr_seq_we_o`) so the controller's live CL/CWL/BL decode tracks what was actually programmed.

The init sequencer sits above the scheduler in the command-priority hierarchy. `init_done_o` also drives `refresh_ctrl.enable_i` — periodic refresh does not start until init reports done.

History (why the sequence is “full”). An earlier version was a “simplified” 4-MR shadow-only walk that never issued MRS/PRECHARGE/REFRESH to the DRAM. On the DFI-loopback sim the memory model stores/returns data regardless of init state, so it passed. On real DDR2 the read DLL never locked (no proper reset + refresh sequence) and no IDELAY tap found a read eye. The full sequence documented here fixed on-board bring-up.

24.2 Synthesis Parameters

Parameter	Default	Effect
`ROW_WIDTH`	14	Width of the `init_cmd_row_o` request field (carries MR data; must hold DDR2 MR0=0x533 which needs bit 10, so a narrower COL-based path would truncate).
`NUM_BANKS`	8	Number of DRAM banks.
`BKW`	derived	`$clog2(NUM_BANKS)` — width of `init_cmd_bank_o` (bank / MR index).

There is no `MEMTYPE` synthesis parameter — the sequence is selected at run-time by the `memtype_i` input, so one elaboration supports both families.

24.3 Interface

24.3.1 Clock / reset

Signal	Direction	Description
`mc_clk`	input	Memory-controller clock.
`mc_rst_n`	input	Active-low async reset (house macros).

24.3.2 Configuration

Signal	Direction	Width	Description
`memtype_i`	input	enu	`MEMTYPE_DDR2` selects the DDR2 sequence;

Signal	Direction	Width	Description
		m	anything else selects the LPDDR2 MRW chain.

24.3.3 JEDEC init-sequence waits (CSR-backed, MC cycles)

All countdowns run internally as 16-bit values; the 8-bit inputs are zero-extended. Defaults live in the CSR (mentioned as 512 / 256 / 8 / 8 / 16), so these were promoted from hardcoded constants to `INIT_TIMING`-programmable.

Signal	Width	Internal name	JEDEC parameter
`t_init_wait_i`	16	`W_INIT`	CKE / tINIT settle (also LPDDR2 tINIT4).
`t_dll_wait_i`	16	`W_DLL`	DLL lock tDLLK (also LPDDR2 tZQINIT).
`t_mrd_wait_i`	8	`W_MRD`	tMRD, post mode-register-set.
`t_rp_wait_i`	8	`W_RP`	tRP, post precharge.
`t_rfc_wait_i`	8	`W_RFC`	tRFC, post auto-refresh.

24.3.4 DFI status

Signal	Direction	Description
`dfi_init_start_o`	output	Asserted (registered) once `r_state != S_RESET`. Tells the PHY to begin its own init.
`dfi_init_complete_i`	input	PHY reports its DLL-lock / IO training complete. Gates the exit from `S_DFI_INIT`.

24.3.5 MR-shadow write port (muxed with CSR by `pumice_scheduler_layer`)

Signal	Direction	Width	Description
`mr_seq_we_o`	output	1	Write-enable strobe into `mode_register`.
`mr_seq_index_o`	output	5	MR index (0..3) being shadowed.
`mr_seq_data_o`	output	16	MR data being shadowed.

24.3.6 DRAM command request into the scheduler (issued while init_busy)

Signal	Direction	Width	Description
`init_cmd_valid_o`	output	1	Single-cycle command pulse.

Signal	Direction	Width	Description
init_cmd_op_o	output	dram_op_e	OP_PREA, OP_REF, or OP_MRS.
init_cmd_bank_o	output	BKW	MR index for MRS (DDR2) / bank field.
init_cmd_row_o	output	ROW_WIDTH	MR data for MRS on the wide row path (see MR tables).

24.3.7 Legacy ZQCL handshake (DDR3+; unused for DDR2/LPDDR2)

Signal	Direction	Description
zqcl_req_o	output	Tied to 0 — DDR2 has no ZQCL command.
zqcl_grant_i	input	Unused (tied off via a _unused sink).

24.3.8 Status

Signal	Direction	Description
init_busy_o	output	Registered; high whenever r_state != S_DONE. Reset value is 1.
init_done_o	output	Registered; high when r_state == S_DONE. Drives refresh_ctrl.enable_i.

24.4 FSM

A single 5-bit state_e enum drives everything. Each command state is occupied for exactly one cycle (an unconditional transition to S_WAIT), so the combinational command decode produces a single-cycle pulse per command. S_WAIT holds for the programmed inter-command delay: it counts r_wait down and, when it reaches zero, jumps to r_next (the resume state latched by the command state).

State	Value	Role	Wait after (into S_WAIT)
S_RESET	0	Reset entry; unconditionally advances.	— (→ S_DFI_INIT)
S_DFI_INIT	1	Wait for dfi_init_complete_i, then arm W_INIT.	W_INIT
S_PREA1	2	DDR2: Precharge All (pre-EMR).	W_RP → S_EMR2

State	Value	Role	Wait after (into S_WAIT)
S_EMR3	3	DDR2: MRS EMR(3).	W_MRD → S_EMR1
S_EMR2	4	DDR2: MRS EMR(2).	W_MRD → S_EMR3
S_EMR1	5	DDR2: MRS EMR(1).	W_MRD → S_MR0_DLL
S_MR0_DLL	6	DDR2: MRS MR0 + DLL reset (MR0.VAL	0x100; reset 0x533).
S_PREA2	7	DDR2: Precharge All (pre-refresh).	W_RP → S_REF1
S_REF1	8	DDR2: Auto Refresh #1.	W_RFC → S_REF2
S_REF2	9	DDR2: Auto Refresh #2.	W_RFC → S_MR0
S_MR0	10	DDR2: MRS MR0, DLL-reset bit cleared (MR0.VAL; reset 0x433).	W_MRD → S_OCD_DEF
S_OCD_DEF	11	DDR2: MRS EMR(1) + OCD default (0x380).	W_MRD → S_OCD_EXIT
S_OCD_EXIT	12	DDR2: MRS EMR(1) + OCD exit (0x000).	W_MRD → S_DONE
S_WAIT	13	Inter-command countdown; r_wait==0 → r_next.	—
S_DONE	14	Terminal; init_done_o high, self-loop.	—
S_L_RESET	15	LPDDR2: MRW(MR63) Reset.	W_INIT → S_L_ZQ
S_L_ZQ	16	LPDDR2: MRW(MR10) ZQ Init Calibration.	W_DLL → S_L_MR1
S_L_MR1	17	LPDDR2: MRW(MR1) BL8 / nWR3.	W_MRD → S_L_MR2
S_L_MR2	18	LPDDR2: MRW(MR2) RL3 / WL1.	W_MRD → S_L_MR3
S_L_MR3	19	LPDDR2: MRW(MR3) drive strength 40ohm.	W_MRD → S_DONE

memtype_i only branches once: on exit from S_DFI_INIT, r_next is set to S_PREA1 for DDR2 or S_L_RESET otherwise. From there each path is a fixed chain of command states separated by S_WAIT.

S_DONE is a hard self-loop — there is no restart, no init_error, and no software-triggered re-run in the RTL. A fresh init requires mc_rst_n.

24.5 DDR2 Sequence

Mirrors LiteDRAM's `get_ddr2_phy_init_sequence` (upstream `litedram/init.py`), the reference proven on the Nexys A7:

1. Assert `dfi_init_start_o`; wait `dfi_init_complete_i` (PHY runs its own DLL-lock / IO training). Then wait `W_INIT` (tINIT / CKE settle).
2. Precharge All (`S_PREA1`), wait `W_RP`.
3. Load the extended mode registers in JEDEC JESD79-2 order **EMR(2), EMR(3), EMR(1)** — the state chain is `S_EMR2 → S_EMR3 → S_EMR1` — each followed by `W_MRD`. (Note the enum numbering: `S_EMR2` = 4, `S_EMR3` = 3, so the values are out of numeric order but the executed order is 2 → 3 → 1.)
4. Load MR0 + DLL reset (`S_MR0_DLL`, MR0.VAL | 0x100; reset 0x533: BL8 / CL3 / tWR3, DLL_RESET); wait `W_DLL` for the DLL to lock (~200 DRAM clocks).
5. Precharge All (`S_PREA2`), wait `W_RP`.
6. Auto Refresh x2 (`S_REF1`, `S_REF2`), each followed by `W_RFC`.
7. Load MR0 with the DLL-reset bit cleared (`S_MR0`, MR0.VAL; reset 0x433); wait `W_MRD`.
8. EMR(1) + OCD Default (`S_OCD_DEF`, 0x380) → EMR(1) + OCD Exit (`S_OCD_EXIT`, 0x000).
9. `S_DONE`: `init_done_o = 1`.

MR data is carried on `init_cmd_row_o` (ROW_WIDTH wide, MR index on `init_cmd_bank_o`), because MR0 = 0x533 sets bit 10 and a 10-bit column-based path would truncate it.

24.5.1 DDR2 MR values

Localparams, transcribed exactly:

Symbol	Value	State	Meaning	Shadowed?
`DDR2_MR0_DLL`	`MR0.VAL | 0x100` (reset 0x0533)	`S_MR0_DLL`	MR0: BL8 / CL3 / tWR3, DLL_RESET ORed in.	Yes (MR0)
`DDR2_MR0`	`MR0.VAL` (reset 0x043	`S_MR0`	MR0 with DLL_RESET cleared.	Yes (MR0)

Symbol	Value	State	Meaning	Shadowed?
	3)			
DDR2_MR1	0x0000	S_EMR1, S_OCD_EXIT	EMR(1): Rtt disabled, ODS full.	Yes (MR1)
DDR2_MR1_OCD	0x0380	S_OCD_DEF	EMR(1) + OCD calibration default.	No — transient calibration; shadow left at final EMR value.
DDR2_MR2	0x0000	S_EMR2	EMR(2).	Yes (MR2)
DDR2_MR3	0x0000	S_EMR3	EMR(3).	Yes (MR3)

`DDR2_MR0_DLL` decomposes as `log2(BL=4)=2 | (CL=3<<4)=0x30 | (tWR=3<<9)=0x400 = 0x433`, plus `reset_dll = 1<<8 = 0x100`, giving 0x533. OCD default = `EMR | (7<<7) = 0x380`; OCD exit = `EMR = 0`.

24.6 LPDDR2 Sequence

JEDEC JESD209-2F §3.4.1 power-up + §3.5 mode registers:

1. Assert `dfi_init_start_o`; wait `dfi_init_complete_i`; wait `W_INIT` (tINIT settle).
2. MRW(MR63) = Reset (`S_L_RESET`, OP don't-care); wait `W_INIT` (tINIT4).
3. MRW(MR10) = 0xFF ZQ Init Calibration (`S_L_ZQ`); wait `W_DLL` (tZQINIT).
4. Configure the device: MRW(MR1) = BL8 / nWR3 (`S_L_MR1`, 0x23), MRW(MR2) = RL3 / WL1 (`S_L_MR2`, 0x01), MRW(MR3) = DS 40ohm (`S_L_MR3`, 0x02) — each followed by `W_MRD` (tMRW).
5. `S_DONE`.

The CSR waits are reused: `W_INIT` covers tINIT4, `W_DLL` covers tZQINIT, `W_MRD` covers post-MRW settling.

LPDDR2 mode-register writes must reach indices up to MR63, which a 3-bit bank port cannot express. So the index (MA) and data (OP) are packed together into the wide row request by the `mrw_row()` function as `{MA[5:0], OP[7:0]}` (`init_cmd_row_o[13:8]` = MA index, `init_cmd_row_o[7:0]` = OP data); `dfi_cmd_formatter` unpacks this for the LPDDR2 CA MRW word.

24.6.1 LPDDR2 MR values

Symbol	OP value	State	Meaning	Shadowed?
`LPDDR2_MR63_OP`	`0x00`	`S_L_RESET`	MRW(63) Reset (OP don't-care).	No — issued, not shadowed (MR63 exceeds the 5-bit index).
`LPDDR2_MR10_OP`	`0xFF`	`S_L_ZQ`	MR10 ZQ Init Calibration.	No — issued, not shadowed.
`LPDDR2_MR1_OP`	`0x23`	`S_L_MR1`	MR1: nWR3 | BL8.	Yes (MR1).
`LPDDR2_MR2_OP`	`0x01`	`S_L_MR2`	MR2: RL3 / WL1.	Yes (MR2).
`LPDDR2_MR3_OP`	`0x02`	`S_L_MR3`	MR3: DS 40ohm.	Yes (MR3).

Only MR1 / MR2 / MR3 update the `mode_register` shadow (`mr_seq_we_o`), because those are the registers that feed the CL/CWL/BL decode. MR63 and MR10 are issued to the DRAM but not shadowed.

24.7 Init-Busy Gating

`init_busy_o` is high from reset (its reset value is 1) through every state except `S_DONE`. While busy:

- The parent `pumice_scheduler_layer` forwards the sequencer's command request to `dfi_cmd_formatter` and issues nothing of its own — the scheduler parks so exactly one command reaches DFI per `init_cmd_valid_o` pulse.
- `refresh_ctrl.enable_i` is low (driven by `init_done_o`), so periodic refresh does not start until init completes.

When the FSM reaches S_DONE, init_busy_o drops and init_done_o rises; the scheduler and refresh controller take over normal operation.

24.8 Verification Notes (cocotb test plan)

Scenario	What it proves
DDR2 init from cold reset to init_done_o	Full DDR2 state chain executes in order.
Observe the DDR2 command stream: PREA, MRS×3 (EMR2/EMR3/EMR1), MRS MR0_DLL, PREA, REF×2, MRS MR0, MRS OCD_DEF, MRS OCD_EXIT	Exact JEDEC order + op/index/data on each init_cmd_* pulse.
DDR2 MR values on init_cmd_row_o match 0x533 / 0x433 / 0x380 / 0x000 (MR0.VAL resets)	Wide row path carries bit 10 without truncation.
LPDDR2 init from cold reset to init_done_o	Full LPDDR2 MRW chain executes.
LPDDR2 mrw_row() packing: MR63/MR10 reach the DRAM; only MR1/MR2/MR3 set mr_seq_we_o	Index/data packing + selective shadowing.
dfi_init_complete_i held low → FSM stalls in S_DFI_INIT, init_busy_o stays high	DFI handshake gating.
Each command state produces exactly one init_cmd_valid_o cycle, then S_WAIT	Single-cycle pulse behavior.
Program short t_*_wait_i values → inter-command S_WAIT countdown matches	CSR-backed wait timing.
mr_seq_* shadow tracks the MRS writes (MR0/MR1/MR2/MR3 for DDR2)	Shadow lockstep with issued commands; OCD_DEF not shadowed.
After init_done_o, refresh_ctrl.enable_i asserts and refresh begins	Init → refresh handoff.

24.9 Open Questions / Future Work

- **No restart / error path.** `S_DONE` is terminal and there is no `init_error`, timeout, or software-triggered re-run — a fresh init requires `mc_rst_n`. If a DFI handshake never completes, the FSM simply parks in `S_DFI_INIT` forever. A future variant could add a timeout on `dfi_init_complete_i` and a status/restart CSR, but that machinery is not in the current RTL.
- **MR4 readback (LPDDR2).** LPDDR2 supports reading MR4 for device temperature class. The current sequencer only writes mode registers; it does no MRR readback. Folding a temperature read into or after init is possible future work but is not implemented.
- **ZQCL / ZQCS.** The `zqcl_req_o` / `zqcl_grant_i` ports exist only as a legacy DDR3+ handshake and are tied off; DDR2 has no ZQCL and LPDDR2 does its ZQ init via the MR10 MRW. If the family is extended to DDR3/DDR4, this port becomes a real ZQ-calibration handshake and the FSM would gain a ZQ state.
- **Per-rank MRS.** The sequencer issues a single stream of commands with no rank iteration; multi-rank support would require a rank field and a per-rank MRS loop. Out of scope for the current single-rank Nexys A7 target.

25 Power-Down Controller (powerdown_ctrl)

Module: powerdown_ctrl.sv **Location:** rtl/fub/ **Category:** FUB **Parent macro:** pumice_scheduler_layer **Status:** Live but optional — a single-channel idle-detect power-down requester. Not instantiated in the default top build. Deep Power Down, per-rank control, and refresh/init interlocks are TODO (see Open Questions).

> **Renamed / rescoped.** An earlier SWAG called this power_state_fub and described a per-rank FSM array with a separate DEEP_POWER_DOWN state, a quiet-point detector, self-refresh coordination handshake with the refresh manager, split APD/SRF idleness counters, an init-time CKE handoff mux, and a large CSR/IRQ interface. **None of that is implemented.** The actual RTL is a single channel-wide 4-state FSM (S_AWAKE / S_ARMING / S_REQ / S_ASLEEP) that detects controller idleness and requests either Precharge Power Down (PDE) or Self Refresh (SR). CKE is dropped for all ranks together. This chapter describes what the RTL actually does.

25.1 Purpose

powerdown_ctrl is a small idle-detector that asks the scheduler for permission to put the DRAM into a low-power state, then drops the DFI clock-enable (CKE) when granted. It supports two low-power flavors, selected by two enable inputs:

1. **PDE (Precharge Power Down)** — CKE low, banks may stay precharged; wakes on any new controller activity.
2. **SR (Self Refresh)** — CKE low while the DRAM self-refreshes internally. SR takes priority over PDE when both are enabled.

The FSM only drops and raises CKE. It does **not** issue the SREFE/SREFX DRAM commands, does **not** run any exit-timing counters (tXP / tXSDLL / tXSR), and does **not** coordinate a handshake with the refresh controller. All command issue, precondition enforcement (all banks precharged before SR), and exit timing are the responsibility of the parent scheduler (pumice_scheduler_layer) — this FUB is purely a request/grant + CKE-drive block. It trusts the scheduler not to issue a grant before DRAM init completes.

25.2 Synthesis Parameters

Parameter	Default	Effect
NUM_RANKS	1	Number of ranks. CKE is driven identically to all ranks (see below).
CS_WIDTH	NUM_RANKS	Width of the dfi_cke_o output vector.

There is no MEMTYPE parameter and no build-time DPD/PDE/SR selection — the enable inputs select behavior at run time.

25.3 FSM

The block is a **single** channel-wide FSM (state_e, 3-bit encoded), not a per-rank array. There is no separate Deep Power Down state.

State	Encoding	Meaning	dfi_cke_o
S_AWAKE	3'd0	CKE high, normal operation; idle counter held at 0	'1
S_ARMING	3'd1	Controller idle; counting cycles toward entry	'1
S_REQ	3'd2	pdn_req_o high, awaiting pdn_grant_i	'1
S_ASLEEP	3'd3	Granted; CKE low, in PDE or SR per latched r_kind	'0

Reset state is S_AWAKE with dfi_cke_o = '1 (CKE high out of reset).

25.3.1 PDE-vs-SR Selection

The request is enabled when either mode is enabled: w_request_enabled = enable_pde_i || enable_sref_i.

- enable_sref_i high → on reaching the idle threshold, request **SR** (pdn_kind_o = 1). SR has priority over PDE.
- enable_sref_i low, enable_pde_i high → request **PDE** (pdn_kind_o = 0).
- Neither enabled → never request; the FSM stays in S_AWAKE.

The kind is latched into r_kind at the S_ARMING → S_REQ transition (r_kind <= enable_sref_i) and is held through S_ASLEEP.

25.3.2 Transitions

```
S_AWAKE:
  r_idle_cnt <- 0
  if (w_request_enabled && controller_idle_i) -> S_ARMING

S_ARMING:
  if (!controller_idle_i || !w_request_enabled) -> S_AWAKE, r_idle_cnt
<- 0
  else if (r_idle_cnt >= idle_threshold_i)      -> S_REQ, r_kind <-
enable_sref_i
  else                                          r_idle_cnt <-
r_idle_cnt + 1

S_REQ:
  if (!controller_idle_i || !w_request_enabled) -> S_AWAKE, r_idle_cnt
<- 0
  else if (pdn_grant_i)                         -> S_ASLEEP
                                                   (r_grants_sr++ if
r_kind else r_grants_pde++)

S_ASLEEP:
  if (!controller_idle_i)                       -> S_AWAKE, r_idle_cnt
<- 0
  // wakes on ANY new activity; SR exit timing (tXSDLL/tXSR) is
enforced
  // by the scheduler, not here
```

Any loss of `controller_idle_i` (or de-assertion of both enables) at `S_ARMING` or `S_REQ` backs the FSM off to `S_AWAKE`. From `S_ASLEEP`, the FSM wakes on any new controller activity and returns to `S_AWAKE`; the scheduler is responsible for honoring exit latency before issuing commands to the DRAM.

The grant counters `r_grants_pde` / `r_grants_sr` (16-bit) increment on each successful grant and are exposed as observability outputs.

25.3.3 CKE Behavior

`dfi_cke_o` is a strict-flop output: `'1` out of reset and whenever the FSM is not in `S_ASLEEP`; `'0` in `S_ASLEEP`. `CKE` is `CS_WIDTH`-wide but every bit is driven identically — the block powers down **all ranks together**. Per-rank CKE control is a documented TODO.

All outputs are registered (strict-flop), so `pdn_req_o`, `pdn_kind_o`, `sref_active_o`, `dfi_cke_o`, and the `obs_*` signals lag the internal state by one cycle.

25.4 Interface

Clock `mc_clk`, active-low reset `mc_rst_n`.

25.4.1 Control / Configuration Inputs

Signal	Direction	Width	Description
`idle_threshold_i`	input	16	Idle cycles to count in `S_ARMING` before requesting power-down
`enable_pde_i`	input	1	Enable Precharge Power Down requests
`enable_sref_i`	input	1	Enable Self Refresh requests; takes priority over PDE
`controller_idle_i`	input	1	Controller idle indication from the scheduler

25.4.2 Request / Grant + CKE

Signal	Direction	Width	Description
`pdn_req_o`	output	1	High while in `S_REQ` (requesting power-down)
`pdn_kind_o`	output	1	Requested kind: `0` = PDE, `1` = SR
`pdn_grant_i`	input	1	Scheduler grant; `S_REQ → S_ASLEEP` on assertion
`sref_active_o`	output	1	High while asleep in SR (`S_ASLEEP && r_kind == 1`)
`dfi_cke_o`	output	`CS_WIDTH`	DFI clock-enable; `'0` in `S_ASLEEP`, else `'1`

25.4.3 Observability (future CSR readout)

These outputs are provided for future CSR readback; there is no CSR/APB interface wired into this FUB today.

Signal	Direction	Width	Description
`obs_state_o`	output	3	Current FSM state (`state_e` encoding)
`obs_idle_cnt_o`	output	16	Current idle counter value
`obs_grants_pde_o`	output	16	Count of PDE grants taken
`obs_grants_sr_o`	output	16	Count of SR grants taken

25.5 Verification Notes (cocotb test plan)

The block is small and self-contained; a directed FSM testbench is sufficient.

Scenario	What it proves
Reset → `S_AWAKE`, `dfi_cke_o = '1`, all `obs_*` cleared	Reset behavior and CKE-high default
Neither enable set; hold idle → FSM never leaves `S_AWAKE`	No request when disabled
`enable_pde_i` only; idle for `idle_threshold_i` cycles → `pdn_req_o`, `pdn_kind_o = 0`	PDE arming/threshold and request
`enable_sref_i` set (with or without PDE) → `pdn_kind_o = 1`	SR priority over PDE
`S_REQ` then `pdn_grant_i` → `S_ASLEEP`, `dfi_cke_o = '0`, `sref_active_o` per kind	Grant, sleep, CKE drop, SR-active flag
Activity (`!controller_idle_i`) during `S_ARMING` → back to `S_AWAKE`, counter cleared	Arming back-off
Activity during `S_REQ` (before grant) → back to `S_AWAKE`	Request withdrawal on new activity
Wake from `S_ASLEEP` on `!controller_idle_i` → `S_AWAKE`, `dfi_cke_o = '1`	Wake path
Both enables dropped mid-arming / mid-request → back-off to `S_AWAKE`	Enable de-assertion handling
Multiple sleep cycles → `obs_grants_pde_o` / `obs_grants_sr_o` increment correctly	Grant counters
`idle_threshold_i = 0` → request on first idle cycle in `S_ARMING`	Threshold edge case

25.6 Open Questions / Future Work

All of the following are called out in the RTL header comments and are **not** implemented today:

- **Deep Power Down (DPD).** LPDDR2-only. Would need an `enable_dpd_i` input and `dfi_dram_clk_disable_o` cooperation from `dfi_signal_pack`. There is no DPD state in the current FSM.
- **Per-rank power-down.** The block currently powers down **all ranks together** (identical CKE on every `dfi_cke_o` bit). A per-rank FSM array with independent CKE routing is future work.
- **`dfi_init_complete` interlock.** The FSM currently trusts the scheduler not to issue `pdn_grant_i` before DRAM init completes. A hard interlock input is future work.
- **Self-refresh command / handshake ownership.** SREFE/SREFX command issue, the all-banks-precharged precondition, and exit-timing enforcement (tXP / tXSDLL / tXSR) live in the scheduler, not here. There is no `refresh_ctrl` coordination handshake in this FUB.
- **CSR / observability wiring.** The `obs_*` outputs are intended for future CSR readback but are not connected to any APB register block yet.
- **Not in the default top build.** The module is live and verifiable standalone, but is not instantiated in the default `pumice` top today.

---

26 Mode Register (mode_register)

Module: mode_register.sv **Location:** rtl/fub/ **Category:** FUB **Parent macro:** pumice_scheduler_layer **Status:** implemented (DDR2 + LPDDR2 decode)

26.1 Purpose

Per-rank Mode Register shadow + live decode of MR-derived timing values for use by the rest of the controller.

On mr_we_i, write mr_data_i into shadow MR[mr_index_i] for mr_rank_i (indices < MAX_MR_IDX only). init_sequencer drives this during DRAM bring-up; a CSR/APB hot-update path drives it later.

The memtype_i input (memtype_e: MEMTYPE_DDR2 / MEMTYPE_LPDDR2) selects the decode branch for every output. The live decode reads from rank 0 (multi-rank designs must program matching MR values across ranks; mixed-per-rank MRs are a TODO).

26.2 Live Decoded Outputs

Output	Source (DDR2)	Source (LPDDR2)
cl_o	MR0[6:4]	RL, from the MR2[3:0] RL&WL enum
cwl_o	CL − 1	WL, from the MR2[3:0] RL&WL enum
bl_o	MR0[2:0]	MR1[2:0] (4/8; BL16 clips to 8)
al_o	MR1[5:3]	0 (not used)
drv_strength_o	MR1[1]	0 (informational)
odt_o	MR1[6,2]	0

For LPDDR2, CL and CWL are **not** independent MR fields — they are both derived from a single MR2[3:0] “RL & WL” enum per JESD209-2F:

MR2[3:0]	RL (cl_o)	WL (cwl_o)
0001	3	1
0010	4	2
0011	5	2
0100	6	3
0101	7	4
0110	8	4

(default / illegal codes fall back to RL3/WL1.)

For DDR2, `cl_o` is the raw MR0[6:4] field and `cwl_o` is simply `CL − 1` (saturating at 0). `bl_o` decodes MR0[2:0] = `010` → BL4, `011` → BL8.

All outputs are strict-flop registered. Consumed by:

- `pumice_cmd_arbiter` / `pumice_scheduler_layer` — use `cl_o`, `cwl_o`, `al_o` to time RD/WR latencies.
- write data path (`pumice_dfi_wr_serializer` via `t_phy_wrlat`) — sized against `cwl_o` for the WR-to-wrdata window.
- read data path (`pumice_dfi_rd_aligner` via `t_rddata_en`) — sized against `cl_o` for the RD-to-rddata window.
- `dfi_cmd_formatter` — uses `bl_o` to size column commands.

26.3 Burst Length: Fixed per Instance, Parameterized for Family Reuse

A memory controller supports exactly **one** DRAM burst length, chosen at init and fixed for the life of the instance — the datapath (beat sequencing, prefetch depth, column-stride math, tWR/tRTP-derived timing windows) is sized around that single value. pumice does **not** switch BL per transaction; the host/AXI burst is arbitrary and `axi_intake` splits it into fixed-BL DRAM commands.

`bl_o` is therefore a **build/init constant**, not a per-command control. It is decoded from the mode register rather than hardcoded so the **same RTL retargets across the DDR family** — the only thing that changes is the programmed value and the device/beat widths:

Family	Burst length	Source	pumice-beat count (via `bl_dram_beats`)
DDR2	BL4 (fixed)	MR0[2:0]=`010`	`bl_o >> log2(DRAM_BEAT_WIDTH/DRAM_DEVICE_WIDTH)`
DDR3	BL8 (fixed)	MR0[1:0]	same expression, `bl_o=8`
DDR4	BL8 (fixed)	MR0[1:0]	same expression, `bl_o=8`

Everything downstream keys off the decoded `bl_o` and the device/beat widths (see §15 “Narrow-Device (x16) Support”), so no burst-length constants are baked into the sequencers or the address decode. `BYTE_OFFSET_WIDTH` keys off **device width, not BL**, so the column granularity is already BL-agnostic.

BC4 / burst-chop (DDR3/DDR4): out of scope — always issue the full fixed BL (BL8 at the RTL reset; the Nexys A7 board runs BL4). DDR3 and DDR4 allow BC4 and BL8-on-the-fly (A12). That is the *only* case where one instance would see two effective burst lengths per transaction, which would break the fixed-BL datapath invariant. The design decision is to **not** support BC4: always transfer the full fixed BL8. Revisit only if a workload demonstrably needs the partial-burst bandwidth (it usually does not).

26.4 Scope

- `MAX_MR_IDX=17` (indices 0..16) covers both DDR2 (MR0..MR3) and LPDDR2 (up to MR16). The shadow array is `[NUM_RANKS][MAX_MR_IDX]`.
- **LPDDR2 BL16 clips to BL8** because `bl_o` is 4-bit. A follow-up widens `bl_o` to `[4:0]` and updates the 3 downstream macros that consume it.
- `mr_req_o` is tied 0 — no hot MR updates are issued via the scheduler. Init does the MR loads directly through this FUB's CSR write port. The `mr_req_*` channel is still flopped (all-zero) for port consistency and lands when the APB CSR slave provides a write-during-traffic path with a quiet-point handshake.

26.5 Tests

Verified by `dv/tests/fub/test_mode_register.py`: `smoke_ddr2`, `ddr2_cl_sweep`, `reset_values`, `ddr2_bl_sweep`, `ddr2_al_sweep`, `multi_rank` (plus LPDDR2 RL/WL-enum coverage as the suite is extended for the `memtype_i` LPDDR2 branch).

> **Status (2026-10-10): this block lives in common-ip now.** The RTL is `mc_storage_layer`; its microarchitecture is owned by common_mas mc_storage_layer — Storage Layer and Entry-Pool FUBs. This chapter keeps the pumice provenance and measured history; for current interfaces and parameters, read the common chapter.

27 Write Data Path (pumice_wr_data_cam + pumice_dfi_wr_serializer)

Modules: pumice_wr_data_cam.sv, pumice_dfi_wr_serializer.sv **Location:** rtl/fub/ **Category:** FUB **Parents:** pumice_axi4_layer (CAM), pumice_dfi_layer (serializer) **Status:** implemented

The old single-block wr_beat_sequencer / wr_data_path_fub no longer exists. In the rearchitected controller the write data path is split across two clock domains and two FUBs:

- pumice_wr_data_cam (MC domain, inside pumice_axi4_layer) — a write-command CAM plus a wr-data SRAM with three de-FSM'd movers.
- pumice_dfi_wr_serializer (DFI domain, inside pumice_dfi_layer) — a purely mechanical DFI-word streamer that drives dfi_wrdata.

The single clock-domain crossing between them is pumice_dfi_cdc (async gaxi FIFOs). This chapter documents both halves.

Architectural context: HAS §3.7 and _SWEEP_GROUND_TRUTH.md §8. Both halves follow the repository "streaming pipeline + flags" model — no enumerated datapath FSM in the CAM movers (the DFI serializer has a tiny 3-state pacing FSM only, described below). Sequence is enforced by data-flow handshakes and burst beat-counters, not by slot state latches.

27.1 Purpose

The write data path stages the AXI write burst that arrives on the AW/W channels, holds it while the scheduler picks a moment to commit it to DRAM, and — when the DFI layer is ready — streams it onto the DFI write bus one DFI word per cycle.

pumice_wr_data_cam is the MC-domain staging structure. It stores each pending write's key {bank, row, col}, its AXI id, a free-running age, and the burst's beats in an SRAM slot. It exposes associative query ports so the pumice_cmd_arbiter can find row-hit writes, an oldest-entry port as a scheduler fallback, and a snarf (read-your-write) forwarding port for pumice_rd_intake. On commit it drains the SRAM slot into the DFI layer.

pumice_dfi_wr_serializer is the DFI-domain endpoint. On a WR command strobe (wr_fire_i) it waits t_phy_wrlat DFI cycles, then pops one DFI word per cycle from its write-data FIFO onto dfi_wrdata / dfi_wrdata_en / dfi_wrdata_mask until the burst's last word.

27.2 pumice_wr_data_cam — three movers over one SRAM

The CAM never runs a datapath state machine. It has one SRAM (r_sram[N_SRAM_SLOTS*BL], plus a companion strobe array r_strb) and three movers, each of which is just a burst beat-counter reading the head of a request FIFO:

Mover	Trigger source	Direction	Beat counter
fill	u_fill_q (insert-order slot FIFO)	wd_data_i → SRAM	r_fill_beat
snarf	u_snarf_q (accept-order slot FIFO)	SRAM → snarf_rd	r_sn_beat
commit	u_drain_q (scheduled-slot FIFO)	SRAM → cm_rd	r_cm_beat

Each mover streams straight off its request-FIFO head, which is stable until popped. The head slot IS the “active” slot; “active” simply means the FIFO is non-empty; the FIFO is popped on the burst’s last beat. This is the de-FSM’d streaming reader pattern — no active/slot latch.

27.2.1 Entry state

Per entry (NUM_ENTRIES, default 8): r_valid, r_bank, r_row, r_col, r_id, r_age, plus:

- r_ptr — SRAM slot index, set on the entry’s first fill beat.
- r_pv — pointer-valid (first beat has been staged).
- r_fdone — **fill COMPLETE** (wd_last has been seen). An entry may be scheduled or snarfed only once r_fdone is set; otherwise the commit-drain mover could outrun a gapped fill and read stale SRAM beats.
- r_sched — the scheduler has committed this slot. It is set the cycle commit_valid_i fires and immediately excludes the entry from the sched_lu / oldest ports, so the arbiter can pick the next entry the very next cycle (clean one-command-per-clock). It clears on evict.

r_age_ctr is a free-running counter; relative age is w_rel[i] = r_age_ctr - r_age[i], which is wrap-safe.

27.2.2 SRAM slot pre-allocation

N_SRAM_SLOTS may be less than NUM_ENTRIES. r_sram_occ is a per-SRAM-slot occupancy bitmap. On the first fill beat of an entry a free slot is chosen (w_slot_free, highest-index-first scan), recorded into r_ptr, and marked occupied. The slot is freed on the entry’s commit evict. wd_ready_o deasserts on the first beat if no SRAM slot is free.

27.2.3 Age-based selectors

Three selectors run combinationally over the entry array (all skip already-scheduled and not-fill-complete entries where applicable):

- **oldest port** — oldest valid, fill-complete, unscheduled entry (max relative age). Scheduler fallback.
- **scheduler lookups** (`N_SCHED_LU`, default 4) — for each `{bank, row}` query, the oldest matching fill-complete unscheduled entry. This gives the arbiter in-order commit per row.
- **snarf lookup** — the *youngest* match (min relative age), so a read sees the latest write data.

27.3 Snarf: write-to-read forwarding (was the standalone `wr2rd_forward`)

There is no standalone `wr2rd_forward` FUB anymore. Write-to-read forwarding is the **snarf mover inside `pumice_wr_data_cam`**. When `pumice_rd_intake` presents an AR's decoded key on the snarf probe port, the CAM combinationally searches for a matching in-flight write. The match is deliberately narrow — a write is snarfable only when all three hold:

1. **Not yet scheduled** (`!r_sched`). A scheduled write is draining/evicting to DRAM, so its CAM data is racy.
2. **Same AXI id** (`r_id[i] == snarf_probe_id_i`). Same-id write-before-read is the only AXI-ordered case where the read is *required* to observe the write; cross-id has no ordering guarantee, so a cross-id read takes the DRAM path instead.
3. **Same burst length** — `snarf_probe_len_i == BL-1`. Every admitted write is exactly `BL` beats (ragged bursts are rejected upstream in `pumice_wr_intake`), so this reduces to `arlen == BL-1`. A short or long read must not snarf a full-BL write.

The match must also be fill-complete (`r_fdone`). Among candidates the youngest is chosen. `snarf_hit_o` asserts when a valid probe finds a match with matching length. On `snarf_accept_i`, the matched slot is pushed into `u_snarf_q`; the snarf mover then streams `snarf_rd_data_o` beats (non-destructively — snarf does not evict the write) in accept order.

> The old chapter's "last-write-wins highest-slot-index" rule and the "any matching in-flight write regardless of id" behavior are gone. The live policy is youngest-match, same-id, same-BL, unscheduled only.

27.4 Commit drain and B response

The arbiter's commit does two things atomically: it sets `r_sched` on the slot (immediate exclusion from the pick ports) and enqueues the slot into `u_drain_q`. The commit mover works through scheduled slots at its own pace, streaming `cm_rd_data_o` / `cm_rd_strb_o` to the DFI write path and popping `u_drain_q` on the last beat. On that last beat the entry is evicted (`r_valid`, `r_pv`, `r_fdone`, `r_sched` cleared; SRAM slot freed) and `commit_done_valid_o` / `commit_done_id_o` strobe, which drives the `pumice_wr_intake` B response. Decoupling the scheduled-mark from the drain is what allows one-command-per-clock scheduling.

27.5 `pumice_dfi_wr_serializer` — the DFI-domain endpoint

This FUB is purely mechanical and lives in the DFI clock domain. Its internal datapath unit is the DFI word (`DFI_DATA_WIDTH`, default 128), which already carries all `DFI_RATE` phases; `dfi_signal_pack` splits it to the pins, so there is **no per-beat packing here**.

Interface (abridged):

Signal	Dir	Description
`t_phy_wrlat_i`	in	WR-cmd → wrdata_en delay (DFI cycles)
`wr_fire_i`	in	WR command strobe (from `pumice_dfi_cmd_path`)
`wd_valid/ready/data/strb/last`	in/out	Pre-staged write-data FIFO (DFI-word granular)
`dfi_wrdata_o`	out	`= wd_data_i` while driving
`dfi_wrdata_en_o`	out	all-ones while driving, else 0
`dfi_wrdata_mask_o`	out	`~wd_strb_i` while driving (AXI wstrb=1 write → DFI mask=1 means "mask", i.e. mask = ~strb)

It has a small 3-state pacing FSM (`S_IDLE`, `S_WAIT`, `S_DRIVE`) — the only FSM in the whole write path, and it exists to time `t_phy_wrlat`, not to route data:

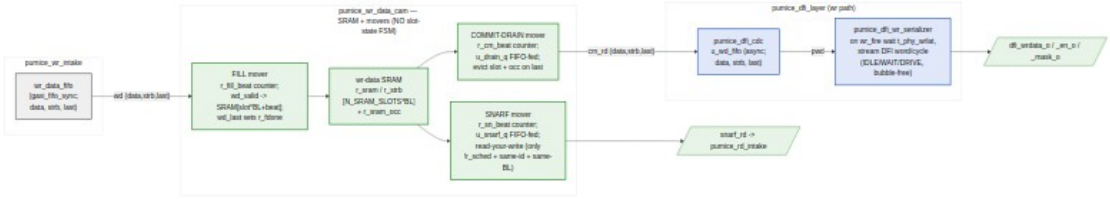
- `S_IDLE`: on an available fire, if `t_phy_wrlat==0` drive word 0 combinationally this cycle (via `w_start_now`) and go straight to `S_DRIVE` (or stay idle for a single-word burst); if `==1` go to `S_DRIVE`; else load `r_wait = t_phy_wrlat-1` and go `S_WAIT`.
- `S_WAIT`: count down, enter `S_DRIVE` at fire+wrlat.

- `S_DRIVE`: pop one DFI word per cycle. On the last word, if another fire is pending continue **seamlessly** into the next burst (zero bubbles); else return to `S_IDLE`.

A `r_pending` counter (3-bit) absorbs `wr_fire` strobes that arrive while a burst is in flight. This matters: the DFI cmd path paces column commands exactly `BL_WORDS` DFI cycles apart (the burst's DQ-bus occupancy), so the next burst's first word is due the cycle right after the current burst's last word. Because the data is pre-staged in the FIFO (the CAM drains it when the command is scheduled), at `wr_fire` the burst is already waiting, so the drive never stalls and bursts run back-to-back with zero bubbles.

> Fixed bug (see project notes): a fire arriving mid-burst used to be dropped, stranding the write in the PHY. It is now counted in `r_pending`.

27.6 Block Pipeline View



Write Data Path — streaming CAM movers + DFI-word serializer, no datapath FSM

Source: 15_wr_data_path_pipeline.mmd

27.7 `pumice_wr_data_cam` interface

27.7.1 Insert (from `pumice_wr_intake` aw_push)

Signal	Dir	Width	Description
`ins_valid_i`	in	1	Allocate an entry
`ins_ready_o`	out	1	Free entry AND fill-FIFO room
`ins_bank_i`	in	`BKW`	Decoded bank
`ins_row_i`	in	`ROW_WIDTH`	Decoded row
`ins_col_i`	in	`COL_WIDTH`	Decoded column
`ins_id_i`	in	`IW`	AXI id

27.7.2 Fill data (from pumice_wr_intake wdata pop)

Signal	Dir	Width	Description
wd_valid_i	in	1	Write-data beat valid
wd_ready_o	out	1	Fill-FIFO head present AND SRAM slot free
wd_data_i	in	DW	Beat data
wd_strb_i	in	SW	Byte strobes
wd_last_i	in	1	Last beat (sets r_fdone, pops fill FIFO)

27.7.3 Snarf lookup + stream (to/from pumice_rd_intake)

Signal	Dir	Description
snarf_probe_*	in	Probe key {bank,row,col}, id, len
snarf_hit_o	out	Valid probe matched (youngest, same-id, same-BL)
snarf_accept_i	in	Read admitted as a snarf; push slot to snarf FIFO
snarf_rd_valid/ ready/data/last	out /in	Snarf data stream (accept order)

27.7.4 Oldest port + scheduler lookups (to pumice_cmd_arbiter)

Signal	Dir	Description
oldest_valid_o + oldest_{bank,row, col,id,slot}_o	out	Oldest fill-complete unscheduled entry
sched_lu_valid_i[N] + sched_lu_{bank,ro w}_i	in	Per-port {bank,row} queries
sched_lu_hit_o[N] + sched_lu_{slot,co l,id,age}_o	out	Oldest matching entry per port

27.7.5 Commit / drain (to pumice_dfi_layer write path)

Signal	Dir	Description
commit_valid_i	in	Arbiter commits a slot
commit_ready_o	out	Room in the drain FIFO
commit_slot_i	in	Slot to schedule
cm_rd_valid/	out	Commit-drain data stream

Signal	Dir	Description
ready/data/strb/last	/in	
commit_done_valid_o + commit_done_id_o	out	Evict strobe → drives B response
busy_o	out	Any pending fill / snarf / drain / oldest

27.8 pumice_dfi_wr_serializer parameters

Parameter	Default	Effect
DFI_DATA_WIDTH	128	DFI-word width (= DRAM_BEAT_WIDTH*DFI_RATE)
DFI_RATE	2	Phases per DFI word (= DFI_EN_WIDTH)
DFI_STRB_WIDTH	DW/8	Byte-mask width
WRLAT_W	8	Width of t_phy_wrlat_i

27.9 Verification Notes (cocotb test plan)

Scenario	What it proves
Insert + fill a BL burst; r_fdone sets only on wd_last	Fill mover + fill-complete gate
Commit an entry; SRAM drains in order; evict + commit_done on last	Commit mover + B-response strobe
Snarf hit: same-id, same-BL, unscheduled write returns its data	Snarf youngest-match forwarding
Snarf miss: cross-id read, or scheduled write, or mismatched len	Snarf policy exclusions
Oldest / sched-lookup exclude scheduled + not-fill-complete entries	Pick-port correctness (1 cmd/clock)
SRAM slot exhaustion: wd_ready_o / ins_ready_o deassert	Pre-allocation backpressure
DFI serializer: t_phy_wrlat sweep (0, 1, N); first word at fire+wrlat	Write-latency alignment
Back-to-back WR fires paced BL_WORDS apart: zero-bubble drive	r_pending seamless continuation

Scenario	What it proves
`wr_fire` mid-burst is counted, not dropped	`r_pending` regression
`dfi_wrdata_mask_o == ~wd_strb_i` while driving	Strobe → mask inversion

27.10 Open Questions / Future Work

- **Cross-burst commit interleave.** The commit mover drains one scheduled slot at a time. Interleaving beats from multiple scheduled slots would need a per-slot beat-counter shadow. Punt to a later performance mode.
- **Multi-outstanding DFI reads/writes.** The serializer's `r_pending` counter is 3-bit; deeper pipelining is bounded by the DFI cmd path's pacing, not by this FUB.
- **Strobe-less writes.** Hosts that always write full beats could elide the strobe SRAM and mask path at elaboration. Minor area win; punt.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

> **Status (2026-10-10): this block lives in common-ip now.** The RTL is `mc_axi4_layer`; its microarchitecture is owned by common_mas mc_axi4_layer — AXI4 host interface and read-return ring. This chapter keeps the pumice provenance and measured history; for current interfaces and parameters, read the common chapter.

28 Read Data Path (pumice_dfi_rd_aligner + pumice_rd_return_ring)

Modules: pumice_dfi_rd_aligner.sv, pumice_rd_return_ring.sv, pumice_rd_cmd_cam.sv **Location:** rtl/fub/ **Category:** FUB **Parents:** pumice_dfi_layer (aligner), pumice_axi4_layer (ring, CAM) **Status:** implemented

The old single-block rd_cl_aligner / rd_data_path_fub no longer exists. In the rearchitected controller the read data path is split across two clock domains and three FUBs:

- pumice_dfi_rd_aligner (DFI domain, inside pumice_dfi_layer) — drives dfi_rddata_en at the right cycle for an issued READ and captures dfi_rddata words into the read return FIFO.
- pumice_rd_return_ring (MC domain, inside pumice_axi4_layer) — the outstanding-read reorder buffer. Returns arrive in *issue* order into the slot named by the read's ticket, and the ring drains from its head in *AR* order to pumice_rd_intake. See §22.
- pumice_rd_cmd_cam (MC domain, inside pumice_axi4_layer) — the scheduling window only. Its entry is freed the cycle the column issues.

Corrected 2026-09-09. This chapter previously said the CAM was the reorder buffer and buffered returned data per entry. That was true before the return ring landed (2026-09-08) and is not now: holding the read in the CAM for its whole round trip made the CAM depth the in-flight limit and capped read bandwidth by Little's law at roughly 180 MB/s on the board (pre-ring-fix state, 2026-09; the return-ring fix took board reads to ~572 MB/s).

The single clock-domain crossing between the domains is pumice_dfi_cdc (async gaxi FIFOs). This chapter documents the aligner half and the hand-off; the ring has its own chapter.

Architectural context: HAS §3.7 and _SWEEP_GROUND_TRUTH.md §8. The CAM has no datapath FSM — the movers are burst beat-counters over the SRAM. The DFI aligner has a small pacing FSM for the dfi_rddata_en window only. Sequence is enforced by data-flow handshakes, not by slot state.

28.1 Purpose

The read data path is the miss path: reads that cannot be satisfied by a snarf from the write CAM (see §17) go to DRAM. `pumice_rd_cmd_cam` is the mirror of `pumice_wr_data_cam` — entries keyed `{bank, row, col}` with a free-running age, an oldest port, and `N_SCHED_LU` scheduler lookups, so reads row-hit-schedule exactly like writes. It also acts as a **reorder buffer**: it accepts AR inserts in AR order, lets the scheduler issue them in an order it chooses (row-hit optimized), receives DRAM data in *issue* order, and drains it back to the host in *AR/oldest* order.

`pumice_dfi_rd_aligner` is the DFI-domain endpoint. On a RD command strobe (`rd_fire_i`) it drives the `dfi_rddata_en` capture window starting `t_rddata_en` DFI cycles later, and pushes one DFI word per `dfi_rddata_valid` cycle into the read return FIFO as `{data, resp, last}`.

Compared to the write CAM there is **no snarf port** here — snarf is a write-CAM concept (§17). Data flows *in* from the DFI return and *out* to `pumice_rd_intake`.

28.2 `pumice_rd_cmd_cam` — reorder buffer, two movers over one SRAM

The CAM stores each burst's return data in an SRAM (`r_sram[N_SRAM_SLOTS*BL]`) and runs two de-FSM'd movers, each a burst beat-counter:

Mover	Trigger source	Direction	Beat counter
return-fill	`u_issue_q` (issue-order slot FIFO)	dfi_ret → SRAM	`r_ret_beat`
drain	oldest-valid pick, data-ready gated	SRAM → drain	`r_dr_beat`

28.2.1 Entry state and age

Per entry (`NUM_ENTRIES`, default 8): `r_valid`, `r_issued` (scheduler has issued it to DRAM), `r_ready` (return data complete), `r_bank`, `r_row`, `r_col`, `r_id`, `r_resp`, `r_age`, plus `r_ptr` (SRAM slot, set on the first return beat) and `r_pv` (pointer valid). `r_age_ctr` is free-running; relative age `w_rel[i] = r_age_ctr - r_age[i]` is wrap-safe.

SRAM slot pre-allocation is identical to the write CAM: `r_sram_occ` bitmap, highest-index-first free scan, allocate on first return beat, free on drain evict.

28.2.2 The three age selectors

- **issue-side oldest** — oldest valid **not-yet-issued** entry (max rel). Scheduler fallback; drives the `oldest_*` port.

- **scheduler lookups** (N_SCHED_LU, default 4) — per {bank, row} query, the oldest not-issued match. Row-hit read scheduling.
- **drain-side oldest** — oldest valid entry over **all** valid entries (max rel), regardless of issued/ready. This is w_dro_slot.

28.2.3 Ordering guarantee

Inserts happen in AR order and each new insert is *younger*, so the drain-side oldest pick w_dro_slot is stable across a burst: the entry stays valid until its own last-beat evict, and later inserts can never become "more oldest". The drain mover therefore needs no active latch — the draining slot IS the oldest-valid pick. It only fires when that oldest entry's data is staged: w_dr_go = w_dro_found && r_ready[w_dro_slot]. This is what enforces AR-order release even though DRAM returns arrive in issue order.

28.2.4 Issue-order return fill

When the scheduler issues a read it notifies the CAM (issue_valid_i / issue_slot_i), which sets r_issued[slot] and pushes the slot into u_issue_q. DRAM returns arrive in the same order the reads were issued, so the return-fill mover writes each return burst into the issue-FIFO head slot's SRAM region. On the burst's dfi_ret_last, it sets r_ready, captures r_resp, and pops u_issue_q.

28.3 pumice_dfi_rd_aligner — the DFI-domain endpoint

This FUB mirrors pumice_dfi_wr_serializer. Its internal unit is the DFI word (= dfi_rddata width); BL_WORDS = BL/DFI_RATE words per burst.

It has two independent concerns:

1. dfi_rddata_en window (small pacing FSM). A 3-state FSM (S_IDLE, S_WAIT, S_EN) plus a pending-fire counter r_epend:

- S_IDLE: on an available fire, if t_rddata_en==0 drive word 0's enable combinationally (w_en_now) and seed the remaining-cycle counter for BL_WORDS-1 more en cycles; if ==1 go straight to S_EN; else load r_ewait = t_rddata_en-1 and go S_WAIT.
- S_WAIT: count down, enter S_EN at fire+t_rddata_en.
- S_EN: assert dfi_rddata_en_o for the window. On the last en-cycle, if another fire is pending reseed for a contiguous back-to-back window (zero bubbles); else return to S_IDLE.

Like the write serializer, the DFI cmd path paces column commands `BL_WORDS` apart, so windows abut with zero bubbles. A `rd_fire` arriving mid-window is counted in `r_epend` (previously it was dropped, leaving the second read with no capture window and stranding it in the PHY).

2. Capture (flag-and-counter only). On each `dfi_rddata_valid` (`w_word_valid = | dfi_rddata_valid_i`) a word is pushed to the read FIFO: `rd_data_o = dfi_rddata_i`, `rd_resp_o = OKAY`, and `rd_last_o` asserts on word `BL_WORDS-1`. A single word counter `r_rcnt` tracks progress; it wraps on the last word.

> v1 note: `rd_resp_o` is hardwired `RESP_OKAY` — the DFI rddata-error signal is not yet wired. The aligner supports one outstanding read window (single-issue, tRTW/tCCD spaced by the scheduler / `global_timers`).

28.4 Block Pipeline View



Read Data Path — DFI en-window aligner + issue-order fill + AR-order drain reorder buffer

Source: 16_rd_data_path_pipeline.mmd

28.5 `pumice_rd_cmd_cam` interface

28.5.1 Insert (from `pumice_rd_intake` ar_push, AR order)

Signal	Dir	Width	Description
`ins_valid_i`	in	1	Allocate an entry
`ins_ready_o`	out	1	Free entry available
`ins_bank/ row/col/id_i`	in	key + id	Decoded key + AXI id

28.5.2 Scheduler ports (to `pumice_cmd_arbiter`)

Signal	Dir	Description
`sched_lu_valid_i[`	in	Per-port {bank,row} queries

Signal	Dir	Description
N] + sched_lu_{bank,row}_i		
sched_lu_hit_o[N] + sched_lu_{slot,col,id,age}_o	out	Oldest not-issued match per port
oldest_valid_o + oldest_{bank,row,col,id,slot}_o	out	Oldest not-issued entry

28.5.3 Issue notify (from scheduler)

Signal	Dir	Description
issue_valid_i	in	Scheduler issued this slot to DRAM
issue_ready_o	out	Room in the issue-order FIFO
issue_slot_i	in	Slot issued (records issue order for return fill)

28.5.4 DFI return (from pumice_dfi_layer read path, issue order)

Signal	Dir	Description
dfi_ret_valid_i	in	Return beat valid
dfi_ret_ready_o	out	Issue-FIFO head present AND SRAM slot free
dfi_ret_data_i	in	Return beat data
dfi_ret_resp_i	in	Return response (captured on last)
dfi_ret_last_i	in	Last beat (sets r_ready, pops issue FIFO)

28.5.5 Drain (to pumice_rd_intake R source, AR order)

Signal	Dir	Description
drain_valid_o	out	Oldest entry with data ready
drain_ready_i	in	Downstream accepts
drain_data_o	out	Beat data
drain_id_o	out	AXI id (rid echo)
drain_resp_o	out	rresp
drain_last_o	out	Last beat (evicts entry, frees SRAM slot)
busy_o	out	Oldest-valid present OR issue FIFO non-empty

28.6 pumice_dfi_rd_aligner parameters

Parameter	Default	Effect
DFI_DATA_WIDTH	128	DFI-word width
DFI_RATE	2	Phases per DFI word (= DFI_EN_WIDTH, DFI_VALID_WIDTH)
BL_WORDS	4	DFI words per read burst (= BL/DFI_RATE)
RDEN_W	8	Width of t_rddata_en_i

28.7 Out-of-Order Completion (across AXI IDs)

The CAM preserves AXI ordering the way the standard requires: per-ID reads complete in issue order (a single in-flight read returns all its beats in burst order), while cross-ID reads may complete out of order because the scheduler is free to issue them in row-hit order. The reorder buffer's AR-order drain (oldest-first, data-ready gated) makes the release order deterministic; the drain_id_o echo tags each beat with its original AXI id.

28.8 Verification Notes (cocotb test plan)

Scenario	What it proves
Insert AR, issue, DFI return, drain: BL burst round-trips	Full miss-path smoke
Returns in issue order buffered into correct SRAM slots	Issue-order return fill
Out-of-issue-order scheduling but AR-order drain release	Reorder-buffer ordering guarantee
Drain gated on r_ready: oldest entry with data not yet complete stalls	Data-ready gate
SRAM slot exhaustion deasserts dfi_ret_ready_o	Return-side pre-allocation backpressure
DFI aligner t_rddata_en sweep (0, 1, N); en-window at fire+t_rddata_en	Read-latency window alignment
Back-to-back RD fires paced BL_WORDS apart:	r_epend seamless

Scenario	What it proves
contiguous en windows	continuation
`rd_fire` mid-window counted, not dropped	`r_epend` regression
`rd_last_o` on DFI word `BL_WORDS-1`	Word counter / burst boundary

28.9 Open Questions / Future Work

- **DFI rddata error propagation.** `rd_resp_o` is fixed OKAY in v1; wire the PHY rddata-error to `rd_resp_o` (and thence `r_resp` → AXI rresp) during bring-up.
- **Multi-outstanding read windows.** The aligner is single-issue in v1; the `r_epend` counter already tolerates paced back-to-back fires, but overlapping windows would need a deeper capture-side counter.
- **Read-path parallelism for reorder / col-major.** Deferred perf item — the drain mover releases one oldest burst at a time.

29 DFI Command Formatter (dfi_cmd_formatter + dfi_signal_pack)

Module: dfi_cmd_formatter.sv (command decode) / dfi_signal_pack.sv (final DFI pipe stage) **Location:** rtl/fub/ **Category:** FUB **Parent macro:** pumice_dfi_layer **Status:** Implemented (DDR2 truth table + bit-exact LPDDR2 CA-bus encoding; both memtypes functional)

> **Replaces the retired command encoder.** The original cmd_encoder_fub (with its ddr2_cmd_encoder / lpddr2_cmd_encoder generate-branch sub-modules and a separate odt_ctrl_fub) is gone. The live design is dfi_cmd_formatter.sv — a single module with a runtime memtype_i branch (not an elaboration-time generate) — followed by dfi_signal_pack.sv, the final registered DFI-bus pipeline stage.
>
> **LPDDR2 is now fully implemented.** The retired chapter listed LPDDR2 as "deferred to v2." The live formatter builds the bit-exact JESD209-2F Table 60 CA-bus word for every command; the transcription lives in docs/uarch/LPDDR2_CA_ENCODING.md, and the DV BFM (lpddr_ca.py) encodes/decodes against the identical layout.
>
> **ODT is absorbed here.** There is no standalone ODT block (see the retired ODT chapter). dfi_cmd_formatter drives dfi_odt_o; the DDR2 ODT rule follows from the same truth table. In v1 dfi_odt_o is driven to 0 (the decode leaves w_p0_odt at its NOP default for every op); the JEDEC ODT-timing hooks live in mode_register's odt_o decode.

29.1 Purpose

dfi_cmd_formatter translates the scheduler's abstract command — (cmd_op, rank, bank, row, col, len) plus an implicit auto-precharge encoded in the op (RDA/WRA vs RD/WR) — into wire-level DFI v2.1 control-bus signals, packed across DFI_RATE phases. For DDR2 it drives the classic {ras_n, cas_n, we_n} strobes plus dfi_address / dfi_bank; for LPDDR2 it packs the multiplexed 10-bit CA bus (two edges = a flat 20-bit word) onto dfi_address.

dfi_signal_pack is the final pipeline register on the DFI v2.1 bus. It latches the formatter's command bus together with the write/read data-enable signals and drives reset-safe NOP values during reset. It owns dfi_dram_clk_disable (currently held 0).

Both modules run in the DFI clock domain inside pumice_dfi_layer (see the DFI layer / gearing chapter); the formatter is instantiated via pumice_dfi_cmd_path.

29.2 Synthesis Parameters (`dfi_cmd_formatter`)

Parameter	Default	Effect
`NUM_RANKS`	1	Width of the per-rank `dfi_cs_n` / `dfi_odt` output
`NUM_BANKS`	8	Bank-field width (`BKW`)
`ROW_WIDTH`	14	Row operand width (also carries DDR2 MR data and LPDDR2 MRW field)
`COL_WIDTH`	10	Column operand width
`BURST_LEN_WIDTH`	8	`cmd_len_i` width (currently unused — tied into `unused_v1`)
`DFI_RATE`	2	Phases per DFI-layer word; sets the multi-phase bus widths
`DFI_ADDR_WIDTH`	14	Per-phase address width; the LPDDR2 CA word needs `DFI_ADDR_BUS_W >= 20`
`DFI_BANK_WIDTH`	3	Per-phase bank width
`DFI_CTRL_WIDTH`	1	Per-phase width of each ras/cas/we strobe
`DFI_CS_WIDTH`	`NUM_RANKS`	Per-phase CS/ODT width

The multi-phase bus widths (`DFI_*_BUS_W = DFI_*_WIDTH * DFI_RATE`) are derived. `memtype_i` is a **runtime input** (`MEMTYPE_DDR2` / `MEMTYPE_LPDDR2`), not a parameter — both encoding paths are synthesized and the branch is selected live, so one bitstream can serve either family.

29.3 Command Interface and Runtime Phase Placement

The formatter takes a valid/ready command handshake from the DFI-layer command FIFO:

Signal	Width	Description
`cmd_valid_i`	1	Command present
`cmd_ready_o`	1	Always 1 (registered) — formatter never stalls

Signal	Width	Description
cmd_op_i	dram_op_e	The chosen operation
cmd_rank_i	RKW	Target rank (selects the active CS_n)
cmd_bank_i	BKW	Target bank (MR index for MRS)
cmd_row_i	ROW_WIDTH	Row (ACT); DDR2 MR data; LPDDR2 packed MRW field
cmd_col_i	COL_WIDTH	Column (RD/WR)
cmd_len_i	BURST_LEN_WIDTH	Burst length (reserved; unused in v1)
rd_phase_i	PHW	DFI sub-phase carrying the READ command
wr_phase_i	PHW	DFI sub-phase carrying the WRITE command

rd_phase_i / wr_phase_i are CSR-driven (DFI_PHASE CSR). The decoded command is placed on wr_phase for WR/WRA, rd_phase for RD/RDA, and phase 0 for everything else. This matches a PHY that consumes a per-command rdphase/wrphase off the DFI bus. Defaults are 0, which reproduces the legacy "everything on phase 0" behavior — notably the LiteDRAM a7ddrphy takes the command on phase 0 and applies its rdphase internally, so on the board target these stay 0. (See the gearing chapter §"DFI_PHASE CSR".)

29.4 DDR2 Encoding Branch

For memtype_i == MEMTYPE_DDR2, a combinational block builds the phase-0 command fields (w_p0_*). The default is an all-deselected NOP; when a command is valid, w_p0_cs_n is set to the active-rank mask (w_active_rank_mask, bit r low for the target rank) and the strobes/address are driven per the JEDEC JESD79-2 truth table (transcribed verbatim from the RTL):

Op	cs_n	ras_n	cas_n	we_n	bank	address
OP_NOP	active mask	1	1	1	0	0
OP_ACT	active mask	0	1	1	bank	row
OP_RD	active mask	1	0	1	bank	col (A10 = 0)
OP_RDA	active mask	1	0	1	bank	col with bit 10 set (auto-PRE)
OP_WR	active	1	0	0	bank	col (A10 = 0)

Op	cs_n	ras_n	cas_n	we_n	bank	address
	mask					
OP_WRA	active mask	1	0	0	bank	col with bit 10 set (auto-PRE)
OP_PRE	active mask	0	1	0	bank	0 (A10 = 0, single-bank)
OP_PREA	active mask	0	1	0	0	bit 10 set (all-bank)
OP_REF	active mask	0	0	1	0	0
OP_MRS	active mask	0	0	0	MR idx	MR data from `cmd_row_i`

`cs_n` is per-rank: bit `r` is 0 for the selected rank, 1 elsewhere; all-1 is an all-deselected NOP. The A10 auto-precharge bit is set by OR-ing `1 << 10` into the address for RDA/WRA, and doubles as the all-bank flag for PREA.

MRS carries data on the ROW field, not the column. The RTL comment is explicit: MR0 = 0x533 needs bit 10 (tWR[1]), which a `COL_WIDTH` (10-bit) field would truncate — so the formatter reads MR data from `cmd_row_i` (ROW_WIDTH) and the MR index from `cmd_bank_i`. This matches `init_sequencer`, which drives MR data on `init_cmd_row_o`.

Unhandled DDR2-irrelevant ops (`OP_REFPB`, `OP_ZQCS`/`ZQCL`, self-refresh entry/ exit) fall through the `default` arm and emit NOP — they are driven via CKE / a separate sequencer, not the strobe truth table.

29.5 LPDDR2 Encoding Branch (bit-exact JESD209-2F Table 60)

LPDDR2 has no `ras_n`/`cas_n`/`we_n`. The command AND a scrambled address are multiplexed onto a 10-bit CA bus over two clock edges (rising `r`, falling `f`), carried on the DFI bus as a flat 20-bit word. The formatter builds two 10-bit half-words `w_ca_r` / `w_ca_f` and concatenates them:

```
assign w_lpddr2_ca = {w_ca_f, w_ca_r};   // [19:10] = falling, [9:0] = rising
```

This is the exact layout locked in `docs/uarch/LPDDR2_CA_ENCODING.md` and enforced by a round-trip conformance test against the DV decoder (`lpddr_ca.py`). Per JEDEC, any other bit ordering is prohibited, so bit-exactness is a spec requirement.

Command decode is by the rising-edge opcode bits {CA0r, CA1r, CA2r, CA3r}, transcribed from the RTL `unique case`:

Op	Rising opcode bits set	Payload placement
OP_ACT	CA1r = 1	bank → CA7r..CA9r; row hi R8..R12 → CA2r..CA6r; row lo R0..R7 → CA0f..CA7f; R13 → CA8f; R14 → CA9f
OP_RD / OP_RDA	CA0r = 1, CA2r = 1 (read)	bank → CA7r..CA9r; C1,C2 → CA5r,CA6r; AP → CA0f; C3..C11 → CA1f..CA9f
OP_WR / OP_WRA	CA0r = 1, CA2r = 0 (write)	same column/bank/AP placement as read
OP_PRE	CA0r=1, CA1r=1, CA3r=1 (AB=0)	bank → CA7r..CA9r
OP_PREA	CA0r=1, CA1r=1, CA3r=1, CA4r=1 (AB=1)	bank don’t-care
OP_REF	CA2r=1, CA3r=1 (all-bank)	—
OP_REFPB	CA2r=1 (per-bank; CA3r=0)	bank implied by device counter
OP_MRS (MRW)	all opcode bits 0	MA0..MA5 → CA4r..CA9r; MA6,MA7 → CA0f,CA1f; OP0..OP7 → CA2f..CA9f
default (NOP)	CA0r=CA1r=CA2r=CA3r=1	—

Auto-precharge is `w_ca_ap = (op == OP_RDA) || (op == OP_WRA)`, placed on CA0f. C0 is implied 0 and never transmitted (only C1..C11 ride the bus). Row/column/ bank are zero-extended (`w_row15`, `w_col12`) so wider geometries (R14, C10, C11) light up their reserved pins cleanly.

MRW field packing. LPDDR2 mode-register addresses reach MR63 (MA[5:0]), which a 3-bit bank port cannot express. The scheduler therefore carries the MRW fields in the ROW request as `{MA[5:0], OP[7:0]}` — `w_mr_ma = {2'b0, cmd_row_i[13:8]}`, `w_mr_op = cmd_row_i[7:0]` — matching `init_sequencer`'s `mrw_row()` packing. MA[7:6] are 0 (MR <= 63).

29.6 Multi-Phase Packing and CS_n

After the decode, a combinational stage packs the command into the multi-phase buses (`w_dfi_*`). Every phase starts at NOP (cs_n = 1, ras/cas/we = 1); then:

- **DDR2:** the decoded `w_p0_*` fields are placed on the target phase `w_cmd_phase` (rd_phase for reads, wr_phase for writes, phase 0 otherwise); the other phases stay NOP.
- **LPDDR2:** the whole 20-bit CA word is written to `w_dfi_address[19:0]` (low bits), ras/cas/we stay idle, and `dfi_cs_n` is asserted for the target rank on phase 0 when the op is not NOP. The two CA edges are already inside the word, so there is no per-DFI-phase command placement for LPDDR2.

The active-rank CS_n mask is built once (`w_active_rank_mask`): bit `r` = 0 when `r == cmd_rank_i`, else 1. For `NUM_RANKS == 1` only bit 0 exists.

29.7 `dfi_signal_pack` — Final Registered DFI Stage

`dfi_signal_pack` is a pure one-cycle registered pipeline. It latches the formatter's command bus plus the data-enable/mask signals from the write/read serializers and drives them to the PHY the next DFI cycle. Its value is the reset-safe defaults it guarantees during reset / before first issue:

Output	Reset value	Meaning
`dfi_cs_n_o`	all-1	all-deselected
`dfi_ras_n_o/cas_n_o/we_n_o`	all-1	NOP
`dfi_cke_o`	0	DRAM held CKE-low until init
`dfi_odt_o`	0	ODT off
`dfi_wrdata_en_o` / `dfi_rddata_en_o`	0	no data movement
`dfi_wrdata_mask_o`	all-1	mask all bytes (write nothing)
`dfi_dram_clk_disable_o`	0	clock enabled (power-state TODO)

It owns `dfi_dram_clk_disable` for a future power-state machine (held 0 today). Phase staggering and CKE power-down driving are v2 TODOs noted in the RTL header; today it is a transparent width-preserving flop.

29.8 Interface (`dfi_cmd_formatter` outputs)

Signal	Width	Description
`dfi_address_o`	DFI_ADDR_BUS_W	DDR2 row/col operand per phase; LPDDR2 flat CA word
`dfi_bank_o`	DFI_BANK_BUS_W	Per-phase bank (DDR2)
`dfi_cas_n_o`	DFI_CTRL_BUS_W	Per-phase CAS strobe
`dfi_ras_n_o`	DFI_CTRL_BUS_W	Per-phase RAS strobe
`dfi_we_n_o`	DFI_CTRL_BUS_W	Per-phase WE strobe
`dfi_cs_n_o`	DFI_CS_BUS_W	Per-phase per-rank chip-select
`dfi_odt_o`	DFI_CS_BUS_W	Per-phase per-rank ODT (0 in v1)

All outputs are strict-flopped; `dfi_signal_pack` widens/relatches them onto the PHY pins unchanged.

29.9 Timing Budget

The formatter is combinational decode into a single output flop; `dfi_signal_pack` adds one more flop. The worst path is the LPDDR2 ACT with full row-bit spreading across CA0r/CA0f (bank + 5 rising row bits + 8 falling row bits), which is a few LUT levels feeding the output register — well within the DFI clock budget. The formatter is not the controller's critical path; that is the arbiter upstream.

29.10 Verification Notes (cocotb test plan)

Scenario	What it proves
DDR2 ACT bank 3 row 0x1234 →	DDR2 ACT truth-table

Scenario	What it proves
`{ras,cas,we}={0,1,1}`, address=0x1234, bank=3	row
DDR2 RDA bank 3 col 0x40 → `{ras,cas,we}={1,0,1}`, address bit 10 set	DDR2 auto-precharge via A10
DDR2 PREA → `{ras,cas,we}={0,1,0}`, address bit 10 set	DDR2 all-bank precharge via A10
DDR2 MRS MR0=0x533 → MR data on address (bit 10 present, not truncated)	MRS data on ROW field
LPDDR2 ACT bank 3 row 0x1234 → `w_lpddr2_ca` bit-exact vs `lpddr_ca.py` decode	LPDDR2 ACT CA packing
LPDDR2 WRA bank 3 col 0x40 → AP bit (CA0f) set	LPDDR2 auto-precharge via CA
LPDDR2 REFpb (CA2r=1, CA3r=0) vs REFab (CA2r=1, CA3r=1)	LPDDR2 refresh CA decode
LPDDR2 MRW MR63 → MA/OP packed on CA4r..CA9r / CA0f.. per Table 60	LPDDR2 MRW field packing
Multi-rank ACT rank 1 → `dfi_cs_n` bit 1 low, bit 0 high	Per-rank CS_n mask
rd_phase=1 → RD command lands on DFI phase 1, other phases NOP	Runtime phase placement
NOP / `!cmd_valid` → all-deselected (cs_n all-1), strobes all-1	Idle pattern
Reset → `dfi_signal_pack` drives cs_n=1, cke=0, wrdata_mask=all-1	Reset-safe NOP defaults

29.11 Open Questions / Future Work

- **ODT driving.** `dfi_odt_o` is held 0 in v1 (the decode leaves the ODT default at NOP). Multi-rank DDR2 needs the JEDEC cross-termination window (ODT-high on the non-accessed rank during a read, on the accessed rank during a write). The hooks exist (`mode_register.odt_o` decodes the DDR2 MR1 ODT bits); wiring the timed ODT window through the formatter is future work — see the absorbed ODT chapter.

- **`cmd_len_i` unused.** The burst-length input is reserved (tied into `unused_v1`); burst geometry is handled by the data-path serializers, so the formatter does not need it today.
- **Phase staggering / clk-disable.** `dfi_signal_pack` is a plain relatch; per-phase output staggering and `dfi_dram_clk_disable` driving (for power-down) are v2 items noted in the RTL header.
- **LPDDR2 BL16.** `mode_register.bl_o` is 4-bit and clips BL16 to BL8; only BL4/BL8 are wired end-to-end. Widening the burst path is a separate item.

> **Status (2026-10-10): this block lives in common-ip now.** The RTL is `mc_dfi_2p1_layer`; its microarchitecture is owned by common_mas mc_dfi_2p1_layer — DFI 2.1 Command / Write / Read Pipeline. This chapter keeps the pumice provenance and measured history; for current interfaces and parameters, read the common chapter.

30 DFI Layer and Host-Width Gearing (pumice_dfi_layer + pumice_top_geared)

Module: mc_dfi_2p1_layer.sv (common-ip macro) / pumice_top_geared.sv (top wrapper)
Location: projects/components/mem-ctrl-ip/common-ip/rtl/macro/ and rtl/top/
Category: Macro / Top wrapper **Status:** Implemented (single-CDC DFI datapath; formal-IP host-width gearing wrapper)

> **Replaces the retired gear_dfi_fub.** The original chapter described a phase-packing FUB (gear_dfi_fub / dfi_signal_pack) that spread a single MC command and burst-of-N data beats into per-phase DFI slots. That phase-packing role is real but small — it lives inside dfi_signal_pack and the DFI-layer sub-FUBs. This chapter now covers the two live, distinct concepts that carry the word "gearing" in pumice, and it keeps them separate:
>
> 1. **The DFI layer** (pumice_dfi_layer) — the single controller/PHY clock-domain crossing plus the DFI-clock command/write/read datapath. This is where the internal DW-wide word is split into DFI_RATE DRAM beats.
> 2. **Host-width gearing** (pumice_top_geared) — an OPTIONAL top wrapper that lets a host SoC use any AXI data width, inserting the repo's formally verified AXI data-width converters between the host and the fixed-width core.
>
> These are orthogonal: (1) is about DRAM-beat phasing on the PHY side; (2) is about AXI beat width on the host side.

30.1 Concept 1 — The DFI Layer (pumice_dfi_layer)

30.1.1 Purpose

pumice_dfi_layer presents the controller-clock command/wrdata/rddata streams on one side and the DFI 2.1 pin bus on the other. It holds the **single** controller/PHY clock-domain crossing and the DFI-clock-side datapath. Its internal datapath unit is the DFI word (dfi_wrdata width = all DFI_RATE phases): one FIFO word equals one DFI cycle, which keeps the datapath bubble-free.

Two clock domains:

- **ctl_clk** — command from the scheduler, write data from the WR CAM, read data to the RD CAM, and the init_start / init_complete handshake.

- **`dfi_clk`** — the DFI command bus, `dfi_wrdata`/en/mask, `dfi_rddata`/en/valid, and `dfi_init_start`/complete.

30.1.2 Internal structure (verified against the RTL)

`pumice_dfi_layer` instantiates exactly four sub-FUBs:

Sub-FUB	Domain	Role
`pumice_dfi_cdc`	ctl↔dfi	The **single** clock crossing — async gaxi FIFOs only (cmd, wrdata, rddata, plus init-start/complete bit crossings)
`pumice_dfi_cmd_path`	dfi_clk	cmd FIFO → DFI command bus + `wr_fire`/`rd_fire` strobes; instantiates `dfi_cmd_formatter` (§2.14) and `dfi_signal_pack`
`pumice_dfi_wr_serializer`	dfi_clk	wrdata FIFO → `dfi_wrdata` at `t_phy_wrlat`
`pumice_dfi_rd_aligner`	dfi_clk	`dfi_rddata` → rddata FIFO at `t_rddata_en`

The CDC (`pumice_dfi_cdc`) is the only place the domains meet; everything downstream of it runs entirely in `dfi_clk`. The command path emits `wr_fire_o` / `rd_fire_o` strobes that time the serializer and aligner relative to the issued command.

30.1.3 Key parameters

Parameter	Default	Effect
`NUM_RANKS`	1	CS/rank fan-out
`NUM_BANKS`	8	Bank-field width
`ROW_WIDTH`	14	Row / DFI address width
`COL_WIDTH`	10	Column width
`DFI_RATE`	2	DRAM beats per DFI word (phase count)
`DRAM_BEAT_WIDTH`	64	One DRAM beat's data width
`BL`	8	DRAM beats per burst; `BL_WORDS = BL / DFI_RATE` DFI words/burst
`CMD/WD/RD_FIFO_DEPTH`	8/16/16	CDC FIFO depths
`N_FLOP_CROSS`	2	Synchronizer flop stages in the async FIFOs

Derived DFI geometry: `DFI_DATA_WIDTH = DRAM_BEAT_WIDTH * DFI_RATE`, `DFI_STRB_WIDTH = DFI_DATA_WIDTH/8`, `DFI_EN_WIDTH = DFI_VALID_WIDTH = DFI_RATE`. The FIFO payloads pack the command (`CMD_DW`), `{last,strb,data}` for writes (`WD_DW`), and `{last,resp,data}` for reads (`RD_DW`).

30.1.4 DFI-clock runtime knobs

Signal	Width	Description
`memtype_i`	enum	DDR2 / LPDDR2 selector for the command path
`rd_phase_i`	PHW	DFI sub-phase carrying the READ command (see §2.14)
`wr_phase_i`	PHW	DFI sub-phase carrying the WRITE command
`t_phy_wrlat_i`	[7:0]	PHY write latency — when the serializer launches wrdata (CSR reset 0; Nexys A7 tuple programs 1)
`t_rddata_en_i`	[7:0]	When the aligner strobes `dfi_rddata_en` (valid = en + PHY read_latency); board tuple 6

These are the PHY-integration knobs surfaced through the CSR (`DFI_PHASE`, `DFI_TIMING`); they carry the PHY-specific latencies rather than baking them into the datapath, which is what let on-board bring-up dial in `t_rddata_en` / phase placement without an RTL change.

30.2 Concept 2 — Host-Width Gearing (`pumice_top_geared`)

30.2.1 Purpose

The controller core is fixed at its natural width `DW = DRAM_BEAT_WIDTH * DFI_RATE` (default 128): one AXI beat == one DFI word == `DFI_RATE` DRAM beats, and one AXI burst (`BL/DFI_RATE` beats) == one DRAM burst (`BL` beats). `pumice_top_geared` wraps `pumice_top` with a **free** `HOST_AXI_DATA_WIDTH`, inserting the repo's formally verified data-width converters (`axi4_dwidth_converter_wr` / `_rd`) between a host-width AXI slave and the fixed-`DW` core.

This is the family-wide gearing path (DDR2/DDR3/DDR4/LPDDR2), reusing proven + formal IP rather than re-verifying a bespoke gearbox inside the freshly stabilized controller datapath.

> **The old coupling is gone.** Do not describe the retired "AXI_DATA_WIDTH == DRAM_BEAT_WIDTH" constraint. Internal geometry is `DW = DRAM_BEAT_WIDTH * DFI_RATE`; the DW→phase split happens inside the DFI layer / `dfi_signal_pack`, and the host AXI width is decoupled from it by this wrapper.

30.2.2 GEAR-1 bypass (bit-identical)

When HOST_AXI_DATA_WIDTH == DW, a generate selects the g_direct branch: the host AXI signals are wired straight through to the core with no converter and no added latency. Existing DW-width builds are therefore bit-identical whether they instantiate pumice_top directly or through pumice_top_geared.

30.2.3 Geared branch

When HOST_AXI_DATA_WIDTH != DW, the g_conv branch instantiates the two formal converters:

```
axi4_dwidth_converter_wr
#(.S_AXI_DATA_WIDTH(HOST_AXI_DATA_WIDTH), .M_AXI_DATA_WIDTH(DW), ...)
axi4_dwidth_converter_rd
#(.S_AXI_DATA_WIDTH(HOST_AXI_DATA_WIDTH), .M_AXI_DATA_WIDTH(DW), ...)
```

The host slave ports (s_axi_* at HOST_AXI_DATA_WIDTH) feed the converters; the converters' master ports (the internal c_* nets at DW) feed pumice_top. The CSR cpuif and the DFI pin bus pass straight through the wrapper unmodified.

Burst geometry. The core still requires one AXI burst == one DRAM burst at its DW side ((awlen+1) * DFI_RATE == BL). The host issues bursts at host width; the converter translates them to DW-width bursts of that geometry. Host burst sizing is the host's contract (the same spirit as the core's ragged-burst check). Verified for host widths in {64, 128, 256}. Full scope in docs/AXI_DRAM_GEARING_SCOPE.md.

30.2.4 Key parameters

Parameter	Default	Effect
HOST_AXI_DATA_WIDTH	128	Host AXI data width (free); == DW triggers the bypass
DW (derived)	DRAM_BEAT_WIDTH * DFI_RATE	Fixed core width
core geometry params	—	AXI_ID_WIDTH, AXI_ADDR_WIDTH, NUM_RANKS, NUM_BANKS, ROW_WIDTH, COL_WIDTH, DFI_RATE, DRAM_BEAT_WIDTH, BL, ... mirror pumice_top

30.3 Narrow-Device (x16) Support — the Two Width Granularities

A distinction that is separate from both concepts above but which the DFI layer must respect: the **pumice DRAM beat** (`DRAM_BEAT_WIDTH`, one DFI phase's data) is not necessarily the **physical DRAM device word** (`DRAM_DEVICE_WIDTH`, the DQ width — e.g. 16 for an x16 device). When a beat is wider than the device (a 32-bit beat over an x16 device packs `K = DRAM_BEAT_WIDTH / DRAM_DEVICE_WIDTH = 2` physical DDR words), three parts of the pipeline must reason in **device-word** units. Getting this wrong is invisible in a DFI-level behavioral sim but corrupts real silicon — the Nexys A7 x16 bring-up hit all three:

1. **Burst length (beats per command).** A JEDEC BL from MR0 (`bl_o`, BL4/BL8) is in physical device beats; the core scales it to pumice-beat units before feeding the burst-split and serializer, else an x16 BL4 is over-counted and the controller drives/captures an extra DFI cycle.
2. **Column address stride.** DRAM columns are device-word granular, so `addr_mapper`'s `BYTE_OFFSET_WIDTH` must be `log2(DRAM_DEVICE_WIDTH/8)`, not `log2(DRAM_BEAT_WIDTH/8)`; using the beat width makes a split burst's chunk overwrite the previous chunk's tail (a ~50% read scramble on silicon).
3. **Read-data / valid alignment.** A PHY may present read data ahead of, or on a different phase than, `rddata_valid`. This is handled at runtime by the `t_rddata_en` knob and the `DFI_PHASE` CSR (below) rather than baked-in latencies.

Default `DRAM_DEVICE_WIDTH = DRAM_BEAT_WIDTH` (K=1) makes all of the above bit-identical to the wide-device behavior.

30.3.1 `DFI_PHASE` CSR (rd_phase / wr_phase)

`dfi_cmd_formatter` places the READ command on `rd_phase` and the WRITE command on `wr_phase` (defaults 0; all other commands on phase 0), runtime-settable via the `DFI_PHASE` CSR through `pumice_dfi_layer`'s `rd_phase_i` / `wr_phase_i`. This matches a PHY that consumes a per-command rdphase/wrphase off the DFI bus. The LiteDRAM a7ddrphy instead takes the command on phase 0 and applies its rdphase internally, so on that PHY `rd_phase` stays 0 — the CSR exists for PHYs that do not.

30.4 Interface Summary

30.4.1 `pumice_dfi_layer` — controller side (`ctl_clk`)

Signal	Direction	Description
`cmd_valid_i` / `cmd_ready_o` / `cmd_data_i[CMD_DW]`	in/out/in	Command stream from the scheduler
`wd_valid_i` / `wd_ready_o` / `wd_data_i[WD_DW]`	in/out/in	Write data from the WR CAM drain ({last,strb,data})
`rd_valid_o` / `rd_ready_i` / `rd_data_o[RD_DW]`	out/in/out	Read data to the RD CAM ({last,resp,data})
`init_start_i` / `init_complete_o`	in/out	Init handshake, controller side

30.4.2 `pumice_dfi_layer` — PHY side (`dfi_clk`)

DFI command bus (`dfi_address_o`, `dfi_bank_o`, `dfi_cas_n_o`, `dfi_ras_n_o`, `dfi_we_n_o`, `dfi_cs_n_o`, `dfi_odt_o`), write data (`dfi_wrdata_o`, `dfi_wrdata_en_o`, `dfi_wrdata_mask_o`), read data (`dfi_rddata_en_o`, `dfi_rddata_i`, `dfi_rddata_valid_i`), and the DFI init handshake (`dfi_init_start_o`, `dfi_init_complete_i`), plus the runtime knobs above.

30.4.3 `pumice_top_geared`

Host AXI4 slave at `HOST_AXI_DATA_WIDTH`, PeakRDL cpuif passthrough (`s_cpuif_*`), `init_done_o`, and the DFI 2.1 pin bus straight through. Internally wires either directly (`g_direct`) or via the two converters (`g_conv`) to `pumice_top` at `DW`.

30.5 Verification Notes (cocotb test plan)

Scenario	What it proves
Command/wrdata/rddata cross the CDC bubble-free at `DFI_RATE = 2`	Single-CDC datapath, one FIFO word = one DFI cycle
Write burst: `wr_fire` → serializer launches `dfi_wrdata` at `t_phy_wrlat`	Write-latency alignment

Scenario	What it proves
Read burst: `rd_fire` → aligner strobes `dfi_rddata_en` at `t_rddata_en`	Read-capture alignment
`rd_phase = 1`: RD command lands on DFI phase 1	`DFI_PHASE` CSR routing
x16 device (K=2): BL4 uses one DFI cycle, columns stride by device words	Two-granularity handling
`pumice_top_geared` with `HOST_AXI_DATA_WIDTH == DW`: `g_direct` bypass	GEAR-1 bit-identical to bare `pumice_top`
Host 64 → `DW` 128 via `axi4_dwidth_converter_wr/_rd`: burst geometry preserved	Geared write/read path
Host 256 → `DW` 128: read data reassembled correctly at host width	Wide-host gearing
CSR cpuif and DFI pins identical whether geared or direct	Passthrough correctness

30.6 Open Questions / Future Work

- **Multi-command-per-cycle.** The command path places one issued command per DFI word today (multi-phase content passes through, but a single command occupies a word). Emitting multiple commands per DFI cycle is a scheduler-side feature; the bus widths are already in place.
- **`dfi_dram_clk_disable` / power-down.** `dfi_signal_pack` holds clk-disable at 0; wiring the power-state machine through the DFI layer is future work.
- **Higher gear ratios.** `DFI_RATE = 4` (and DDR3+ higher ratios) scale the DFI geometry linearly; validate the CDC FIFO depths and serializer/aligner timing when the family controller targets them.
- **Host width coverage.** Gearing is verified for host ∈ {64, 128, 256}; add 512 when a host SoC requires it. See `docs/AXI_DRAM_GEARING_SCOPE.md`.

31 pumice_training_layer (LPDDR2 calibration / training macro)

Module: pumice_training_layer.sv **Location:** rtl/macro/ **Category:** Layer-4 macro (maintenance traffic for LPDDR2) **FUBs bundled:** pumice_zq_ctrl + pumice_lp_cal, wired into the common mc_training_layer shell (the shell provides the one-active trn_cmd mux and the mc_clk/dfi_clk CDC) **Status:** implemented and FUB-tested (ZQ ctrl, lp_cal); sim-verified against pumice_scheduler_layer training priority/gate

> The common training shell is documented in common_mas chapter 5: mc_training_layer.

31.1 Purpose

LPDDR2 needs two kinds of calibration traffic that DDR2 simply doesn't have: periodic ZQ calibration (ZQCS/ZQCL — LPDDR2 has no dedicated command, so both are MRW writes to MR10) and DQ calibration (MRR reads of MR32/MR40 against which firmware sweeps the PHY read-delay taps). Both are maintenance-class: they need the bus to themselves, they can wait, and they must never invent their own command encodings. This layer is where they live.

pumice_training_layer holds the two FUBs, muxes both onto the single trn_cmd maintenance channel into the scheduler layer's arbiter, and owns all of the mc_clk/dfi_clk clock crossing for the read-aligner calibration sideband. Run gating (init_done && memtype == MEMTYPE_LPDDR2) lives inside the two FUBs, not here — the layer is a passive wiring and arbitration shell. For DDR2 builds both FUBs are inert: no ZQ pin, no MRR.

The DQ-vs-tap sweep itself deliberately stays outside the core (the scoria/ andesite doctrine: cores are DFI-clean, no FPGA-primitive logic). The layer exposes cal_start/cal_abort CSRs and the MRR engine; board firmware walks the taps and compares each lane against patterns A/B.

31.2 FUBs

FUB	Role
pumice_zq_ctrl	Periodic ZQ calibration. Counts down zq_interval, requests the arbiter on expiry, holds the post-grant bus-quiet window (t_zqcs/t_zqcl)

FUB	Role
	that comes back as `cal_busy`. Mode-C deferral holds the request under sustained demand and escalates an overdue deferral to ZQCL.
`pumice_lp_cal`	One-shot DQ-calibration sequencer. On `cal_start`: MRR to MR32, capture the first return beat, wait `t_mrr`, MRR to MR40, capture, done. Sticky `cal_done`/`cal_err`; abort via `cal_abort`.

31.3 The One-Active Mux

Both FUBs want the same `trn_cmd` channel, so the layer arbitrates: ZQ wins, because it is periodic maintenance with a real expiry behind it, while `lp_cal` is a firmware-initiated one-shot that can wait a few hundred cycles without anything drifting. Selection is a single `w_zq_select = w_zq_req` term; the channel payload is just the selected FUB's payload, with one wrinkle — the command kind:

`trn_cmd_*` output	ZQ selected	lp_cal selected
`trn_cmd_op_o`	`w_zq_op` (always `OP_MRS`)	`OP_MRS`
`trn_cmd_row_o`	`{4'd0, MR10, OP}` (`0x56` ZQCS / `0xAB` ZQCL)	`{4'd0, MA, 8'd0}` (MR32=32 / MR40=40, OP=0)
`trn_cmd_mrr_o`	`0` (MRW)	`1` (MRR)
`trn_cmd_bank_o`	`0`	`0`

Grant is broadcast back to the selected FUB only: `w_zq_grant = trn_cmd_grant_i && w_zq_select`, `w_lp_grant = trn_cmd_grant_i && !w_zq_select`. The arbiter passes the granted op verbatim onto the command output — neither the mux nor the arbiter re-encodes maintenance commands.

31.4 External Boundaries

- **Maintenance channel out (to `pumice_scheduler_layer`):** `trn_cmd_req_o`, `trn_cmd_op_o`, `trn_cmd_bank_o[2:0]`, `trn_cmd_row_o[17:0]`, `trn_cmd_mrr_o`; `trn_cmd_grant_i` back. Wired in `pumice_core.sv` exactly as andesite's core wires its training layer. The arbiter grants only when all banks are idle, nothing row-affecting is in flight or inside its guard window, and no REF recovery is running — see ch02/07 and ch04/05.
- **CSR controls in (from `pumice_top` hwif):** `zq_en_i`, `zq_defer_en_i`, `zq_interval_i[31:0]`, `t_zqcs_i[15:0]`, `t_zqcl_i[15:0]`, `zq_overdue_max_i[12:0]`, `cal_start_i`, `cal_abort_i`, `t_mrr_i[15:0]`, `t_readout_i[15:0]` — the `CAL_*` register block (§4.2). Config-drive, like every other CSR field: no staging, no quiet point.
- **DFI-layer calibration sideband (dfi_clk domain):** `cal_expect_o` down to the read aligner, `cal_data_i`/`cal_valid_i` back with the captured beat. See ch02/18.
- **Telemetry out (CAL_STATUS and MRR data registers):** `cal_busy_o` (ZQ hold or lp_cal running), `cal_done_o`/`cal_err_o` (sticky), `mrr32_data_o`/`mrr40_data_o` (full DFI read width each), `zq_busy_o`, `zq_overdue_o`, `zqcs_total_o[15:0]`.

31.5 CDC Discipline

All mc_clk/dfi_clk crossing for the calibration sideband lives here, using the repo's standard cells:

- `cal_expect_o` crosses down through `sync_pulse` (3 stages) — a single mc_clk assertion from `pumice_lp_cal` becomes exactly one dfi_clk pulse, which arms the aligner sideband.
- Captured data crosses up through `cdc_synchronizer` (`WIDTH = DFI_DATA_WIDTH`, 3 flops). This is safe because the aligner holds the captured beat stable until the next arm — a multi-flop synchronizer is sufficient, no handshake required.
- `cal_valid_i` crosses up through `sync_pulse` so the mc_clk-side FSM sees a single-cycle "data landed" strobe.

31.6 Parameters

Parameter	Default	Meaning
DFI_DATA_WIDTH	128	Full DFI read-data width; sizes cal_data_i, mrr32_data_o, mrr40_data_o, and the up-synchronizer.

31.7 Tests

- dv/tests/macro/test_pumice_scheduler_layer.py::cocotb_test_training_priority_and_gate — training channel sits between refresh and demand, grants only when the bus is safe, demand blocked during the hold window, fire==valid invariant.
- FUB suites: dv/tests/fub/test_pumice_zq_ctrl.py, dv/tests/fub/test_pumice_lp_cal.py (see the per-FUB chapters that follow).

31.8 Open Questions / Future Work

- **Multi-rank ZQ overlap.** Pumice board targets are single-rank, so a shared-ZQ-resistor overlap policy is untested; the rank loop is left as a documented generate hook in pumice_zq_ctrl.
- **Temperature-driven refresh.** The MRR engine makes MR4 reads possible, but tREFI derating on temperature class is not built.
- **Self-refresh-exit ZQCL.** The power-down path is dormant in pumice, so no ZQCL-on-self-refresh-exit trigger exists.

32 ZQ Calibration Controller (pumice_zq_ctrl)

Module: pumice_zq_ctrl.sv **Location:** rtl/fub/ **Category:** FUB **Parent macro:** pumice_training_layer **Status:** implemented and FUB-tested (LPDDR2); inert for DDR2 builds — no ZQ pin

> This FUB never emits a DRAM command. It raises a request, and on grant the arbiter issues the command from the FUB's payload verbatim. LPDDR2 has no ZQCS/ZQCL opcode, so both calibrations are MRW writes to MR10 (OP = 8'h56 for ZQCS, 8'hAB for ZQCL) — the FUB packs the MR index and OP into its row output and stops there.

32.1 Purpose

Periodic ZQ calibration for LPDDR2, as maintenance traffic. JESD209-2F §5.13.3 expects ZQCS (short) at regular intervals and ZQCL (long) to re-establish the ±15% RON window after calibration has been deferred too long. The block is a port of scoria_zq_ctrl to LPDDR2 semantics: wait the programmed interval, ask for the bus, hold the post-grant quiet window, start the next interval. It is deliberately small.

The run condition is zq_en_i && init_done_i && (memtype_i == MEMTYPE_LPDDR2) && interval != 0, so the block is completely inert for DDR2 builds. LPDDR2-S4 wants all banks precharged for ZQ and LPDDR2-N allows Idle-or-Active; the conservative all-banks-idle grant contract the arbiter enforces satisfies both.

32.2 Parameters

Parameter	Default	Meaning
MR10_INDEX	10	MR index for ZQ calibration; lands in zq_row_o[13:8].

32.3 The Four FSM States

r_state is 2 bits. Unlike the FSM-free refresh_ctrl, this block has a genuinely sequential lifecycle — wait, ask, hold, and a deferred-wait state for the placement policy:

```
ZQ_IDLE  — counting down r_interval to the next calibration
ZQ_REQ   — requesting the arbiter, waiting for grant
ZQ_HOLD  — granted; holding the post-grant quiet window before reload
ZQ_DEFER — interval expired but demand is high, holding off (Mode C)
```

- **ZQ_IDLE** decrements r_interval. On zero: if zq_defer_en_i && demand_i, enter ZQ_DEFER; otherwise ZQ_REQ. The counter seeds from t_zqcs_interval_i at reset (not zero) so the first interval is full length even if enable comes up late — same latent-coupling removal scoria made.
- **ZQ_REQ** holds zq_req_o until zq_grant_i, then loads r_hold with t_zqcs_i or t_zqcl_i (selected by r_is_zqcl), bumps r_total, and enters ZQ_HOLD. While waiting with demand_i high, r_overdue sets — a request that withdrew itself under load would starve silently, so the starvation is made visible.
- **ZQ_HOLD** counts r_hold to zero, then returns to ZQ_IDLE and reloads r_interval. The hold itself does not block traffic; the block just declines to start the next interval early. The actual bus-quiet enforcement is the arbiter's job (below).
- **ZQ_DEFER** exits to ZQ_REQ when demand drops, when zq_defer_en_i is cleared, or when overdue_max_i != 0 && r_defer_cnt >= overdue_max_i. Any exit with r_defer_cnt != 0 requests **ZQCL** instead of ZQCS — that is the escalation path. r_overdue is set the whole time the block sits deferred.

32.4 Interface

32.4.1 Run gating and configuration (from CAL_* CSRs via the training layer)

Signal	Direction	Width	Description
mc_clk	in	1	controller clock
mc_rst_n	in	1	active-low reset
zq_en_i	in	1	periodic ZQ enable (CAL_CTRL.zq_en)
init_done_i	in	1	scheduler init complete
memtype_i	in	1	MEMTYPE_LPDDR2 required
t_zqcs_interval_i	in	32	MC cycles between calibrations; 0 disables
t_zqcs_i	in	16	post-grant hold, ZQCS
t_zqcl_i	in	16	post-grant hold, ZQCL
zq_defer_en_i	in	1	Mode-C deferral under sustained demand
overdue_max_i	in	13	max deferral cycles; 0 = no cap
demand_i	in	1	scheduler has read/write work (tied 0 in the

Signal	Direction	Width	Description
			layer today — see Notes)

32.4.2 Arbiter handshake and telemetry

Signal	Direction	Width	Description
zq_req_o	out	1	request; high in ZQ_REQ
zq_grant_i	in	1	arbiter grant (single cycle)
zq_op_o	out	—	always OP_MRS — LPDDR2 ZQ is pure MRW
zq_bank_o	out	3	0; the MR index lives in the row field
zq_row_o	out	18	{4'd0, MR10_INDEX[5:0], OP[7:0]} — MRW row packing
zq_is_zqcl_o	out	1	1 = the current request is a ZQCL
cal_busy_o	out	1	high in ZQ_HOLD; the bus-quiet window
obs_zqcs_total_o	out	16	calibrations issued since reset (CAL_STATUS.zqcs_total)
obs_overdue_o	out	1	interval expired without grant, or deferred (CAL_STATUS.zq_overdue)

32.5 Timing / Behavior

One grant per calibration. The interval reloads at the end of ZQ_HOLD, so grant-to-next-request spacing is t_zqcs_interval_i + hold cycles. t_zqcs_interval_i counts mc_clk cycles; convert from the JEDEC tZQCS interval at the MC frequency.

Traffic blocking is the arbiter's job, not this FUB's: cal_busy_o from the training layer folds into the arbiter's w_out_safe term, which blocks every demand-class fire during the hold window. That keeps the BUG-003 discipline intact — w_fire_out == cmd_valid_o && cmd_ready_i holds by construction, and the new gate never touches the pick cone.

MR10 writes are transient commands: they flow through the arbiter maintenance path and are never shadowed into the mode_register FUB, same rule init_sequencer's ZQ step follows today.

32.6 Verification Notes (cocotb test plan)

Verified by dv/tests/fub/test_pumice_zq_ctrl.py: interval expiry → request, grant → hold → reload, Mode-C deferral under sustained demand, overdue expiry selects ZQCL, zq_row_o MRW

packing (MR10 + 0x56/0xAB), obs_zqcs_total_o accounting, and DDR2-inert behavior (memtype_i != MEMTYPE_LPDDR2 never requests).

32.7 Notes

- **ZQ never preempts.** It raises zq_req_o and waits, matching the family doctrine that maintenance is request-and-wait traffic.
- **demand_i is tied off in the layer today.** The training layer grounds demand_i (1'b0), so the ZQ_DEFER entry condition (zq_defer_en_i && demand_i at interval expiry) can never fire: expiry goes straight to ZQ_REQ, the ZQCL escalation path is unreachable, and obs_overdue_o never sets. Mode-C and the overdue cap are dormant until something wires this port to the scheduler's demand indicator. The RTL keeps the FSM so the policy is one wire away.
- **Overdue is a starvation indicator, not an error.** High obs_overdue_o under load says calibration is losing arbitration; it should be bounded by overdue_max_i or the workload starves ZQ entirely.

32.8 Open Questions / Future Work

- **Demand wiring.** demand_i grounded in pumice_training_layer; feed it from the scheduler's demand indicator to make Mode-C placement live.
- **Multi-rank.** Single-rank today; the rank loop for a shared ZQ resistor is a documented generate hook.

33 LPDDR2 DQ-Calibration Sequencer (pumice_lp_cal)

Module: pumice_lp_cal.sv **Location:** rtl/fub/ **Category:** FUB **Parent macro:** pumice_training_layer **Status:** implemented and FUB-tested (LPDDR2); inert for DDR2 builds — no MRR

> The DQ-vs-tap sweep lives outside the core, on purpose. This FUB is the MRR engine and capture sideband only: it issues the two mode-register reads, catches the first return beat of each, and hands the data to firmware. The PHY read-delay tap walk and the per-lane compare against patterns A/B run in board firmware driving the FPGA IDELAY primitives — the core stays DFI-clean.

33.1 Purpose

One-shot DQ calibration for LPDDR2 via MRR (Mode Register Read, JESD209-2F §5.12.2). MR32 returns calibration pattern A (1010 on DQ0, mirrored on DQ8 for x16); MR40 returns pattern B (0011). Firmware compares each DQ lane against the expected pattern across its tap sweep and picks the working window. The FSM here just runs the sequence:

1. Wait for a cal_start_i pulse, then wait for the arbiter grant (which guarantees all banks idle, no ACT/PRE in flight or guard window, no REF recovery, and a drained write path — the JESD209-2F MRR spacing rules RD→MRR ≥ BL/2, WR→MRR ≥ WL+1+BL/2+tWTR come along for free).
2. Issue MRR to MR32: OP_MRS + cmd_mrr_o = 1, row {4'd0, MA=32, OP=0}. Pulse cal_expect_o for one cycle to arm the DFI-layer read aligner.
3. Capture the first dfi_rddata_valid beat (MRR data lands RL·tCK + tDQSCK + tDQSQ after the command; BL=4, but only beat 0 carries the MR data).
4. Wait t_mrr_i cycles (JEDEC tMRR = 2 clocks), issue MRR to MR40, capture beat 0 the same way.
5. Set sticky cal_done_o. If either readout times out (t_readout_i cycles with no valid beat), set sticky cal_err_o and finish anyway.

The run condition is init_done_i && (memtype_i == MEMTYPE_LPDDR2); DDR2 has no MRR, so the FUB is inert there.

33.2 Parameters

Parameter	Default	Meaning
DFI_DATA_WIDTH	128	Full DFI read-data width; sizes the captured-beat registers.
MR32_INDEX	32	Pattern-A mode-register index.
MR40_INDEX	40	Pattern-B mode-register index.

33.3 The Eight FSM States

r_state is 3 bits, one state per beat of the handshake:

```
CAL_IDLE           – waiting for cal_start_i
CAL_WAIT_GRANT32   – requesting the arbiter for the MR32 MRR
CAL_PULSE_EXPECT32 – granted; pulsing cal_expect_o to arm the aligner
CAL_WAIT_DATA32    – waiting for the captured MR32 beat / readout
timeout
CAL_WAIT_TMRR      – t_mrr_i spacing before the second MRR
CAL_WAIT_GRANT40   – requesting the arbiter for the MR40 MRR
CAL_PULSE_EXPECT40 – granted; pulsing cal_expect_o again
CAL_WAIT_DATA40    – waiting for the captured MR40 beat / readout
timeout
```

Grant loads r_timeout from t_readout_i; each wait-data state decrements it and escalates to cal_err + done at zero. Both captured beats and their valids are held in registers; data is stable until the next cal_start clears the valids.

cal_abort_i (or run-gate loss) returns the FSM to CAL_IDLE immediately. The sticky done/err bits are **not** cleared by abort — only a fresh cal_start or soft reset moves them, matching the CAL_STATUS "sticky, cleared by soft reset" contract. The captured-data valids, though, are cleared by abort, so a cancelled sequence can't present stale captures as fresh.

33.4 Interface

33.4.1 Control, timing, and run gating

Signal	Direction	Width	Description
mc_clk	in	1	controller clock
mc_rst_n	in	1	active-low reset
cal_start_i	in	1	start one shot (CAL_CTRL.cal_start, self-clearing)
cal_abort_i	in	1	cancel the current sequence

Signal	Direction	Width	Description
			(CAL_CTRL.cal_abort)
init_done_i	in	1	scheduler init complete
memtype_i	in	—	MEMTYPE_LPDDR2 required
t_mrr_i	in	16	MRR-to-MRR spacing, MC cycles (CAL_TRAIN_TIMING.t_mrr, reset 2)
t_readout_i	in	16	MRR issue → data timeout (CAL_TRAIN_TIMING.t_readout)

33.4.2 Maintenance command channel (to the training-layer mux, then the arbiter)

Signal	Direction	Width	Description
cmd_req_o	out	1	request in the WAIT_GRANT states
cmd_grant_i	in	1	arbiter grant back through the mux
cmd_op_o	out	—	always OP_MRS — MRR rides the MRS opcode + mrr flag
cmd_bank_o	out	3	0; the MR index lives in the row field
cmd_row_o	out	18	{4'd0, MA[5:0], 8'd0} — OP field = 0 for MRR
cmd_mrr_o	out	1	formatter MRR select; 1 whenever this FUB drives

33.4.3 Calibration capture handshake (mc_clk domain, synchronized by the layer)

Signal	Direction	Width	Description
cal_expect_o	out	1	single-cycle arm for the rd_aligner sideband
cal_data_i	in	DFI_DATA_WIDTH	captured beat, synchronized up from dfi_clk
cal_data_valid_i	in	1	single-cycle “data landed” strobe

33.4.4 Status and captured data

Signal	Direction	Width	Description
mrr32_data_o	out	DFI_DATA_WIDTH	captured MR32 beat 0 (CAL_MRR32_DATA)
mrr40_data_o	out	DFI_DATA_WIDTH	captured MR40 beat 0 (CAL_MRR40_DATA)
mrr32_valid_o	out	1	MR32 capture is fresh
mrr40_valid_o	out	1	MR40 capture is fresh
cal_busy_o	out	1	FSM not in IDLE
cal_done_o	out	1	sticky — sequence finished (either outcome)
cal_err_o	out	1	sticky — a readout timed out

33.5 Timing / Behavior

The FSM is single-issue and never overlaps the two MRRs — tMRR spacing is a hard FSM state, not a counter check folded into a shared datapath. Timeout coverage: `t_readout_i = 0` makes the wait-data states time out on the first cycle without data, so programming zero buys an immediate error rather than an infinite wait (worth knowing before bumping the timeout down).

The arbiter's grant contract is what makes the capture safe: no read may be outstanding when calibration is active, so the aligner sideband cannot collide with the normal read FIFO push path (see ch04/05).

33.6 Verification Notes (cocotb test plan)

Verified by `dv/tests/fub/test_pumice_lp_cal.py`: the one-shot sequence (start → grant32 → arm → capture → tMRR → grant40 → arm → capture → done), readout-timeout error path, abort mid-sequence, MRR row packing (MA + OP=0), and DDR2-inert behavior.

33.7 Open Questions / Future Work

- **Timeout default.** `CAL_TRAIN_TIMING.t_readout` resets to 0, so the readout-timeout error path fires immediately (a degenerate default that software must reprogram); firmware must program a real window (covering `RL·tCK + tDQSCK + tDQSQ` at the current frequency) before starting a calibration.

- **x16 DQ8 mirror.** The RTL captures the full DFI beat, so the x16 DQ[8]-mirrored pattern lands in the CSR with everything else; the per-lane compare in firmware handles it.
- **Back-to-back sequences.** A second `cal_start` while busy is ignored (the FSM only accepts start in IDLE); firmware paces shots through `CAL_STATUS.cal_busy`.

---

34 Transaction Queue (retired — replaced by the two command CAMs)

RETIRED — no standalone FUB

The unified transaction queue (`txn_queue_fub`) described by the SWAG was **never built as a separate block**. The scheduling-view function it was meant to provide is delivered directly by the two command CAMs:

- **Pending writes** live in `pumice_wr_data_cam` — `{bank, row, col}` key, free-running age, and the burst data in SRAM.
- **Pending reads** live in `pumice_rd_cmd_cam` — `{bank, row}` key, free-running age, per-entry return buffering.
- **Per-direction completion** is FIFO-based in the intakes (`pumice_wr_intake` / `pumice_rd_intake`): the write B FIFO and the read order + rd-data FIFOs.

This chapter is retained to document *where the queue's responsibilities went* and *why*. The SWAG's entry-format, four-state lifecycle, `cam_slot_idx` reverse pointer, and `row_hit_cached` broadcast are all superseded and should not be treated as the current architecture.

Status: Retired

34.1 What the CAMs do instead

The SWAG separated a narrow "scheduling view" (the queue) from a wide "metadata view" (the CAMs) to keep the scheduler's per-cycle comparator fan-in small. The live design collapses the two: each CAM entry carries **both** the scheduling key and the metadata, and the scheduler reads the CAMs directly through purpose-built lookup ports rather than a wide parallel-entry snapshot.

34.1.1 Scheduler visibility (replaces the parallel `q_entries` bus)

Instead of a wide per-entry snapshot bus, each CAM exposes:

- **`N_SCHED_LU` lookup ports** (`N_LU = NUM_BANKS`, one per bank). The arbiter drives each port with `{bank j, that bank's open row}` and the CAM returns the oldest matching not-yet-committed entry — slot, column, id, and age. This is the row-hit query that used to be the `row_hit_cached` field.

- **An `oldest` port** — the oldest schedulable entry, used as the arbiter's ACT fallback target.

The scheduler thus gets exactly the row-hit and fallback candidates it needs, computed inside the CAM combinationally, with no separate queue structure and no `row_hit_cached` coherency broadcast.

34.1.2 Entry lifecycle (replaces FREE/PENDING/ISSUED/COMPLETING)

The four-state queue lifecycle is replaced by per-entry flags in each CAM:

SWAG queue state	Live equivalent (write CAM)	Live equivalent (read CAM)
FREE	`!r_valid`	`!r_valid`
PENDING	`r_valid && r_fdone && !r_sched`	`r_valid && !r_issued`
ISSUED	`r_sched` (enqueued to drain FIFO)	`r_issued` (enqueued to issue FIFO)
COMPLETING	commit-drain streaming, then evict	data returning, then oldest-drain

The exclusion of already-committed/issued entries from the lookup/oldest ports (`r_sched` on the write side, `r_issued` on the read side) is what guarantees the arbiter never picks the same slot twice — the role the `pick_valid_mask` played in the SWAG.

34.1.3 Age (replaces the saturating age counter)

Both CAMs use a free-running `AGE_WIDTH` counter and wrap-safe relative age (`rel = age_ctr - entry_age`). Oldest = max `rel`. There is no saturation, no `AGE_MAX` parameter, and no `age_max_runtime` CSR clip; wrap-safety makes saturation unnecessary.

34.1.4 Backpressure (replaces `q_high_water` / `q_full`)

There is no queue occupancy watermark. Backpressure is structural: the write path stalls when the wr-data CAM has no free entry or its fill FIFO is full; the read path stalls when the rd-cmd CAM is full or the intake's order FIFO is full. The AXI channels back-pressure through the intake FIFOs.

34.2 Why the split disappeared

The queue existed to make a *single* wide scheduling snapshot cheap. Once the scheduler was rearchitected to query per-bank (one lookup per open row) rather than scan every entry, the wide snapshot was no longer needed — the CAM's own associative match is the scheduling view. Keeping a second copy of `{rank, bank, row}` in a queue, plus the `cam_slot_idx` reverse pointer and the `row_hit_cached` broadcast coherency machinery, would have been pure overhead.

See ch02/07 for the arbiter that consumes the CAM lookup ports and ch02/04 / ch02/05 for the CAM internals.

---

35 Bank Timer (bank_timer + pumice_bank_timers)

Module: bank_timer.sv (per-bank) / pumice_bank_timers.sv (aggregator) **Location:** rtl/fub/ **Category:** FUB **Parent macro:** pumice_scheduler_layer **Status:** Implemented (FSM-free countdown-timer model)

> **Replaces the retired bank machine.** The original architecture placed a per-(rank, bank) seven-state FSM (bank_machine_fub) between the scheduler and the DFI encoder. That block **no longer exists**. Per-bank JEDEC timing is now enforced by bank_timer.sv — a set of preset/decrement countdown timers with a trivial row-open register and **no state machine at all**. pumice_bank_timers.sv stamps one bank_timer per (rank, bank) and fans the scheduler's command-event strobes to the addressed instance.
>
> The retirement was deliberate: the old FSM double-registered readiness (state register + next-state combinational + accepts flop), so the scheduler saw readiness two cycles stale. That staleness was the root cause of the refresh-vs-ACT and column-vs-PRE hazards observed during bring-up. The FSM-free model collapses the readiness path to a **single register stage** (the timers themselves), so the arbiter sees the just-issued command's effect exactly one cycle later.

35.1 Purpose

bank_timer tracks one DRAM bank's JEDEC "safe" timing. For each JEDEC per-bank constraint it holds a down-counter that is preset on the triggering command and free-runs toward zero (saturating at 0). A constraint is "safe" when its counter reads 0. The per-command readiness outputs (safe_act, safe_rd, safe_wr, safe_pre) are a **combinational AND** of the relevant timers plus a one-bit row-open flag — no FSM, no multi-stage lag.

The scheduler (pumice_cmd_arbiter, the single command issuer in the design) drives one set_* strobe per issued command. Each timer reloads its config value on its trigger; the safe_* outputs therefore reflect the just-issued command one cycle later. The arbiter ANDs these per-bank safe_* signals with the channel-wide turnaround windows from global_timers (§2.10) to decide which command may fire.

pumice_bank_timers is a thin aggregator: it instantiates NUM_RANKS × NUM_BANKS copies of bank_timer and routes the arbiter's single command-event bus (evt_* + evt_rank_i / evt_bank_i) to the one addressed instance. All other instances see their set_* strobes deasserted that cycle and simply continue counting down.

35.2 Synthesis Parameters

35.2.1 bank_timer

Parameter	Default	Effect on this FUB
ROW_WIDTH	14	Width of open_row_o register and the row_i input
TW	8	Timer width (bits) for all five countdown timers

35.2.2 pumice_bank_timers

Parameter	Default	Effect
NUM_RANKS	1	Rank dimension of the instance array; width of evt_rank_i sel
NUM_BANKS	8	Bank dimension of the instance array; width of evt_bank_i sel
ROW_WIDTH	14	Passed to each bank_timer
RKW	clog2(NUM_RANKS)	Rank-select width (min 1)
BKW	clog2(NUM_BANKS)	Bank-select width

There is **no per-instance rank/bank identifier parameter**. Identity comes from the genvar position in pumice_bank_timers' nested generate loop; the arbiter selects an instance by comparing evt_rank_i / evt_bank_i against that position (w_sel). All five timers share the same TW-bit width; the JEDEC cycle counts are supplied at runtime on the t_*_i ports (CSR-backed), not fixed as parameters.

35.3 Instantiation Pattern

From pumice_bank_timers.sv — the nested generate that stamps the array:

```
for (genvar k = 0; k < NUM_RANKS; k++) begin : g_rank
    for (genvar b = 0; b < NUM_BANKS; b++) begin : g_bank
        // route the scheduler's command event to THIS bank only
        logic w_sel;
        assign w_sel = (evt_rank_i == RKW'(k)) && (evt_bank_i ==
BKW'(b));
```

```
            bank_timer #(.ROW_WIDTH(ROW_WIDTH)) u_bt (
                .clk(aclk), .rst_n(aresetn),
                .t_rcd_i(t_rcd_i), .t_rp_i(t_rp_i), .t_ras_i(t_ras_i),
                .t_rc_i(t_rc_i), .t_wr_i(t_wr_i), .t_rtp_i(t_rtp_i),
                .set_act_i(evt_act_i && w_sel),
                .set_rd_i (evt_rd_i  && w_sel),
                .set_wr_i (evt_wr_i  && w_sel),
                .set_pre_i(evt_pre_i && w_sel),
                .set_ap_i (evt_ap_i),
                .row_i(evt_row_i),
                .safe_act_o (bank_act_ready_o [k][b]),
                .safe_rd_o  (bank_rdwr_ready_o[k][b]),
                .safe_wr_o  (/* == safe_rd */),
                .safe_pre_o (bank_pre_ready_o [k][b]),
                .row_valid_o(bank_row_active_o[k][b]),
                .open_row_o (bank_open_row_o  [k][b]),
                .state_o    (bank_state_o     [k][b]),
                /* obs_* ... */
            );
        end
    end
```

The per-(rank, bank) readiness outputs are 2-D-packed arrays (`logic [NUM_RANKS-1:0] [NUM_BANKS-1:0]`) consumed directly by `pumice_cmd_arbiter`. There is no aggregation logic beyond the packing — pure structural fan-out. Note that `safe_wr_o` is left unconnected in the aggregator because inside `bank_timer` it is identical to `safe_rd_o` (see the RTL: `assign safe_wr_o = safe_rd_o`); the arbiter uses the single `bank_rdwr_ready_o` bit for both RD and WR column commands.

35.4 The Timer Pool (per bank)

`bank_timer` holds five countdown registers (`r_rcd`, `r_ras`, `r_rc`, `r_rp`, `r_preblk`), each `TW` bits, plus a one-bit `r_row_valid`, the `r_open_row` register, and a single `r_ap_pending` auto-precharge bit. Every timer is a down-counter: reload on its trigger, else decrement while non-zero (saturate at 0).

Timer	JEDEC constraint	Reload trigger	Reload value	Gates
`r_rcd`	tRCD (ACT → RD/WR)	`set_act_i`	`t_rcd_i`	`safe_rd/wr`
`r_ras`	tRAS (ACT → PRE min)	`set_act_i`	`t_ras_i`	`safe_pre`
`r_rc`	tRC (ACT →	`set_act_i`	`t_rc_i`	`safe_act`

Timer	JEDEC constraint	Reload trigger	Reload value	Gates
	ACT same bank)			
r_rp	tRP (PRE → ACT)	set_pre_i OR auto-PRE fire	t_rp_i	safe_act
r_preblk	tRTP (RD) / tWR (WR)	set_rd_i / set_wr_i	t_rtp_i / t_wr_i	safe_pre

The t_*_i reload values are supplied by the parent (pumice_bank_timers forwards its own 8-bit t_*_i inputs, which are CSR-backed JEDEC timing fields). They are sampled live on each reload — the timer does not stage them.

Note: the retired FSM's separate t_cl_cnt / t_cwl_cnt (column-busy windows) are **gone**. Column-to-column pacing (tCCD) and the read-drives-DQ / write-recovery windows are enforced globally in global_timers (tCCD/tWTR/tRTW) and, for the write-recovery-before-precharge case, folded into the r_preblk timer here. There is no per-bank "RD_BUSY / WR_BUSY" occupancy state anymore.

35.5 Row-Open Register and Auto-Precharge (no FSM)

Row state is a two-element register — r_row_valid + r_open_row — plus one r_ap_pending bit. There is no enumerated state; the priority in the RTL is **ACT** > **explicit PRE** > **auto-PRE fire** > **column**:

```
if (set_act_i) begin
    r_row_valid  <= 1'b1;
    r_open_row   <= row_i;
    r_ap_pending <= 1'b0;
end else if (set_pre_i) begin
    r_row_valid  <= 1'b0;
    r_ap_pending <= 1'b0;
end else if (w_ap_fire) begin
    r_row_valid  <= 1'b0;
    r_ap_pending <= 1'b0;
end else if (set_rd_i || set_wr_i) begin
    r_ap_pending <= set_ap_i;   // latch the RDA/WRA qualifier
end
```

Auto-precharge (RDA / WRA) is a single bit, not a state. set_ap_i qualifies a RD/WR issue; if set, r_ap_pending latches. The auto-precharge "fires" combinationally once the read/write recovery window and tRAS have both elapsed:

```
assign w_ap_fire = r_ap_pending && (r_preblk == '0) && (r_ras == '0);
```

When `w_ap_fire` asserts, the row closes internally (`r_row_valid <= 0`) and `r_rp` reloads with `t_rp_i` — exactly as an explicit PRE would. No PRE command is consumed from the scheduler and no extra state is entered. This is the same degeneracy the old FSM's "auto-pre pending" trick achieved, but here it is one flop and one AND gate.

Open-page behavior. A bare RD/WR (with `set_ap_i` low) only reloads `r_preblk` and leaves `r_row_valid` set, so the row stays open. Successive column commands to the open row stream at the channel-wide tCCD rate enforced by `global_timers`; the bank timer imposes no additional per-column pacing.

35.6 Combinational safe_* Outputs

The readiness outputs are pure combinational functions of the timer/flag registers — one register stage behind the arbiter's issue decision:

```
// ACT: bank precharged (row closed), tRP since PRE, tRC since last
ACT.
assign safe_act_o = !r_row_valid && (r_rp == '0) && (r_rc == '0);
// RD/WR: row open, tRCD met, not auto-precharging.
assign safe_rd_o  = r_row_valid && (r_rcd == '0) && !r_ap_pending;
assign safe_wr_o  = safe_rd_o;
// PRE: row open, tRAS + tRTP/tWR met, not auto-precharging.
assign safe_pre_o = r_row_valid && (r_ras == '0) && (r_preblk == '0)
&& !r_ap_pending;
```

The `!r_ap_pending` term on `safe_rd_o` / `safe_pre_o` is what makes an auto-precharging bank refuse further column commands and a redundant explicit PRE while its internal auto-PRE is still pending — the arbiter naturally skips the bank until the row closes and `safe_act_o` re-asserts.

Because these are combinational off single-stage registers, the arbiter's per-cycle pick sees the effect of the command it issued on cycle *T* on cycle *T+1*. There is no second flop between the timer and the arbiter, so a command issued this cycle cannot be double-issued next cycle — the class of hazard that motivated retiring the FSM.

35.7 Derived State (observability only)

A `bank_state_e` (`state_o`) is produced **purely for observability** — no downstream logic reads it. It is decoded combinationally from the row-valid flag and the timers:

```
if (r_row_valid) state_o = (r_rcd != '0) ? BANK_ACTIVATING :
BANK_ACTIVE;
else             state_o = (r_rp  != '0) ? BANK_PRECHARGING :
BANK_IDLE;
```

This gives waveform viewers and CSR telemetry the familiar IDLE / ACTIVATING / ACTIVE / PRECHARGING names without the design actually implementing them as a sequencer. It is a decode of the timers, not the source of truth.

The remaining observability outputs are single-bit non-zero flags on the timers and the auto-pre bit: `obs_rcd_nz_o`, `obs_preblk_nz_o`, `obs_ras_nz_o`, `obs_ap_pending_o` (per bank, fanned out to `obs_*` arrays in the aggregator).

35.8 Interface (`pumice_bank_timers`)

35.8.1 Clocks / Reset

Signal	Direction	Description
`aclk`	input	Controller clock
`aresetn`	input	Active-low async reset

35.8.2 Timing config (CSR-backed, per-bank JEDEC cycle counts)

Signal	Width	Constraint
`t_rcd_i`	8	tRCD
`t_rp_i`	8	tRP
`t_ras_i`	8	tRAS
`t_rc_i`	8	tRC
`t_wr_i`	8	tWR (WR cmd → earliest PRE, incl WL+BL/2)
`t_rtp_i`	8	tRTP (RD cmd → earliest PRE)

35.8.3 Command-event strobes from the arbiter

Signal	Width	Description
`evt_act_i`	1	ACT issued this cycle
`evt_rd_i`	1	RD issued this cycle
`evt_wr_i`	1	WR issued this cycle
`evt_pre_i`	1	PRE issued this cycle
`evt_ap_i`	1	Auto-precharge qualifier (with RD/WR)

Signal	Width	Description
evt_rank_i	RKW	Rank of the issued command (selects instance)
evt_bank_i	BKW	Bank of the issued command (selects instance)
evt_row_i	ROW_WIDTH	Row operand (latched on ACT)

35.8.4 Per-bank readiness to the arbiter (combinational, single-stage)

Signal	Width	Description
bank_act_ready_o[NR][NB]	1 each	safe_act per bank
bank_rdwr_ready_o[NR][NB]	1 each	safe_rd (== safe_wr) per bank
bank_pre_ready_o[NR][NB]	1 each	safe_pre per bank
bank_row_active_o[NR][NB]	1 each	row_valid per bank
bank_open_row_o[NR][NB]	ROW_WIDTH each	Open row per bank
bank_state_o[NR][NB]	bank_state_e	Decoded state (observability)

35.8.5 Observability

obs_act_cnt_nz_o, obs_preblk_nz_o, obs_ras_nz_o, obs_ap_pending_o — each a [NR][NB] bit array, mirroring the per-bank obs_* outputs.

35.9 Timing Budget

The path from a command event to updated readiness is single-cycle:

```
cycle T:   arbiter asserts evt_act_i with evt_rank_i=r, evt_bank_i=b
cycle T:   w_sel picks instance (r,b); r_rcd/r_ras/r_rc load,
r_row_valid <- 1
cycle T+1: safe_act_o[r][b] = 0 (r_rc/row_valid now block ACT),
           safe_rd_o[r][b]  gated by r_rcd countdown
```

safe_*_o is combinational from the registers, so the arbiter consumes the post-event readiness on the very next cycle — no race allowing a second issue to the same bank while its constraint is still counting. This one-cycle discipline is the whole point of the FSM-free rework: readiness is exactly one register stage deep, versus the retired FSM's two (state + accepts).

35.10 Verification Notes (cocotb test plan)

Scenario	What it proves
ACT → wait tRCD → RD; RD blocked until `r_rcd == 0`	tRCD gate on `safe_rd`
ACT → RDA; row auto-closes once `r_preblk` and `r_ras` clear	`w_ap_fire` closes row, reloads `r_rp`; no scheduler PRE
ACT → WRA; `r_preblk` loads tWR; auto-PRE waits for tWR AND tRAS	Write-recovery folded into `r_preblk`
Bare RD (set_ap=0) keeps row open; second RD to same row allowed	Open-page: `r_row_valid` stays set
ACT → PRE (explicit) → ACT; second ACT blocked until `r_rp == 0`	tRP gate on `safe_act`
Two ACTs same bank; second blocked until `r_rc == 0`	tRC gate on `safe_act`
`safe_rd` deasserts while `r_ap_pending` set (mid auto-PRE)	Auto-precharging bank refuses new column commands
Aggregator: `evt_bank_i=b` only reloads instance b; others count down	`w_sel` routing correctness
`state_o` decode matches timers (ACTIVATING while `r_rcd != 0`)	Observability decode
Reset during any countdown → all timers and row flag clear	Reset behavior

35.11 Open Questions / Future Work

- **Timer width.** `TW = 8` covers the DDR2/LPDDR2 board targets (tRC ≈ tens of cycles). A DDR3/DDR4 family part at a higher CK:MC ratio could need wider timers; `TW` is already a parameter, so this is a re-elaboration, not a rework.
- **Per-bank vs shared tRC.** tRC is enforced per bank here. If a future family needs same-bank-group tRC differentiation (DDR4 bank groups), that logic would land in `global_timers`, not here — `bank_timer` stays bank-local by design.
- **Auto-PRE vs explicit PRE priority.** The RTL priority is ACT > explicit PRE > auto-PRE. If the arbiter ever issued an explicit PRE to a bank with a pending auto-PRE, the explicit PRE wins and `r_ap_pending` clears. The

arbiter does not do this today (it treats an auto-precharging bank as unschedulable via `safe_*`), but the RTL is safe if it ever did.

---

36 ODT Control (absorbed — no standalone block)

ABSORBED — No standalone FUB

There is **no `odt_ctrl` module** in the live RTL. The dedicated ODT-control FUB the original architecture planned was never built as a separate block. ODT responsibility is split between two modules that already exist:

- **`dfi_cmd_formatter.sv`** (see the command-formatter chapter) owns the `dfi_odt_o` output. ODT follows deterministically from the issued command, so it belongs on the same output stage as the ras/cas/we truth table rather than in a separate FSM.
- **`mode_register.sv`** decodes the DDR2 ODT rule bits from the mode registers into an `odt_o` field (see the mode-register chapter).

This chapter is retained only to explain where ODT lives and to record the current (minimal) state honestly.

Status: Absorbed / minimal — ODT is decoded but not yet actively driven

---

36.1 Where ODT Lives Today

Concern	Live location	State
`dfi_odt_o` bus (per-rank, per-phase)	`dfi_cmd_formatter.sv` output	Driven to 0 in v1 (decode leaves `w_p0_odt` at the NOP default for every op)
`dfi_odt_o` reset value	`dfi_signal_pack.sv`	0 (ODT off during reset / before init)
DDR2 ODT-rule decode from MRs	`mode_register.sv` `odt_o[1:0]`	Decoded from MR1 (`{w_mr1[6], w_mr1[2]}`); informational, not wired to the pin driver
LPDDR2 ODT	`mode_register.sv`	`odt_o = 0` (LPDDR2 typically point-to-point)

`mode_register` exposes `odt_o` as one of its live decoded outputs, but the RTL comment marks it "informational; not used" — the value is computed from the DDR2 MR1 ODT bits but is not currently consumed to time a termination window. `dfi_cmd_formatter` drives `dfi_odt_o` from

`w_p0_odt`, which the DDR2 decode never raises above its NOP default, so on the DDR2/LPDDR2 board targets (both effectively single-rank point-to-point) ODT stays off.

36.2 Why ODT Is Off on the Board Target

ODT is a JEDEC impedance-matching feature: a DDR2 device contains internal termination resistors that the controller selectively enables to terminate the DQ/DQS bus. The rule is asymmetric and only matters with more than one device on the bus:

- **During a read**, the accessed rank drives DQ; *other* ranks should terminate.
- **During a write**, the controller drives DQ; the *target* rank should terminate.

For a single-rank point-to-point system there is nothing else on the bus, so ODT serves no purpose and 0 is correct. The pumice board bring-up (Nexys A7, single x16 DDR2 device) is exactly this case — which is why the v1 controller ships with ODT decoded-but-not-driven and passes on silicon.

36.3 What a Full ODT Implementation Would Add

If a multi-rank DDR2 target is brought up, the timed ODT window would be added inside `dfi_cmd_formatter`'s output stage (not as a new block), consuming `mode_register.odt_o` plus CL/CWL and burst length to compute per-rank turn-on/ turn-off. The JEDEC cross-termination pattern (ODT-high on the non-accessed rank during a read, on the accessed rank during a write) would drive `w_p0_odt` per target rank instead of the current constant 0. Because the rule follows deterministically from the issued op and the MR-decoded termination value, it stays in the formatter's truth table — the reason the standalone block was never needed.

36.4 Verification Notes (cocotb test plan)

Scenario	What it proves
Single-rank DDR2: `dfi_odt_o` stays 0 for all traffic	Current behavior (point-to-point board target)
Reset: `dfi_signal_pack` drives `dfi_odt_o = 0`	Reset-safe ODT-off
`mode_register` MR1 ODT bits decode into `odt_o`	Decode present for future use
LPDDR2: `mode_register.odt_o = 0`	No ODT on LPDDR2 point-to-point

36.5 Open Questions / Future Work

- **Timed ODT window (multi-rank DDR2).** Not implemented. Would live in `dfi_cmd_formatter`'s output stage, driven by `mode_register.odt_o` + CL/CWL + BL. Add when a multi-rank DDR2 board is targeted.
- **Dynamic ODT (DDR3+).** DDR3 introduced Rtt_Wr vs Rtt_Nom; DDR2/LPDDR2 have only Rtt_Nom. A DDR3+ family controller would extend the decode.
- **ODT during refresh/precharge.** Some boards want ODT held during refresh to avoid a floating bus. Revisit only if signal-integrity characterization flags it on a multi-rank target.

---

37 AXI4 Slave Protocol

This chapter is the **wire-level contract** of the controller's host AXI4 slave face – what's supported, what's relaxed, what's omitted, and the timing / ordering / backpressure semantics the integrator can rely on.

The interface is implemented by `pumice_axi4_layer` (section 2.1): a pair of AMBA burst splitters feed the dumb `pumice_wr_intake` / `pumice_rd_intake` blocks (each wrapping `axi4_slave_wr` / `axi4_slave_rd`), which push into the write / read CAMs.

37.1 Compliance Profile

Aspect	Support level
AXI4 (ARM IHI 0022) signal set	Full AW/W/B/AR/R
AXI4-Lite	Not supported (use the register cpuif for control)
AXI4-Stream	Not applicable
AXI5	Not supported

37.2 Data Width

The core's AXI data width is the **DFI word**: `DW = DRAM_BEAT_WIDTH * DFI_RATE` (128 by default). One AXI beat == one DFI word == `DFI_RATE` DRAM beats. If the SoC master runs a different width, wrap the core in `pumice_top_geared`, which inserts the formally-verified `axi4_dwidth_converter_wr/_rd` between a host-width AXI slave and the fixed-`DW` core (`HOST_AXI_DATA_WIDTH == DW` is a bit-identical generate bypass). See `docs/AXI_DRAM_GEARING_SCOPE.md`.

Hard width rule (compile-enforced). `HOST_AXI_DATA_WIDTH : DW` **MUST** be an exact power-of-two ratio (`AXI:DFI = G:1` or `1:G`, `G ∈ {1,2,4,8,…}`). An `initial assert … $fatal` in `pumice_top_geared` **fails elaboration / Vivado synthesis** on any other pairing — a bad width choice is a compile error, not a silent broken build. This mirrors LiteDRAM exactly (AXI frontend 1:1 with its native port; all width change via a power-of-two stride converter). Because the DFI word is the atomic memory-side transfer, an AXI beat must be a whole power-of-two number of DFI words. `HOST_AXI_DATA_WIDTH` and `DW` are the **only** compile-time width parameters; **gear ratio and burst length are runtime CSRs** (`gear_ratio`, `bl`), built for max and selected at runtime — a wrong value is bad config programming, never a parameter/synthesis mismatch.

37.3 Burst Splitting

Each host burst is split at DRAM-burst-byte boundaries by `axi_master_wr_splitter` / `axi_master_rd_splitter` (`ALIGN_MASK = BL*(DRAM_BEAT_WIDTH/8) - 1`), so every command handed to an intake is exactly one DRAM burst. This is how an arbitrary AXI `awlen`/`arlen` maps onto the fixed DRAM burst length.

37.4 Supported Features

- INCR burst type (mandatory)
- Arbitrary INCR burst length (split into DRAM-burst commands as above)
- ID-carried requests; per-ID in-order completion (AXI4 mandate)
- Cross-ID reordering at the DRAM layer, with AXI response ordering honored at the response side (per HAS 3.1)
- `awqos`/`arqos`/`awregion`/`arregion` carried through the splitters (observed, not yet used for scheduler priority)
- Single-bit `awuser`/`aruser`/`wuser` ports carried through the front-end (echoed on B/R `buser`/`ruser`)

37.5 Unsupported / Ignored

- AXI4 exclusive accesses (`awlock`, `arlock` observed but ignored; treated as normal accesses)
- AXI4 cache-coherent behavior (`awcache`/`arcache` observed, no behavior)
- FIXED / WRAP burst types are not a design target; the front-end is built for INCR traffic

37.6 Backpressure Semantics

The slave asserts backpressure via the standard AXI handshake. Backpressure originates in the intake FIFOs and CAM fill:

Channel	Backpressure reason
AW	AW-meta FIFO full in `pumice_wr_intake`, or `pumice_wr_data_cam` has no free slot
W	Write-data FIFO / write-data SRAM full in the write path
AR	`pumice_rd_cmd_cam` has no free slot
B	Standard `bready` handshake

Channel	Backpressure reason
R	Standard `rready` handshake; the drain mover in `pumice_rd_cmd_cam` stalls until `rready`

Total in-flight depth per direction is set by `NUM_ENTRIES` (CAM depth) with `N_SRAM_SLOTS` burst-data SRAM slots per CAM.

37.7 Response Ordering

1. **Per-ID in-order** – always enforced. Two accesses with the same ID complete in issue order.
2. **Cross-ID** – reordering is allowed at the DRAM layer (the scheduler picks by bank/row/age), but the CAMs drain and the intakes emit responses so that the AXI-visible ordering rules hold.

37.8 Read-Your-Write Forwarding (snarf)

`pumice_rd_intake` probes `pumice_wr_data_cam` before a read is scheduled. On a hit against an unscheduled write with the same id and same burst length, the read is streamed straight from the write CAM's SRAM (the **snarf mover**) with no DRAM round-trip. On a miss the read goes through the normal DRAM path.

37.9 Timing Contract

Path	Behavior
`awvalid` -> `awready`	<= 1 cycle when the AW-meta FIFO has space
`awready` accept -> CAM push	1 cycle
CAM push -> DRAM issue	scheduler / refresh dependent (typically 0 to ~256 cycles)
Last W beat -> B response	commit + write-recovery window + scheduler backlog
`arvalid` -> first R beat	DRAM access latency (typically 30-100 ns) + bus rounding, or ~0 on a snarf hit
Successive R beats (same burst)	1 per `aclk` cycle (sustained streaming)

The slave does not embed an AXI register slice. If a target needs one for closure, add an external `axi_register_slice` from the AMBA library, or use `pumice_top_geared` (whose dwidth converters register the boundary).

37.10 Error Responses

Condition	Response	Notes
Traffic before `init_done_o`	held / SLVERR	Poll `STATUS.init_done` (or the `init_done_o` pin) before issuing traffic
DFI read return flagged bad by the aligner	`SLVERR`	Propagated on the affected R beats

DECERR is never returned – the address space is monolithic.

37.11 Open Questions / Future Work

- **QoS-driven priority.** `awqos`/`arqos` are carried but not yet consumed by `pumice_cmd_arbiter`. A future revision would fold them into the pick.
- **Wider USER signals.** The front-end carries single-bit user today; wider sideband would need a `USER_WIDTH` parameter threaded through the intakes and CAMs.
- **AXI register-slice option.** Currently external / via the geared wrapper; a build-time embed could be added for very-high-frequency targets.

38 DFI v2.1 Master Protocol

This chapter is the wire-level contract of the controller's DFI 2.1 master face – what's driven, what's tied, what's sampled, what's deferred.

The interface is implemented by `pumice_dfi_layer` (section 2.4): `pumice_dfi_cmd_path` (with `dfi_cmd_formatter` + `dfi_signal_pack`) drives the command bus, `pumice_dfi_wr_serializer` drives write data, `pumice_dfi_rd_aligner` captures read data, and `pumice_dfi_cdc` is the single clock crossing to `aclk`.

38.1 DFI Spec Reference

Canonical: the DFI 2.1 specification (Denali cleartext copy at `/home/seang/github/cold_storage/MemorySpecs/`). The controller covers the subset of DFI 2.1 sub-interfaces needed for DDR2/LPDDR2 operation; higher-version sub-interfaces are not driven.

38.2 Sub-Interface Support Summary

Sub-Interface	Direction	Supported?	Notes
Command	Controller -> PHY	Yes	Per-phase; command placed on `wr_phase`/`rd_phase`
Write-Data	Controller -> PHY	Yes	`dfi_wrdata` presented at `t_phy_wrlat` after WR fire
Read-Data	PHY -> Controller	Yes	`dfi_rddata_en` at `t_rddata_en` after RD fire; captured on `dfi_rddata_valid`
Status (init)	Both	Yes	`dfi_init_start` / `dfi_init_complete` only
Update	Both	No	`ctrlupd_*` / `phyupd_*` not driven in v1
Training	Both	No	PHY-side (`a7ddrphy` self-trains at startup)
Frequency Change	Both	No	No use case
Low-Power	Controller -	No	No PHY-side low-power coordination

Sub-Interface	Direction	Supported?	Notes
	> PHY		
Error / CRC	-	No	Not used in DDR2/LPDDR2 here

38.3 Clock and CDC

The DFI pin bus is driven in the **`dfi_clk`** domain. Unlike the classic "DFI is on the controller clock" model, this controller keeps the whole DFI datapath on a dedicated PHY clock and places the **single** controller-to-PHY CDC inside `pumice_dfi_layer` (`pumice_dfi_cdc`, async gaxi FIFOs only). The command, write-data, and read-data streams plus the init handshake cross there; the internal datapath unit is the whole DFI word, so one FIFO word is one `dfi_clk` cycle and the path is bubble-free at rate. `a7ddrphy` handles any further gearing to the DRAM clock.

38.4 Command Sub-Interface

38.4.1 Phase Topology

The command bus is `per-phase x DFI_RATE` wide. `dfi_address_o` is `ROW_WIDTH * DFI_RATE`, `dfi_bank_o` is `$clog2(NUM_BANKS) * DFI_RATE`, and each control strobe (`dfi_cas_n_o` / `dfi_ras_n_o` / `dfi_we_n_o`) is `1 * DFI_RATE`. `pumice_dfi_cmd_path` places the active JEDEC command on the `wr_phase` / `rd_phase` slot (programmed by the `DFI_PHASE` CSR) and drives NOP-equivalent idle on the other phases.

38.4.2 Chip-Select and ODT

`dfi_cs_n_o` and `dfi_odt_o` are each `NUM_RANKS * DFI_RATE` wide (rank inner, phase outer, as the `DFI_CS_BUS_W = NUM_RANKS * DFI_RATE` packing implies). The formatter drives the addressed rank's CS_n active for a single-rank command; init-time REF broadcasts. ODT is driven inside `dfi_cmd_formatter` / `mode_register` – there is no standalone `odt_ctrl` block.

38.4.3 DDR2 vs LPDDR2 Encoding

For `memtype_i = DDR2`, commands use the classic `ras/cas/we/cs` strobe encoding. For `memtype_i = LPDDR2`, `dfi_cmd_formatter` takes the memtype branch and emits bit-exact JESD209-2F CA-bus commands (Table 60): the 10-bit CA bus over 2 edges packed as a flat 20-bit word on `dfi_address`, with `ras/cas/we` idle. See `docs/uarch/LPDDR2_CA_ENCODING.md`.

38.5 Write-Data Sub-Interface

`pumice_dfi_wr_serializer` presents `dfi_wrdata` / `dfi_wrdata_en` / `dfi_wrdata_mask` exactly `t_phy_wrlat` `dfi_clk` cycles after the `wr_fire` strobe from the command path. The write burst arrives from the write CAM as whole DFI words `{last, strb, data}`; the serializer splits into the per-phase data / enable / mask.

dfi_wrdata_en_o is DFI_RATE wide (one enable per phase). A mask bit of 1 means “do not write this byte” per DFI convention; the mask payload carried from the CAM is aligned to that convention.

38.6 Read-Data Sub-Interface

pumice_dfi_rd_aligner drives dfi_rddata_en_o (DFI_RATE wide) exactly t_rddata_en dfi_clk cycles after the rd_fire strobe, then captures dfi_rddata_i when dfi_rddata_valid_i asserts, packing whole DFI words ({last, resp, data}) into the read FIFO for the crossing back to aclk.

38.6.1 Read-Data Error Handling

dfi_rddata carries no error bit in DFI 2.1. A missing / malformed read is surfaced by the aligner’s resp field and propagated as SLVERR on the affected AXI R beats. On-silicon this path was hardened during board bring-up: t_rddata_en and the DFI read-data delay are runtime CSRs, and rd_phase = 0 is correct for a7ddrphy (which handles read gearing internally).

38.7 Status / Init Sub-Interface

dfi_init_start_o and dfi_init_complete_i are the only status signals. The init_sequencer (in pumice_scheduler_layer) asserts dfi_init_start and walks the JEDEC MRS init, waiting on dfi_init_complete from the PHY plus the programmed init-timing CSRs. These cross the CDC in pumice_dfi_cdc.

38.8 Pin Tie-Offs

Pin	Tie-off (when applicable)
dfi_ras_n_o / dfi_cas_n_o / dfi_we_n_o	Idle in LPDDR2 (command rides the CA bus on dfi_address)
Update / frequency / low-power buses	Not present / not driven

38.9 Open Questions / Future Work

- **DFI error / update.** ctrlupd_* / phyupd_* and a dfi_error input are not wired in v1; a later revision could add quiet-point update handshaking and route read errors to an IRQ.
- **Per-rank CS width form.** The controller produces the flat NUM_RANKS * DFI_RATE vector form; a PHY expecting a different packing needs a thin wrapper at the boundary.

- **DFI 3+ migration.** When the DDR3-LPDDR3 family controller arrives, the sub-interface set grows substantially. Swap `pumice_dfi_layer` and add a dedicated DFI section in that MAS; keep this one DDR2/LPDDR2-scoped.

---

39 Register Access Interface (PeakRDL cpuif)

Per HAS §2.4 §3 for the per-signal port list and HAS §4.3 for the architectural view. This chapter is the **wire-level contract** for the controller's register access interface.

The register block is a PeakRDL-generated module (`pumice_csr`, from `rtl/macro/pumice_csr.rdl`) instantiated directly in `pumice_top`. It exposes a **PeakRDL passthrough CPU interface** (`s_cpuif_*`), not a hand-written APB slave. Any APB (or AXI-Lite) bridge is **external/optional** — the SoC attaches whatever register transport it uses and drives the cpuif. All register storage lives in the `aclk` (MC) domain; there is no separate `apb_pclk` domain and no on-die CDC inside the register block.

39.1 The cpuif Contract

`pumice_top` presents the PeakRDL passthrough interface on `aclk`/`aresetn`:

Direction	Signal	Meaning
in	`s_cpuif_req`	Request strobe
in	`s_cpuif_req_is_wr`	1 = write, 0 = read
in	`s_cpuif_addr[11:0]`	Byte address (12-bit, 4 KB region)
in	`s_cpuif_wr_data[31:0]`	Write data
in	`s_cpuif_wr_biten[31:0]`	Per-bit write-enable (bit strobe)
out	`s_cpuif_req_stall_wr`	Back-pressure a write request
out	`s_cpuif_req_stall_rd`	Back-pressure a read request
out	`s_cpuif_rd_ack`	Read response valid
out	`s_cpuif_rd_err`	Read decode error (unmapped address)
out	`s_cpuif_rd_data[31:0]`	Read data
out	`s_cpuif_wr_ack`	Write response valid

Direction	Signal	Meaning
out	`s_cpuif_wr_err`	Write decode error (unmapped address)

The interface is single-request/single-response with a stall handshake. Registers are 32-bit, naturally aligned on 4-byte boundaries. Sub-word writes use `s_cpuif_wr_biten` (per-bit granularity); a full-word write drives all 32 biten bits.

39.2 Attaching an APB / AXI-Lite Bridge

Because the interface is a generic passthrough cpuif, the register transport is an integration choice made **outside** this controller:

- An APB3/APB4 slave bridge translates `psel`/`penable`/`pwrite`/`paddr`/`pwdata`/`prdata`/`pslverr` to the cpuif and back. `pslverr` maps from `s_cpuif_rd_err`/`s_cpuif_wr_err`.
- An AXI-Lite slave bridge is equally valid; the cpuif does not assume APB.
- If the register bus is in a different clock domain than `aclk`, the CDC lives in that external bridge — not in `pumice_csr`.

No `pumice_top` parameter selects the transport; the bridge is a separate module in the SoC's register fabric.

39.3 Address Space

The register block decodes a 4 KB region (12-bit address). Layout, matching `rtl/macro/pumice_csr.rdl`:

Address range	Region
0x000 – 0x008	Control / Status / Status-history
0x010 – 0x01C	Timing parameters (RC/RCD/RP/RAS, RFC/REFI, RRD/FAW/WTR/CCD, CL/CWL/WR)
0x020 – 0x02C	Mode Register values (MR0..MR3)
0x030 – 0x038	LPDDR2-specific (PASR bank/segment masks, temp derate)
0x040 – 0x05C	Scheduler / Page / Refresh / Addr-map / Init tuning + init timing
0x054, 0x060, 0x064	tRTP/tRTW, DFI command phase, PHY/DFI data timing
0x080 – 0x09C	Per-bank row-hit observation (NUM_BANKS = 8)
0x0A0 – 0x0BC	LPDDR2 calibration / training control (`CAL_CTRL`,

Address range	Region
	CAL_ZQ_INTERVAL, CAL_ZQ_TIMING, CAL_TRAIN_TIMING, CAL_MRR32_DATA[4])
0x0E0 – 0x0F0	CAL_MRR40_DATA[4] + CAL_STATUS (training telemetry)
0x0C0 – 0x0DC	Retired — was OBS_REF_LATENCY[8]; removed 2026-09-28 (BUG-020), never reused
0x100 – 0x138	System observation / telemetry
0x1C0 – 0x1E0	Packed observation-word harvest (9 words)
0xFF0 – 0xFF4	Module identification + build hash

The complete register/field table is in §4.2; it is generated from the RDL and must match it exactly.

39.4 Behavior

39.4.1 Access Cycles

Each request drives s_cpuif_req with s_cpuif_req_is_wr, s_cpuif_addr, and (for writes) s_cpuif_wr_data/s_cpuif_wr_biten. The block responds with s_cpuif_rd_ack/s_cpuif_wr_ack (and optionally asserts a stall while it is busy). Reads return s_cpuif_rd_data; unmapped accesses assert s_cpuif_rd_err/s_cpuif_wr_err.

39.4.2 Sub-Word Accesses

s_cpuif_wr_biten carries a per-bit write mask, so byte- or field-level writes are supported natively. A bridge that lacks byte strobes simply drives all biten bits on every write.

39.4.3 Error Conditions

The block asserts a decode error (s_cpuif_rd_err / s_cpuif_wr_err) for:

Condition	Notes
Access to an unmapped address (gaps / RSVD)	rd_err/wr_err; read data is 0
Write to a read-only register/field	Write is dropped; field unchanged

Writes to reserved (sw = r) fields inside an otherwise-writable register are ignored per the RDL field access; the register keeps its value. There is no scheme/rule sub-decode error (the old ADDR_MAP_TUNING scheme decode is retired — address mapping is now the plain ADDR_MAP.bank_lsb field, see §4.2/§4.4).

39.5 Config-Drive Model

`pumice_top` wires the register block's `hwif_out.*` outputs **by name** directly into `pumice_core` (see the top-level instantiation in `rtl/top/pumice_top.sv`). Configuration is therefore live combinational drive from the register storage into the datapath — software writes a field and the corresponding core input tracks it. There is no staging/commit or quiet-point protocol in this architecture (see §4.3). Status/observation readback (`hwif_in.*`) is currently tied off in `pumice_top`.

39.6 Open Questions / Future Work

- **Observation readback.** `hwif_in` is presently tied to 0; wiring the live status/telemetry counters back into the read path is a follow-up.
- **Bridge selection.** The passthrough cpuif is transport-agnostic. A packaged APB and AXI-Lite bridge pair (with the CDC option) would simplify SoC integration.
- **Quiet-point commit.** The old two-cell staging/commit model was removed with the config-drive refactor. If a future field must not change mid-transaction, a per-field commit gate can be reintroduced.

---

40 Register Map

This MAS chapter is the **authoritative field-level register map**, transcribed directly from `rtl/macro/pumice_csr.rdl` and mirrored by the generated `dv/tbclasses/pumice_regmap.py`. If this table and the RDL ever disagree, the RDL wins. Offsets are byte addresses in the 4 KB (12-bit) region.

40.1 Source of Truth

The register map is a SystemRDL source, `rtl/macro/pumice_csr.rdl`. It is compiled by `bin/peakrdl_generate.py` into:

Artifact	Consumer
`regs/generated/rtl/pumice_csr.sv` + `pumice_csr_pkg.sv`	The PeakRDL passthrough register block instantiated in `pumice_top` (`hwif_in`/`hwif_out` structs)
`dv/tbclasses/pumice_regmap.py`	The DV `RegisterMap` by-name access model (offset/field/default)

`pumice_top` drives `hwif_out.*` fields **by name** straight into `pumice_core` — there is no hand-written register file and no APB slave inside the controller (see §4.1). DV programs every register by name via `pumice_regmap.py`; hardcoded offsets are forbidden.

40.2 Register Summary

Offset	Register	Purpose
0x000	`CTRL`	Init / power / soft-reset request bits
0x004	`STATUS`	Init / power / version status (RO, hw-written)
0x008	`STATUS_HISTORY`	Last 8 power-state transitions (RO)
0x010	`TIMINGS_RC_RCD_RP_RAS`	tRC / tRCD / tRP / tRAS
0x01	`TIMINGS_RFC_REFI`	tRFC / tREFI

Offset	Register	Purpose
4		
0x018	TIMINGS_RRD_FAW_WTR_CCD	tRRD / tFAW / tWTR / tCCD
0x01C	TIMINGS_CL_CWL_WR	CL / CWL / tWR / tRFCpb
0x020	MR0	Mode Register 0 value
0x024	MR1	Mode Register 1 value
0x028	MR2	Mode Register 2 value
0x02C	MR3	Mode Register 3 value
0x030	PASR_BANK_MASK_RANK0	LPDDR2 PASR per-bank mask (rank 0)
0x034	PASR_SEG_MASK_RANK0	LPDDR2 PASR segment mask (rank 0)
0x038	TEMP_DERATE_RANK0	LPDDR2 MR4 temperature class (rank 0, RO)
0x040	SCHED_TUNING	Retired — reserved in full (scheduling is SCHED_POLICY)
0x044	(unmapped)	was PAGE_PRED_TUNING — retired with the HAPPY predictor
0x048	REFRESH_TUNING	Page-policy override (the refresh fields are retired)
0x04C	ADDR_MAP	Address-map: bank_lsb + XOR-hash (replaces ADDR_MAP_TUNING)
0x068	SCHED_POLICY	Axis 1 scheduling: order_mode / prio_sub / row_sel / col_sel / access_pref / qos_en / age_thresh
0x06C	SCHED_WR_WM	Write-batching drain watermarks

Offset	Register	Purpose
0x070	PAGE_POLICY_CFG	Axis 2 paging mode select + adapt_access counter shape
0x074	PAGE_TIMEOUT_CFG	fixed_open / adapt_time timeout bounds
0x078	PAGE_ADAPT_CFG	adapt_time mistake-counter thresholds
0x07C	*(retired)*	was PAGE_RBL_CFG; left a HOLE, not reused – an old host reading it gets nothing rather than another register's meaning
0x140	REF_CTRL	Axis 3 refresh mode + JEDEC postpone/pull-in credits
0x144	REF_TIMING_PB	REFpb intervals
0x050	INIT_TUNING	ZQ retries + per-step init timeout
0x054	TIMINGS_RTP_RTW	tRTP / tRTW
0x058	INIT_TIMING0	Init waits: tINIT / tDLLK
0x05C	INIT_TIMING1	Init waits: tMRD / tRP / tRFC
0x060	DFI_PHASE	DFI READ/WRITE command sub-phase placement
0x064	PHY_TIMING	t_phy_wrlat / t_rddata_en / memtype / refresh_burst
0x080..0x09C	OBS_ROW_HIT[8]	Per-bank row-hit count (RO, free-running)
0x0A0	CAL_CTRL	ZQ + DQ calibration control strobes and enables
0x0A4	CAL_ZQ_INTERVAL	ZQCS interval in MC cycles, 0 = disabled

Offset	Register	Purpose
0x0A8	CAL_ZQ_TIMING	ZQ post-grant hold windows (t_zqcs / t_zqcl)
0x0AC	CAL_TRAIN_TIMING	MRR spacing (t_mrr) and readout timeout (t_readout)
0x0B0..0x0BC	CAL_MRR32_DATA[4]	Captured MRR-MR32 beat 0, 32-bit slices (RO)
0x0E0..0x0EC	CAL_MRR40_DATA[4]	Captured MRR-MR40 beat 0, 32-bit slices (RO)
0x0F0	CAL_STATUS	Training-layer telemetry (busy / done / err / overdue / count)
0x0C0..0x0DC, 0x100..0x1E0	*(retired holes)*	27 OBS_* registers removed 2026-09-28 — see BUG-020
0xFF0	ID	Module ID (version / memtype / n_phases / 0xD2)
0xFF4	BUILD	Build hash

40.3 Field-Level Detail

All registers are 32-bit; unlisted bits are reserved (RSVD, sw = r). “Default” is the RDL reset value.

40.3.1 CTRL @ 0x000 (rw)

Bits	Field	Default	Notes
0	init_start	0	Write 1 to start init (swmod)
1	init_force_restart	0	Write 1 to force re-init mid-sequence (swmod)
4	pwr_req_low_power	0	Request power-down
5	pwr_req_dpd	0	Request DPD (LPDDR2 only)

Bits	Field	Default	Notes
6	pwr_req_active	0	Request return to ACTIVE
7	pwr_req_self_refresh	0	Request self-refresh
31	soft_reset	0	Write 1 to assert internal soft reset (self-clearing, swmod)

40.3.2 STATUS @ 0x004 (RO, hw-written)

Bits	Field	Notes
0	init_done	Init complete
1	init_error	Init error
7:4	power_state	Current power-state FSM state (encoded)
8	pasr_active	LPDDR2: PASR mask is non-zero
23:16	init_step_dbg	Current init step number (bring-up)
31	version_match	Build matches expected version

40.3.3 STATUS_HISTORY @ 0x008 (RO)

Bits	Field	Notes
31:0	history	8 x 4-bit power-state history; most recent in [3:0]

40.3.4 TIMINGS_RC_RCD_RP_RAS @ 0x010 (rw)

Bits	Field	Default (dec)	Notes
7:0	tRC	60	MC cycles
15:8	tRCD	15	
23:16	tRP	15	
31:24	tRAS	40	

40.3.5 TIMINGS_RFC_REFI @ 0x014 (rw)

Bits	Field	Default (dec)	Notes
15:0	tRFC	16	or tRFCab; mission-mode REF recovery (arbiter

Bits	Field	Default (dec)	Notes
			down-counter)
31:16	tREFI	1950	

40.3.6 TIMINGS_RRD_FAW_WTR_CCD @ 0x018 (rw)

Bits	Field	Default (dec)
7:0	tRRD	6
15:8	tFAW	35
23:16	tWTR	4
31:24	tCCD	4

40.3.7 TIMINGS_CL_CWL_WR @ 0x01C (rw)

Bits	Field	Default (dec)	Notes
7:0	CL	6	CAS latency
15:8	CWL	4	CAS write latency
23:16	tWR	15	Write recovery
31:24	tRFCpb	70	LPDDR2 per-bank tRFC

40.3.8 MR0..MR3 @ 0x020 / 0x024 / 0x028 / 0x02C (rw)

Bits	Field	Default	Notes
15:0	VAL	0	Mode-register value loaded during init

40.3.9 PASR_BANK_MASK_RANK0 @ 0x030 (rw)

Bits	Field	Default	Notes
7:0	pasr_banks	0	LPDDR2 MR16; bit N=1 masks bank N

40.3.10 PASR_SEG_MASK_RANK0 @ 0x034 (rw)

Bits	Field	Default	Notes
7:0	pasr_segs	0	LPDDR2

Bits	Field	Default	Notes
			segment mask

40.3.11 TEMP_DERATE_RANK0 @ 0x038 (RO, hw-written)

Bits	Field	Notes
1:0	temp_class	LPDDR2 MR4: 00 nominal, 01 2x refresh, 10 4x refresh

40.3.12 SCHED_TUNING @ 0x040 (retired — reserved in full)

Bits	Field	Default	Access	Notes
31:0	RSVD	0	r	Reserved (retired scheduler knobs)

RETIRED 2026-09-09. Every field of this register belonged to the pre-rearchitecture scheduler: `lookahead_active`, `force_inorder`, `age_max_runtime`, `txn_queue_high_water` and the `lookahead_max_obs` echo (plus `happy_enable`, retired earlier with the HAPPY predictor). The CAM + arbiter scheduler has never read any of them, so a write here was a silent no-op — the host's `inorder` characterization preset was programming this bit and getting FR-FCFS on silicon.

Scheduling is SCHED_POLICY @ 0x068: `order_mode` selects in_order / age_threshold, and FR-FCFS reorders across the whole CAM so there is no lookahead window to size. The address is kept reserved rather than reused so the map does not shift.

40.3.13 PAGE_PRED_TUNING @ 0x044 (rw)

Bits	Field	Default (dec)	Notes
15:0	warmup_cycles	1024	
23:16	hysteresis	2	

40.3.14 REFRESH_TUNING @ 0x048 (rw)

Bits	Field	Default	Access	Notes
1:0	RSVD_1_0	0	r	Reserved (was `refpb_policy_or`)
3:2	page_policy_or	0	rw	00 build-time, 01 OPEN, 10 CLOSE, 11 reserved (was HYBRID)
15:4	RSVD_15_4	0	r	Reserved (was `refresh_defer_active`)

Bits	Field	Default	Access	Notes
31:16	RSVD_31_16	0	r	Reserved (was zqcs_freq_hz)

PARTIALLY RETIRED 2026-09-09. page_policy_or is live and is the static OPEN/CLOSE override. The three refresh fields are not: refresh mode and the JEDEC postpone/pull-in credits are REF_CTRL @ 0x140, and no ZQCS engine ever consumed the interval.

40.3.15 ADDR_MAP @ 0x04C (rw) — replaces the retired ADDR_MAP_TUNING

Bits	Field	Default	Notes
4:0	bank_lsb	0x0A (=COL_WIDTH, ROW_MAJOR)	Bank-field LSB in the byte-offset-stripped word address; RTL clamps to [0, COL_WIDTH]
8	hash_en	0	Enable bank XOR-hash: bank ^= fold(row) ^ hash_seed
23:16	hash_seed	0	XOR-hash seed (seed[BW-1:0])

There is no scheme selector, scheme_or, or synth_mask_obs. ROW_MAJOR / BANK_INTERLEAVE / XOR_HASH are just settings of this one register (see §4.4 and rtl/fub/addr_mapper.sv).

40.3.16 INIT_TUNING @ 0x050 (rw)

Bits	Field	Default (dec)	Notes
3:0	zq_retries	3	ZQ retries (1..8)
15:8	init_timeout_ms	10	Init timeout ms

40.3.17 TIMINGS_RTP_RTW @ 0x054 (rw)

Bits	Field	Default (dec)	Notes
7:0	tRTP	4	Read to

Bits	Field	Default (dec)	Notes
			precharge
15:8	tRTW	6	Read to write

40.3.18 INIT_TIMING0 @ 0x058 (rw)

Bits	Field	Default (dec)	Notes
15:0	t_init_wait	512	CKE/tINIT settle
31:16	t_dll_wait	256	DLL lock (tDLLK)

40.3.19 INIT_TIMING1 @ 0x05C (rw)

Bits	Field	Default (dec)	Notes
7:0	t_mrd_wait	8	post mode-reg (tMRD)
15:8	t_rp_wait	8	post precharge (tRP)
23:16	t_rfc_wait	16	post refresh (tRFC)

40.3.20 DFI_PHASE @ 0x060 (rw)

Bits	Field	Default	Notes
2:0	rd_phase	0	READ command DFI sub-phase
6:4	wr_phase	0	WRITE command DFI sub-phase

Sliced to `clog2(DFI_RATE)` bits downstream; upper bits ignored when `DFI_RATE` is small. On the Nexys A7 a7ddrphy, rd_phase=0 (the PHY handles rdphase internally).

40.3.21 PHY_TIMING @ 0x064 (rw)

Bits	Field	Default (dec)	Notes
7:0	t_phy_wrlat	0	WR cmd -> dfi_wrdata_en (Nexys A7 bring-up tuple programs 1)

Bits	Field	Default (dec)	Notes
15:8	t_rddata_en	6	RD cmd -> dfi_rddata_en window
16	memtype	0	0 = DDR2, 1 = LPDDR2
23:20	refresh_burst	1	REFs drained per request (1..8)

40.3.22 Advanced-mode registers (0x068-0x07C, 0x140-0x144)

Added to this chapter 2026-09-09. These registers have existed in `pumice_csr.rdl` since the TASK-001 mode work and are cited by ch02/07 Command Arbiter and ch02/08 Page Policy, but the register map never listed them. They are the three mode axes: scheduling (`SCHED_POLICY`, `SCHED_WR_WM`), paging (`PAGE_*_CFG`) and refresh (`REF_*`). Every field encodes 0 as "build default", so a zeroed register block is bit-identical to the pre-mode controller.

40.3.23 SCHED_POLICY @ 0x068 (rw)

Axis 1 (Rixner FR-FCFS variants). All fields 0 = build default.

Bits	Field	Default	Access	Notes
1:0	order_mode	0x0	rw	0=build default, 1=in_order, 2=fr_fcfs, 3=age_threshold
3:2	prio_sub	0x0	rw	Priority sub-policy: 0=default, 1=none, 2=load_over_store, 3=age_boost
5:4	row_sel	0x0	rw	Row-arbiter select: 0=default(oldest), 1=most_pen

Bits	Field	Default	Access	Notes
				ding, 2=fewest_pending
7:6	col_sel	0x0	rw	Column-arbiter select: 0=default(oldest), 1=most_pending, 2=fewest_pending
9:8	access_pref	0x0	rw	Address-arbiter class preference: 0=default, 1=column_first, 2=row_first, 3=precharge_first
10:10	RSVD_10	0x0	r	Reserved (was auto_precharge_en, never consumed: auto-precharge is driven by PAGE_POLICY_CFG.policy_mode – static_close and the modes 5..7

Bits	Field	Default	Access	Notes
				predictors)
11:11	qos_en	0x0	rw	1 = factor AxQOS into the pick (highest ready first, age tie-break)
15:12	RSVD_15_12	0x0	r	Reserved
23:16	age_thresh	0x0	rw	age_threshold mode: age (MC cycles/16) above which a reference is boosted
31:24	RSVD_31_24	0x0	r	Reserved

40.3.24 SCHED_WR_WM @ 0x06C (rw)

Write-batching drain hysteresis. 0/0 = build default (no batching).

Bits	Field	Default	Access	Notes
7:0	wr_high_wm	0x0	rw	Start back-to-back write drain when the write buffer crosses this
15:8	wr_low_wm	0x0	rw	Stop the drain when occupancy falls to this
31:16	RSVD	0x0	r	Reserved

40.3.25 PAGE_POLICY_CFG @ 0x070 (rw)

Axis 2 mode select + adapt_access counter shape. 0 = build default.

Bits	Field	Default	Access	Notes
2:0	policy_mode	0x0	rw	0=build default, 1=static_open, 2=static_close, 3=fixed_open, 4=adapt_time, 5=adapt_access, 6/7 RETIRED 2026-09-26 (were rbl_static/rbl_dyn)
3:3	policy_scope	0x0	rw	0 = per-bank decision state, 1 = global
5:4	RSVD_5_4	0x0	r	Reserved (was ctr_width; the adapt_access counter is the 2-bit saturating counter of the paper, not selectable)

Bits	Field	Default	Access	Notes
9:6	ctr_open_max	0x0	rw	adapt_access : counter value at/above which the row is CLOSED
13:10	ctr_init	0x0	rw	adapt_access : counter init value
31:14	RSVD	0x0	r	Reserved

40.3.26 PAGE_TIMEOUT_CFG @ 0x074 (rw)

fixed_open / adapt_time timeout register bounds (MC cycles).

Bits	Field	Default	Access	Notes
7:0	tr_init	0x0	rw	TR init (fixed_open uses this alone; ~tRC)
15:8	tr_min	0x0	rw	adapt_time TR lower clamp
23:16	tr_max	0x0	rw	adapt_time TR upper clamp
31:24	tr_step	0x0	rw	adapt_time TR adjustment step

40.3.27 PAGE_ADAPT_CFG @ 0x078 (rw)

adapt_time (Happy adaptive-timeout) mistake-counter thresholds.

Bits	Field	Default	Access	Notes
3:0	mc_high_thr	0x0	rw	MC high

Bits	Field	Default	Access	Notes
				threshold (TR += step above)
7:4	mc_low_thr	0x0	rw	MC low threshold (TR -= step below)
11:8	mc_init	0x0	rw	MC init value
15:12	RSVD	0x0	r	Reserved
31:16	check_interval	0x0	rw	Cycles between MC evaluations

40.3.28 0x07C — retired

Was PAGE_RBL_CFG (RBLA miss-counter table shape). Modes 6/7 (rbl_static/rbl_dyn) were **RETIRED 2026-09-26**: measured on silicon at txn_scale=1000 on a workload built specifically to suit them (TASK-011), mode 6 lost 26% of bandwidth (195.2 -> 144.2 MB/s) by paying +22,827 ACTs to save precharges that never materialised, and mode 7's hill-climb drove its threshold to "never close early", landing bit-identical to plain open page (pre-sdpram-fix 2026-10-02; off-datapath fix, reproduced 2026-10-03 — FINDINGS_litedram_ab_2026-10-03.md). A write of 6 or 7 now falls through to the build default.

40.3.29 REF_CTRL @ 0x140 (rw)

Axis 3 mode + JEDEC +-8 credit limits. tREFI/tRFCab live in TIMINGS_RFC_REFI (not duplicated).

Bits	Field	Default	Access	Notes
1:0	mode	0x0	rw	0=build default (REFab), 1=REFab, 2=REFpb round-robin (LPDDR2)
3:2	RSVD_3_2	0x0	r	Reserved
7:4	postpone_limit	0x0	rw	Max

Bits	Field	Default	Access	Notes
				refreshes postponed under demand (0..8; 0 = strict)
11:8	pullin_limit	0x0	rw	Max refreshes pulled in on idle (0..8; 0 = strict)
12:12	perbank_supported	0	r	Capability strap: 1 = the DRAM supports per-bank refresh
31:13	RSVD_31_13	0x0	r	Reserved

40.3.30 REF_TIMING_PB @ 0x144 (rw)

REFpb intervals (MC cycles). All-bank tREFI/tRFCab stay in TIMINGS_RFC_REFI.

Bits	Field	Default	Access	Notes
15:0	trefi_pb	0x0	rw	tREFIpb (~tREFI/8; 0 = derive from tREFI)
23:16	trfc_pb	0x0	rw	tRFCpb recovery
31:24	RSVD	0x0	r	Reserved

40.3.31 LPDDR2 Calibration / Training registers (0x0A0–0x0F0)

Added to this chapter at rev 0.8 with the training layer (`pumice_training_layer`: `pumice_zq_ctrl` + `pumice_lp_cal`). The window sits in the free range between `OBS_ROW_HIT` (0x080–0x09F) and the retired 0x0C0 hole; retired holes are never reused (pumice doctrine).

Field names, widths, and reset values below are transcribed from `rtl/macro/pumice_csr.rdl` — the RDL wins on any drift. See ch02/23, ch02/24, and ch04/05.

40.3.32 CAL_CTRL @ 0x0A0 (rw)

Bits	Field	Default	Access	Notes
0	zq_en	0	rw	Enable periodic ZQCS/ZQCL
1	zq_defer_en	0	rw	Mode-C deferral: hold ZQ under demand
3:2	RSVD_3_2	0	r	Reserved
4	cal_start	0	rw	Write 1 to start a DQ-cal sequence (self-clearing command, swmod)
5	cal_abort	0	rw	Write 1 to abort the current DQ-cal sequence (swmod)
15:6	RSVD_15_6	0	r	Reserved
28:16	zq_overdue_max	0	rw	Max ZQ deferral cycles before overdue (0 = no cap)
31:29	RSVD_31_29	0	r	Reserved

The `cal_start`/`cal_abort` bits are write-one strobes: they self-clear and never read back set. Run gating lives in the FUBs (`zq_en` additionally requires `init_done` + LPDDR2 + nonzero interval; `cal_start` requires `init_done` + LPDDR2), so writing `cal_start` on a DDR2 build is a silent no-op.

40.3.33 CAL_ZQ_INTERVAL @ 0x0A4 (rw)

Bits	Field	Default	Access	Notes
31:0	VAL	0	rw	ZQCS interval in MC cycles, 0 = disabled

40.3.34 CAL_ZQ_TIMING @ 0x0A8 (rw)

Bits	Field	Default	Access	Notes
15:0	t_zqcs	0	rw	ZQCS post-grant hold (bus-quiet window)
31:16	t_zqcl	0	rw	ZQCL post-grant hold

Both holds count mc_clk cycles from the grant; the busy indication folds into the arbiter's `w_out_safe` so demand-class traffic cannot fire inside the window.

40.3.35 CAL_TRAIN_TIMING @ 0x0AC (rw)

Bits	Field	Default (dec)	Access	Notes
15:0	t_mrr	2	rw	MRR-to-MRR spacing (MC cycles)
31:16	t_readout	0	rw	MRR issue → data timeout

`t_mrr = 2` matches JEDEC tMRR. A `t_readout` of 0 makes the lp_cal wait-data states time out on the first cycle without data — program a real window (covering `RL·tCK + tDQSCK + tDQSQ`) before starting a calibration.

40.3.36 CAL_MRR32_DATA[4] @ 0x0B0 += 0x4 (RO)

Bits	Field	Default	Access	Notes
31:0	VAL	0	r	32-bit slice of captured MRR32 data (pattern A)

First captured DFI read-data beat for MRR MR32, as four 32-bit slices (`CAL_MRR32_DATA0..3` at 0x0B0/0x0B4/0x0B8/0x0BC). Hardware-written; reset value 0.

40.3.37 CAL_MRR40_DATA[4] @ 0x0E0 += 0x4 (RO)

Bits	Field	Default	Access	Notes
31:0	VAL	0	r	32-bit slice of captured MRR40 data (pattern B)

First captured DFI read-data beat for MRR MR40, as four 32-bit slices (`CAL_MRR40_DATA0..3` at 0x0E0/0x0E4/0x0E8/0x0EC).

40.3.38 CAL_STATUS @ 0x0F0 (RO, hw-written)

Bits	Field	Default	Notes
0	zq_busy	0	ZQ calibration busy (post-grant hold window)
1	cal_busy	0	DQ calibration busy (lp_cal FSM not idle)
2	cal_done	0	DQ calibration done (sticky)
3	cal_err	0	DQ calibration error (sticky)
4	zq_overdue	0	ZQ interval expired without grant
15:5	RSVD_15_5	0	Reserved
31:1	zqcs_total	0	ZQCS/ZQCL commands issued since reset

Bits	Field	Default	Notes
6			

Sticky means what it says: `cal_done`/`cal_err` survive `cal_abort` and are cleared by soft reset (a fresh `cal_start` also re-arms them). The full DFI-width captured beats land in `CAL_MRR32_DATA` / `CAL_MRR40_DATA`; the x16 DQ[8] pattern-A mirror is in the captured data, not in this register.

40.3.39 Observation registers (RO)

Offset	Register / array	Field	Notes
0x080..0x09C	`OBS_ROW_HIT[8]`	`VAL[31:0]`	Per-bank ROW HITS: column ops issued to a bank whose row was already open. Free-running; subtract two reads for a window
0x0C0..0x0DC	*(retired)*	—	was `OBS_REF_LATENCY[8]` — never driven, no consumer; removed 2026-09-28 (BUG-020)
0x100..0x118	*(retired)*	—	was `OBS_TXN_QUEUE_DEPTH_*` / `OBS_REFRESH_PENDING_MAX` / `OBS_REFRESH_DEFER_HIST_0..3` — never driven (BUG-020)
0x120	*(retired)*	—	was `OBS_PAGE_PRED_ACCURACY` — retired with the HAPPY predictor; see `PAGE_STATS_*`
0x130..0x138	*(retired)*	—	was `OBS_AXI_{R,W}_LATENCY_*` — AXI latency belongs to an external meter (`axi_bus_meter`, `axi_perf_latency_hist`), not the controller (BUG-020)
0x1C0..0x1E0	*(retired)*	—	was `OBS_WORDS[9]` — its aggregator was deleted in e8908eebf (2026-07-21); the window outlived its source (BUG-020)

0x148 | `PAGE_STATS_HIT` | `VAL` | EVERY column op issued — **not** a hit count despite the name. For row hits read `OBS_ROW_HIT[8]`. Do **not** derive them as `PAGE_STATS_HIT -`

SCHED_STATS_ACT: that goes negative under a background-close mode, because a row can be opened, timed out and reopened before its column command issues (ISSUE-014) |
0x14C | PAGE_STATS_MISS | VAL | ACT after a conflict PRE (row thrash) |
0x150 | PAGE_STATS_EMPTY | VAL | ACT to an idle bank (cold open) |
0x154 | SCHED_STATS_ACT | VAL | ACT commands issued |
0x158 | SCHED_STATS_PRE | VAL | PRE + PREA commands issued |
0x15C | REF_STATS_REF | VAL | REF commands issued — **FREE-RUNNING**, see the note below |
0x160 | STALL_BP | VAL | picked, DFI said no |
0x164 | STALL_REFRESH | VAL | blocked by a refresh request/drain |
0x168 | STALL_TURNAROUND | VAL | blocked by tWTR / tRTW |
0x16C | STALL_TCCD | VAL | blocked by tCCD |
0x170 | STALL_ACTLIMIT | VAL | blocked by tFAW / tRRD |
0x174 | STALL_BANKTIMER | VAL | blocked by per-bank tRCD/tRP/tRAS, incl. final-stage rejects |
0x178 | STALL_NOREQ | VAL | nothing to issue — **contaminated by host idle**, see below |
0x17C | *(retired)* | — | was PAGE_RBL_CFG |
0x180 | REF_STATS_REF_BUSY | VAL | REF commands issued **with work pending** |

40.3.39.1 Reading the stall counters

The seven STALL_* registers are a **priority classification** of every cycle the arbiter did not fire: each stalled cycle is charged to exactly one cause, tested in the order listed. Two consequences that have both produced wrong conclusions:

- **STALL_NOREQ advances during host idle.** It ticks whenever the arbiter has nothing to do, which includes all the UART round-trip time between two host reads. A host-bracketed delta therefore reads ~99% idle regardless of what the workload did. The other six sit after the !w_any_pending branch and can only tick with work pending, so they are clean and comparable; **rank those six and leave noreq out.**
- **A high share proves a term was UNSATISFIED, never that relieving it would help.** static_close reads actlimit at 70% of stalls, yet relaxing tFAW/tRRD from 6/2 to 1/1 leaves both throughput and the counter bit-identical. The classifier has no category for pipeline serialisation, so those cycles fall through to whichever timing term happens to be unsatisfied. **Confirm every stall verdict by moving the knob.**

40.3.39.2 REF_STATS_REF vs REF_STATS_REF_BUSY

Refresh is autonomous, so REF_STATS_REF free-runs and a host-bracketed delta times the UART round trips rather than the workload — measured on the board, a 186 us window carried a delta implying 479 ms, a **2584x** overstatement, and the number was near-identical across every config precisely because it was timing the HOST.

REF_STATS_REF_BUSY @ 0x180 counts only refreshes that issued **with at least one CAM entry schedulable**. That is contamination-free by construction — the CAMs are empty while the host is

idle — and it is also the number worth having: a refresh during idle costs the workload nothing, one during traffic costs bandwidth. On silicon the two read **8,814 vs 2,570,069**, a 292x ratio. Use BUSY to attribute refresh cost; use the free-running one for absolute accounting only.

40.3.40 ID @ 0xFF0 (RO) — reset 0xD2020001

Bits	Field	Value	Notes
7:0	version	0x01	Build version
15:8	memtype	0x00	0 = DDR2, 1 = LPDDR2
23:16	n_phases	0x02	Gear ratio (1/2/4)
31:24	module_id	0xD2	Fixed 0xD2

40.3.41 BUILD @ 0xFF4 (RO)

Bits	Field	Default	Notes
31:0	VAL	0	Build hash word

40.4 Multi-Rank / Multi-Bank Registers

The RDL declares per-bank observation arrays with `NUM_BANKS = 8` for rank 0 (`OBS_ROW_HIT[8]` at 0x080, `OBS_REF_LATENCY[8]` at 0x0C0). PASR / temperature registers are declared per rank as `*_RANK0` for the default single-rank build. The generated `pumice_regmap.py` flattens arrays into indexed register names (e.g., `OBS_ROW_HIT0_ROW_HIT`, `OBS_REF_LATENCY7_REF_LAT`, `OBS_WORDS8_WORD`). Multi-rank builds add the corresponding `*_RANK{N}` registers; there is no separate capability vector register in this RDL — software reads memtype/n_phases from `ID` (0xFF0).

40.5 Reset Values

Reset defaults come straight from the RDL `= <value>` initializers (echoed in the `default`/`offset` entries of `pumice_regmap.py`). They are chosen so a "do nothing" bring-up sees a sane DDR2 baseline; DV programs the workload-specific values by name before triggering init.

40.6 Open Questions / Future Work

- **Observation readback wiring.** The RO/telemetry registers are declared and generated, but `pumice_top` currently ties `hwif_in` to 0 (see §4.1). Connecting the live counters is a follow-up.

- **Multi-rank register generation.** The single-rank build declares only `*_RANK0`. A `NUM_RANKS`-driven RDL loop for the PASR/temp/observation windows is the natural extension.
- **RDL <-> regmap drift check.** `pumice_regmap.py` is generated from the RDL; a CI diff would catch manual edits.

---

41 Runtime CSR Fields (Config-Drive Model)

Per HAS §5.1 for the build-time-vs-runtime principle and §4.2 for the field-level register map. This MAS chapter is the **implementation protocol** for how runtime CSR fields reach the live datapath.

In the rearchitected controller, `pumice_top` wires the register block's `hwif_out.*` fields **by name** directly into `pumice_core` (see `rtl/top/pumice_top.sv`). There is **no** staging/commit two-cell architecture, **no** `CTRL.config_apply`, **no** `STATUS.config_settled`, and **no** quiet-point drain FSM. A CSR write updates the field's storage and the corresponding core input tracks it combinationally.

41.1 What "Runtime" Means Here

Most of the controller's behavior is programmed at runtime by CSR fields, not fixed by parameters. The following are all live CSR-driven core inputs (see the `u_core` port map in `rtl/top/pumice_top.sv`):

Domain	CSR field(s)	Core input
Memory type	`PHY_TIMING.memtype`	`memtype_i` (0=DDR2, 1=LPDDR2)
Page policy	`REFRESH_TUNING.page_policy_or`	`page_policy_i`
Address mapping	`ADDR_MAP.bank_lsb` / `.hash_en` / `.hash_seed`	`bank_lsb_i` / `hash_en_i` / `hash_seed_i`
Core JEDEC timing	`TIMINGS_RC_RCD_RP_RAS`, `TIMINGS_CL_CWL_WR.tWR`, `TIMINGS_RTP_RTW`, `TIMINGS_RRD_FAW_WTR_CCD`, `TIMINGS_RFC_REFI.tREFI`	`t_rcd_i`/`t_rp_i`/ `t_ras_i`/`t_rc_i`/ `t_wr_i`/`t_rtp_i`/ `t_rtw_i`/`t_faw_i`/ `t_rrd_i`/`t_wtr_i`/ `t_ccd_i`/`t_refi_i`
Refresh burst	`PHY_TIMING.refresh_burst`	`refresh_burst_i`
Init waits	`INIT_TIMING0` / `INIT_TIMING1`	`t_init_wait_i`/ `t_dll_wait_i`/

Domain	CSR field(s)	Core input
		t_mrd_wait_i/ t_rp_wait_i/ t_rfc_wait_i
DFI phase	DFI_PHASE.rd_phase / .wr_phase	rd_phase_i/wr_phase_i (sliced to clog2(DFI_RATE))
PHY data timing	PHY_TIMING.t_phy_wrlat / .t_rddata_en	t_phy_wrlat_i/ t_rddata_en_i

41.1.1 What a JEDEC timing value MEANS: blocking cycles, so spacing is N+1

Every JEDEC timing field is a count of **MC (controller) cycles**, and the hardware treats it as a count of cycles to BLOCK. A window programmed to N is therefore enforced as **N+1 cycles of command spacing**: the counter is loaded with N when the event happens, and the gate opens the cycle after it would reach zero.

This holds for all ten enforced windows, in both timing blocks:

Block	Windows
bank_timer (per bank)	tRC, tRP, tRCD, tRAS, tRTP, tWR
global_timers (per rank / global)	tRRD, tCCD, tWTR, tRTW

It is measured, not asserted: formal/pumice/bank_timer and formal/pumice/global_timers carry a_*_bound_n1 properties proving the bound, and cover statements proving it is TIGHT (exactly N+1, never more – the same assertions with +2 fail on every window).

Nothing compensates for the extra cycle. The board host programs raw JEDEC-derived cycle counts with no -1 (build-perf/host/pumice_device.py: tCCD=ck(DDR2_CK_MIN["tCCD"]), tRCD=ns(part["tRCD"])), so each window on silicon is one MC cycle longer than JEDEC requires – 13.3 ns at 75 MHz. That is always SAFE, never a violation, and it costs bandwidth. Whether to recover it is pumice TASK-034, and it is a board measurement rather than a code change: the empirical rtw_guard in the tRTW derivation was tuned WITH this cycle present.

If you are programming a part from its datasheet, write the JEDEC value. Do not subtract one to "correct" for this – the CSR would then stop reading as the part's timing, which is worse than the cycle it saves.

CL/CWL/BL are **not** driven from the timing CSRs at the core boundary — they are decoded from the mode-register shadow inside the scheduler layer (mode_register.sv) as the init sequencer programs MR0..MR3 (see §5.1). The TIMINGS_CL_CWL_WR.CL/.CWL fields are informational in this build.

41.2 Commit Semantics

Because the fields drive the core directly, a write takes effect as soon as it lands in the register storage. There is no explicit apply step. Two practical timing classes:

Class	Fields	When it takes effect
Sampled at an event boundary	JEDEC timings, refresh interval/burst, page policy, scheduler knobs	On the next counter reload / arbiter decision that consumes the value
Sampled continuously	`memtype`, `bank_lsb`/`hash_*`, DFI phase, PHY data timing	Combinationally; affects the next command that uses the mapper/formatter

41.3 Safe-Change Guidance

The controller does not enforce a quiet point, so **software** is responsible for not changing a field mid-transaction when that would corrupt in-flight state:

- **Timings, page policy, scheduler knobs, refresh interval**: safe to change during light traffic; the new value is picked up at the next event boundary. For a clean swap, quiesce AXI traffic first (stop issuing, let outstanding responses drain) then write.
- **`memtype`, address-map (`bank_lsb`/`hash_*`)**: these define how addresses decode and how commands are encoded. Change them **only before init** (or with the datapath fully idle). Changing address mapping under traffic re-decodes in-flight addresses inconsistently.
- **DFI phase / PHY data timing**: board bring-up knobs; set once during bring-up and leave fixed. `rd_phase`/`t_rddata_en`/`t_phy_wrlat` are matched to the attached PHY (the Nexys A7 a7ddrphy bring-up tuple, board-validated 2026-07-21: `rd_phase=0`, `t_rddata_en=6`, `t_phy_wrlat=1`, plus harness `DFI_TUNING.rddata_delay=7`).

41.4 Recommended Programming Order

For a clean bring-up the register writes happen while init is still gated (before `CTRL.init_start`):

```
// 1. Select memory type and address mapping (must be set before init)
csr_write(PHY_TIMING, memtype_field | t_rddata_en_field |
t_phy_wrlat_field | refresh_burst_field);
```

```
csr_write(ADDR_MAP,   bank_lsb_field | hash_en_field |
hash_seed_field);

// 2. Program JEDEC timings for the attached part
csr_write(TIMINGS_RC_RCD_RP_RAS, ...);
csr_write(TIMINGS_RFC_REFI,      ...);
csr_write(TIMINGS_RRD_FAW_WTR_CCD, ...);
csr_write(TIMINGS_CL_CWL_WR,     ...);
csr_write(TIMINGS_RTP_RTW,       ...);
csr_write(INIT_TIMING0, ...);  csr_write(INIT_TIMING1, ...);

// 3. Program MR0..MR3 and (LPDDR2) PASR
csr_write(MR0, ...); csr_write(MR1, ...); csr_write(MR2, ...);
csr_write(MR3, ...);

// 4. Trigger init — the values above are already live at the core
boundary
csr_write(CTRL, CTRL_INIT_START);
```

There is no apply/poll handshake between the writes — each write is already visible to the core.

41.5 Open Questions / Future Work

- **Mid-traffic re-tuning guard.** A future revision could add an optional per-field commit gate (re-introducing a lightweight quiet point) for fields that software wants to change under load. Not present in this build.
- **CL/CWL CSR coupling.** The `TIMINGS_CL_CWL_WR.CL/.CWL` fields are currently informational; the live decode comes from the MR shadow. Deciding whether the CSR or the MR shadow is authoritative is a cleanup item.
- **Observation readback.** `STATUS/OBS_*` are declared but `hwif_in` is tied off in `pumice_top` (see §4.1); wiring them back is a follow-up.

42 Family Config Knobs (DDR2 / LPDDR2)

Per HAS §5.4 for the family-wide config philosophy. This MAS chapter is the **implementation-level** detail for **this controller** specifically: which CSR fields select DDR2-vs-LPDDR2 behavior and which fields are present-but-inert on this generation.

The controller is a two-family design (DDR2 + LPDDR2). The single most important family selector is `PHY_TIMING.memtype` (0 = DDR2, 1 = LPDDR2), which `pumice_top` maps to the core's `memtype_i` and which the init sequencer and DFI command formatter branch on. There is **no** separate family capability vector register; `ID` @ 0xFF0 echoes the build's memtype and phase count.

42.1 The Family Selector: `PHY_TIMING.memtype`

memtype	Family	What it changes
0	DDR2	`init_sequencer` runs the DDR2 MRS/precharge/refresh chain; `dfi_cmd_formatter` drives the DDR2 RAS/CAS/WE command bus
1	LPDDR2	`init_sequencer` runs the LPDDR2 MRW chain (MR63/MR10/MR1/MR2/MR3); `dfi_cmd_formatter` packs the JESD209-2F CA-bus word onto `dfi_address`

`memtype` is set before init and left fixed (see §4.3). DDR2 is the board target; LPDDR2 is the family-reuse path. Both pass the full sim suite.

42.2 Field-by-Field Applicability

Fields below are the real CSR fields in `rtl/macro/pumice_csr.rdl` (see §4.2). "DDR2 / LPDDR2" notes whether the field is meaningful on each family in this build.

42.2.1 SCHED_TUNING (0x040) — RETIRED, reserved in full

Every field of this register belonged to the pre-rearchitecture scheduler and the CAM + arbiter has never read any of them (retired 2026-09-09). Scheduling applicability is `SCHED_POLICY` below.

42.2.2 SCHED_POLICY (0x068)

Field	Applicability
order_mode	Both; 0 FR-FCFS, 1 in_order, 3 age_threshold. **Build-tiered:** in_order is per-channel FIFO on the base build; GLOBAL read-vs-write age order additionally requires +define+PUMICE_ENHANCED
age_thresh	Both; age (MC cycles / 16) above which a reference is boosted, for order_mode 3
prio_sub	Both; read-vs-write key within a class (load_over_store / none / age_boost)
row_sel / col_sel	Both; most- / fewest-pending row and column selection
access_pref	Both; column_first (default), row_first, precharge_first
qos_en	Both; makes AxQOS the outer pick key

42.2.3 REFRESH_TUNING (0x048)

Field	Applicability
page_policy_or	Both; drives page_policy_i (00 build-time, 01 OPEN, 10 CLOSE, 11 reserved — was HYBRID)
refresh_defer_active	**Retired 2026-09-09** — reserved; the JEDEC credits are REF_CTRL.postpone_limit / pullin_limit
zqcs_freq_hz	**Retired 2026-09-09** — reserved; no ZQCS engine consumed it
refpb_policy_or	**Retired 2026-09-09** — reserved; refresh mode is REF_CTRL.mode (02 = REFpb round-robin, LPDDR2)

42.2.4 ADDR_MAP (0x04C) — family-agnostic

Field	Applicability
bank_lsb	Both; the single address-map knob. = COL_WIDTH -> ROW_MAJOR; smaller -> interleave. No scheme selector exists.
hash_en	Both; enables the bank XOR-hash (the old XOR_HASH “scheme”)
hash_seed	Both; XOR-hash seed

DDR2/LPDDR2 have no bank groups, so there is no bank-group field. The retired ADDR_MAP_TUNING register (with scheme_or / synth_mask_obs / xor_seed_runtime) does not exist; all address mapping is the ADDR_MAP register above.

42.2.5 INIT_TUNING (0x050) / INIT_TIMING0/1 (0x058/0x05C)

Field	Applicability
`zq_retries`, `init_timeout_ms`	Both
`t_init_wait`, `t_dll_wait`	Both; on LPDDR2 the sequencer reuses `t_init_wait` for tINIT4 and `t_dll_wait` for tZQINIT (see §5.1)
`t_mrd_wait`, `t_rp_wait`, `t_rfc_wait`	DDR2 uses all three; LPDDR2 reuses `t_mrd_wait` for post-MRW (`t_rp_wait`/`t_rfc_wait` are inert on the LPDDR2 chain)

42.2.6 PHY_TIMING (0x064)

Field	Applicability
`memtype`	The family selector (above)
`t_phy_wrlat`	Both; WR cmd -> dfi_wrdata_en (0 = a7ddrphy pre-pull)
`t_rddata_en`	Both; RD cmd -> dfi_rddata_en window
`refresh_burst`	Both; REFs drained per request

42.2.7 LPDDR2-only registers

Register	Applicability
`PASR_BANK_MASK_RANK0` (0x030)	LPDDR2 partial-array self-refresh (MR16); inert on DDR2
`PASR_SEG_MASK_RANK0` (0x034)	LPDDR2 PASR segment mask; inert on DDR2
`TEMP_DERATE_RANK0` (0x038)	LPDDR2 MR4 temperature class (RO); inert on DDR2
`TIMINGS_CL_CWL_WR.tRFCpb`	LPDDR2 per-bank tRFC; unused by DDR2
`CTRL.pwr_req_dpd`	LPDDR2 Deep Power Down request; inert on DDR2

42.3 Feature Discovery

There is no `0xFF8` capability vector in this RDL. Software identifies the build via the `ID` register at 0xFF0:

Bits	Field	Meaning
7:0	`version`	Build version (0x01)
15:8	`memtype`	Build memtype echo (0 DDR2 / 1 LPDDR2)
23:16	`n_phases`	Gear ratio (1 / 2 / 4)
31:24	`module_id`	Fixed 0xD2

Note the `ID.memtype` field is a build-time echo (default reflects the DDR2 build); the **runtime** family select is `PHY_TIMING.memtype`, which software programs before init.

42.4 Not Present in This Generation

The following belong to higher DDR/LPDDR generations and have **no** field in this controller's CSR map: fine-granularity refresh (FGR), write-leveling, MPR, CA training, command-bus training (CBT), bank groups, and inline ECC. They were speculative entries in an earlier family registry and are omitted here rather than tied off.

42.5 Open Questions / Future Work

- **Multi-rank family fields.** PASR/temperature registers are declared for rank 0 only; a `NUM_RANKS` loop in the RDL would add `*_RANK{N}`.
- **Capability register.** If a family capability vector proves useful for a generic SoC driver, it can be re-added as an RO register; for now `ID` carries the essentials.
- **DPD power-state path.** `CTRL.pwr_req_dpd` is defined; the LPDDR2 DPD entry/exit datapath is a follow-up (see §5.2).

43 Training-Layer Interface Contracts

Wire-level contracts for the interfaces the LPDDR2 calibration/training layer added at rev 0.8. Block behavior lives in ch02/23, ch02/24, and ch02/25; the registers are in §4.2. Every row below is transcribed from committed RTL — the file:line citations are the contract.

43.1 Contract: scheduler command FIFO word (MRR flag)

Source. `rtl/macro/pumice_scheduler_layer.sv` — pack at the `w_cmd_wr_data` assign, unpack at `w_cmd_rd_data`.

The arbiter-to-DFI command FIFO word grew one bit for training: the MRR flag rides on top of the demand-path fields. MSB first:

Field	Width	Meaning
`mrr`	1	1 = MRR (mode-register read); 0 = normal / MRW
`ap`	1	auto-precharge verdict carried with the pick
`col`	`COL_WIDTH`	column address
`row`	`ROW_WIDTH`	row address (MR index + OP packing for MRS/MRR)
`bank`	`BKW`	bank address
`rank`	`RKW`	rank (single-rank today)
`op`	4	`dram_op_e`; MRR rides as `OP_MRS` + `mrr = 1`

`CMD_W = 4 + RKW + BKW + ROW_WIDTH + COL_WIDTH + 1 + 1`. `dram_op_e` is not widened — all 16 opcodes were consumed, and MRR never enters the demand path. The word is a `gaxi_fifo_sync` data width; the only pack/unpack sites are the scheduler layer and `pumice_dfi_cmd_path`, so no external consumer sees the packing.

The arbiter registers the flag with the decision: `r_cmd_mrr <= w_do_trn ? trn_cmd_mrr_i : 1'b0` — demand-class commands always carry `mrr = 0`, so a set flag by construction means a maintenance MRR got through.

43.2 Contract: formatter MRR CA word

Source. `rtl/fub/dfi_cmd_formatter.sv` — `OP_MRS` branch; JESD209-2F §5.12.

For `OP_MRS`, the formatter already builds the MRW CA word; MRR is the same word with one bit inverted:

CA bit	MRR value	Meaning
CA3r	H (cmd_mrr_i)	0 = MRW, 1 = MRR — the only difference
CA4r..CA9r	MA[5:0]	mode-register address, low half
CA0f	MA[6]	mode-register address, high half
CA1f	MA[7]	mode-register address, high half
CA2f..CA9f	OP[7:0]	operation field; 0 for MRR (mrr_row()), 0x56/0xAB for ZQCS/ZQCL
CA0r..CA2r	L	fixed low for MRS-class

So the MRR command word is CA0r=L, CA1r=L, CA2r=L, CA3r=H with the MR select {CA1f, CA0f, CA9r..CA4r} = MA[7:0] — identical field packing to MRW, CA3r inverted. The ZQCS/ZQCL opcodes themselves remain NOP at the formatter (default branch): LPDDR2 ZQ is pure MRW, so no formatter change was needed for ZQ.

43.3 Contract: read-aligner calibration sideband

Source. rtl/fub/pumice_dfi_rd_aligner.sv — calibration capture block; CDC owned by pumice_training_layer.

Signal	Domain	Direction (aligner)	Contract
cal_expect_i	dfi_clk	in	single-cycle arm; set the capture flag
cal_data_o	dfi_clk	out	first dfi_rddata_valid beat after the arm, held until the next arm
cal_valid_o	dfi_clk	out	single-cycle pulse when the beat is captured

Behavior: cal_expect_i sets r_cal_armed; the next cycle with any dfi_rddata_valid_i beat latches dfi_rddata_i into r_cal_data, raises cal_valid_o for one cycle, and clears the flag. The normal read FIFO push path is untouched — the arbiter grant contract guarantees no read is outstanding when calibration is active, so the sideband cannot collide with a returning demand read. Crossing into mc_clk for pumice_lp_cal is a cdc_synchronizer on data (held stable by this register) plus a sync_pulse on valid.

43.4 Contract: `trn_cmd` maintenance arbitration

Source. `rtl/fub/pumice_cmd_arbiter.sv` — priority pick, `w_trn_safe`, grant output; channel driven by `pumice_training_layer`.

The training layer is one more maintenance client on the arbiter, in priority order:

```
init > refresh > trn (training) > demand
```

Same shape scoria uses (ZQ below refresh there, same here). The contract rows:

#	Term	Contract
1	Request payload	`trn_cmd_req_i/op_i/bank_i/row_i/mrr_i` from the training-layer mux; the granted op passes **verbatim** onto the command output — the arbiter never re-encodes maintenance commands
2	Grant safety (`w_trn_safe`)	all banks idle (`!w_any_active`), no ACT/PRE in flight (`!w_inflight_preact`), both guard windows clear (`r_guard0/1 == 0`), no REF recovery (`!w_rfc_busy`), no in-flight write data (satisfied by the scheduler draining the write path before `trn_cmd_req_i` asserts)
3	Priority	training picks only when init is done and no refresh request/drain is pending; it never preempts either, and demand never preempts it
4	Grant return	`trn_cmd_grant_o = w_fire_out && r_do_trn` — the grant is the fired command, so grant and issue cannot diverge
5	Post-grant hold	`cal_busy` from the training layer folds into `w_out_safe`; all demand-class fires are blocked during the ZQ/lp_cal bus-quiet window

Row 5 is the BUG-003 discipline applied again: the new gate lives in `w_out_safe`, never in the pick cone, so `w_fire_out == cmd_valid_o && cmd_ready_i` holds by construction.

43.5 Contract: CSR map additions

Source. `rtl/macro/pumice_csr.rdl` — full field-level detail in §4.2.

Seven registers added at 0x0A0–0x0F0 (free range between `OBS_ROW_HIT` 0x080–0x09F and the retired 0x0C0 hole; retired holes are never reused):

Offset	Register	Acc	Reset	Contents
0x0A0	`CAL_CTRL`	rw	`0x00000000`	`zq_en[0]`, `zq_defer_en[1]` (Mode C), `cal_start[4]` (self-clearing strobe), `cal_abort[5]` (strobe), `zq_overdue_max[28:16]` (0 = no cap)
0x0A4	`CAL_ZQ_INTERVAL`	rw	`0x00000000`	32-bit ZQCS interval in MC cycles, 0 = off
0x0A8	`CAL_ZQ_TIMING`	rw	`0x00000000`	`t_zqcs[15:0]`, `t_zqcl[31:16]` post-grant hold windows
0x0AC	`CAL_TRAIN_TIMING`	rw	`0x00020000`	`t_mrr[15:0]` = 2 (JEDEC tMRR), `t_readout[31:16]` MRR issue→data timeout
0x0B0–0x0BC	`CAL_MRR32_DATA[4]`	ro	`0` (hw-written)	first captured DFI read-data beat for MRR MR32 (pattern A), four 32-bit slices
0x0E0–0x0EC	`CAL_MRR40_DATA[4]`	ro	`0` (hw-written)	first captured DFI read-data beat for MRR MR40

Offset	Register	Acc	Reset	Contents
				(pattern B), four 32-bit slices
0x0F0	CAL_STATUS	ro	0x00000000	zq_busy[0], cal_busy[1], cal_done[2] (sticky), cal_err[3] (sticky), zq_overdue[4], zqcs_total[31:16]

The sticky done/err bits survive `cal_abort` and clear on soft reset (a fresh `cal_start` re-arms them). Config-drive applies like everywhere else in this map: a written field tracks straight into the datapath, no staging or quiet point (see §4.1).

44 Initialization Sequence

Per HAS §6.2 for the architectural sequence and §2.12 for the FUB-level detail. This chapter is the **software-side cookbook** — what bring-up firmware actually does to get from power-on to "DRAM ready for AXI traffic." The sequence is driven by `rtl/fub/init_sequencer.sv`.

44.1 Pre-Reset Setup

Before `aresetn` (the MC-domain reset) deasserts, the SoC's PMU should:

1. Apply DRAM power rails (VDD, VDDQ; LPDDR2 also VDD1/VDD2/VDDCA)
2. Wait for power rails to be stable (typically 10–100 µs per JEDEC)
3. Bring the register transport out of reset (the PeakRDL cpuif is R/W-able; the init sequencer sits in `S_RESET`/`S_DFI_INIT`)
4. Program the family selector and address map: `PHY_TIMING.memtype`, `ADDR_MAP.bank_lsb`/`hash_*` (these must be set before init — see §4.3)
5. Load timing values (`TIMINGS_*`, `INIT_TIMING0/1`, `MR0..MR3`, PASR if LPDDR2)
6. Release the datapath reset

Because config is driven by name into the core (see §4.3), the values are already live at the core boundary when init runs — there is no staging/commit step.

44.2 Init Trigger

```
void start_dram_init(void) {
    // Family + address map first (must be set before init)
    csr_write(PHY_TIMING, MEMTYPE(DDR2) | T_RDDATA_EN(6) |
T_PHY_WRLAT(1) | REFRESH_BURST(1));
    csr_write(ADDR_MAP,   BANK_LSB(COL_WIDTH));   // ROW_MAJOR; add
HASH_EN|HASH_SEED if wanted

    // Load timings (example values for DDR2-800)
    csr_write(TIMINGS_RC_RCD_RP_RAS,   TIMING_PACK(60, 15, 15, 40));
// tRC, tRCD, tRP, tRAS
    csr_write(TIMINGS_RFC_REFI,        REFI_PACK(16, 1950));
// tRFC, tREFI
    csr_write(TIMINGS_RRD_FAW_WTR_CCD, RRD_PACK(6, 35, 4, 4));
// tRRD, tFAW, tWTR, tCCD
    csr_write(TIMINGS_CL_CWL_WR,       CL_PACK(6, 4, 15, 70));
```

```
// CL, CWL, tWR, tRFCpb
    csr_write(TIMINGS_RTP_RTW,         RTP_PACK(4, 6));
// tRTP, tRTW

    // JEDEC init-sequence waits (or leave RDL defaults
512/256/8/8/16)
    csr_write(INIT_TIMING0, INIT_T0_PACK(512, 256));
// tINIT, tDLLK
    csr_write(INIT_TIMING1, INIT_T1_PACK(8, 8, 16));
// tMRD, tRP, tRFC

    // MR values: for DDR2 the sequencer uses its own JEDEC-correct MR
words
    // (0x0533/0x0433/0x0380/0x0000 at the MR0.VAL reset default); the
MR0..MR3 CSRs are the software-visible
    // shadow. For LPDDR2 the sequencer drives the MRW OP values
internally.
    csr_write(MR0, mr0_value); csr_write(MR1, mr1_value);
    csr_write(MR2, mr2_value); csr_write(MR3, mr3_value);

    // (LPDDR2) PASR if needed
    csr_write(PASR_BANK_MASK_RANK0, pasr_bank_mask);

    // Optional: ZQ retry count / timeout
    csr_write(INIT_TUNING, INIT_TUNING_PACK(3, 10));  // 3 retries,
10ms timeout

    // Trigger init
    csr_write(CTRL, CTRL_INIT_START);

    // Poll for completion
    while (1) {
        uint32_t s = csr_read(STATUS);
        if (s & STATUS_INIT_DONE)  break;             // DRAM ready
        if (s & STATUS_INIT_ERROR) {                  //
STATUS.init_step_dbg shows where
            log_error("DRAM init failed at step %d",
STATUS_INIT_STEP_DBG(s));
            return -EIO;
        }
    }
}
```

Note (this build): `STATUS` readback (`hwif_in`) is currently tied off in `pumice_top` (see §4.1); `init_done` is also exposed directly as the top-level `init_done_o` port. Poll whichever is wired on the platform.

44.3 Init Sequence Details

The step-by-step JEDEC sequence is a hardware FSM in `init_sequencer.sv` (`r_state`); software does not interleave commands during init. The sequencer both **issues** the commands to the DRAM (through the scheduler to `dfi_cmd_formatter`) and updates the mode-register shadow so the live CL/CWL/BL decode tracks what was programmed. The `memtype_i` input selects the DDR2 or LPDDR2 chain.

44.3.1 DDR2 chain (mirrors LiteDRAM's proven Nexys A7 sequence)

1. Assert `dfi_init_start_o`; wait `dfi_init_complete_i` (PHY DLL-lock / IO training), then wait tINIT (`t_init_wait`, CKE settle)
2. **Precharge All** (wait tRP)
3. **EMRS(2)** = 0, **EMRS(3)** = 0, **EMRS(1)** = 0 — JEDEC MRS order EMR2 -> EMR3 -> EMR1, each followed by tMRD
4. **MRS(0)** + **DLL reset** (`MR0.VAL | 0x100`, reset default 0x0533: BL8/CL3/tWR3/DLL_RESET); wait tDLLK (DLL lock)
5. **Precharge All** (wait tRP)
6. **Auto Refresh x2** (each followed by tRFC)
7. **MRS(0)** without DLL-reset (`MR0.VAL`, reset default 0x0433 — clears the reset bit); wait tMRD
8. **EMRS(1)** + **OCD Default** (0x0380) -> **EMRS(1)** + **OCD Exit** (0x0000)
9. `init_done_o = 1`

44.3.2 LPDDR2 chain (JESD209-2F power-up + MR configuration)

1. Assert `dfi_init_start_o`; wait `dfi_init_complete_i`; tINIT settle
2. **MRW(MR63)** = Reset (wait tINIT4, reuses `t_init_wait`)
3. **MRW(MR10)** = 0xFF ZQ Init Calibration (wait tZQINIT, reuses `t_dll_wait`)
4. **MRW(MR1)** = BL8/nWR3 (0x23), **MRW(MR2)** = RL3/WL1 (0x01), **MRW(MR3)** = DS 40ohm (0x02) — each followed by tMRW (reuses `t_mrd_wait`)
5. `init_done_o = 1`

The MR index (MA, up to MR63) and data (OP) are packed as `{MA[5:0], OP[7:0]}` into the row request field; `dfi_cmd_formatter` unpacks it for the LPDDR2 CA-bus MRW word (a 3-bit bank port cannot reach MR10/MR63). Only MR1/MR2/MR3 update the CL/CWL/BL decode shadow; MR63/MR10 are issued but not shadowed.

The init waits are the `INIT_TIMING0/1` CSR fields (`t_init_wait`, `t_dll_wait`, `t_mrd_wait`, `t_rp_wait`, `t_rfc_wait`); RDL defaults (512/256/8/8/16 MC cycles) comfortably cover the JEDEC minimums at the board frequency. For fast sim, program small values before triggering init.

44.4 Multi-Rank Init Differences

The current `init_sequencer.sv` targets a single rank (the board build is `NUM_RANKS = 1`). For a `NUM_RANKS > 1` build the intended extension is:

- MRS/MRW steps iterate over ranks (one rank per step iteration)
- ZQ calibration is per-rank (each rank has its own reference)
- Each rank's PASR mask is programmed via its own `PASR_*_RANK{N}` register (see §4.2 open items)

Software programs the MR values once; per-rank iteration is absorbed by the sequencer. Per-rank register generation is a follow-up (the RDL declares only `*_RANK0` today).

44.5 ZQ Retry

The `INIT_TUNING.zq_retries` field (default 3) is exposed for the retry count; DDR2 uses OCD calibration during init (no ZQCL) and LPDDR2 uses the MR10 ZQ Init step. Raise the retry budget if signal-integrity issues cause occasional failures.

44.6 Post-Init Power-State

After `init_done`, the controller is in ACTIVE for all ranks with CKE high. The refresh manager starts its tREFI counter and begins normal refresh scheduling. The scheduler is unblocked and any AXI bursts that accumulated during init begin issuing.

44.7 Software Wait Times

Phase	Wait (real silicon, 200 MHz MC clock)
Power-up settling	200 µs
MRS loads + per-rank iteration	~10 cycles × NR × 4 MRs ≈ 160 cycles
ZQ calibration	~1 µs
Initial refresh batches	~10 µs
Total init duration	~250 µs (single-rank), ~280 µs (4-rank)

The init duration scales weakly with rank count — the per-rank serialization is dwarfed by the fixed settling delays.

44.8 Reset Recovery

If init fails (`STATUS.init_error = 1`):

```c
void recover_init_failure(void) {
    uint8_t step = STATUS_INIT_STEP_DBG(csr_read(STATUS));
    log_error("Init failed at step %d", step);

    // Inspect step-specific telemetry (per step table)
    // ...

    // Force re-init
    csr_write(CTRL, CTRL_INIT_FORCE_RESTART);

    // Wait a few cycles for FSM to settle
    for (int i = 0; i < 10; i++) asm volatile("nop");

    // Re-init
    start_dram_init();
}
```

A hard fault (e.g., persistent ZQ calibration failure) requires SoC-level intervention — typically a power-cycle and re-bring-up.

44.9 Open Questions / Future Work

- **Init telemetry capture.** When init fails, telemetry beyond `init_step_dbg` would help (DFI signal trace, per-rank fault detection). The current debug surface is sparse; expand if bring-up calls it out.
- **Init via JTAG.** Some platforms init via a JTAG side-band rather than CPU+APB. The CSR map supports this through any APB-attached transport; no controller changes needed.
- **Partial init for warm reset.** Coming out of self-refresh doesn't require full init. The current init engine doesn't have a "warm reset" path. Add when DDR3+ needs it for frequency change.

45 Refresh and Power-State Programming

Per HAS §3.4 / §3.5 for architecture and §2.11 for FUB detail. This chapter is the **software-side** view of refresh and power configuration. Register writes are the PeakRDL cpuif (`csr_write`/`csr_read`); there is no apply/commit handshake — fields drive the core live (see §4.3).

45.1 Tuning Refresh Behavior

```
void configure_refresh(refresh_config_t* cfg) {
    // Base tREFI / tRFC (also set during init; retunable at runtime)
    csr_write(TIMINGS_RFC_REFI, REFI_PACK(cfg->trfc, cfg->trefi));

    // REFRESH_TUNING packs deferral count, page/refpb policy, ZQCS
interval.
    // JEDEC refresh credits: how many REFs may be postponed under
demand and
    // pulled in when idle (0..8 each; 0 = strict tREFI).
    uint32_t v = POSTPONE_LIMIT(cfg->postpone) | PULLIN_LIMIT(cfg-
>pullin);

    // Refresh mode: 01 REFab, 02 REFpb round-robin (LPDDR2; needs the
    // REF_CTRL.perbank_supported capability strap). 0 = build
default.
    v |= MODE(cfg->refpb ? 2 : 1);

    csr_write(REF_CTRL, v);   // live on the next refresh event
boundary
}
```

Updated 2026-09-09. This recipe previously programmed `REFRESH_TUNING.refresh_defer_active`, `.refpb_policy_or` and `.zqcs_freq_hz`. All three are retired and reserved: refresh mode and the JEDEC credits are `REF_CTRL`, and no ZQCS engine ever consumed the interval. Writing the old fields was a silent no-op. The DARP and OLDEST_FIRST per-bank policies are not commodity-legal on this part and were never built — see `ADVANCED_MODES_ROADMAP.md`.

`PHY_TIMING.refresh_burst` (1..8) additionally controls how many REFs are drained per refresh request.

45.1.1 When to Tune

Workload	Recommended config
Streaming (DMA, video)	`defer_active` = 8, ZQCS = 1 Hz
Low-latency bursty (CPU access)	`defer_active` = 1 (no batching), ZQCS = 0.1 Hz
Power-sensitive (sleep-heavy)	`defer_active` = 4, ZQCS = 0.1 Hz
Real-time / safety-critical	`defer_active` = 1, deterministic refresh latency

45.2 LPDDR2 PASR (Partial Array Self-Refresh)

PASR masks DRAM regions that hold no data, reducing refresh power.

```
void set_pasr_rank0(uint8_t bank_mask, uint8_t seg_mask) {
    csr_write(PASR_BANK_MASK_RANK0, bank_mask);
    csr_write(PASR_SEG_MASK_RANK0,  seg_mask);
    // Propagated to DRAM via MR16/MR17 at the next self-refresh
entry.
}
```

The single-rank build exposes `PASR_BANK_MASK_RANK0` / `PASR_SEG_MASK_RANK0`; multi-rank `*_RANK{N}` registers are a follow-up (see §4.2). The PASR mask is propagated lazily — typically at the next self-refresh entry — to avoid an extra bus-blocking MR write during normal operation.

45.3 Temperature Compensation (LPDDR2)

LPDDR2 devices expose temperature classification in MR4. The SoC reads this via PHY-side mechanisms (out of scope for the controller; the PHY signals temperature change via interrupt or polled register).

Software updates the controller's tREFI scaling:

`TEMP_DERATE_RANK0.temp_class` (0 = nominal, 1 = 2x refresh, 2 = 4x refresh) is a **read-only, hardware-written** field in this build — the controller captures the LPDDR2 MR4 class rather than software programming it. Software reads it to inform its own tREFI scaling via `TIMINGS_RFC_REFI`:

```
uint8_t temp = csr_read(TEMP_DERATE_RANK0) & 0x3;
// Scale tREFI down for 2x/4x refresh as temp rises
csr_write(TIMINGS_RFC_REFI, REFI_PACK(trfc, trefi >> temp));  // live
next reload
```

45.4 Self-Refresh Entry

For periods of inactivity, software (or auto-detection) can put the DRAM into self-refresh:

The power-state request bits live in CTRL: `pwr_req_low_power`, `pwr_req_self_refresh`, `pwr_req_active`, `pwr_req_dpd`. `STATUS.power_state[7:4]` reports the current encoded state (RO, hw-written).

```
void enter_self_refresh(void) {
    csr_write(CTRL, CTRL_PWR_REQ_SELF_REFRESH);
    while (STATUS_POWER_STATE(csr_read(STATUS)) !=
POWER_STATE_SELF_REFRESH);
}

void exit_self_refresh(void) {
    csr_write(CTRL, CTRL_PWR_REQ_ACTIVE);
    while (STATUS_POWER_STATE(csr_read(STATUS)) !=
POWER_STATE_ACTIVE);
}
```

Note: any AXI traffic to a rank will auto-wake it from SR. Explicit exit is only needed when the application wants to be ready before issuing.

There is no `POWER_TUNING` register in this build — idle-threshold auto-APD/SRF is not a CSR knob here. Power-state entry is software-requested via `CTRL`. (An auto-low-power threshold register is a possible follow-up; see below.)

45.5 DPD (LPDDR2 Deep Power Down)

DPD is the deepest power state on LPDDR2 — DRAM is fully off; software must full-init to exit. The request bit is `CTRL.pwr_req_dpd` (inert on DDR2). Check the build memtype via `ID` (0xFF0) rather than a capability vector (there is no `0xFF8`):

```
void enter_dpd(void) {
    if (((csr_read(ID) >> 8) & 0xFF) != MEMTYPE_LPDDR2) {
        log_error("DPD is LPDDR2-only");
        return;
    }
    csr_write(CTRL, CTRL_PWR_REQ_DPD);
    while (STATUS_POWER_STATE(csr_read(STATUS)) != POWER_STATE_DPD);
}

void exit_dpd(void) {
    start_dram_init();   // full re-init
}
```

DPD is rare in practice (DRAM fully unused for hours). Note the runtime family select is `PHY_TIMING.memtype`; `ID.memtype` is the build-time echo.

45.6 Open Questions / Future Work

- **Auto power-down thresholds.** This build has no `POWER_TUNING` register — APD/SRF is request-driven. Adding idle-threshold CSRs for automatic low-power entry is a candidate feature.
- **Per-rank power control.** `CTRL.pwr_req_*` is channel-wide. Per-rank request registers are a multi-rank follow-up.
- **Observation readback.** `STATUS.power_state` and `STATUS_HISTORY` are declared but `hwif_in` is tied off in `pumice_top` today (see §4.1). The `OBS_REFRESH_DEFER_HIST_*` registers this bullet used to name were REMOVED on 2026-09-28 for exactly that reason (pumice BUG-020) – this chapter had recorded the tie-off while other chapters went on presenting them as live telemetry. Refresh remains observable through `REF_STATS_REF` and `REF_STATS_REF_BUSY`, which are driven.

46 Multi-Rank Programming

Per HAS §2.1 (multi-rank as differentiator), HAS §3.3 / §3.4 / §3.6 (architecture). FUB-level detail in MAS §2 (`pumice_bank_timers` per (rank,bank), `refresh_ctrl` round-robin, `dfi_cmd_formatter` CS_n / ODT).

Note: the default board build is single-rank (`NUM_RANKS = 1`). This chapter documents the intended multi-rank programming model; the single-rank RDL declares only `*_RANK0` registers, and per-rank register generation is a follow-up (see §4.2).

46.1 Discovery

There is no capability vector register in this build. Software knows `NUM_RANKS` from the build parameter (it is a synthesis-time geometry parameter, see §6.1); the `ID` register (0xFF0) reports memtype and phase count but not rank count. Treat the build parameter as authoritative.

46.2 Per-Rank Mode Register Loads

During init, the sequencer (intended multi-rank extension) iterates the MRS/MRW loads across ranks. Software writes MR0..MR3 once:

```
csr_write(MR0, mr0_value);   // applied to all ranks during init
csr_write(MR1, mr1_value);
csr_write(MR2, mr2_value);
csr_write(MR3, mr3_value);
```

For DDR2 the sequencer uses its own JEDEC-correct MR words; the `MR*` CSRs are the software-visible shadow (see §5.1). Per-rank MR variation (rare) is a future feature.

46.3 Per-Rank PASR (LPDDR2)

Already covered in §5.2. In the single-rank build:

```
csr_write(PASR_BANK_MASK_RANK0, mask);
```

A multi-rank build adds `PASR_BANK_MASK_RANK{N}` registers (each rank has independent MR16/MR17).

46.4 ODT

ODT is handled inside `dfi_cmd_formatter` / `mode_register` (there is no standalone ODT control block and no `RANK_TUNING.odt_rule_or` CSR). `dfi_odt_o` is driven per command. Board-specific ODT rule selection is not a runtime CSR knob in this build; it is baked into the formatter's per-command ODT logic. For boards with non-standard impedance budgets, adjust the formatter's ODT policy at build time.

46.5 Per-Rank Disable

There is no `RANK_TUNING` rank-enable CSR in this build. Rank enable/disable is a multi-rank feature to be added alongside per-rank register generation. In the single-rank build there is one rank and it is always active.

46.6 Address Mapping for Multi-Rank

The rank field always sits in the high bits of the (byte-offset-stripped) word address, above the row — its position is invariant. The single `ADDR_MAP.bank_lsb` knob only slides the **bank** field within the column region (see §4.4 and `rtl/fub/addr_mapper.sv`); the rank field is unaffected by `bank_lsb`. The `hash_en` XOR-hash folds row bits into the bank index only — it does not touch the rank field.

There is no rank-interleave mode. Software-managed rank interleaving (the OS striping allocations across rank-aligned regions) is the workaround if consecutive-line rank striping is desired.

46.7 Refresh Behavior with Multi-Rank

Per HAS §3.4 and MAS §2 (`refresh_ctrl`, intended multi-rank extension):

- REFab dispatches per-rank in round-robin (not all-rank simultaneously)
- REFpb (LPDDR2) selects (rank, bank) tuples via `REF_CTRL.mode` = 2 (round-robin), gated by the `REF_CTRL.perbank_supported` capability strap. (`REFRESH_TUNING.refpb_policy_or` was retired 2026-09-09.)
- Per-rank PASR masks are honored independently

A multi-rank system distributes refresh across the timeline so any single rank only blocks for tRFC per refresh — non-target ranks keep operating.

46.8 Power State Per-Rank

- Channel-wide CSR `CTRL.pwr_req_*` applies to all ranks (there is no per-rank power request CSR)
- `STATUS.power_state[7:4]` reports the encoded state

- Per-rank auto-low-power is not a CSR feature in this build (there is no `POWER_TUNING`; see §5.2)

46.9 Per-Rank / Per-Bank Observation

The RDL declares one per-bank observation array for rank 0: `OBS_ROW_HIT[8]` (0x080..0x09C), driven from the issued command stream. The generated regmap flattens it to indexed names:

```
uint32_t get_row_hits(uint8_t bank) {
    return csr_read(OBS_ROW_HIT0_ROW_HIT + bank * 4);   // free-
running
}
```

Free-running: read twice around a window and subtract, masking the 32-bit wrap.

`OBS_REF_LATENCY[8]` (0x0C0..0x0DC) was removed on 2026-09-28 – it was never driven and had no consumer (pumice BUG-020).

46.10 Multi-Rank Bring-Up Checklist

Step	Why
Verify the build's `NUM_RANKS` matches the expected DIMM rank count	Build / board mismatch
Program `PASR_*_RANK{N}` if using LPDDR2	PASR for LPDDR2 bring-up
Verify per-rank ZQ calibration succeeds during init	Each rank's drive impedance
Stress-test rank-switching (read alternating ranks)	tRTRS / tCS timing

46.11 Open Questions / Future Work

- **Per-rank register generation.** The RDL declares only `*_RANK0`; a `NUM_RANKS` loop is needed for PASR/temp/observation and rank-enable.
- **Per-rank MR override.** Some DIMMs have rank-specific tuning (rare).
- **Per-rank power control.** `CTRL.pwr_req_*` is channel-wide; per-rank request registers are a follow-up.
- **Runtime ODT rule select.** ODT is currently baked into `dfi_cmd_formatter`; a runtime CSR knob could be reintroduced if board diversity needs it.

47 Error Handling

The controller's error-reporting paths and the recommended software response model. Register accesses are the PeakRDL cpuif (`csr_write`/`csr_read`); config fields drive the core live with no apply/commit step (see §4.3).

47.1 Error Categories

Category	Detection	Reporting
Init failure	`init_sequencer` timeout / ZQ retries exhausted	`STATUS.init_error = 1`; `STATUS.init_step_dbg` shows where
DFI rddata-valid timeout	RD issued, no rddata in the expected window	`SLVERR` on the AXI R (bring-up: check DFI phase / `t_rddata_en`)
AXI burst boundary violation	Per §3.1 (one AXI burst == one DRAM burst)	`SLVERR` on B/R
ZQ calibration failure (silicon)	DFI signal-integrity, caught at PHY layer	Not this controller
Multi-bit DRAM error	Out of scope (no inline ECC)	Would need ECC sideband

Note: `STATUS` is currently a declared RO register whose `hwif_in` is tied off in `pumice_top` (see §4.1). `init_done` is exposed directly as the top-level `init_done_o` port. There is no dedicated IRQ port list wired in this build; software polls `STATUS`/`init_done_o`. A queue-overflow / refresh-miss IRQ is a possible follow-up.

47.2 Software Response Patterns

47.2.1 Init Failure

```
void handle_init_error(void) {
    uint32_t s = csr_read(STATUS);
    uint8_t step = STATUS_INIT_STEP_DBG(s);

    log_error("DRAM init failed at step %d", step);
```

```
    // init_sequencer step encodings (r_state), for reference:
    //   S_DFI_INIT  -> PHY init did not complete (check PHY config)
    //   S_EMR*/S_MR0* / S_L_* -> MRS/MRW issue phase (verify MR
values, CA-bus)
    //   S_REF1/S_REF2 -> refresh phase
    // Force re-init (config CSRs are preserved)
    csr_write(CTRL, CTRL_INIT_FORCE_RESTART);
}
```

47.2.2 DFI Read-Data Timeout (board bring-up)

A read that returns no data almost always means the DFI read window is misaligned for the attached PHY. The knobs are `DFI_PHASE.rd_phase`, `PHY_TIMING.t_rddata_en`, and `PHY_TIMING.t_phy_wrlat`:

```
// Nexys A7 a7ddrphy known-good: rd_phase=0, t_rddata_en=6,
t_phy_wrlat=1 (+ harness rddata_delay=7)
csr_write(DFI_PHASE,   RD_PHASE(0) | WR_PHASE(0));
csr_write(PHY_TIMING,  MEMTYPE(DDR2) | T_RDDATA_EN(6) | T_PHY_WRLAT(1)
| REFRESH_BURST(1));
```

Set these during bring-up before triggering init; they take effect immediately (config-drive, no commit step).

47.2.3 Refresh Pressure

Watch `REF_STATS_REF_BUSY` against `REF_STATS_REF`: the ratio is how often a refresh fired with work already pending. If it climbs, reduce batching:

(`OBS_REFRESH_PENDING_MAX` at 0x108 used to be named here. It was removed on 2026-09-28 – it was never driven and always read zero, so this step could not have told anyone anything. pumice BUG-020.)

```
uint32_t v = csr_read(REFRESH_TUNING);
v = (v & ~REFRESH_DEFER_ACTIVE_MASK) | REFRESH_DEFER_ACTIVE(1);  // no
batching
csr_write(REFRESH_TUNING, v);   // live on the next refresh event
boundary
```

47.3 STATUS_HISTORY for Bring-Up

The `STATUS_HISTORY` register (0x008) captures the last 8 power-state transitions (4 bits each, most recent in [3:0]). Useful when the controller oscillates between power states:

```
void dump_state_history(void) {
    uint32_t hist = csr_read(STATUS_HISTORY);
    for (int i = 0; i < 8; i++)
        log_info("State[-%d]: %s", i, power_state_name((hist >> (i*4))
```

```
& 0xF));
}
```

(As with STATUS, this readback depends on hwif_in being wired — a follow-up per §4.1.)

47.4 Diagnostic Sequence for Suspected Bug

```
1. Read STATUS.init_done (or the init_done_o port) — must be 1
2. Read STATUS.power_state — should be ACTIVE
3. Read STATUS_HISTORY — look for oscillation
4. Read STALL_* — stall-cause attribution (backlog shows up here)
5. Read OBS_ROW_HIT[bank] — per-bank row-hit distribution (free-
running:
   read twice and subtract)
6. Read PAGE_STATS_{HIT,MISS,EMPTY} + SCHED_STATS_{ACT,PRE} — page
behaviour
7. Read REF_STATS_REF / REF_STATS_REF_BUSY — refresh cost against
demand
```

This is the bring-up team's first-pass diagnostic flow.

47.5 Soft Reset vs. Re-Init

CTRL.soft_reset (bit 31, self-clearing) requests an internal soft reset that re-clears datapath state without disturbing the CSR contents. Use it when the controller is in an inconsistent state but the software's config is correct:

```
void soft_reset(void) {
    csr_write(CTRL, CTRL_SOFT_RESET);
    for (int i = 0; i < 16; i++) asm volatile("nop");  // self-
clearing
    csr_write(CTRL, CTRL_INIT_START);                  // CSRs
preserved
}
```

For hard cases (silicon bug, persistent failure), the SoC PMU drives the asynchronous reset and software re-brings-up from scratch.

47.6 Open Questions / Future Work

- **IRQ port.** This build polls STATUS/init_done_o. A latched error/IRQ output (init error, refresh miss, rddata timeout) is a candidate feature.
- **Observation readback wiring.** The STATUS/OBS_* diagnostics assume hwif_in is connected; it is tied off today (§4.1).
- **Per-rank error reporting.** When multi-rank lands, localizing a fault to a rank would help board bring-up.

48 Build-Time Configuration Reference

Per HAS §5.2 for the parameter table and §5.1 for the build-vs-runtime philosophy. This chapter is the **build-script-author's** view: the synthesis-time parameters and how the data-width geometry works. Most behavior is runtime CSR-driven (see §4/§5), so the build parameter set is deliberately small — mostly memory geometry.

48.1 Parameters

The synthesis-time parameters are the ports of `pumice_top` (`rtl/top/pumice_top.sv`). They are geometry, not policy — memtype / timings / page policy / address map are all runtime CSR fields.

```
pumice_top #(
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .NUM_RANKS       (1),      // board build is single-rank
    .NUM_BANKS       (8),
    .ROW_WIDTH       (14),
    .COL_WIDTH       (10),
    .DFI_RATE        (2),      // DFI phases per MC clock (gear ratio)
    .DRAM_BEAT_WIDTH (64),     // bits per DRAM beat (device data bus)
    .DRAM_BL         (8),      // DRAM burst length (DDR2 uses 4; 8 is
the default)
    .NUM_ENTRIES     (8),      // CAM depth
    .N_SRAM_SLOTS    (8)
    // DW is DERIVED: DW = DRAM_BEAT_WIDTH * DFI_RATE
) u_pumice ( ... );
```

There is **no** `MEMTYPE`, `PAGE_POLICY`, `SCHEDULER_MODE`, `ODT_RULE_*`, `ADDR_MAP_SCHEMES_SYNTH`, or `DFI_ADDR_WIDTH` parameter — those are runtime CSR fields or derived from geometry. `memtype` is `PHY_TIMING.memtype`; page policy is `REFRESH_TUNING.page_policy_or`; address mapping is `ADDR_MAP.bank_lsb`.

48.2 Data-Width Geometry (DW = DRAM_BEAT_WIDTH x DFI_RATE)

The controller's internal / AXI-facing data width is **derived**, not a free parameter:

```
DW = DRAM_BEAT_WIDTH * DFI_RATE          // e.g. 64 * 2 = 128
(default)
```

- **One AXI beat == one DFI word == DFI_RATE DRAM beats.**
- **One AXI burst (BL / DFI_RATE beats) == one DRAM burst (BL beats).**

- The DW -> per-phase split happens in `dfi_signal_pack` / the DFI layer (`DFI_DATA_WIDTH = DW`, `DFI_EN_WIDTH = DFI_RATE`).

The old "AXI_DATA_WIDTH == DRAM_BEAT_WIDTH" coupling is **gone** — do not describe it. The host AXI data width the core presents is exactly `DW`.

48.2.1 Host-Width Freedom (`pumice_top_geared`)

To let a host SoC use a convenient fixed AXI width regardless of the DRAM point, wrap `pumice_top` in `pumice_top_geared` (`rtl/top/pumice_top_geared.sv`), which adds a free `HOST_AXI_DATA_WIDTH` parameter:

- It inserts the repo's **formally-verified** `axi4_dwidth_converter_wr` / `_rd` between a host-width AXI slave and the fixed-`DW` core.
- `HOST_AXI_DATA_WIDTH == DW` is a **generate bypass** — bit-identical to bare `pumice_top`, no converter, no added latency.
- Verified host widths: 64, 128, 256.
- Burst geometry contract is unchanged at the core's DW side; the converter re-sizes host bursts to DW-width bursts.

Doc: `docs/AXI_DRAM_GEARING_SCOPE.md`.

48.3 Parameter Sanity

The natural elaboration-time constraints:

```
generate
    if (NUM_RANKS != 1 && NUM_RANKS != 2 && NUM_RANKS != 4)
        $error("NUM_RANKS must be 1, 2, or 4");
    if (NUM_BANKS != 4 && NUM_BANKS != 8)
        $error("NUM_BANKS must be 4 or 8");
    if (DFI_RATE != 1 && DFI_RATE != 2 && DFI_RATE != 4)
        $error("DFI_RATE must be 1, 2, or 4 (gear ratio)");
    // BL must be an integer multiple of DFI_RATE so a DRAM burst is a
whole
    // number of AXI beats.
endgenerate
```

`DFI_RATE` must equal the attached PHY's phase count (e.g., the on-board 300 MT/s point is `DFI_RATE = 4` at sys = 37.5 MHz / CK = 150 MHz; regenerating for a different phase count alone produces a broken hybrid — see project notes).

48.4 Filelists

FUB filelists live in `rtl/filelists/`. The live module hierarchy (top -> core -> `pumice_axi4_layer` / `pumice_scheduler_layer` / `pumice_dfi_layer`) is fixed; the DDR2 vs

LPDDR2 command paths coexist in `dfi_cmd_formatter` and branch on `memtype`, so a single build supports both families with dead-code elimination handling the unused branch. `page_predictor.sv` / `powerdown_ctrl.sv` are optional/not in the default top build (confirm via the filelists).

48.5 Open Questions / Future Work

- **Multi-rank build.** `NUM_RANKS > 1` requires the per-rank register generation and per-rank init iteration described in §4.2 / §5.3.
- **Curated small-area filelist.** Excluding the optional predictor/power-down FUBs shrinks the smallest build.
- **Geared-top width matrix.** Extending the verified `HOST_AXI_DATA_WIDTH` set beyond {64,128,256} as needed.

49 Runtime Configuration Reference

Per §4.2 for the full CSR map and §4.3 for the config-drive model (fields drive the core live; no apply/commit step). This chapter is the **driver author's** cookbook for runtime tunables: what to tune, when, and what to watch for. Register access is the PeakRDL cpuif (`csr_write`/`csr_read`).

49.1 Bring-Up Configuration Order

When bring-up software first comes up, recommended order (address map and memtype are set **before** init — see §5.1):

1. **Family + address map (pre-init)** — `PHY_TIMING.memtype`; `ADDR_MAP.bank_lsb` / `.hash_en` / `.hash_seed`.
2. **JEDEC timings** — the `TIMINGS_*` registers. These do **not** default to anything usable for a given part and clock; leaving them at reset runs the DRAM at the RDL defaults, which is how the board spent a long time at a fraction of its bandwidth. Derive them from the part and the MC clock.
3. **Static page policy** — `REFRESH_TUNING.page_policy_or` (00 build-time, 01 OPEN, 10 CLOSE; 11 reserved).
4. **Adaptive page policy (optional)** — `PAGE_POLICY_CFG.policy_mode` (0 build default, 1 static_open, 2 static_close, 3 fixed_open, 4 adapt_time, 5 adapt_access; 6/7 retired 2026-09-26). Program the table shape **before** the mode select so the predictor starts from a known table: `PAGE_POLICY_CFG.ctr_open_max` / `.ctr_init` for mode 5, `PAGE_TIMEOUT_CFG` for modes 3/4.
5. **Scheduling order** — `SCHED_POLICY.order_mode` (0 FR-FCFS, 1 in_order, 3 age_threshold) plus `.age_thresh` for mode 3. See the build-tier note below.
6. **Refresh** — `REF_CTRL.mode` / `.postpone_limit` / `.pullin_limit`, and `TIMINGS_RFC_REFI.tREFI`; `PHY_TIMING.refresh_burst`.

Build tier. `order_mode = 1` (in_order) is per-channel FIFO on the base bitstream: each CAM issues only its oldest entry and the arbiter's normal read/write preference chooses the side. GLOBAL read-versus-write age order additionally requires a `+define+PUMICE_ENHANCED` build. A base bitstream accepts the write either way, so software cannot detect the difference by readback — check the build, not the register.

Each write is live immediately at the core boundary; there is no `config_apply` and no quiet-point drain. Quiesce AXI traffic before changing a field that would corrupt in-flight state (see §4.3).

49.2 Address-Map Tuning (single knob)

Address mapping is `ADDR_MAP.bank_lsb` alone (plus the optional XOR-hash). There is no scheme selector, `scheme_or`, or `xor_seed_runtime`:

```
// ROW_MAJOR: bank field above the whole column
csr_write(ADDR_MAP, BANK_LSB(COL_WIDTH));

// Max BANK_INTERLEAVE: bank field just above the burst's low column bits
csr_write(ADDR_MAP, BANK_LSB(log2_cols_per_burst));

// XOR_HASH folded on top of any placement
csr_write(ADDR_MAP, BANK_LSB(COL_WIDTH) | HASH_EN | HASH_SEED(seed));
```

RTL clamps `bank_lsb` to `[0, COL_WIDTH]`; keep `log2(DRAM_BL) <= bank_lsb <= COL_WIDTH` so a DRAM burst stays inside one bank (see §4.4 and `rtl/fub/addr_mapper.sv`). `DRAM_BL`, not `BL/DFI_RATE`: the mapper indexes device words, so a burst spans `DRAM_BL` columns here (pumice BUG-022, measured in `dv/tests/fub/test_addr_mapper.py::minimum_bank_lsb_is_measured`). Change address mapping only before init or with the datapath idle.

49.3 Characterization Sweep Order

Sweep order	Knob	Why
1	`ADDR_MAP.bank_lsb`	Largest impact on row-hit / bank parallelism
2	`ADDR_MAP.hash_en` / `.hash_seed`	Defeat power-of-two-stride hot-banking
3	`REFRESH_TUNING.page_policy_or`	OPEN vs CLOSE for the workload mix
4	`PAGE_POLICY_CFG.policy_mode`	Adaptive paging: 4 adapt_time, 5 adapt_access (6/7 retired)
5	`SCHED_POLICY.access_pref`	column_first vs row_first vs precharge_first
6	`REF_CTRL.postpone_limit` / `.pullin_limit`	Refresh latency vs sustained BW
7	`SCHED_POLICY.order_mode` / `.ag`	Ordering guarantee vs bandwidth

Sweep order	Knob	Why
	e_thresh	
8	SCHED_WR_WM.wr_high_wm / .wr_low_wm	Write-batching turnaround

49.4 Telemetry to Watch

Observation registers per §4.2. All of these are DRIVEN and free-running: read two and subtract for a window.

Telemetry register	What it tells you	Tune action
OBS_ROW_HIT[bank]	Per-bank row hits	address mapping (bank_lsb/hash), page policy
PAGE_STATS_{HIT,MISS,EMPTY}	Column ops, and the cause of each activation	page policy, tr_init
SCHED_STATS_{ACT,PRE}	Activations and closes	page policy, tr_init
REF_STATS_REF / _REF_BUSY	Refreshes, and those firing with work pending	tREFI, REF_CTRL.postpone_limit
STALL_*	Stall-cause attribution	whichever cause dominates

AXI latency and queue occupancy are not here on purpose. They are properties of the interface, measured outside the controller by axi_bus_meter and axi_perf_latency_hist (the characterisation harness instantiates both). The OBS_AXI_* and OBS_TXN_QUEUE_DEPTH_* registers this table used to list were removed on 2026-09-28 – they were never driven and always read zero, so every "tune action" in those rows was keyed on a constant (pumice BUG-020). | OBS_WORDS[9] | Packed obs_* harvest | FUB-internal diagnostics |

49.5 Workload-Specific Recipes

49.5.1 Streaming (DMA, video, audio capture)

```
// Maximize row-hit, credit refresh, interleave banks
csr_write(REFRESH_TUNING, PAGE_POLICY_OR(1 /*OPEN*/));
csr_write(SCHED_POLICY,   ORDER_MODE(0 /*FR-FCFS*/));   // reorders
across the whole CAM
csr_write(REF_CTRL,       POSTPONE_LIMIT(8) | PULLIN_LIMIT(8));
csr_write(ADDR_MAP,       BANK_LSB(log2_cols_per_burst));   // bank-
interleave (pre-init)
```

49.5.2 Low-Latency Bursty (CPU)

```
// Adaptive close: let the per-row predictor decide when to shut the
page.
// Table shape FIRST, then the mode select.
csr_write(PAGE_POLICY_CFG, CTR_OPEN_MAX(2) | CTR_INIT(0));
csr_write(PAGE_POLICY_CFG, CTR_OPEN_MAX(2) | CTR_INIT(0) |
POLICY_MODE(5 /*adapt_access*/));
csr_write(REFRESH_TUNING,  PAGE_POLICY_OR(1 /*OPEN*/));
csr_write(REF_CTRL,        POSTPONE_LIMIT(1) | PULLIN_LIMIT(1));
csr_write(ADDR_MAP,        BANK_LSB(COL_WIDTH) | HASH_EN |
HASH_SEED(seed));  // pre-init
```

49.5.3 Real-Time / Safety-Critical

```
// in_order = per-channel FIFO (global rd-vs-wr age order needs an
// ENHANCED build). Expect a bandwidth cost: under the auto-precharge
paging
// modes each access is two dependent commands, so it pays the
arbiter's pick
// pipeline twice -- see ch02 §7 and pumice ISSUE-006 (was PUMICE-
021).
csr_write(SCHED_POLICY,   ORDER_MODE(1 /*in_order*/));
csr_write(REFRESH_TUNING, PAGE_POLICY_OR(2 /*CLOSE*/));
csr_write(REF_CTRL,       POSTPONE_LIMIT(0) | PULLIN_LIMIT(0));   //
strict tREFI
```

If strict ordering is not actually required and the goal is only bounded latency, prefer `ORDER_MODE(3 /*age_threshold*/)` with `AGE_THRESH(n)`: it runs FR-FCFS until a reference ages past 16*n MC cycles and only then narrows to the aged set, which bounds starvation at a fraction of the bandwidth cost.

49.6 Telemetry-Driven Auto-Tuning Loop

For SoCs with firmware capable of background loops (once observation readback is wired):

```
void periodic_autotune(void) {
    static uint8_t defer = 8;
    uint32_t pending_max = csr_read(OBS_REFRESH_PENDING_MAX);

    if (pending_max > DEFER_BUDGET * 7 / 8)      { if (defer > 1)
defer--; }
    else if (pending_max < DEFER_BUDGET / 4)     { if (defer < 8)
defer++; }

    uint32_t v = csr_read(REFRESH_TUNING);
    v = (v & ~REFRESH_DEFER_ACTIVE_MASK) |
REFRESH_DEFER_ACTIVE(defer);
    csr_write(REFRESH_TUNING, v);   // live on the next refresh event
```

```
boundary
}
```

Run every ~100 ms. Writes are cheap (no commit drain) and workload-dependent tuning is automatic.

49.7 Open Questions / Future Work

- **Observation readback.** The telemetry recipes assume `hwif_in` is wired; it is tied off today (§4.1).
- **Profile-select CSR.** A single "workload profile" field that switches all knobs at once would simplify firmware; adds CSR area. Punt.
- **QoS priority.** `awqos`/`arqos` are on the AXI port but not yet consumed by the scheduler; a v2 hook.

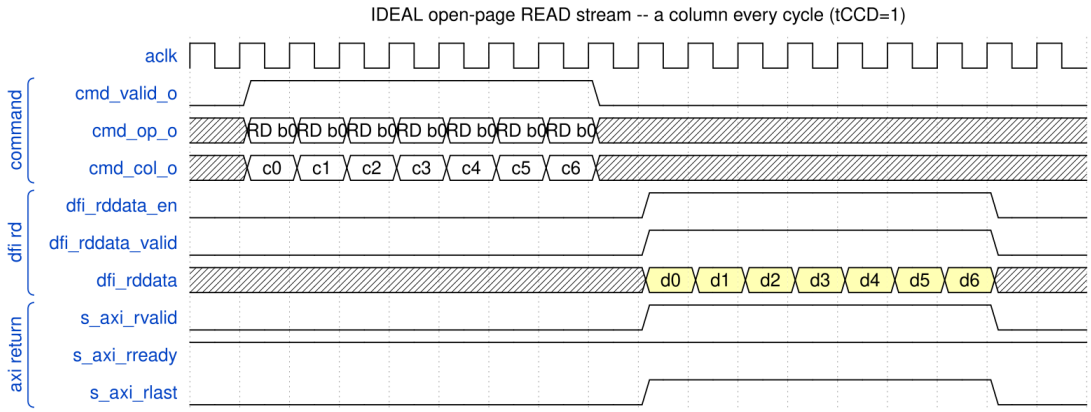
50 Ideal Command Cadence

Every diagram on this page is generated (`design/gen_waves.py`) and rendered (`design/render_waves.py`); none is drawn by hand. They define the cadence the RTL is written to hit, so where a diagram and the RTL disagree, one of them is a bug and the disagreement is the point.

Operating point. Nexys A7, DDR2-300 at aclk 75 MHz, DFI_RATE=2, BL4 on a x16 device with a 32-bit DRAM beat. That gives `BURST_WORDS = 1` DFI word per DRAM burst (`pumice_core.sv:76`), so **one column moves 8 bytes in one aclk cycle** and the peak is 600 MB/s. Timing in aclk cycles: tCCD=1, tRCD=3, tRP=3, tRAS=4, tRC=6, tWTR=2, tRTW=2, tRRD=2, tFAW=6, CL=3, CWL=2, t_rddata_en=6, write_latency=0.

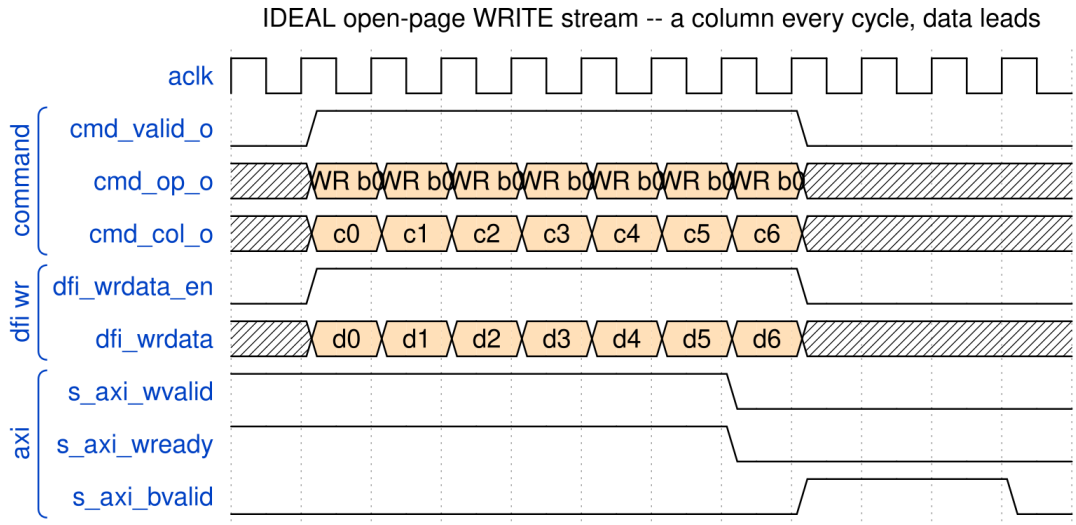
> **Corrected 2026-09-10.** These diagrams previously declared tCCD=2 and drew a column every *other* cycle while captioned "~100% bus util", which cannot both be true. `pumice_core.sv:255` clamps `t_ccd_i` up to `BURST_WORDS`, which is 1 here, and the board sustains ~572 MB/s read (571.3 on the 2026-09-10/14 images, 572.3 re-measured on the post-sdpram-fix image of 2026-10-03 — `docs/char_results/FINDINGS_litedram_ab_2026-10-03.md`). A column every cycle is the ideal, and that is what these now show.

50.1 Streaming



Ideal open-page read stream

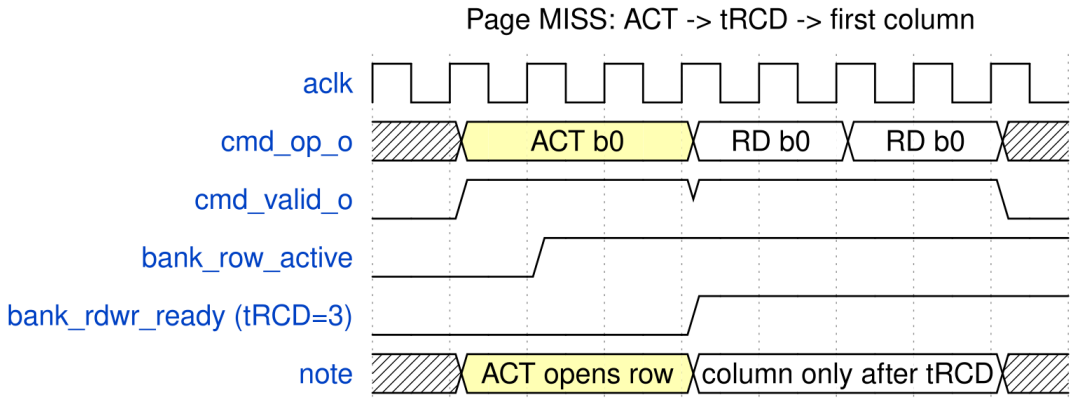
IDEAL open-page READ stream: a column EVERY cycle (tCCD=1 at BL4/x16/32b beat), rvalid continuous after the t_rddata_en+CL fill. 8 B/cycle = 600 MB/s peak; the board measures 572.3 (95%) post-fix 2026-10-03 (571.3 pre-fix). No occupancy stall between same-bank columns – the per-bank mask is AP-gated, so non-AP columns are spaced by tCCD alone.



Ideal open-page write stream

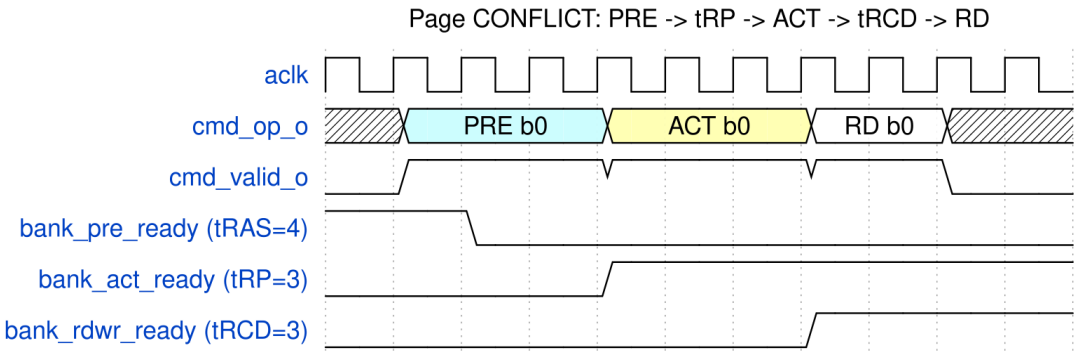
IDEAL open-page WRITE stream: a column EVERY cycle (tCCD=1), wready never drops. Board measures 570.3 MB/s = 95% of the 600 MB/s peak. Write data LEADS the command (rate-matched commit + CMD_DELAY); a WR held at the DFI for want of data stalls the whole in-order command stream behind it.

50.2 Paying for a row



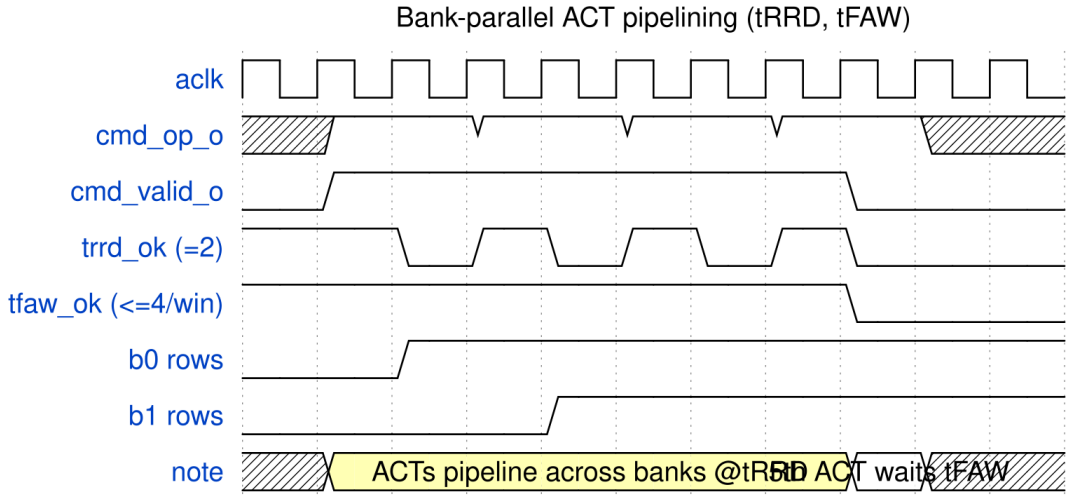
Page miss: activate then column

PAGE MISS: ACT then wait tRCD=3 before the first column. First-access latency only – subsequent hits stream at tCCD (see open_read_stream).



Page conflict: precharge, activate, column

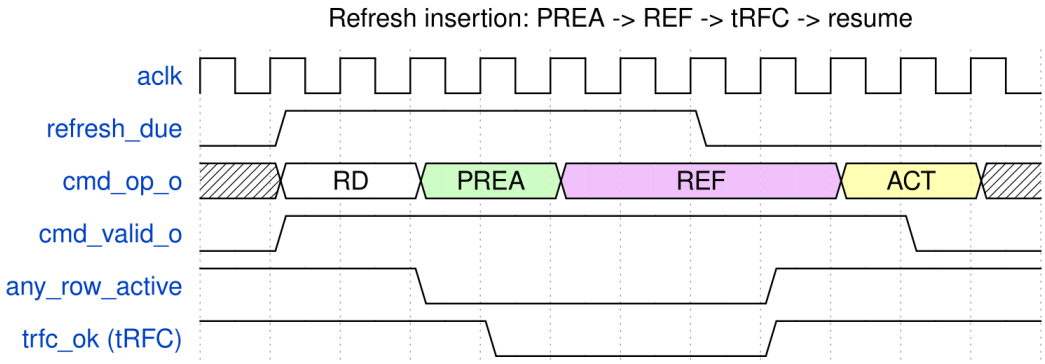
PAGE CONFLICT: PRE (after tRAS) -> tRP -> ACT -> tRCD -> RD. Worst case; open-page avoids it on hits.



Bank-parallel activates

CROSS-BANK ACT pipelining: one ACT per tRRD=2, capped at 4 per tFAW window. Their tRCDs overlap so columns from multiple banks are ready together -> the 4x.

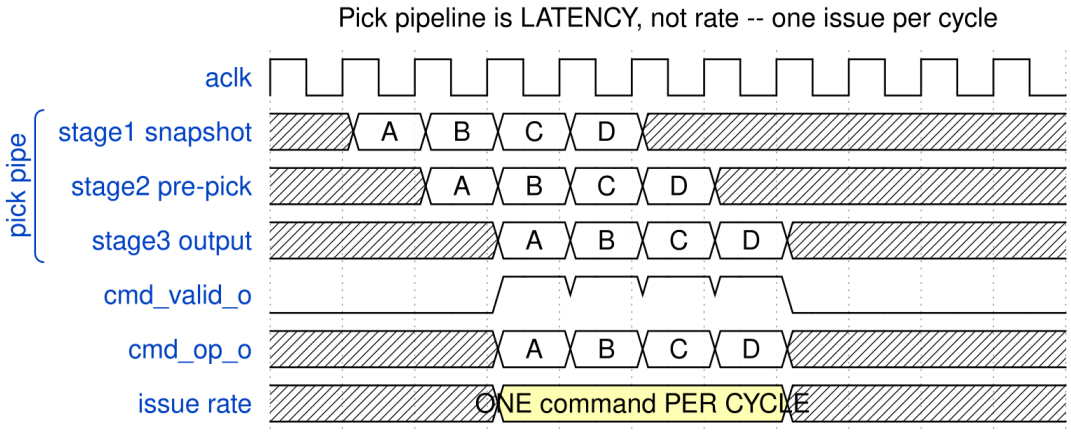
50.3 Maintenance



Refresh insertion

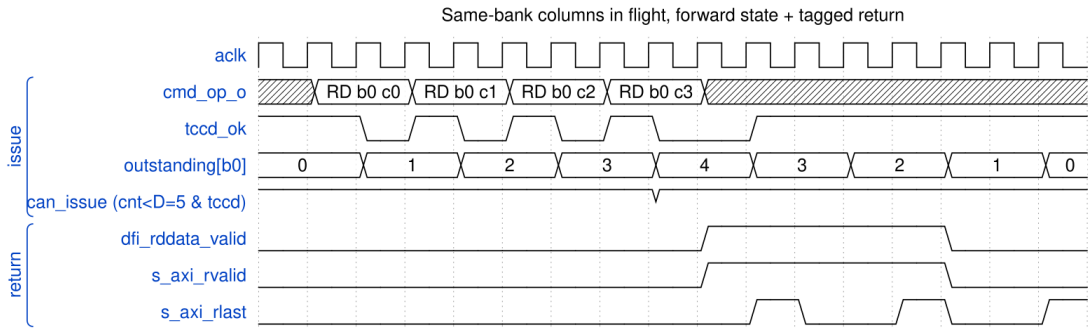
REFRESH: precharge-all -> REFab -> wait tRFC -> resume (re-ACT). The only maintenance bubble; postpone/pullin credits (TASK-001) move it out of demand windows.

50.4 Pipelining and reordering



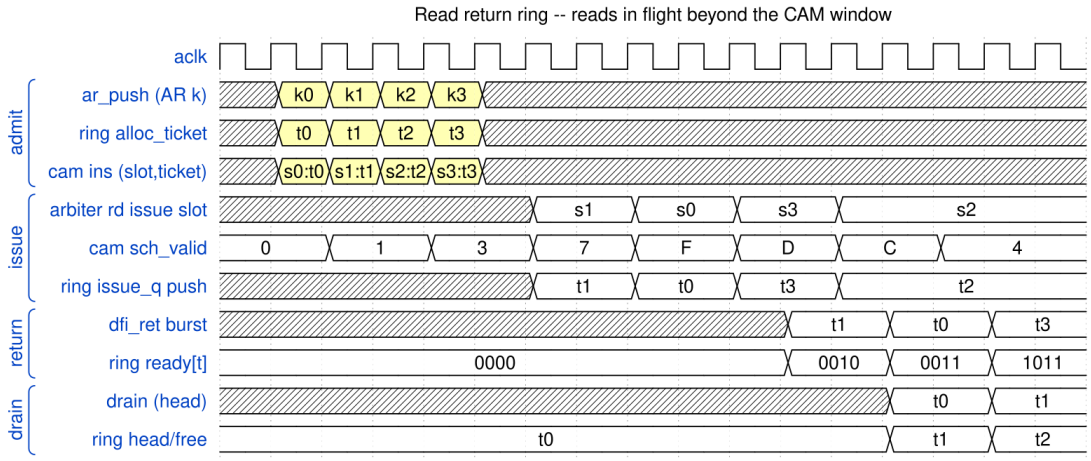
The pick pipeline is latency, not rate

IDEAL pick pipeline: A,B,C,D advance one stage/cycle, one FIRES every cycle after a 3-cycle fill. Pipeline = LATENCY, not throughput. Today's bug: an occupancy mask blocks B until A drains -> 1 per 4 cycles. DELETE it; gate only on DRAM timers.



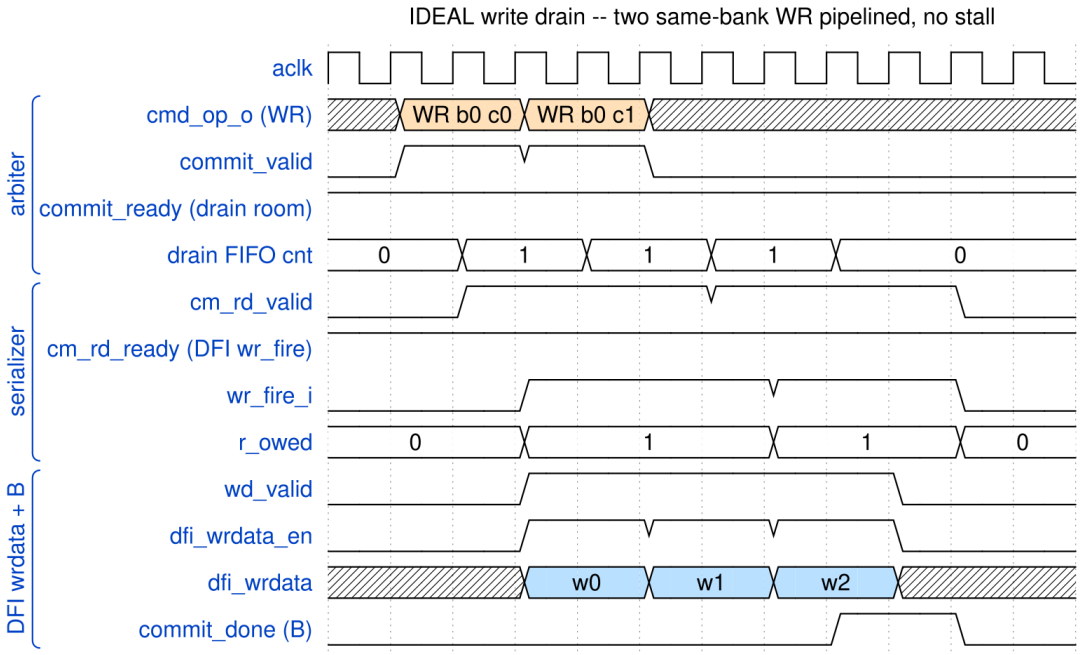
Same-bank columns in flight

DEADLOCK FIX: up to D=ceil((t_rddata_en+CL)/tCCD)=5 same-bank columns in flight; per-bank counter gates issue (cnt<D & tCCD), completion (R-last/B) decrements. Today one-per-bank is forced -> 15%% util. Return path (rd issue-FIFO, aligner MAX_OUTSTANDING) must hold D.



Read return ring

READ RETURN RING: CAM entry lives insert->issue; the ticket (AR-order ring slot) follows the read through DRAM; returns fill by ticket in issue order; the ring drains its head in AR order once complete. In-flight bound = RD_RET_DEPTH (32), not NUM_ENTRIES (8): Little's law 32 x 8 B / 27 cyc > the 8 B/cyc bus.



Ideal write drain

IDEAL write drain: 2 same-bank WR columns pipeline at tCCD; drain FIFO stays shallow, cm_rd_ready/wr_fire keep pace, wrdata streams, one B per host burst. No stall on commit_ready.

50.5 Regenerating

```
python3 design/gen_waves.py          # JSON, self-checking
python3 design/gen_write_path.py     # JSON for the write-path pair
python3 design/render_waves.py       # ->
docs/pumice_mas/assets/waves/*.png
```

`design/check_waves.py` runs inside all three. WaveDrom renders a malformed diagram without complaining – a short `data` list leaves buses blank, ragged rows render out of step, a long caption runs off the image – so an unchecked render is a picture of a bug. It found eleven real defects in this set when it was first written, five of them labels attached to a logic level instead of a bus slot, which silently shifts every label in that row onto the wrong segment.

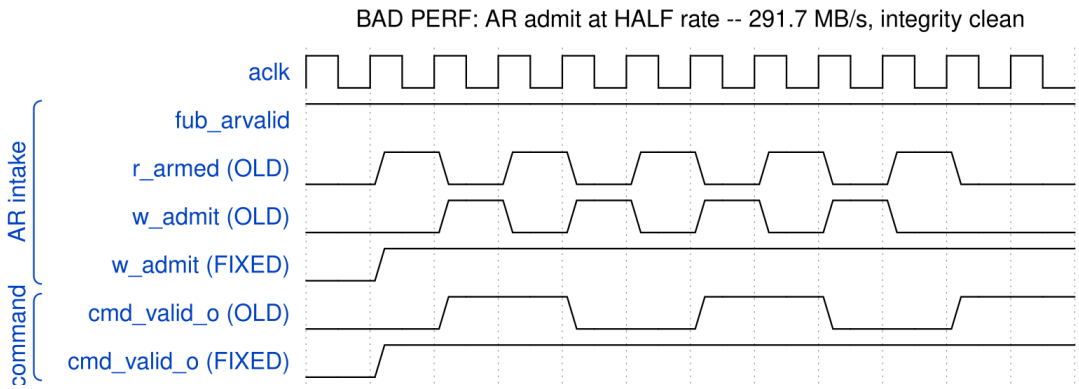
51 Performance Pathologies

A specification made only of ideals cannot be used to recognise a bad capture. These are the shapes that mean something is wrong, each one measured on this board and captioned with the number it produced, so an ILA trace can be matched against it by eye.

They divide into two kinds, and the distinction matters more than it sounds. **Bad performance is correct**: every byte is right, every transaction completes, and the only symptom is the shape. That is the class that ships, because nothing fails. **Pathological** shapes are the ones where the design is wedged or about to be.

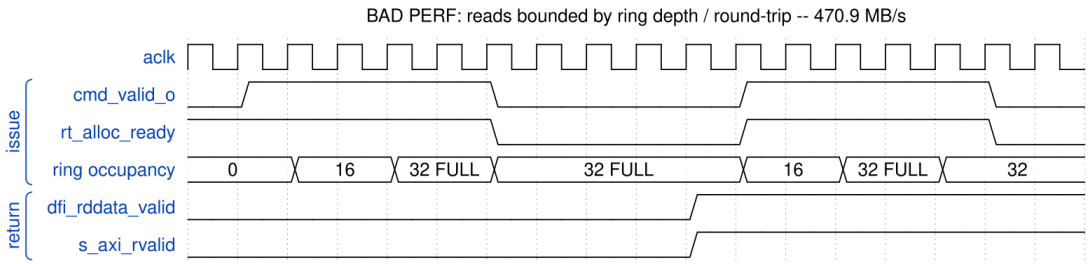
> **Vintage.** These shapes were captured 2026-09, before the sdpram burst-boundary fix (`amba 71d48b6f7`, 2026-10-02). The fix is on the harness UART/debug SRAM, off this datapath, and the 2026-10-03 post-fix board re-run reproduces the healthy endpoints these shapes are measured against (writes 570.3, row_major read 572.3 MB/s — `char-perf/reports/char_postfix_2026-10-03.csv`, FINDINGS_litedram_ab_2026-10-03.md). Captions below keep their capture-date numbers; where a post-fix re-measure exists it is noted inline.

51.1 Bad performance – correct, just slow



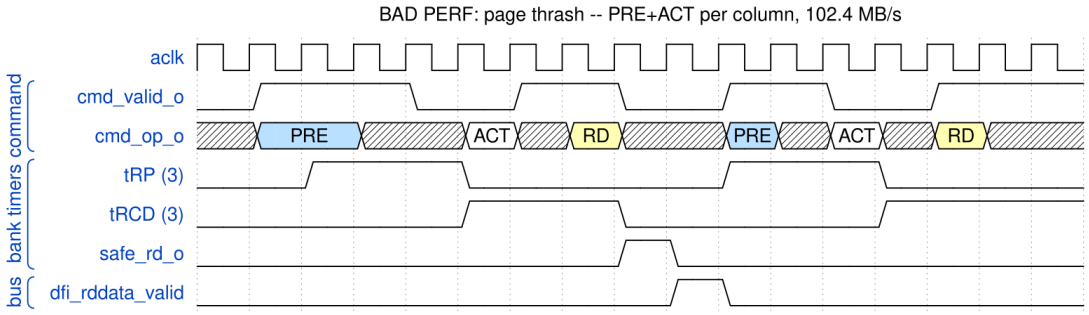
Admit gate at half rate

BAD PERF – read admit at HALF rate. The arm bit was cleared by its own admit and could only re-set the next cycle, so one sub-command (= one DRAM burst = one column here) admitted every TWO cycles. Ceiling 0.5 x 8 B x 75 MHz = 300 MB/s; measured 291.7 against 570 for writes. Integrity was perfect throughout – this shape is the ONLY symptom.



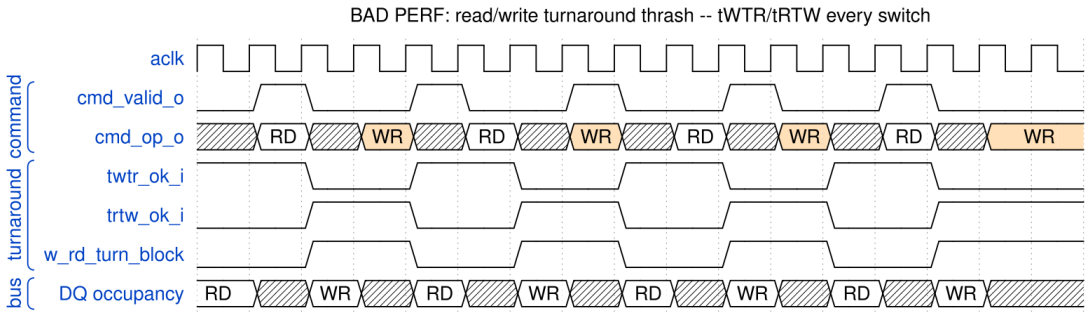
Reads bounded by the return ring

BAD PERF – reads bounded by the RETURN RING, not by tCCD. Tickets allocate at admit and free only when the beat drains ~49 cycles later, so the sustained rate is DEPTH/latency: 32/49 = 0.78 col/cycle = 470.9 MB/s, which is exactly what the board measured at depth 32. Issue goes idle in bursts (alloc_ready low) even though every DRAM timer is clear. Depth 64 -> 571.3 MB/s (572.3 re-measured on the post-fix image, 2026-10-03).



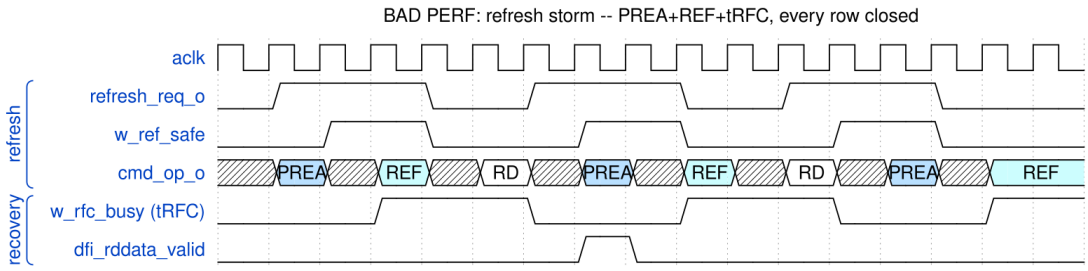
Page thrash

BAD PERF – page thrash (col_major). Every access is a different row in the SAME bank, so each column costs PRE + tRP + ACT + tRCD before it can issue: ~8 cycles of overhead per 8 bytes. Board: 102.4 MB/s read at AxLEN 4 on the 2026-09-10 image (114.7 re-measured 2026-10-03, open_page col_major bl4) against 572.3 for row_major – a 5x penalty with identical DRAM and identical controller settings. The fix is the ADDRESS MAP, not the controller.



Read/write turnaround thrash

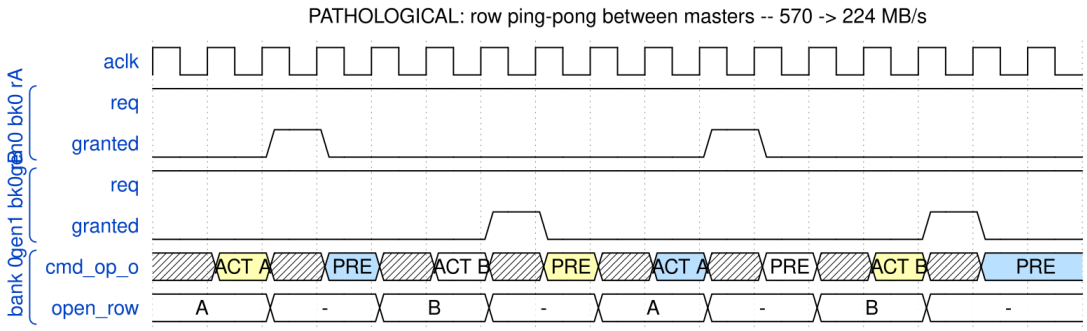
BAD PERF – read/write turnaround thrash. Switching direction every column pays tWTR or tRTW each time and the DQ bus idles in the gap. This is the workload a global reorder window exists to fix: pumice batches same-direction columns and sustains 571.1 MB/s with both directions live (2026-10-04 post-fix re-verification, 570.1 on 2026-09-10), where LiteDRAM's per-bank round-robin pays the turnaround and reaches 285.8 (285.6 on 2026-09-10).



Refresh storm

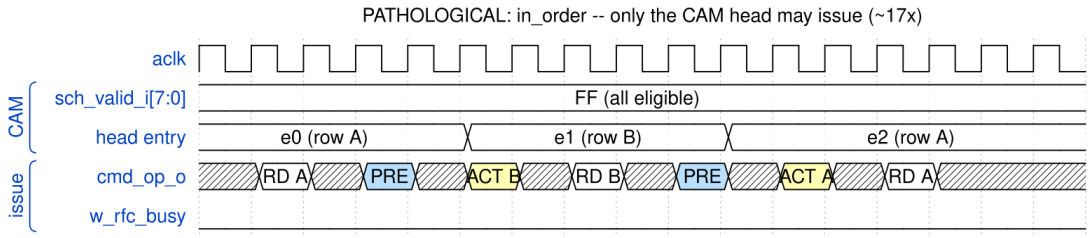
BAD PERF – refresh storm. With tREFI cranked down, every refresh costs PREA + REF + tRFC and closes every open row, so the next access is a guaranteed page miss too. w_rfc_busy blocks ACT and REF alike. Board sweep: fast_refresh vs slow_refresh is the axis that isolates this; the cost is roughly tRFC/tREFI of the bus plus the re-activation of every row it closed.

51.2 Pathological



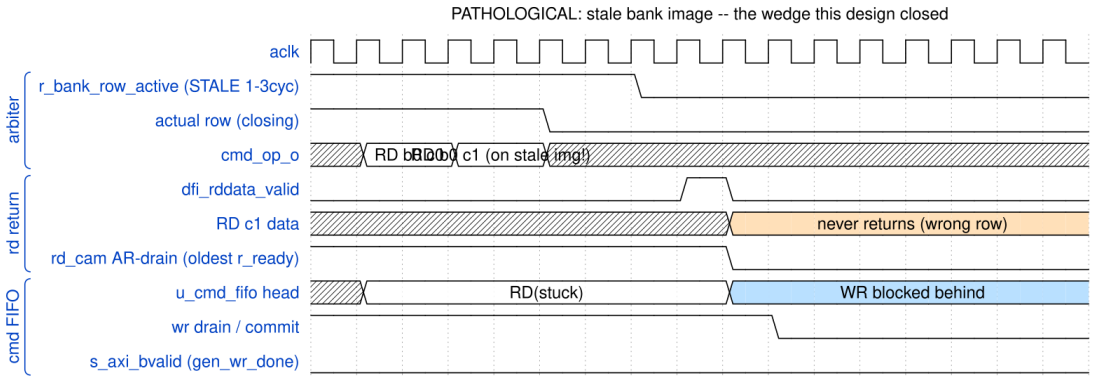
Row ping-pong between masters

PATHOLOGICAL – row ping-pong between masters. Two generators on different ROWS of the same banks force a PRE+ACT pair between every pair of columns; the row buffer never survives a grant. Measured accidentally 2026-09-10: spacing two readers a device/4 region apart put them in the same banks and collapsed row_major from 570 to 224 MB/s. Place concurrent masters in NEIGHBOURING banks, not distant rows.



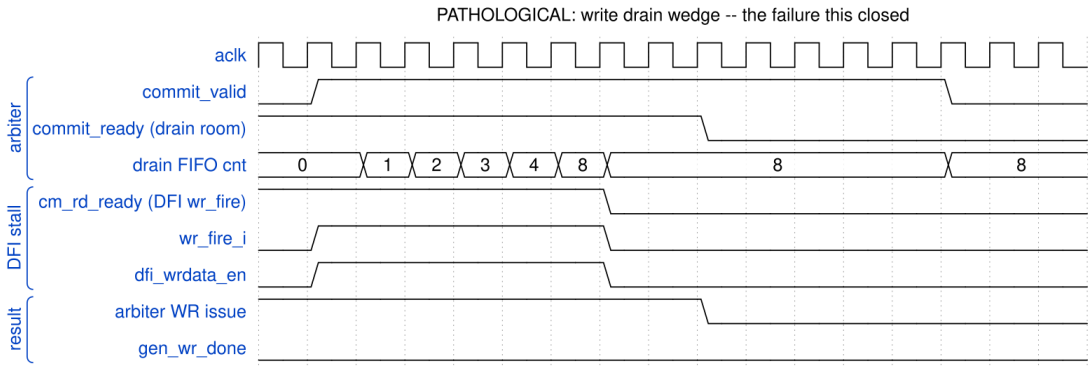
In-order serialization

PATHOLOGICAL – in_order (SCHED_POLICY.order_mode=1). Every entry is eligible and several are page hits, but only the HEAD may issue, so a row-interleaved stream pays PRE+ACT between consecutive columns while the hit sits two entries back. This is the cost of the mode, not a defect: it exists to make ordering observable. Roughly 13x on the board (2026-10-03 post-fix image: 45.0 MB/s vs 572.3 open_page, row_major bl8; was ~17x against the 2026-09 close-page baseline) – use it to prove reordering is what is buying the bandwidth, never in production.



Stale bank image (closed)

CURRENT FAILURE (mask relaxed, no forward-state): 2nd same-bank RD classified on STALE row image -> lands on closing row -> never returns -> in-order AR-drain wedges -> stuck RD head-of-line-blocks WRs in the shared cmd FIFO -> write ‘wedges first’. Reference only.



Write drain wedge (closed)

CURRENT WEDGE (hypothesis, to confirm by waveform): same-bank WR pipelining fills the drain FIFO (8); cm_rd_ready (DFI accepting writes) stalls -> commit_ready=0 -> arbiter WR stops -> gen_wr_done never asserts. MEASURE which CM_RD_STALL candidate holds wr_fire off (see kmap sheet 2).

51.3 How to use these

Take the board's own numbers first (`build-perf/host/pumice_master.py --char`), then match the shape. The bandwidth alone rarely says which of these you have: the admit gate, the ring bound and a refresh storm all present as "reads are slow and the data is fine", and they are told apart by *where* the idle cycles sit – at the intake, at the ticket allocator, or behind a tRFC counter.

The last two diagrams are historical. They are the failure chains this design closed, kept because recognising them is still the fastest way to rule them out.